

STEPSS

Static and Transient Electric Power Systems
Simulation

Petros Aristidou Thierry Van Cutsem



STEPSS

STATIC & TRANSIENT ELECTRIC
POWER SYSTEMS SIMULATION



Static and Transient Electric Power Systems Simulation

Documentation of models and user guide

Petros Aristidou Thierry Van Cutsem

Sustainable Power Systems Lab, Cyprus University of Technology

Friday 21st August, 2026

ACADEMIC PUBLIC LICENSE FOR THE USE OF STEPSS

1. Definitions related to the software

“Software” means a copy of STEPSS, which is distributed under this Academic Public License. “Work based on the Software” means either the Software or any derivative work under copyright law: that is to say, a work containing the Software or a portion of it, either verbatim or with modifications. “Using the Software” means any act of creating executables that contain or directly use libraries that are part of the Software, running any of the tools that are part of the Software, or creating works based on the Software. “You” refers to each licensee.

2. Authors

STEPSS has been developed by Dr Petros Aristidou (email: petros.aristidou@cut.ac.cy) and Dr Thierry Van Cutsem (email: thierry.h.van.cutsem@gmail.com), hereafter referred to as the “Authors”.

3. Intellectual property rights

STEPSS is made up of three modules: Helios (for power flow computations), RAMSES (the solver of differential-algebraic equations), and CODEGEN (a tool to develop models).

Helios is the property of Dr. Petros Aristidou. CODEGEN is the property of Dr. Thierry Van Cutsem. RAMSES is the property of the University of Liège, Belgium, which has granted to both Authors a personal, royalty-free, limited, non-exclusive, non-transferable and non-assignable license to distribute free of charge an executable version of RAMSES in accordance with the terms detailed under Articles 4, 5 and 6 below.

4. License terms

Permission is hereby granted to use the Software free of charge for any non-commercial purpose, including teaching and research at universities, colleges and other educational institutions, research at non-profit research institutions, and personal non-profit purposes. For using the Software for commercial purposes, including but not restricted to consulting activities, design of commercial hardware or software products, or if you are a commercial entity participating in research projects, you have to contact the Authors.

You are not required to accept this License. Nothing else grants you permission to use the Software; the law prohibits this action if you do not accept this License. Therefore, by using the Software (or any work based on the Software), you indicate your acceptance of this License and all its terms and conditions.

5. Warranty

The Software is provided “as is”, without warranty of any kind, express or implied, including but not limited to the warranties of merchantability, fitness for a particular purpose and non-infringement. In no event shall the Authors nor the Intellectual property owners be liable for any claim, damages or other liability, whether in an action of contract, tort or otherwise, arising from, out of or in connection with the software or the use or other dealings in the Software.

6. Limited version of the Software

This free-of-charge version of the software is limited to power system models up to 1000 buses (or nodes) and can be executed with parallelization using no more than two cores. For extensions to larger models or execution using more than two cores, you have to contact the Authors.

Contents

I	Information pertaining to all three modules	1
1	A quick overview of STEPSS	3
1.1	The three modules of STEPSS	4
1.2	Helios module	5
1.3	RAMSES module	5
1.4	CODEGEN module	7
2	Installing STEPSS	9
2.1	Installing STEPSS	10
2.2	Installing a Fortran Compiler	11
3	Driving the engines: GUI and Python	13
3.1	Dynamic Simulation (RAMSES)	13
3.2	Power Flow (Helios)	14
3.3	Typical Workflow	15
3.4	Next Steps	15
4	Organisation of the data files	17
4.1	Records	18
4.2	Comments	19
4.3	Sharing data between files	19
5	Modelling of network components	21
5.1	Buses	22
5.2	Lines and cables	22
5.3	Switches	23
5.4	Transformers	24
5.5	Non-reciprocal two-ports	26
5.6	Shunts	27
II	The STEPSS graphical interface	29
6	First run of STEPSS GUI	31
6.1	Accepting the licence	31
6.2	Start from a bundled example	31
6.3	The four bundled systems	34
6.4	Where the files go	34
6.5	Real-time plotting	34
6.6	Choosing a theme	35
6.7	Next steps	35
7	The STEPSS GUI interface	37
7.1	System Data	37

7.2	Observables	38
7.3	Power Flow Simulation	39
7.4	Dynamic Simulation	42
7.5	Analysis	42
7.6	Codegen	44
7.7	The menus	44
7.8	The status bar	45
8	Running a simulation in STEPSS GUI	47
8.1	1. Check what was loaded	47
8.2	2. Choose what to record	47
8.3	3. Solve the power flow	48
8.4	4. Run the simulation	48
8.5	5. Extract the curves	48
8.6	6. Read the result	49
8.7	Try it with the stabiliser out	49
8.8	Where to go next	51
III	Power Flow Computation with Helios	53
9	Helios data	55
9.1	Load and shunt data	56
9.2	Generator data	57
9.3	Slack-bus specification	58
9.4	Static Var Compensators	58
9.5	Transformer ratio adjustment for voltage control	59
9.6	Phase-shifting transformer ratio adjustment for power control	60
9.7	Bus voltages: initial values and results	62
9.8	Share of records between Helios and RAMSES	62
9.9	Computation control parameters	62
IV	Dynamic Simulation with RAMSES	65
10	Reference frame and model initialization	67
10.1	Phasor approximation and reference frames	68
10.2	Network equations	69
10.3	Initialization procedure	70
10.4	Data format	72
11	Synchronous machines and loads	73
11.1	Nominal system frequency	74
11.2	Synchronous machines	74
11.3	Thévenin equivalents	85
11.4	Impedance loads	86
12	Disturbances	87
12.1	Continue Solver	88
12.2	Continue Display	88
12.3	Set stopping criteria for voltage	88
12.4	Set stopping criteria for machine speed	88

12.5	Stop	88
12.6	Trip line	89
12.7	Trip synchronous machine / injector	89
12.8	Trip two-port	89
12.9	Three phase short-circuit (with use of resistance to ground)	89
12.10	Three phase short-circuit (with use of voltage reached after fault)	89
12.11	Change parameters	90
12.12	Export Jacobian matrix	91
12.13	Small-signal stability analysis	91
12.14	Export load flow	91
13	Solver Settings	93
13.1	Sampling time for observed variables	94
13.2	Display Profiling Results	94
13.3	Run-time observables refresh interval	94
13.4	Observable buffer size	94
13.5	Time constant of load restoration	94
13.6	Omega Reference	94
13.7	Maximum Fault Value	94
13.8	Base Power	94
13.9	Nominal Frequency	94
13.10	Newton Tolerance	94
13.11	Finite Difference Values	94
13.12	Full Jacobian Update	94
13.13	Skip Converged Blocks	95
13.14	Latency Tolerance	95
13.15	Solution Scheme	95
13.16	Number of Threads for parallel computing	95
13.17	Way of injector distribution over parallel threads	95
13.18	Update network elements with frequency	95
13.19	Call Gnuplot	95
13.20	Gnuplot output mode	95
13.21	Minimum branch impedance	95
13.22	Size limit of the small-signal analysis	96
13.23	Latency on subnetworks	96
13.24	Load a compiled model library	96
13.25	License key	96
14	Discrete controller models	97
14.1	Tap Changers	97
14.2	Protection	101
14.3	Monitoring	108
15	Small-signal stability analysis	111
15.1	What is computed	111
15.2	Running an analysis	111
15.3	Output files	112
15.4	Saving a run as one archive	113
15.5	Degenerate modes, and why the <code>smp</code> column matters	114
15.6	Refusals	114
15.7	Worked example	114

15.8	See Also	116
V	Model library	117
16	The model library	119
16.1	How a Model Name Resolves	119
16.2	Exciters	119
16.3	Governors	120
16.4	Injectors	120
16.5	Two-Port Models	120
16.6	Discrete Controllers	121
16.7	The Synchronous Machine	121
16.8	Next Steps	121
17	Synchronous machine models	123
17.1	Model Switches	123
17.2	Park Transformation	123
17.3	Flux-Current Relationships	125
17.4	Saturation Model	125
17.5	Reference Frame	126
17.6	Park Equations	127
17.7	Rotor Motion	127
17.8	State Variables and Equations Summary	128
17.9	Per Unit System and IBRATIO	128
17.10	SYNC_MACH Record	128
17.11	Initialization Output	130
17.12	Parameter Conversion (XT ↔ RL)	131
18	Synchronous machine parameter conversion	133
18.1	Authoritative Field Order	133
18.2	Conversion Algorithm (XT → RL)	134
18.3	Reference Python Implementation	135
18.4	Worked Examples	135
18.5	Comparing with an EMT Simulator	135
18.6	See Also	136
19	IEEE excitation controllers	137
19.1	Model Index	138
19.2	AC Exciter Models	139
19.3	DC Exciter Models	144
19.4	ST (Static) Exciter Models	145
19.5	Other IEEE Models	148
19.6	ENTSO-E Models	150
19.7	PSS Models	151
19.8	OEL and Limiter Models	155
20	Other excitation controllers	157
20.1	CONSTANT (exc_constant)	157
20.2	1ST_ORDER (exc_1storder)	158
20.3	kundur (exc_kundur)	159
20.4	GENERIC1 (exc_generic1)	161

20.5	AVR_DG (exc_AVR_DG)	162
20.6	GENERIC2 (exc_generic2)	163
20.7	GENERIC (exc_GENERIC)	165
20.8	See Also	167
21	IEEE turbine-governor models	169
21.1	DEGOV1 (tor_DEGOV1)	170
21.2	ENTSOE_simp (tor_ENTSOE_simp)	172
21.3	GAST (tor_GAST)	173
21.4	TGOV1 (tor_TGOV1D)	175
21.5	HYGOV (tor_hygov)	177
22	Other turbine-governor models	181
22.1	CONSTANT (tor_constant)	181
22.2	1ST_ORDER (tor_1storder)	182
22.3	THERMAL_GENERIC1 (tor_thermal_generic1)	184
22.4	HYDRO_GENERIC1 (tor_hydro_generic1)	186
22.5	HYDRO_DG (tor_HYDRO_DG)	187
22.6	tor_gasturbm	189
22.7	tor_govclasm	190
22.8	tor_govhydr	191
22.9	tor_govnuc	193
23	Injector models	195
23.1	Load Models	196
23.2	Induction Machine Models	201
23.3	Renewable Generation / Inverter-Based Resources	205
23.4	Energy Storage	218
23.5	Reactive Compensation	219
23.6	Measurement	220
24	Two-port models	223
24.1	HVDC Models	224
VI	Adding user-defined models with CODEGEN	237
25	User-defined models	239
25.1	States and equations	240
25.2	Discrete transitions	243
25.3	Model assembly	245
25.4	Syntax of the model description	246
25.5	Error detection	251
25.6	Data format	252
26	Library of modelling blocks	255
26.1	List of modelling blocks	256
26.2	Information provided for each block	256
26.3	Library	257
26.4	Functions available in models	317
27	Worked CODEGEN examples	321
27.1	Exciter Model: exc_ST1A	321

27.2	Governor Model: <code>tor_hygov</code>	325
27.3	Injector Model: <code>inj_vfd_load</code>	327
27.4	Two-Port Model: <code>twop_HVDC_LCC</code>	329
28	CODEGEN Studio	335
28.1	Installation	335
28.2	Interface Overview	335
28.3	Creating a Model	336
28.4	Saving a Project	340
28.5	Loading a Project	340
28.6	Importing a DSL File	340
28.7	Checking the Model	341
28.8	Exporting a DSL File	341
28.9	Running CODEGEN	342
28.10	File Formats Summary	343
28.11	Keyboard Shortcuts	343
28.12	Settings	343
28.13	Workflow Example: Building an AVR Model	344
28.14	See Also	345
VII	STEPSS in Python	347
29	STEPSS in Python	349
29.1	Package-Level Attributes	349
29.2	Main Classes	349
29.3	Platform Support	349
29.4	Further Reading	350
29.5	Repository	350
30	Installing the Python package	351
30.1	Installation	351
30.2	Linux System Prerequisites	351
30.3	macOS System Prerequisites	352
30.4	Run-time observables	352
30.5	Live plotting	352
30.6	Platform Support	352
30.7	Next Steps	353
31	Python examples	355
31.1	Running a Basic Simulation	355
31.2	Pause and Continue	355
31.3	Querying System State	356
31.4	Adding Disturbances at Runtime	356
31.5	Plotting Results	357
31.6	Live Plotting	358
31.7	Parameter Sweep	359
31.8	Eigenanalysis Workflow	361
31.9	Test System Examples	361
32	Python API reference	365
32.1	<code>stepss.cfg</code> : Test Case Configuration	365

32.2	stepss.sim: Simulation Control	369
32.3	stepss.extractor: Result Extraction	374
32.4	stepss.monitor: Live Plotting	378
32.5	Complete Example	380
32.6	Next Steps	381
33	Power flow from Python	383
33.1	Basic Workflow	383
33.2	Solver Options	383
33.3	Vectorized Results	383
33.4	Modifying the System	384
33.5	Contingency Screening	384
33.6	Exporting Results	384
33.7	Runnable Examples	385
33.8	Further Reading	385
34	uRAMSES, the C and Fortran interface	387
34.1	Prerequisites	387
34.2	Project Structure	388
34.3	Model Types	389
34.4	Building	389
34.5	Adding Custom Models	391
34.6	Using Custom Models	392
34.7	Helper Functions	392
34.8	Repository	392
VIII	Test systems and resources	393
35	Test systems	395
35.1	Which One to Start With	395
35.2	Running Any of Them	395
35.3	Next Steps	396
36	The 5-bus tutorial system	397
36.1	One-line Diagram	397
36.2	Quick Start	397
36.3	Download	398
36.4	See Also	398
37	The Kundur two-area system	399
37.1	Quick Start	399
37.2	Download	399
37.3	Citation	399
37.4	See Also	400
38	The IEEE Nordic test system	401
38.1	System Overview	401
38.2	One-line Diagram	401
38.3	Dynamic Models	401
38.4	Operating Points	403
38.5	Repository Contents	403
38.6	Disturbance Scenarios	404

38.7	Quick Start	404
38.8	What to Observe	405
38.9	License	405
38.10	References	405
38.11	See Also	405
39	The Great Britain network	407
39.1	One-line Diagram	407
39.2	Quick Start	407
39.3	Download	407
39.4	See Also	409
40	Component repositories	411
40.1	Repository Index	411
40.2	Public Repositories	411
41	References	415
41.1	Core RAMSES / STEPSS Publications	415
41.2	Reference Frame and Solver	415
41.3	Project Pages	415
41.4	External Resources	415

Part I

Information pertaining to all three modules

1

A quick overview of STEPSS

1.1. The three modules of STEPSS

STEPSS includes three modules:

- **Helios** (for Power Flow Computation) performs a power flow computation in order to determine the initial operating point of a dynamic simulation;
- **RAMSES** (for RApid Multithreaded Simulation of Electric power Systems) simulates the dynamic evolution of the power system in response to disturbances/actions specified by the user;
- **CODEGEN** (for CODE GENERator) translates a model described by the user in a text file into its equivalent in FORTRAN 2003 language. The latter has to be compiled and linked to a user-defined executable version RAMSES.

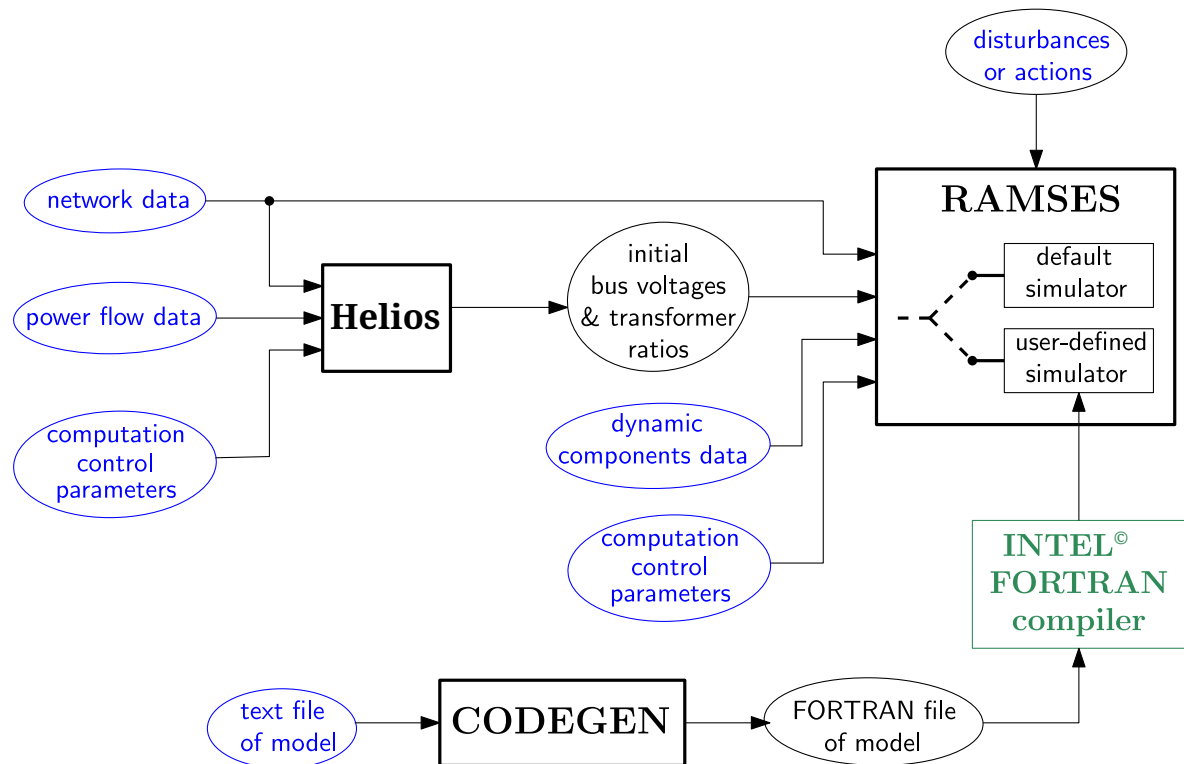


Figure 1.1: STEPSS: Overview of modules and data files. The files shown in blue are provided by the user, those in black are produced internally. The Intel® FORTRAN compiler is not part of STEPSS

The three modules, their relationships and their input/output data files are shown graphically in Fig. 1.1.

Note that each of the three modules can be used separately. Here are examples of such uses:

- Helios alone: a power flow computation is run to inspect the power system state and/or save the corresponding power flow solution for use by RAMSES;
- Helios alone: a sequence of power flow computations is performed until obtaining the desired power system state. The corresponding power flow solution is saved for use by RAMSES;
- RAMSES alone: with the initial power flow solution produced by Helios, several dynamic simulations are run starting from that initial state, i.e. without invoking Helios each time;

- CODEGEN alone: a model is built and saved for future incorporation in a user-defined version of RAMSES.

1.2. Helios module

The power flow computation resorts to the well-known Newton(-Raphson) method. Polar coordinates are used.

As shown in Fig. 1.1, the input information consists of:

- network data relative to buses, lines, transformers, etc.
- power flow data specified at PV, PQ and slack buses, respectively
- Helios control parameters. These are settings such as: tolerances on final power mismatches, thresholds to enforce reactive power limits of generators, etc. These are optional data; if they are not provided, default values (listed in this documentation) are used.

Optionally Helios also adjusts the ratios of some transformers, with the objective:

- for an in-phase transformer: to bring the voltage magnitude at a bus inside a specified deadband
- for a phase-shifting transformer: to bring the active power flow in a network branch inside a specified deadband.

Helios produces an output file including:

- the voltage magnitudes and phase angles at all buses of the network
- the adjustable transformer data with updated values of their ratios.

1.3. RAMSES module

RAMSES is aimed at simulating the dynamic response of power system models derived under the phasor approximation (also known as RMS approximation).

It takes as input (see Fig. 1.1):

- the network data, which are shared with Helios¹
- the data pertaining to the dynamics of the components connected to the network
- RAMSES control parameters. These are settings such as: tolerances for solving the equations, variation of state variables to identify the Jacobians, time steps of plots, reference angular speed, etc.
- the sequence of actions and/or disturbances imposed during the simulation.

The mathematical model involves both differential and algebraic equations. Three algebraization methods are available to integrate the differential equations:

- Backward Euler:

$$x_{k+1} = x_k + h \dot{x}_{k+1}$$

¹with a few exceptions (detailed in this document)

- Trapezoidal method:

$$x_{k+1} = x_k + \frac{h}{2} (\dot{x}_{k+1} + \dot{x}_k)$$

- Second-order Backward Differentiation Formula (BDF2):

$$x_{k+1} = \frac{4}{3}x_k - \frac{1}{3}x_{k-1} + \frac{2h}{3}\dot{x}_{k+1}$$

where h is the time step size. The three methods are implicit, which contributes to numerical robustness. BDF2 is a stiff-decay (or L_1 -stable) integration scheme allowing to somewhat increase the time step size when fast transients are not of interest.

The resulting algebraized and the original algebraic equations are solved all together using a Newton method. Some more information follows.

1.3.1. About the solver

The solver was developed in response to the growing demand for simulations that last longer (e.g. long-term stability studies) or involve larger models (e.g. to account for the impact of active distribution networks). A high computational efficiency is made possible by two acceleration techniques: parallel processing and localization.

Parallel processing is based, first, on the decomposition of the power system model into respectively the network, the “injectors” and the “two-ports”. Injectors and two-ports are solved independently of each other. However, by resorting to the Schur-complement for the network equations, RAMSES yields the exact same solution as a non-decomposed scheme².

Next, the tasks pertaining to injectors and two-ports are assigned to a number of *threads*. Examples of tasks are:

- the update and factorization of injector and two-port Jacobians
- the computation of the mismatch vector of Newton method
- the computation of injector contributions to the Schur-complement matrix
- the solution of local linear systems.

The threads can be executed each on a separate processor, the computational load being balanced among the available processors. The freeware version of STEPSS allows exploiting two processors.

The solver in RAMSES has been developed using a shared-memory parallel programming model with the help of the OpenMP Application Programming Interface. The implementation is general: there is no “hand-crafted” optimization particular to the computer system, the power system or the disturbance.

Localization is based on the fact that, after a disturbance, the various components of a (large enough) system exhibit different levels of dynamic activity.

This property is exploited at each time step:

- to accelerate the Newton scheme: thanks to the decomposed solution scheme, Newton iterations are skipped on injectors and two-ports that have already converged
- to exploit component “latency”: injectors with high dynamic activity are classified as active, the others as latent. Active injectors have their original model simulated, while latent injectors are replaced by automatically calculated, sensitivity-based models to accelerate the simulation. A fast to compute metrics is used to classify the injectors, which seamlessly switch between categories according to their activity.

²The solution scheme is thus of the simultaneous type while offering some advantages of a partitioned scheme

More information on the solver can be found in the following references:

- D. Fabozzi, A. Chieh, B. Haut, and T. Van Cutsem, "Accelerated and localized newton schemes for faster dynamic simulation of large power systems," IEEE Trans. on Power Systems, Vol. 28, No 4, pp. 4936-4947, Dec. 2013
doi: 10.1109/TPWRS.2013.2251915
- P. Aristidou, D. Fabozzi, and T. Van Cutsem, "Dynamic simulation of large-scale power systems using a parallel Schur-complement-based decomposition method," IEEE Trans. on Parallel and Distributed Systems, Vol. 25, No 10, pp. 2561-2570, Sept. 2014,
doi:10.1109/TPDS.2013.252

1.4. CODEGEN module

CODEGEN allows incorporating user-defined models in RAMSES. Different versions of the RAMSES executable can be loaded and executed. This involves a translation of the user model specified in a text file into FORTRAN 2003 code to be compiled and linked to the rest of the executable. The FORTRAN 2003 language can produce very efficient number crunching code.

Don't panic: in the vast majority of cases, the user does not need to even open the FORTRAN file. The user concentrates on the text file describing his/her model, which is rather easy to read.

Thus, the user model is not interpreted but executed.

There are four types of user-defined models:

- excitation controller of synchronous machine: typically the excitation system and the automatic voltage regulator
- torque controller of synchronous machine: typically the turbine and the speed governor
- injector: a component connected to a single AC bus
- two-port: a component connecting two buses.

While the code of the solver is not made public, the models are expected to be freely shared by users. This feature makes STEPSS an **open-source simulation software**, at least for the modelling part. That is what matters to the user, isn't it ?

2

Installing STEPSS

Before going any further, you have to read and accept the legal terms detailed at the beginning of this document.

This chapter covers STEPSS GUI, for which installers are published for Windows, macOS and Linux. The other edition, `stepss`, the Python interface, is installed with `pip` on all three; see its own chapter. Both are documented online at

<https://stepss.sps-lab.org>

Every version of STEPSS is published on its releases page, which is the place to download it from:

<https://github.com/SPS-L/stepss-java-ui/releases>

2.1. Installing STEPSS

STEPSS is installed from an **installer**, which carries its own Java, so nothing else has to be installed and any Java already on the machine is neither needed nor disturbed. Everything required to run the simulations and display the results, executables and libraries included, travels with STEPSS.

On Debian and Ubuntu there is a second route to the same installer, an **APT repository**. It delivers the same package, with the difference that later releases arrive through `apt upgrade` rather than being downloaded by hand.

2.1.1. Using the installer (recommended)

From the releases page, download the file for your system:

- Windows: `STEPSS-version.msi`
- macOS (Apple Silicon): `STEPSS-version.dmg`
- Linux (Debian, Ubuntu): `stepss_version_amd64.deb`

Open it and follow the installer. STEPSS is then started from the Start menu, from Applications, or from your desktop's list of applications, like any other program.

On Linux, install the `.deb` with

```
sudo apt install ./stepss_version_amd64.deb
```

rather than with `dpkg -i`, so that the libraries STEPSS needs are installed with it. It requires Ubuntu 24.04 or newer, or a Debian of the same vintage or later; on an older release, use the archive instead.

To remove it, uninstall it the way you would any other application: *Add or Remove Programs* on Windows, dragging it to the Trash on macOS, `sudo apt remove stepss` on Debian and Ubuntu.

2.1.2. Using the APT repository (Debian and Ubuntu)

On Debian and Ubuntu, STEPSS can be installed as an ordinary package from the repository the laboratory publishes at

<https://apt.sps-lab.org>

It carries the same `.deb` as the previous section. What it adds is that `apt upgrade` then moves you to each later release along with the rest of the system, instead of leaving you to notice that one was published.

The setup is done once. Copy the following block into a terminal:

```
sudo install -d -m 0755 /etc/apt/keyrings
sudo curl -fsSL https://apt.sps-lab.org/sps-l-archive-keyring.asc \
-o /etc/apt/keyrings/sps-l-archive-keyring.asc
```



```

sudo tee /etc/apt/sources.list.d/sps-l.sources >/dev/null <<'EOF'
Types: deb
URIs: https://apt.sps-lab.org
Suites: stable
Components: non-free
Architectures: amd64
Signed-By: /etc/apt/keyrings/sps-l-archive-keyring.asc
EOF

```

STEPSS is then installed, and later upgraded, like anything else:

```

sudo apt update
sudo apt install stepss

```

Start it by typing `stepss`, or from the *Science* section of the applications menu. Remove it with `sudo apt remove stepss`.

If `apt update` answers `NO_PUBKEY 3600A756077AD061`, that is apt naming the key it wanted and could not find. It means one of three things, which look identical from apt: the keyring file is not there, because the download failed and `curl` correctly writes nothing when it does; the Signed-By line is missing from your `.sources` file; or the file holds some other key. Check which:

```
gpg --show-keys /etc/apt/keyrings/sps-l-archive-keyring.asc
```

It should report fingerprint `125ABE6917ED2F2965D6545B3600A756077AD061`, for SPS-L Packaging. If it reports no such file, run the `curl` command again. A different complaint mentioning `add_keyblock_resource` means the file exists but is empty, and has the same remedy.

Four things to know about this route:

- It requires **Ubuntu 24.04 or newer**, or a Debian of the same vintage or later, on **x86-64**. The package is built on Ubuntu 24.04 and names the packages it depends on as that release names them, so Ubuntu 22.04 and Debian 12 cannot install it. There is no Linux arm64 build of the engines, which is why `Architectures: amd64` appears above: without that line, apt on an arm64 machine looks for an index that does not exist and reports an error on every `apt update`.
- The repository serves **the current release only**. To install a specific earlier build, download its `.deb` from the releases page.
- `non-free` is accurate rather than a formality. STEPSS is not free software taken as a whole, and installing it through apt agrees to nothing: the licence appears the first time STEPSS is launched, which is after installation. Read the legal terms at the beginning of this document first. Please also do not mirror the repository, since the licence granted for RAMSES is not transferable.
- It is **not** an official Ubuntu or Debian repository, and nothing in it has been through Canonical's or Debian's review. That is also why the setup above is needed at all.

2.2. Installing a Fortran Compiler

This step can be skipped if you do not plan to develop new models for RAMSES. Everything else in STEPSS works without it.

Compiling a model of your own produces a simulator built from the URAMSES kit that STEPSS carries, and that kit is a **GNU Fortran** build. You therefore need `gfortran`, `GNU make`

and OpenBLAS. The Intel oneAPI compiler and the Microsoft Visual Studio environment it relies on are no longer used and no longer need to be installed.

On Windows, these come from MSYS2:

1. install MSYS2 from <https://www.msys2.org/>
2. open an MSYS2 terminal and install the toolchain:

```
pacman -S mingw-w64-x86_64-gcc-fortran mingw-w64-x86_64-openblas  
make
```

STEPSS looks for MSYS2 in `C:\msys64`, or wherever the `MSYS2_ROOT` environment variable points, because the MSYS2 installer does not add its compiler directory to the system `PATH`.

On Linux, install the same three with your package manager; on Debian and Ubuntu that is

```
sudo apt install gfortran make libopenblas-dev
```

On macOS, use Homebrew:

```
brew install gcc openblas
```

The kits STEPSS carries are specific to a gfortran ABI, and each platform's default compiler matches the kit shipped for it. If yours does not, STEPSS reports the exact compiler version to install rather than failing part way through a build.

3

Driving the engines: GUI and Python

You drive STEPSS one of two ways, and both are covered here.

Edition	Dynamic Simulator (RAMSES)	Power Flow (Helios)	CODEGEN	Use it for
STEPSS GUI	yes	yes	yes	Interactive work: load a case, run it, watch curves, build models
STEPSS in Python	yes	yes		Scripting, parameter sweeps, and the scientific Python stack

Those two are what you install; see Installation. Pick the GUI if you want to see the system respond, Python if you want to automate. Neither wraps the other, so a case built in one runs unchanged in the other.

New to STEPSS? First Run opens a bundled test system and simulates it without your having to prepare any data.

The engines can also be invoked directly from a shell, which is what the **Command line** tab on each section below documents. It is not a third thing to install and most readers will not need it: the executables are not published separately, they ship inside STEPSS GUI, which unpacks them while it runs. It matters when you are wiring STEPSS into a job scheduler or another program.

3.1. Dynamic Simulation (RAMSES)

STEPSS GUI

The window is six tabs, used left to right:

1. **Launch STEPSS**, from your applications if you used an installer, or by double-clicking `stepss.jar`
2. **Get a case. File**, then **Open Examples** ships three complete test systems and fills in every file slot. Otherwise load your own on the **System Data** tab, which takes the system data files and the disturbance file
3. On **Observables**, tick what to record, and optionally set a quantity to plot live while the run proceeds

4. On **Power Flow Simulation**, Run **power flow** to get the operating point
5. On **Dynamic Simulation**, Run **dynamic simulation**
6. On **Analysis**, **Extract curves** to plot the result

The GUI coordinates the power flow and RAMSES through lock files (`.lock_RAMSES`, `.kill_RAMSES`) for graceful process management.

For the full walkthrough on a real case, see [Running a Simulation](#).

STEPSS in Python

1. **Install stepss:**

```
pip install stepss
```

2. **Run a simulation:**

```
import stepss

# Configure the case
case = stepss.cfg()
case.addData("data.dat")
case.addData("volt_rat.dat")
case.addData("settings.dat")
case.addObs("obs.obs")
case.addDst("disturbance.dst")

# Run
ram = stepss.sim()
ram.execSim(case, 150.0)
```

3. **Extract and plot results:**

```
ext = stepss.extractor(case.getTrj())

# Plot bus voltage magnitude
ext.getBus('1041').mag.plot()

# Plot synchronous machine speed
ext.getSync('g1').S.plot()
```

See the [Python API Reference](#) for the complete interface.

3.2. Power Flow (Helios)

STEPSS GUI

1. **Launch STEPSS** and load the network and power-flow data files on the **System Data** tab
2. Go to **Power Flow Simulation** and click **Run power flow**. The pane reports bus voltages and angles, the generator table with its limits, and the system power balance
3. Inspect the result through **Bus overview**, **Branch flows**, **Generators & SVCs**, **Adjustable transformers** and **Global power balance**, which enable once the run succeeds

4. **Add Helios results to data** feeds the solution back into the loaded case, ready for the dynamic simulation

See The Interface for what each control does.

STEPSS in Python

HeliosSession follows load, then modify, then solve, then query:

```
from stepss.helios import HeliosSession

with HeliosSession() as pf:
    pf.load_file('network.dat')
    pf.load_file('powerflow.dat')
    converged = pf.solve()

    v, angle = pf.get_bus_voltage('1041')
    pf.export_voltages('volt_rat.dat') # the LFRESV file RAMSES initialises from
```

export_voltages writes the same LFRESV file the GUI's **Add Helios results to data** feeds back into the case. See Power Flow with Helios for the full interface, including redispatch and contingency screening.

3.3. Typical Workflow

1. **Define the network:** Specify buses, lines, transformers, and shunts
2. **Set up power flow data:** Define generators, loads, and the slack bus
3. **Run the power flow:** compute the initial operating point and export the LFRESV file
4. **Add dynamic models:** Specify synchronous machines, excitation systems, speed governors, loads, and controllers
5. **Configure solver:** Set integration method, time steps, and tolerances
6. **Define disturbances:** Specify faults, line trips, parameter changes, etc.
7. **Run the dynamic simulation:** from **Dynamic Simulation** in the GUI, or `sim.execSim(case)` in Python
8. **Analyze results:** Extract and visualize trajectories of voltages, frequencies, and powers

3.4. Next Steps

- First Run, Open a bundled test system in STEPSS GUI
- Running a Simulation, A complete run, start to finish
- Python Examples, The same ground in a script
- File Formats, Learn about data file syntax
- Network Modeling, Define your power system network
- Power Flow, Power flow data and control parameters
- Dynamic Data Records, Add generators, loads, and controllers

4

Organisation of the data files

Data files are organised into *records* and *comments*, whose syntax is detailed next.

4.1. Records

Each record includes:

- a leading *keyword*, which identifies the information provided in the record
- a number of *fields*. A field is either a real number (numeric field) or a string of characters (character field). There is at least one field
- a *terminating semicolon* (;), which indicates the end of the record.

✚ The following record, with keyword LINE, specifies a transmission line:

```
LINE A-B BUS_A BUS_B 3.0 30.0 150.0 1400.0 1 ;
```

Inside the record, the keyword, the fields and the terminating semicolon are separated by (at least one) space(s). Anything after the semicolon is ignored. The next record (or comment) starts with the next line.

✚ Two incorrect versions of the above sample record, owing to missing spaces:

```
LINE A-B BUS_ABUS_B 3.0 30.0 150.0 1400.0 1 ;
```

```
LINE A-B BUS_A BUS_B 3.030.0 150.0 1400.0 1 ;
```

Inside a data file, a record may span over multiple lines; the semicolon indicates the end of the record. Spanning over several lines is highly recommended for records that include many fields. Note that, depending upon the text editor and its settings, a long record could appear truncated when displayed.

✚ An example of long record spanning over three lines:

```
INJEC GFOL VSC1 A 1.0 1.0 0.0 0.0 0.005 0.15 1.02
1200.0 0.005 0.15 0.573 6.0 0.0033 0.0333 10.0 0.002 -999.0
10.0 0.1667 50.0 0.10 0.4 0.5 1.0 -1000. -2000. 1.001 1 ;
```

The nature of each field, and the total number of fields are specified in this documentation. Some records have optional fields, which are always located at the end of the record.

4.1.1. Constraints on numeric fields

There is almost no constraint on numeric fields. They are written in free format. The notation is that of floating-point numbers in MATLAB: with or without a dot, with or without an exponent, exponent denoted by E or D.

✚ Different valid formats of the same number: 30 30. 30.0 3E01 3.E01 3.0E01 3.E1 3.e1

4.1.2. Constraints on character fields

Character fields are limited to 20 characters. Only the first 20 characters are read; the remaining of the string is just ignored, without warning. It is thus discouraged to have more than 20 characters in a field.

Furthermore, some character fields are limited to eight characters; the remaining of the string is just ignored.

Uppercase letters are significant within a character field. Thus, two fields that differ by the uppercase/lowercase spelling are different.

If a character field includes a space or a slash (/), it must be enclosed with quotes (' or "). In between the two quotes, the leading spaces are significant while the trailing ones are ignored. Keywords do not include spaces; hence, the use of quotes is useless.

Because the semi-column (;) is a special character (used to indicate the end of a record), **it must not be included in any character field**, even within quotes.

4.2. Comments

There are three ways to insert comments in the data files:

1. a line in which the first non blank character is an exclamation mark (!). At most the first 130 characters after the ! are memorised and reproduced on output files
2. a line in which the first non blank character is the sharp character (#): this line is simply ignored by the program
3. anything written after the semicolon that terminates a record is also ignored. This is convenient to store (short) comments next to a record, without interfering with the latter.

✚ Comments starting with # can be used to identify the fields of a record:

```
#          name bus FP  FQ  P  Q SNOM RS   LLS LSR  RR   LLR
INJEC INDMACH1 SM   2  0.2 0.2 0. 0. 0. 0.031 0.1 3.2 0.018 0.180
#                                     H   A   B   LF
                                     0.7 0.5 0.0 0.6 ;
```

Comments do not span over several lines. If several lines are needed, each of them must start with a ! or a #.

Empty lines are just ignored.

4.3. Sharing data between files

Records may be distributed over an arbitrary number of data files, which will be read sequentially. The order in which the records are placed inside the data files does not matter. The order in which the files are read does not matter either.

For instance, a first data file may be devoted to network data, a second one to data for the initial power flow computation, a third one to the dynamic data of the components connected to the network and a last one to simulation control parameters. The second and third files, for instance, may be swapped in the list without any effect.

5

Modelling of network components

The network model includes buses, lines, cables, transformers and shunts. Their models and data are described in the present chapter.

5.1. Buses

In dynamic simulations with RAMSES the only parameter associated with a bus is its nominal voltage (more precisely, the RMS value of the nominal line-to-line voltage). This is used as base voltage to convert parameters from physical to per unit values.

If two buses have different nominal voltages they cannot be connected through a path made up of lines or switches (see next sections). This causes the software to issue an error message and stop.

5.1.1. Data format

The record, with keyword BUS, includes the following fields:

BUS NAME VNOM ;

where:

- NAME is the name of the bus. This is a string of at most 8 characters
- VNOM is the nominal voltage, in kV.

Only one BUS record per bus is allowed.

All buses must be declared through BUS records. If the software finds (in other than BUS records) a bus name not defined through a BUS record, it issues an error message and stops.

Caveat. The above record is used in the RAMSES module. For the initial power flow computation with Helios, an extended version of the BUS record is used with six fields instead of two. Please refer to Section 9.1. Thus, when RAMSES is initialized, if a BUS record has six fields, only the first two are read, the others are ignored.

5.2. Lines and cables

5.2.1. Modelling

Lines and cables both have the same pi-equivalent model, which is shown in Fig. 5.1. Note that shunt conductances are neglected.

Under the phasor approximation, series capacitors can also be modelled with this pi-equivalent, by setting R and C to zero and X to a negative value.

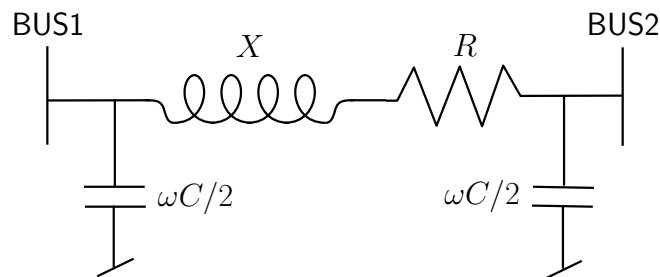


Figure 5.1: Pi-equivalent of lines and cables

5.2.2. Data format

The record, with keyword LINE, includes the following fields:

LINE NAME BUS1 BUS2 R X WC2 SNOM BR ;

where:

- NAME is the name of the line or cable. This is a string of at most 20 characters
- BUS1 is the name of the first bus. This is a string of at most 8 characters defined in a BUS record
- BUS2 is the name of the second bus. This is a string of at most 8 characters defined in a BUS record
- R is the series resistance R , in Ω
- X is the series reactance X , in Ω
- WC2 is the half shunt susceptance $\omega C/2$, in μS (microSiemens)
- SNOM is the nominal apparent power, in MVA. This value is used to display the line loading, or possibly in user-defined models. If not used, it may be set to zero; this will be interpreted as an infinite power
- BR is the on/off status of the line breakers. A zero value indicates that the breakers are open at both ends (line out of service); any other value (e.g. 1) means that both breakers are closed (line in service).

The orientation of the line is arbitrary: BUS1 and BUS2 may be swapped.

Only one LINE record per line is allowed.

All lines are memorized, even those that are out of service. A line out of service is not involved (it appears in the output results with a zero power flow) but it can be put into service in the dynamic simulation.

As mentioned in Section 5.1, a line must not connect two buses with different nominal voltages.

To have a line connected through a single end, add a bus at the open end and set BR to a nonzero value.

5.3. Switches

5.3.1. Modelling

A switch is a connection without impedance between two buses. It is treated as a very short line, more precisely a line with $R = 0$, $\omega C/2 = 0$ and X set to a very low value. Thus it has no active power losses and negligible reactive power losses.

5.3.2. Data format

The record, with keyword SWITCH, includes the following fields:

SWITCH NAME BUS1 BUS2 BR ;

where:

- NAME is the name of the switch. This is a string of at most 20 characters
- BUS1 is the name of the first bus. This is a string of at most 8 characters defined in a BUS record
- BUS2 is the name of the second bus. This is a string of at most 8 characters defined in a BUS record
- BR is the on/off status of the switch. A zero value indicates that the switch is open; any other value means that it is closed.

The orientation of the switch is arbitrary: BUS1 and BUS2 may be swapped.

Only one SWITCH record per switch is allowed.

All switches are memorized, even those which are open. An open switch is not involved (it appears in the output results with a zero power flow) but it can put into service in the dynamic simulation.

As mentioned in Section 5.1, it is not allowed to have a switch connecting two buses with different nominal voltages.

5.4. Transformers

5.4.1. Modelling

Transformers are represented by the two-port shown in Fig. 5.2. Note that R , X , B_1 and B_2 are specified on the “from” side of the transformer.

R corresponds to the copper losses. The iron losses are neglected (no shunt resistance). X is the leakage reactance. B_1 or B_2 are the magnetizing susceptances, which have negative values. Usually one of them is zero. n (resp. ϕ) is the magnitude (resp. the phase angle) of the transformer ratio. A phase-shifting transformer is characterized by a nonzero value of ϕ .

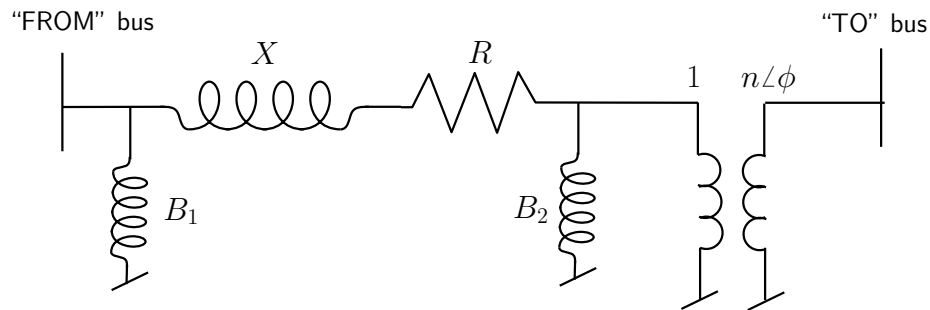


Figure 5.2: Two-port model of transformers

It may be of interest to recall how the values of R , X , B_1 and B_2 relate to the following characteristics, easily obtained from manufacturer data:

- S_{nom} the nominal apparent power
- V_{N1} (resp. V_{N2}) the nominal voltage on the “from” (resp. “to”) side (see Fig. 5.2)
- $R_{baseV_{N1}}$ (resp. $X_{baseV_{N1}}$) the series resistance (resp. leakage reactance) in per unit on the (S_{nom}, V_{N1}) base
- $B_{1baseV_{N1}}$ and $B_{2baseV_{N1}}$ the shunt susceptances in per unit on the (S_{nom}, V_{N1}) base
- V_{o1} and V_{o2} the open-circuit voltages corresponding to the transformer ratio¹.

Let V_{B1} and V_{B2} be the nominal voltages of respectively the “from” and the “to” buses, as specified in their BUS records (see Section 5.1).

In STEPSS R , X , B_1 and B_2 are in percent on the (S_{nom}, V_{B1}) base. The following changes of base have to be used:

$$\begin{aligned} R &= 100 R_{baseV_{N1}} \left(\frac{V_{N1}}{V_{B1}} \right)^2 & X &= 100 X_{baseV_{N1}} \left(\frac{V_{N1}}{V_{B1}} \right)^2 \\ B_1 &= 100 B_{1baseV_{N1}} \left(\frac{V_{B1}}{V_{N1}} \right)^2 & B_2 &= 100 B_{2baseV_{N1}} \left(\frac{V_{B1}}{V_{N1}} \right)^2 \end{aligned}$$

¹Very often the open-circuit voltages coincide with the nominal voltages, i.e. $V_{o1} = V_{N1}$ and $V_{o2} = V_{N2}$

n is in percent on the (V_{B1}, V_{B2}) base. Its value is given by:

$$n = 100 \frac{V_{o2} V_{B1}}{V_{o1} V_{B2}}$$

A second transformer model is available. It is a simplified version with $B_2 = 0$ and $\phi = 0$ in Fig. 5.2 and includes data for Helios to adjust the transformer ratio (see Section 9.5). This model cannot be used for phase-shifting transformers.

5.4.2. Data format

The record, with keyword TRANSFO, includes the following fields:

TRANSFO NAME FROMBUS TOBUS R X B1 B2 N PHI SNOM BR ;

where:

- NAME is the name of the transformer. This is a string of at most 20 characters
- FROMBUS is the name of the “from” bus (see Fig. 5.2). This is a string of at most 8 characters defined in a BUS record
- TOBUS is the name of the “to” bus (see Fig. 5.2). This is a string of at most 8 characters defined in a BUS record
- R is the series resistance, in % on the base detailed above
- X is the leakage reactance, in % on the base detailed above
- B1 is the shunt suceptance, in % on the base detailed above
- B2 is the shunt suceptance, in % on the base detailed above
- N is the magnitude of the transformer ratio in % (dimensionless)
- PHI is the phase angle of the transformer ratio, in degree
- SNOM is the nominal apparent power of the transformer, in MVA. This value must not be zero
- BR is the on/off status of the transformer breakers. A zero value indicates that the breakers are open at both ends (transformer out of service); any other value means that both breakers are closed (transformer in service).

The second transformer model is described by the record with keyword TRFO:

TRFO NAME FROMBUS TOBUS CONBUS R X B N SNOM NFIRST NLAST NBPOS TOLV VDES BR ;

where:

- NAME is the name of the transformer. This is a string of at most 20 characters
- FROMBUS is the name of the “from” bus (see Fig. 5.2). This is a string of at most 8 characters defined in a BUS record
- TOBUS is the name of the “to” bus (see Fig. 5.2). This is a string of at most 8 characters defined in a BUS record
- CONBUS is used by Helios for adjusting n : see Section 9.5. It is not used by RAMSES, but a (dummy) name must be provided

- R is the series resistance, in % on the base detailed above
- X is the leakage reactance, in % on the base detailed above
- B is the shunt suceptance, in % on the base detailed above
- N is the magnitude of the transformer ratio in % (dimensionless)
- SNOM is the nominal apparent power of the transformer, in MVA. This value must not be zero
- NFIRST is used by Helios for adjusting n : see Section 9.5.
- NLAST: same as for NFIRST
- NBPOS: same as for NFIRST
- TOLV: same as for NFIRST
- VDES: same as for NFIRST
- BR is the on/off status of the transformer breakers. A zero value indicates that the breakers are open at both ends (transformer out of service); any other value means that both breakers are closed (transformer in service).

The following remarks apply to both TRANSFO and TRFO records.

The orientation of the transformer is not arbitrary: FROMBUS and TOBUS cannot be swapped.

Only one TRANSFO or TRFO record per transformer is allowed.

All transformers are memorized, even those that are out of service. A transformer out of service is not involved (it appears in the output results with a zero power flow) but it can be put into service in the dynamic simulation.

To have a transformer connected through a single end, add a bus at the open end and set BR to a nonzero value.

5.5. Non-reciprocal two-ports

5.5.1. Modelling

A non-reciprocal two-port is a two-port with a non-symmetric nodal admittance matrix of the form:

$$\mathbf{Y} = \begin{bmatrix} (G_{si} + jB_{si}) + (G_{ij} + jB_{ij}) & -(G_{ij} + jB_{ij}) \\ -(G_{ji} + jB_{ji}) & (G_{ji} + jB_{ji}) + (G_{sj} + jB_{sj}) \end{bmatrix}$$

with:

$$G_{ij} \neq G_{ji} \quad \text{and} \quad B_{ij} \neq B_{ji}$$

where i and j relate to the terminal nodes of the two-port, as shown in Fig. 5.3.

Typically, two-ports are produced when reducing a network that includes phase-shifting transformers, the purpose being to obtain an equivalent.

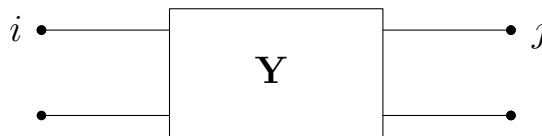


Figure 5.3: A two-port connecting buses i and j

5.5.2. Data format

The record, with keyword NRTP, includes the following fields:

NRTP NAME FROMBUS TOBUS GIJ BIJ GJI BJI GSI BSI GSJ BSJ BR ;

where:

- NAME is the name of the two-port. This is a string of at most 20 characters
- FROMBUS is the name of bus i in Fig. 5.3. This is a string of at most 8 characters defined in a BUS record
- TOBUS is the name of bus j in Fig. 5.3. This is a string of at most 8 characters defined in a BUS record
- GIJ is the G_{ij} conductance, in pu
- BIJ is the B_{ij} susceptance, in pu
- GJI is the G_{ji} conductance, in pu
- BJI is the B_{ji} susceptance, in pu
- GSI is the G_{si} conductance, in pu
- BSI is the B_{si} susceptance, in pu
- GSJ is the G_{sj} conductance, in pu
- BSJ is the B_{sj} susceptance, in pu
- BR is the on/off status of the two-port breakers. A zero value indicates that the breakers are open at both ends (two-port out of service); any other value means that both breakers are closed (two-port in service).

The parameters of the two-port are given in per unit on the following base: nominal bus voltages, system base power. By default a value of 100 MVA is used² but it can be changed: see Section 9.9.

It is allowed for a non-reciprocal two-port to connect two buses with different nominal voltages.

The orientation of the two-port is not arbitrary: FROMBUS and TOBUS cannot be swapped. Only one NRTP record per two-port is allowed.

A non-reciprocal two-port is treated as a piece of equipment; hence, the presence of the BR field. All non-reciprocal two-ports are memorized, even those which are out of service. A non-reciprocal two-port out of service is not involved (it appears in the output results with a zero power flow) but it can be put into service in the dynamic simulation.

5.6. Shunts

5.6.1. Modelling

The shunt element is treated as a purely reactive, constant shunt admittance. Hence, the reactive power Q it produces varies with the square of the voltage V according to:

$$Q = B V^2$$

where B is the susceptance. The element may correspond to either a capacitor ($B > 0$) or a reactor ($B < 0$).

²a typical value for transmission systems

5.6.2. Data format

The record, with keyword SHUNT, includes the following fields:

SHUNT NAME BUS_NAME QNOM BR ;

where:

- NAME is the name of the shunt. This is a string of at most 20 characters
- BUS_NAME is the name of the bus to which the shunt is connected. This is a string of at most 8 characters defined in a BUS record
- QNOM is the nominal reactive power of the shunt, in Mvar. This is the reactive power produced by the shunt under the nominal voltage of the bus, specified in its BUS record. QNOM is positive (resp. negative) for a shunt capacitor (resp. inductor)
- BR is the on/off status of the shunt. A zero value indicates that the shunt is not connected; any other value means that it is in service.

Only one SHUNT record per named shunt is allowed. Multiple shunts at the same bus are allowed, but each with its own name. In this case, the susceptances are added (taking signs into account).

All shunts are memorized, even those which are disconnected. A disconnected shunt is not involved (it appears in the output results with a zero power flow) but it can be put into service in the dynamic simulation.

Caveat. The above record is used in the RAMSES module. However, for the initial power flow computation with Helios, the shunt data are attached to a bus and are specified in an extended version of the BUS record. Please refer to Section 9.1.

Part II

The STEPSS graphical interface

6

First run of STEPSS GUI

STEPSS GUI is the desktop edition: the one an installer puts on your machine, and the one that carries every engine plus CODEGEN. This page covers the first launch and gets you to a working simulation without you supplying any data of your own.

If it is not installed yet, start at Installing STEPSS GUI.

6.1. Accepting the licence

The first launch shows a licence and will not continue until you accept it.

The licence on that screen is **RAMSES's**, not the interface's. The two user interfaces are Apache 2.0, while the dynamic simulator is the property of the University of Liège and free for non-commercial use only, so the dialog names the component rather than the application. The free version is also capped, in ways worth knowing before you build a large case.

License is the one page that states the terms, the caps and which component each applies to. Read it there rather than inferring them from the dialog.

You accept once. Later launches go straight to the window.

6.2. Start from a bundled example

The fastest way to see STEPSS work is not to load anything of your own. The application ships four complete test systems, and opening one copies it to disk and fills in every file slot, so **Run** works immediately.

1. Open the **File** menu.
2. Choose **Open Examples**.
3. Pick a system from the list. The panel beside it describes the case, its size and what the shipped disturbance does.
4. Click **Open example**.

A banner across the top of the window then reports where the copy landed, with an **Open folder** button to reveal it, and **Dismiss** to put it away. Behind it every file slot is already filled in.

The dialog appears on startup by default. Clear **Show this at startup** to stop that, and reach it from **File, Open Examples** whenever you want it back.

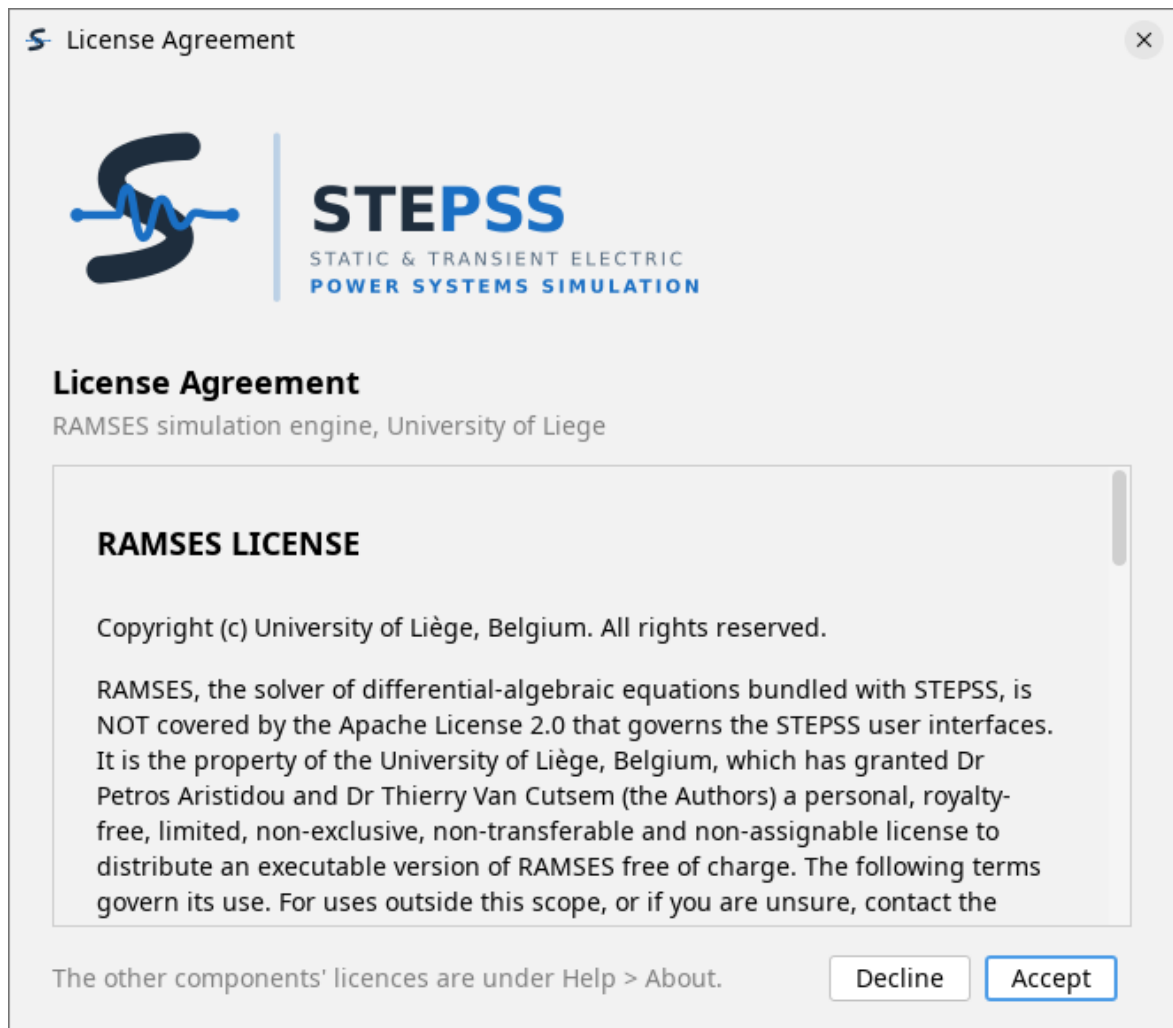


Figure 6.1: The License Agreement dialog on first launch.

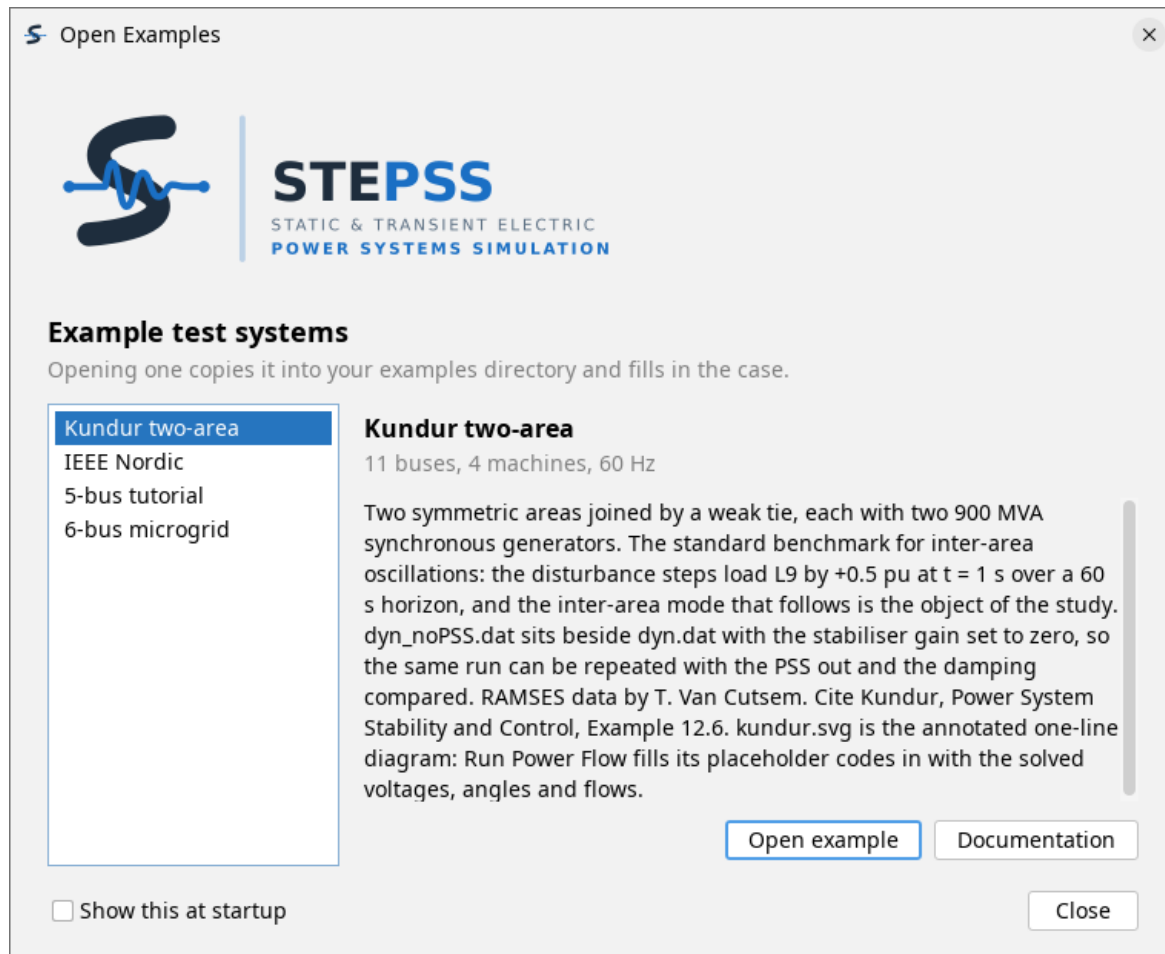


Figure 6.2: The Open Examples dialog.

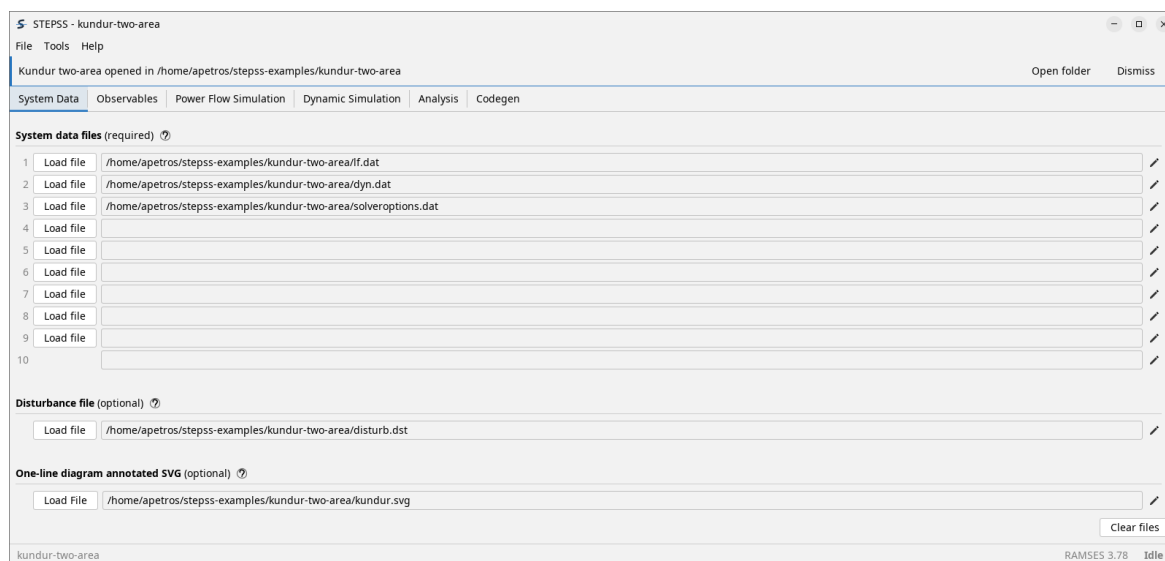


Figure 6.3: The System Data tab immediately after opening the Kundur two-area example.

6.3. The four bundled systems

System	Size	What it is for
5-bus tutorial	5 buses, 1 machine	The teaching case from EEN452 at the Cyprus University of Technology, and the one to open first. One generator with detailed machine, governor and AVR models, a composite load, and an external grid, small enough to read end to end. Its disturbance file is deliberately empty, ready for you to add the records of the study you want.
Kundur two-area	11 buses, 4 machines, 60 Hz	The standard benchmark for inter-area oscillations. Two symmetric areas joined by a weak tie. The shipped disturbance steps a load and the resulting inter-area mode is the object of the study. A second dynamic file with the stabiliser disabled ships beside the first, so the same run can be repeated with the PSS out and the damping compared.
IEEE Nordic	74 buses, 20 machines, 400/220/130 kV	The reference system for long-term voltage stability, from IEEE PES technical report PES-TR19. It opens on the tripped-branch case it is best known for, with further disturbances, a second operating point and a set of load-increase variants shipped alongside.
6-bus microgrid	6 buses, 2 generators, 6/11 kV	A power-flow-only case: two lines, four transformers, loads at buses D and E, and generators at A (slack) and F. It carries no dynamic data and no disturbance scenario, so it runs on Power Flow Simulation and not on Dynamic Simulation . It is the case the annotated one-line diagram is demonstrated with: 6bus.svg is a template of placeholder codes that Run power flow fills in with the solved values.

The first three link to their own page here, which carries the network, the models and the studies the system is used for. The microgrid has no page of its own, because everything it demonstrates belongs to Power Flow, so its name links to its repository instead. **Documentation** in the dialog opens the system's source repository in every case.

6.4. Where the files go

Two directories are remembered separately, and the distinction matters once you open more than one example.

- The **examples directory** is where copies are unpacked. Opening a second example puts it beside the first rather than inside it.
- The **working directory** is where a run reads and writes. Opening an example moves it into that example.

Set either from the **Tools** menu, which also offers **Open working folder** and **Open terminal in working folder** for getting at the files directly.

An example is copied rather than run in place, so editing it is safe and the copy survives quitting. That is the point of copying: work on it freely.

6.5. Real-time plotting

STEPSS GUI plots curves while a simulation runs, which is covered on The Interface. It draws them itself: nothing has to be installed on any platform, and no external plotting program is involved.

Each runtime observable you filled in gets a panel of its own, stacked on a shared time axis, and the window keeps drawing until the run ends. **Save PNG** and **Save CSV** keep the picture or the samples behind it, and **Reset zoom** returns to the full horizon after a drag.

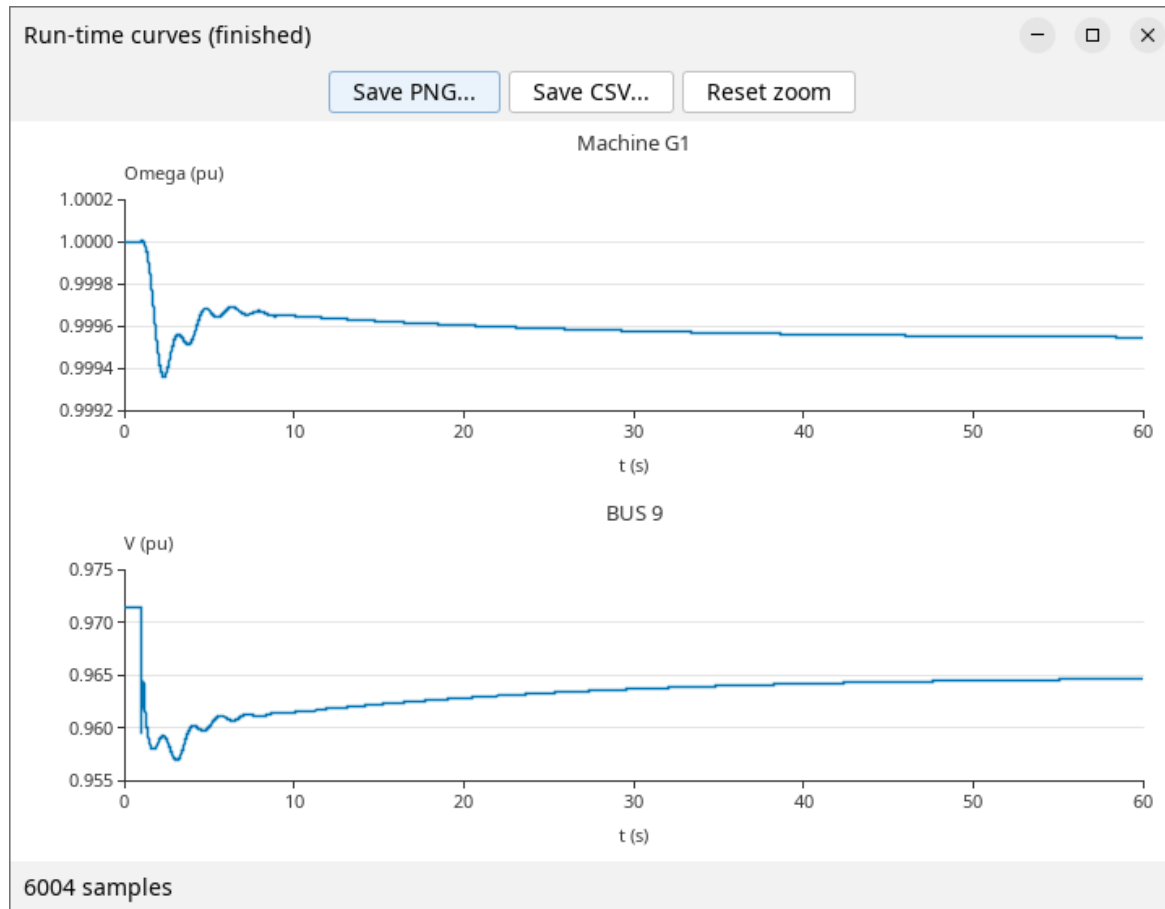


Figure 6.4: The Run-time curves window at the end of a Kundur run, titled Run-time curves (finished).

Run-time curves need a RAMSES release that writes a column map into the header of its observable file. Against an engine that does not, the window says so rather than drawing. Extracting curves after a run is independent of this: that path reads DYNGRAPH's output, which carries no header.

Could not prepare the simulation toolchain

The engines ship inside the application and are unpacked to a temporary directory each time it starts. A **Toolchain error** on launch means that unpacking failed, almost always because the temporary directory is not writable. Fix that and start again.

6.6. Choosing a theme

Tools, then **Dark theme**. The choice is remembered between sessions, along with the window's size and position.

6.7. Next steps

- The Interface, what each tab is for
- Running a Simulation, a complete run on the Kundur case
- File Formats, what goes in the files you loaded

The STEPSS GUI interface

The window is six tabs, and you use them roughly left to right: name the files, choose what to record, solve the initial operating point, run the simulation, then analyse the result. The sixth builds your own models.

This page says what each tab is for. It does not describe the file formats themselves; File Formats and Dynamic Data Records own those.

7.1. System Data

Where the case is named. Nine numbered rows take the **system data files**, and a separate row below takes the **disturbance file**. Both groups are required.

Each row has a **Load file** button and a pencil that opens the file in an editor. **Clear files** empties the tab.

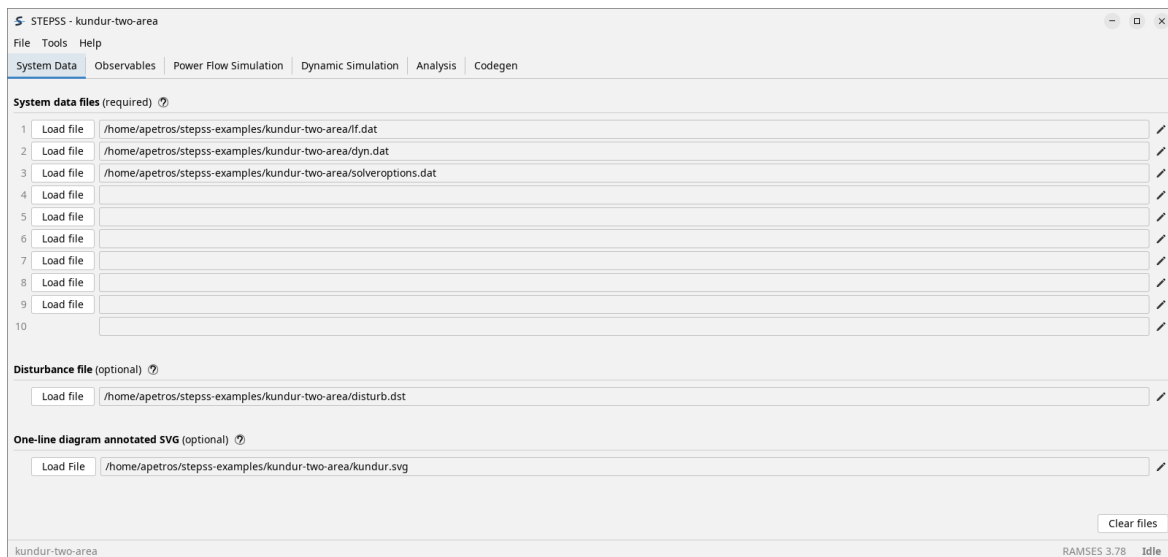


Figure 7.1: The System Data tab with the Kundur two-area case loaded.

Order does not matter and the rows need not be filled contiguously. Which files a case needs depends on the study: a network description, dynamic component data and solver settings are the usual three.

The last row takes an **annotated SVG one-line diagram**, a drawing of the network with placeholder codes typed into it. Every **Run power flow** fills those codes in with the solved

values and opens the result in a window of its own; Annotated One-line Diagram owns the codes and the authoring.

7.2. Observables

Two separate jobs share this tab: what you watch **while** the simulation runs, and what gets written to disk for afterwards.

7.2.1. Runtime observables

Three rows, each pairing a quantity with the name of the equipment to watch. Fill any of them in and STEPSS plots those quantities live as the simulation advances.

The quantities on offer are bus voltage, machine speed, omega-delta and active power-delta of a machine, centre of inertia, wall time, latency, branch active and reactive power at either end, and an injector observable. The name beside it is the equipment's own: a bus name for a bus voltage, a synchronous machine name for a speed or a centre of inertia, and RT for wall time, which plots wall clock against simulation time.

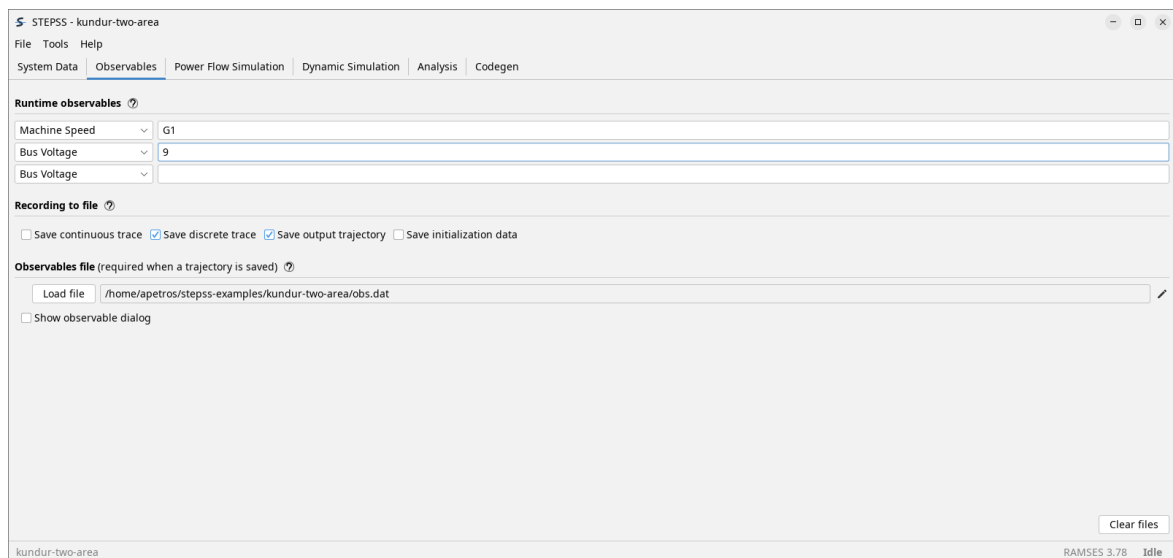


Figure 7.2: The Observables tab.

A quantity may be followed by extra gnuplot commands, separated by /. They are passed through into the .plt script the engine writes, for anyone who opens that file in gnuplot. They have no effect on the curves STEPSS draws.

Live plotting needs nothing installed; see First Run.

7.2.2. Recording to file

Four independent choices, on one row: save the **continuous trace**, the **discrete trace**, the **output trajectory**, and the **initialization data**. The last of those writes the settings, the comments and the initialization data to dump.trace; hover any of the four for what it records.

The **observables file** below them is required whenever a trajectory is saved, because it is what says which quantities the trajectory should contain. Observables File gives its syntax and the eight equipment types it accepts. **Show observable dialog** builds one interactively instead of writing it by hand.

Ticking it opens eight rows under the tab, one per equipment type. Name a piece of equipment and **Add** puts it on that row's list, **All** takes every one of that type in the case, and the list beside it is what will be recorded.

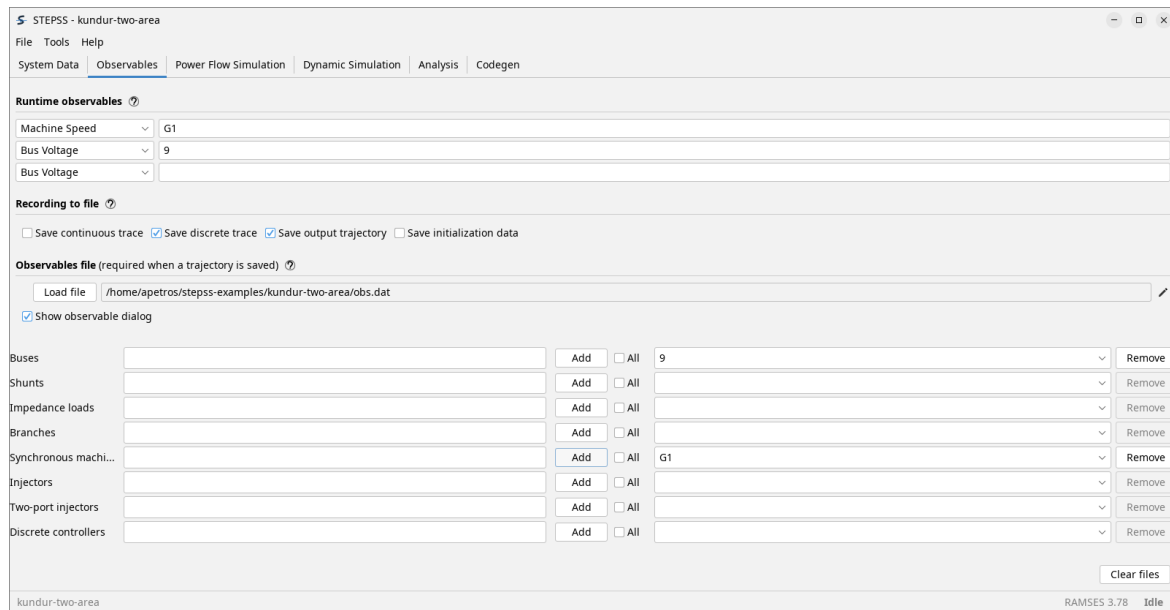


Figure 7.3: The Observables tab with Show observable dialog ticked.

The eight lists are session state and are not written into a saved configuration, so a case saved with the box ticked comes back needing them filled in again.

7.3. Power Flow Simulation

Where the power flow is solved, giving the operating point the dynamic simulation starts from.

Run power flow calls Helios and reports into the pane above: bus voltages and angles, the generator table with its limits, the system power balance, and the files it exported.

Seven buttons enable once a run succeeds:

Button	What it does
Add Helios results to data	Puts the solved <code>in_volt_trfo.dat</code> into system data row ten, so the dynamic simulation starts from this operating point
Save power flow solution	Keeps a copy of that file, which the next run would otherwise overwrite
Bus overview	Bus voltages and angles
Branch flows	P and Q at each end, losses, and loading against rated MVA
Generators & SVCs	Generator and SVC output, against their limits
Adjustable transformers	Transformer ratios, including any Helios adjusted
Global power balance	Generation, load, shunts and network losses

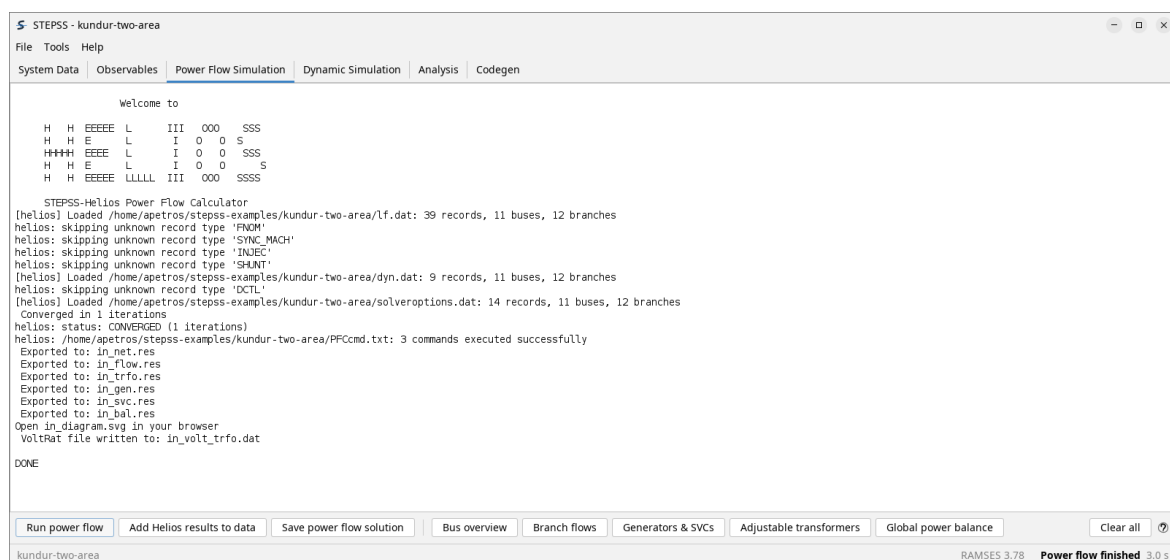


Figure 7.4: The Power Flow Simulation tab after a successful run on the Kundur case.

Each of the four inspection buttons writes its table into the same pane, so the run's log and

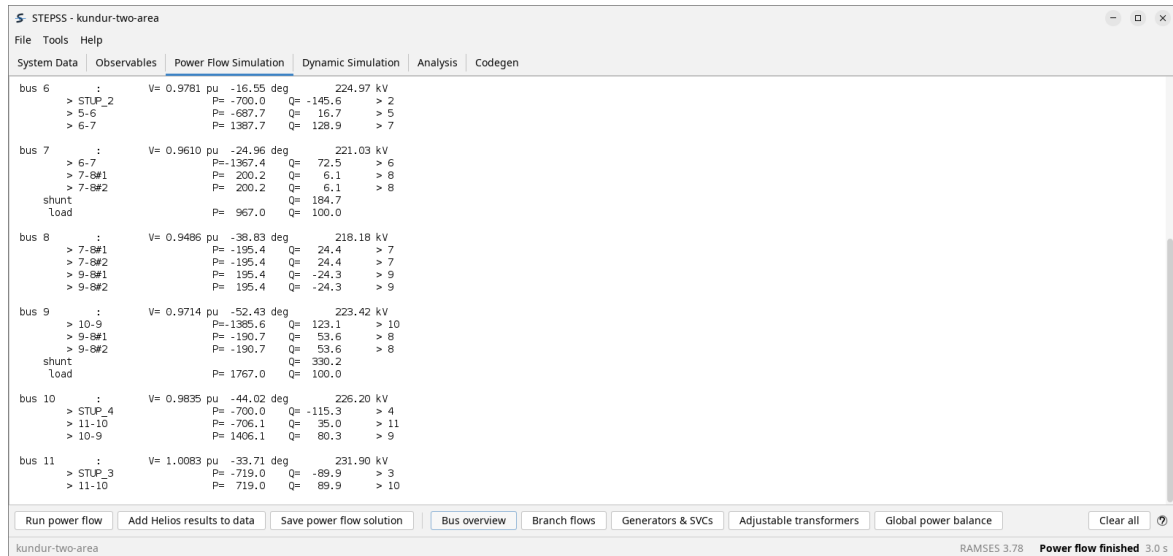


Figure 7.5: The Power Flow Simulation pane showing the bus overview for the Kundur case.

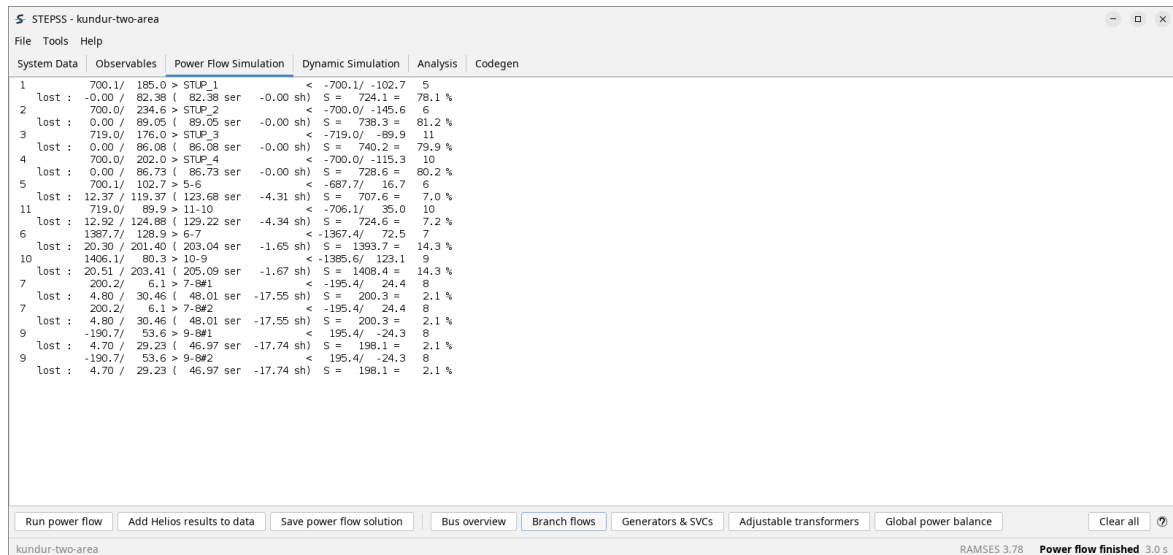
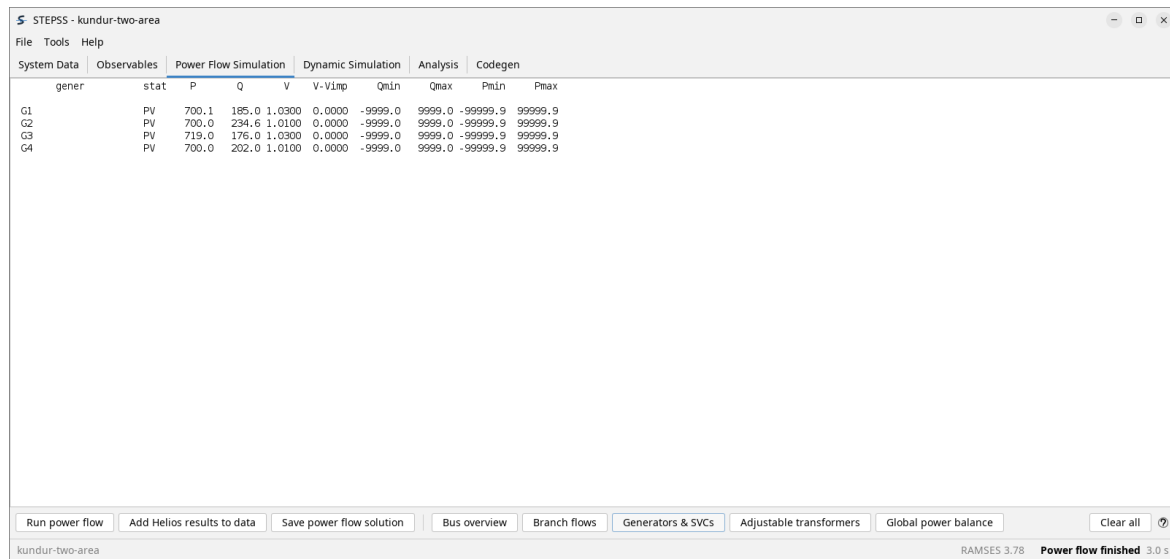


Figure 7.6: The Power Flow Simulation pane showing branch flows for the Kundur case.

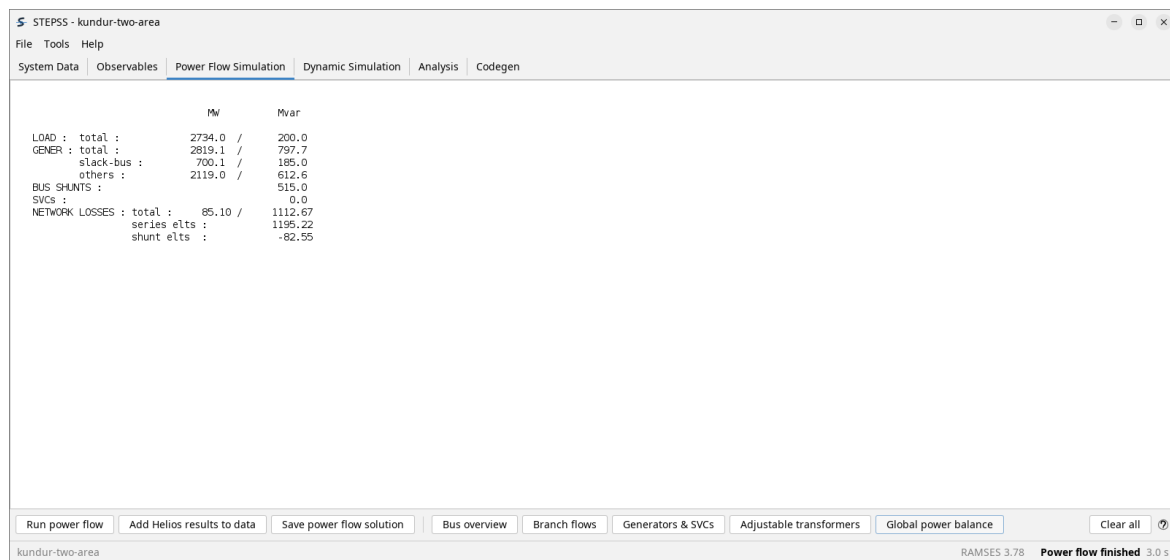


The screenshot shows the 'Power Flow Simulation' tab in the STEPS software. The 'generator' table is displayed with the following data:

gener	stat	P	Q	V	V-Vimp	Qmin	Qmax	Pmin	Pmax
G1	PV	700.1	185.0	1.0300	0.0000	-9999.0	9999.0	-99999.9	99999.9
G2	PV	700.0	234.6	1.0100	0.0000	-9999.0	9999.0	-99999.9	99999.9
G3	PV	719.0	176.0	1.0300	0.0000	-9999.0	9999.0	-99999.9	99999.9
G4	PV	700.0	202.0	1.0100	0.0000	-9999.0	9999.0	-99999.9	99999.9

At the bottom of the window, the status bar indicates 'Power flow finished 3.0 s'.

Figure 7.7: The Power Flow Simulation pane showing the generator table for the Kundur case.



The screenshot shows the 'Global power balance' tab in the STEPS software. The power balance data is as follows:

	MW	Mvar
LOAD : total :	2734.0	200.0
GENER : total :	2819.1	797.7
slack-bus :	700.1	185.0
others :	2119.0	612.6
BUS SHUNTS :		515.0
SVCs :		0.0
NETWORK LOSSES : total :	85.10	1112.67
series elts :		1195.22
shunt elts :		-82.55

At the bottom of the window, the status bar indicates 'Power flow finished 3.0 s'.

Figure 7.8: The Power Flow Simulation pane showing the global power balance for the Kundur case, in MW and Mvar.

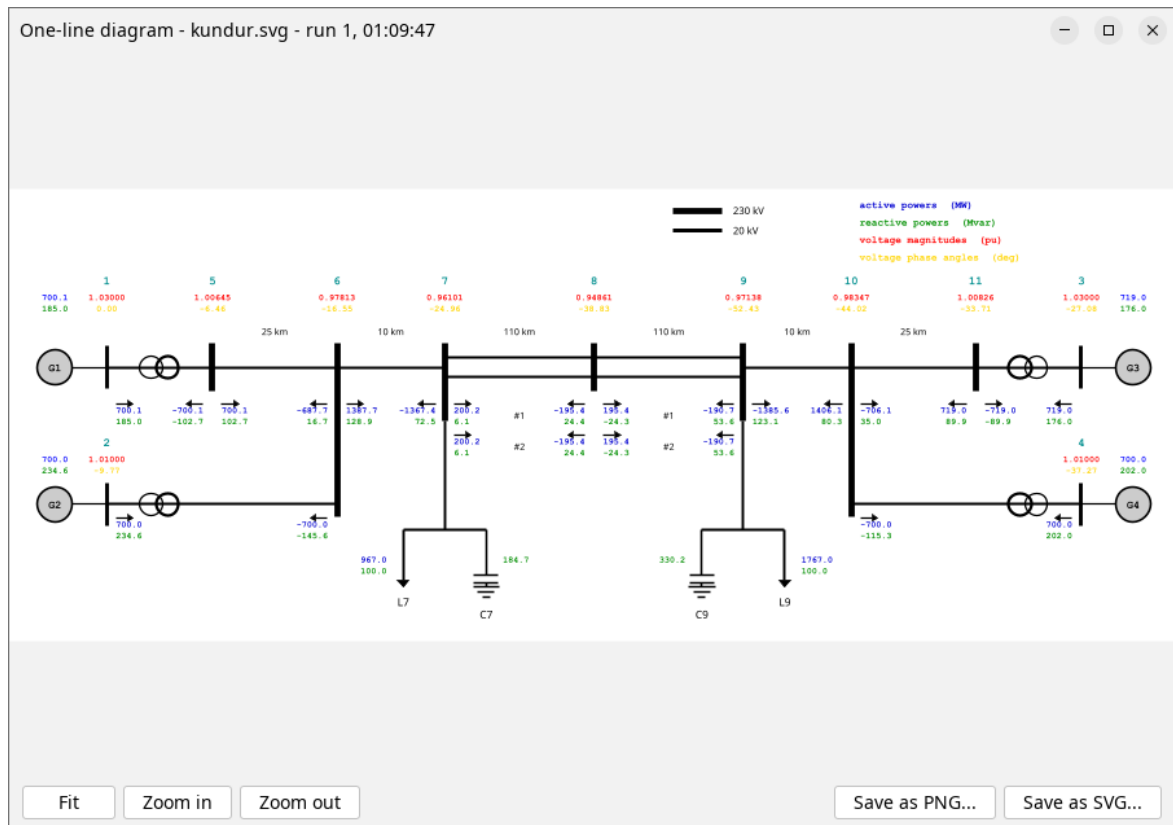


Figure 7.9: The one-line diagram window for the Kundur case, titled One-line diagram, kundur.svg, run 1.

7.4. Dynamic Simulation

Where the case is run. **Run dynamic simulation** starts RAMSES and **Stop simulation** ends it early.

The pane reports the settings in force, the integration method and tolerances, whether parallelism is active, and on completion the elapsed time, the number of time steps, and Jacobian, solution and evaluation counts for the network and the injectors. Those counts are how you tell a cheap run from an expensive one.

The load buttons beside them read back a previous run's output, continuous trace, discrete trace or initialization data, so a result can be examined without re-running it. **Search** finds text in the pane.

7.5. Analysis

Two kinds of analysis, on the same solved case.

Time-domain analysis turns a saved trajectory into curves. **Extract curves** opens a picker and then draws the result in a window of its own, one per extraction, so two extractions can be compared side by side. Each window saves its own figure as PNG, SVG or CSV, and its own .cur and .plt pair for plotting in gnuplot yourself. **Save output trajectory** writes the whole trajectory. **Load trajectory** reads a trajectory from an earlier run. **Close all curve windows** closes every curve window at once. Running a Simulation walks through this.

Small-signal stability analysis runs the engine's eigenanalysis at a chosen instant. Point it at a results directory, set a basename, the analysis time, a real-part limit and a participation factor (PF) threshold, then **Run small-signal stability analysis**. Each run opens its own results window and leaves any already up alone, so two runs can be compared side by side. **Save**

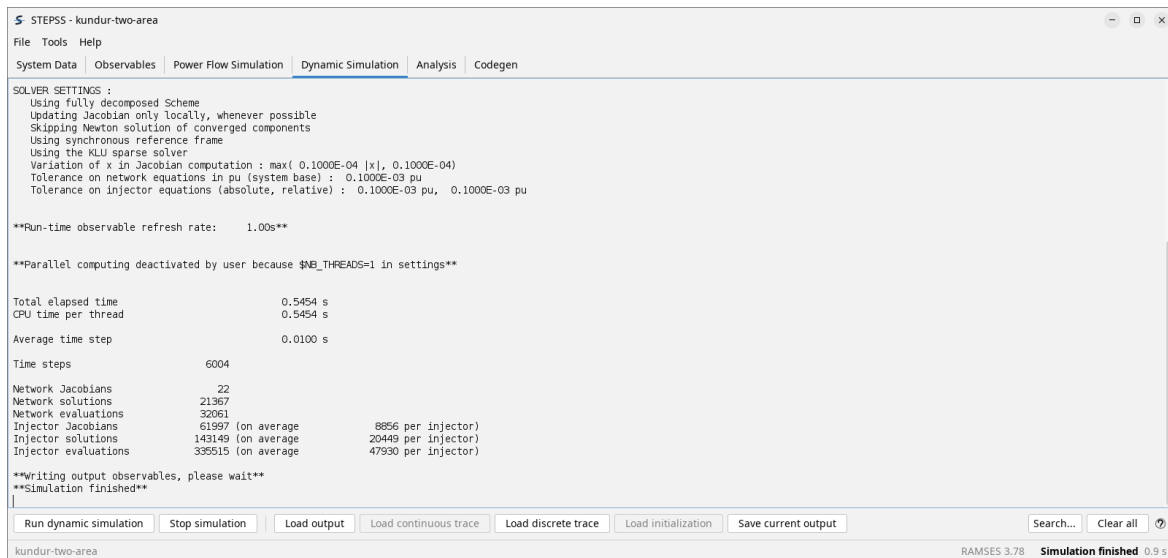


Figure 7.10: The Dynamic Simulation tab after a completed run.

dynamic Jacobian... and **Load dynamic Jacobian...** write that run out as an archive and open it again later, each load in a window of its own too.

The analysis needs `$SCHEME DE` and `$OMEGA_REF SYN`, and the button supplies both: it writes them into one small data file read after your own, where being last is what makes them win. So a case set up for time-domain runs analyses as it stands, and your own solver settings are neither edited nor left changed afterwards. The engine's output goes to the pane on the **Dynamic Simulation** tab, which is where a run that produced nothing says why.

This is performed by RAMSES itself. Eigenanalysis explains the method, the settings and what the results mean.

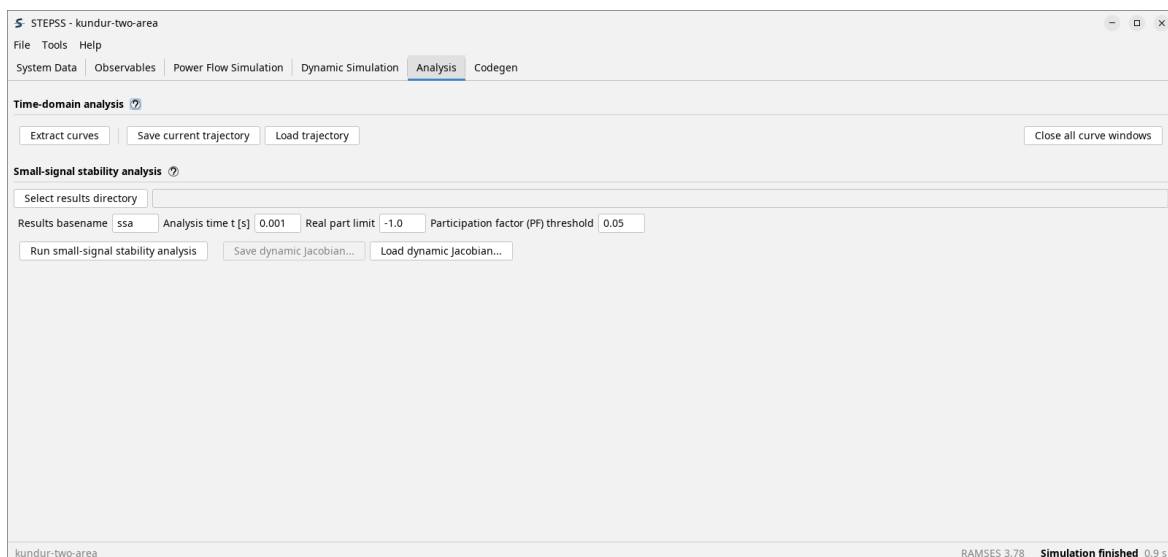


Figure 7.11: The Analysis tab with a completed run available.

The unlabelled field beside **Select results directory** is where results are written, and it defaults to the working directory. The picker that **Extract curves** opens, and the small-signal results window, are both shown on the pages linked above.

7.6. Codegen

Where a model of your own becomes part of the simulator, in the order the buttons run:

Button	What it does
Load files for Codegen	Takes one or more model descriptions in the CODEGEN language
Run Codegen	Translates them into Fortran, reporting each block with its equation and state counts
Display loaded files	Lists the Fortran files generated so far, not the descriptions that were read
Save converted files	Writes the generated .f90 out
Compile	Builds and links it against the engine, giving a custom simulator
Save executable	Keeps that simulator, rather than rebuilding it next time

Each enables as the previous step makes it meaningful. Once a custom simulator exists, simulations run on it rather than on the bundled engine.

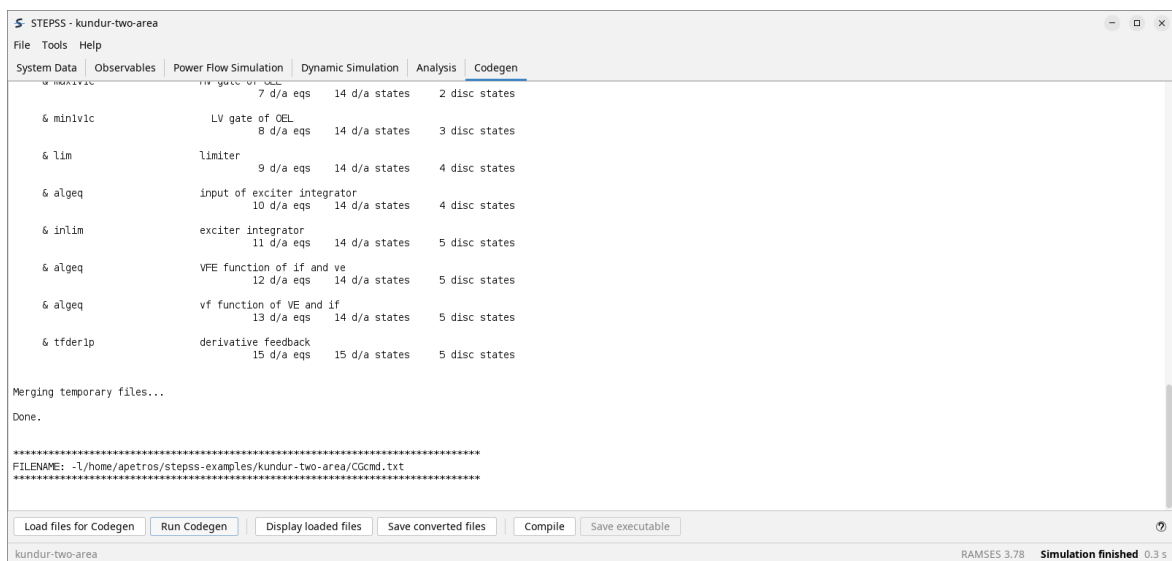


Figure 7.12: The Codegen tab after running CODEGEN on an IEEE AC1A exciter description.

Compiling needs a Fortran toolchain on the machine, which is its own installation step; translating a model does not.

User-Defined Models is the reference for the model language, and CODEGEN Studio assembles models visually instead of by hand.

7.7. The menus

File holds **Save configuration** and **Load configuration**, plus **Open Examples** and **Exit**.

Save configuration writes a .cfg holding everything a run is made of: the ten system data rows, the disturbance and observables files, the three runtime observable rows, and the four recording checkboxes. **Load configuration** puts them back, so a case set up once can be reopened and run in two clicks. A path inside the .cfg's own folder is stored relative to it, which means the folder can be moved, copied or sent to a colleague whole; anything outside that folder is stored as an absolute path and stays tied to the machine that saved it.

Two things the file does not carry. The observable dialog's five picker lists are not saved, so a configuration saved with **Show observable dialog** ticked says so when it loads and the lists need filling in again. And .cfg files written by STEPSS releases before this format are refused

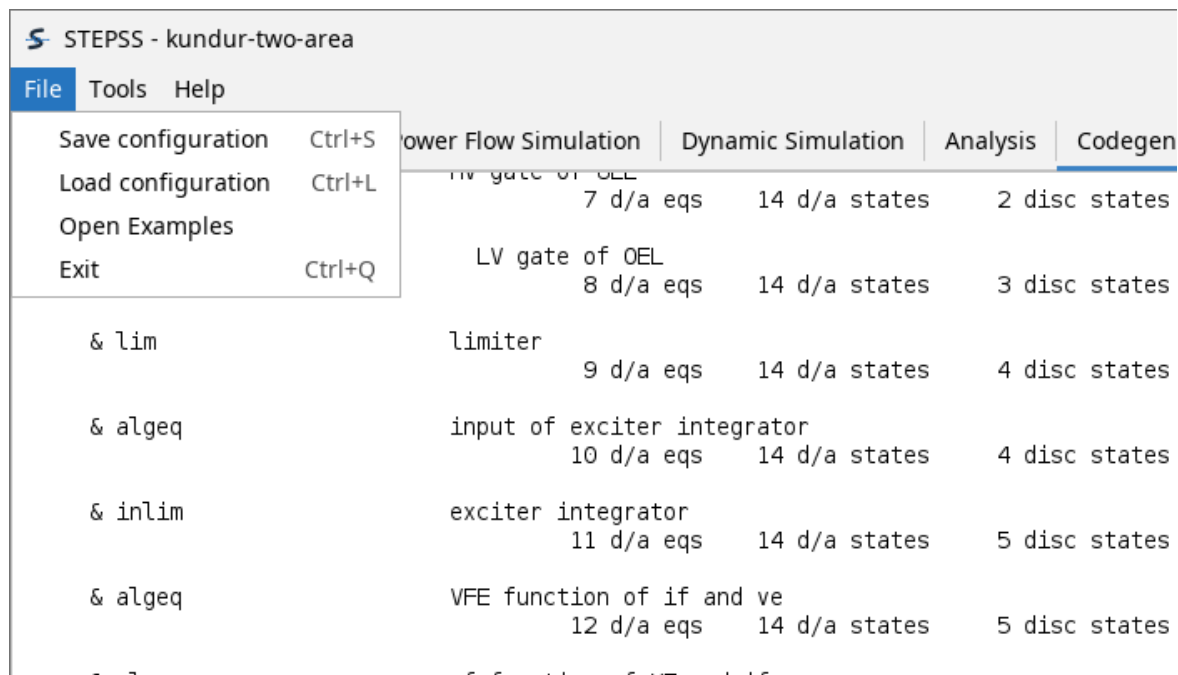


Figure 7.13: The File menu open, holding Save configuration with Ctrl+S, Load configuration with Ctrl+L, Open Examples, and Exit with Ctrl+Q.

rather than loaded: they hold absolute paths belonging to whoever last saved them, so what they name almost never exists on the machine opening them. Set the case up and save it again.

Tools holds, in order, **Save command file** and **Save observables file**, which write out the files a command-line run needs; **Open in editor** and **Select external simulator**; the working-directory controls **Select working directory**, **Open working folder** and **Open terminal in working folder**; **Close all curve windows**; and, below a separator, the **Dark theme** toggle and **Check for updates at startup**. The last two are ticks rather than actions, and both are remembered between sessions.

Save command file writes out the file the engine reads, listing the data, disturbance, trajectory and observables files of the case in the order it expects. It is what a run is made of, in one file. Quick Start compares the two ways to drive the engines.

7.8. The status bar

The working directory on the left; on the right the engine version and what the application is doing. That last field runs from **Idle** through **Solving power flow** to **Power flow finished** and **Simulation finished**, each with the elapsed time, which is the quickest confirmation that a run actually did something.

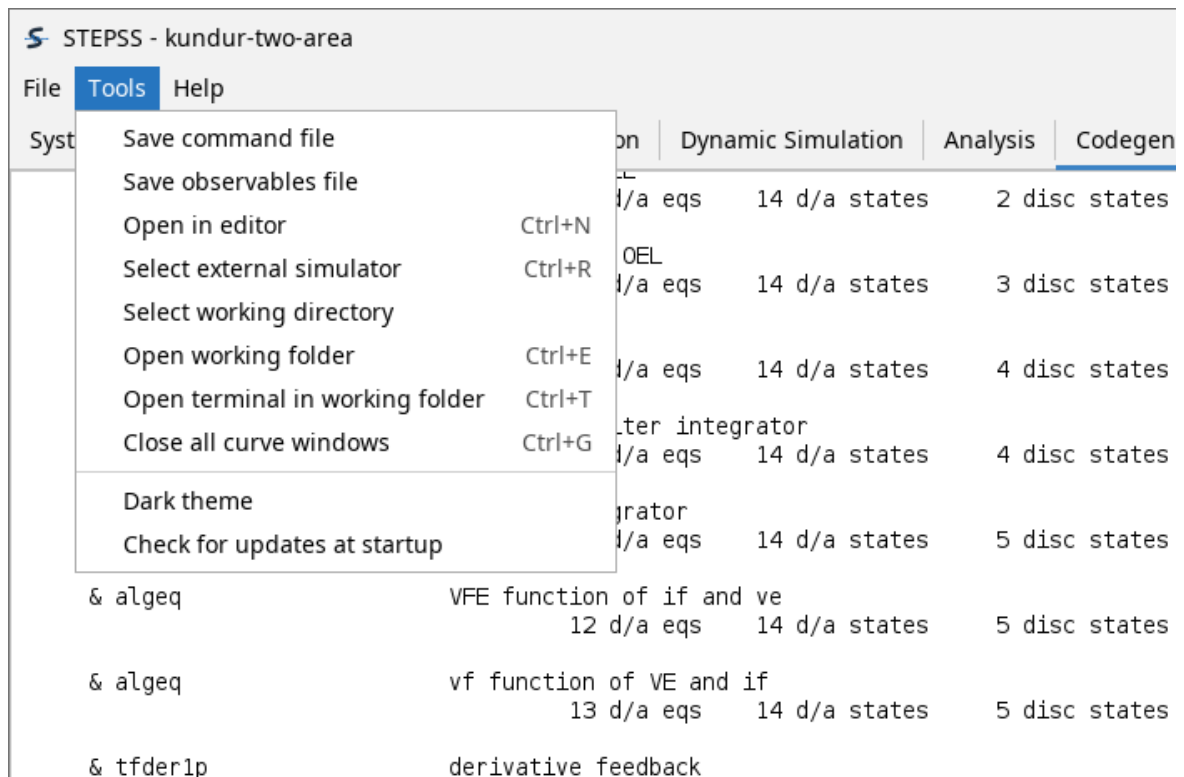


Figure 7.14: The Tools menu open, holding Save command file, Save observables file, Open in editor with Ctrl+N, Select external simulator with Ctrl+R, Select working directory, Open working folder with Ctrl+E, Open terminal in working folder with Ctrl+T and Close all curve windows with Ctrl+G, then below a separator the Dark theme and Check for updates at startup ticks, both unticked.

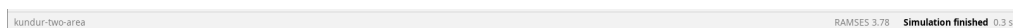
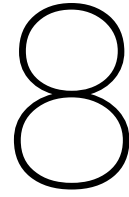


Figure 7.15: The status bar across the foot of the window: the working directory kundur-two-area on the left, and on the right the engine version RAMSES 3.78 followed by Simulation finished and an elapsed time of 0.9 seconds.



Running a simulation in STEPSS GUI

This walks one case end to end: the Kundur two-area system, a load step, and the inter-area oscillation that follows. It needs no data of your own, because the case ships with the application.

If you have not opened it yet, First Run covers **File, Open Examples**. Pick **Kundur two-area** and come back.

8.1. 1. Check what was loaded

Opening the example fills the case in, so there is nothing to do here but look. **System Data** should hold three system data files and one disturbance file:

Slot	File	What it holds
System data 1	lf.dat	The network and the power-flow data
System data 2	dyn.dat	The dynamic models: machines, exciters, governors, loads
System data 3	solveroptions.dat	Solver settings
Disturbance	disturb.dst	What happens, and when

The disturbance is three lines, and worth reading before running anything:

```
0.000 CONTINUE SOLVER BD 0.01 0.0001 0. ALL
1.000 CHGPRM INJ L9 Po +0.5 0
60.000 STOP
```

At $t = 1$ s the active power set point of load L9 is stepped by +0.5 pu, and the run ends at 60 s. Disturbances is the reference for these records.

8.2. 2. Choose what to record

On **Observables**, obs.dat is already loaded and **Save output trajectory** is already checked, which is the pair you need: the trajectory is the file the curves come from, and the observables file says what goes into it.

If you want to watch the run as it happens, set a **runtime observable**. Machine speed of G1 and the voltage of bus 9 are good choices for this case. A refresh interval of 1 second is plenty.

Note

This case sets `$GP_REFRESH_RATE 1.0` in `solveroptions.dat` already. The field on the tab is the same setting, and either place works.

8.3. 3. Solve the power flow

On **Power Flow Simulation**, click **Run power flow**.



Figure 8.1: The Power Flow Simulation tab after a successful run on the Kundur case.

Read the balance at the bottom before going on. For this case the load totals 2734 MW against 2819 MW of generation, the difference being the 85.1 MW of network losses. A balance that does not add up, or a generator sitting on a reactive limit, is a problem to fix here rather than to discover halfway through a dynamic run.

The five inspection buttons beside **Run power flow** become available now: **Bus overview**, **Branch flows**, **Generators & SVCs**, **Adjustable transformers** and **Global power balance**, each of which writes its table into the same pane. The Interface shows what each one produces.

8.4. 4. Run the simulation

On **Dynamic Simulation**, click **Run dynamic simulation**.

The pane opens with the settings actually in force, which is the place to confirm that the case is doing what you meant. This one reports the synchronous reference frame, the KLU sparse solver, local Jacobian updates, and that converged components are skipped.

It also says **parallel computing deactivated by user**, because the case sets `$NB_THREADS 1`. That is a property of this example, not a limit of the engine.

60 seconds of system time takes well under a second of wall time here, in 6004 steps. The counts below are the cost of the run: how often the network and the injector Jacobians were rebuilt and solved. They are what to watch when a bigger case gets slow.

If you set the runtime observables in step 2, a **Run-time curves** window drew them as the run advanced and is still up when it finishes; see Real-time plotting.

8.5. 5. Extract the curves

On **Analysis**, click **Extract curves**. The picker reads the trajectory and offers whatever the observables file recorded.

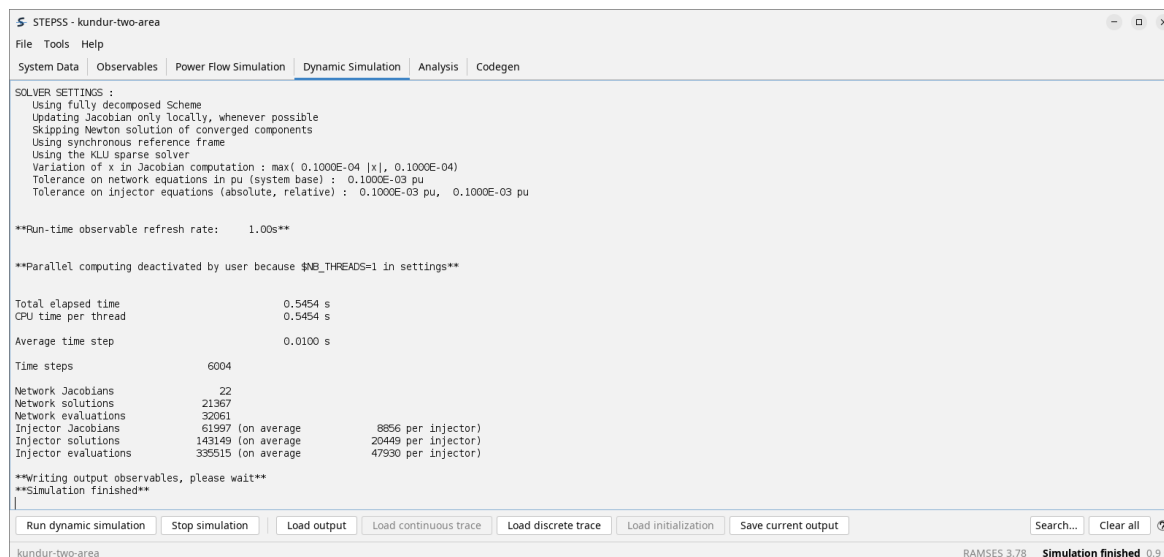


Figure 8.2: The Dynamic Simulation tab after a completed run.

1. Set **Category** to SYNC, which lists the four synchronous machines.
2. Select G1 and set **Observable** to active power produced (MW), then click **Add**.
3. Select G3, leave the observable as it is, and **Add** again.
4. Click **Plot**.

G1 and G3 are the point of the exercise: one sits in each area, so putting them on the same axes shows the oscillation *between* the areas rather than within one.

8.6. 6. Read the result

Three things to see in it.

The two machines swing **in opposition**. When G3 is at a peak, G1 is near a trough. That anti-phase pattern is what makes this an inter-area mode rather than a local one: the two areas are oscillating against each other across the weak tie.

The period is about 1.4 seconds, so roughly 0.7 Hz, which is the range inter-area modes normally sit in.

The oscillation **decays**, and after about ten seconds both machines settle a few megawatts above where they began, sharing the extra load. That decay is the damping, and it is the quantity the next section changes.

8.7. Try it with the stabiliser out

The example ships a second dynamic file, `dyn_noPSS.dat`, identical to `dyn.dat` except that the power system stabiliser gain is set to zero.

On **System Data**, load `dyn_noPSS.dat` into slot 2 in place of `dyn.dat`, then repeat steps 3 to 6. Everything else stays as it is.

The mode is the same and its frequency barely moves, but it is far less damped. Comparing the two runs is the standard demonstration of what a PSS is for, and it is why the case ships both files.

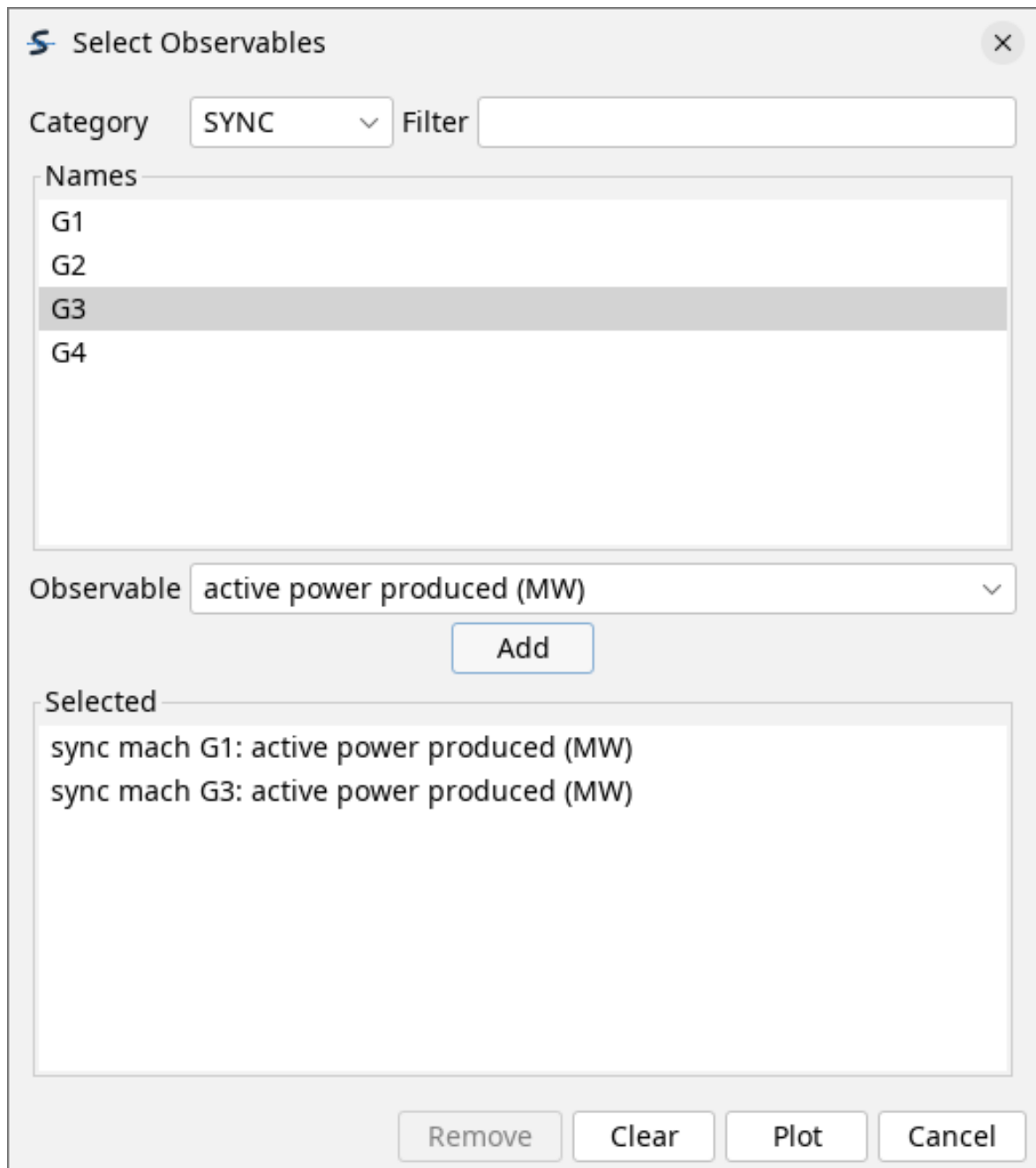


Figure 8.3: The Select Observables dialog with category SYNC chosen, machines G1 to G4 listed, and two entries queued in the Selected list: active power produced in MW for sync mach G1 and for sync mach G3.

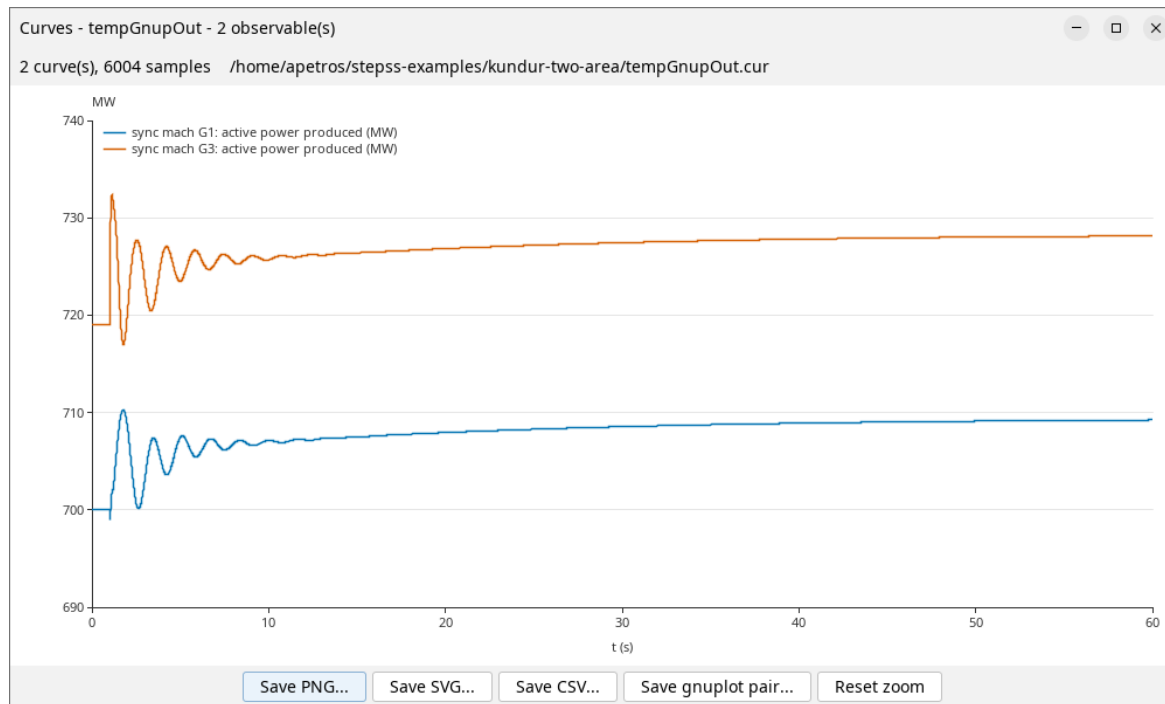


Figure 8.4: Active power produced by generators G1 and G3 against time, drawn as two curves on one set of axes with a legend naming each.

8.8. Where to go next

- Disturbances, to change what happens and when
- Solver Settings, to change how it is solved
- Eigenanalysis, to get this mode's frequency and damping as numbers instead of reading them off a curve
- Kundur Two-Area System, the system itself in detail
- STEPSS in Python, the same run as a script

Part III

Power Flow Computation with Helios

9

Helios data

The following records, documented in Chapter 5 are used by Helios:

BUS

LINE

SWITCH

TRANSFO

TRFO

NRTP.

The additional (mandatory or optional) records only used in power flow computations are documented in this chapter.

9.1. Load and shunt data

Load and shunt data are attached to buses. They are specified in an extended version of the BUS record, as follows:

BUS NAME VNOM PLOAD QLOAD BSHUNT QSHUNT ;

where:

- NAME is the name of the bus. This is a string of at most 8 characters
- VNOM is the nominal voltage, in kV
- PLOAD is the total active power load at the bus, in MW. A positive value corresponds to power drawn from the network
- QLOAD is the total reactive power load at the bus, in Mvar. A positive value corresponds to power drawn from the network
- BSHUNT is the nominal reactive power of the shunt modelled as constant susceptance, in Mvar. This is the reactive power produced by the shunt under the nominal voltage of the bus, specified in its BUS record. BSHUNT is positive (resp. negative) for a shunt capacitor (resp. inductor)
- QSHUNT is the reactive power produced by the shunt modelled as constant power, in Mvar. QSHUNT is positive (resp. negative) for a shunt capacitor (resp. inductor).

The total reactive power Q produced by the two shunt components is given, in Mvar, by:

$$Q = BSHUNT \left(\frac{V}{V_{nom}} \right)^2 + QSHUNT$$

where V is the bus voltage and V_{nom} the corresponding nominal voltage.

If no shunt is connected to the bus, BSHUNT and QSHUNT are set to zero.

If no load is connected to the bus, PLOAD and QLOAD are set to zero.

Let us recall that the PLOAD, QLOAD, BSHUNT and QSHUNT fields are ignored by RAMSES.

9.2. Generator data

Generators are specified by GENER records, as follows:

```
GENER NAME BUS P Q VIMP SNOM QMIN QMAX BR ;
```

where:

- NAME is the name of the generator. This is a string of a most 20 characters
- BUS is the name of the bus which the generator is connected to. This is a string of at most 8 characters
- P is the active power produced by the generator, in MW
- Q is the reactive power produced by the generator, in Mvar. This value is ignored if the VIMP field is nonzero
- VIMP is the voltage imposed by the generator, in pu. If VIMP is zero, the bus is treated as a PQ bus with the reactive power production set to Q; if VIMP is nonzero, the bus is treated as a PV bus and the Q field is ignored
- SNOM is the nominal apparent power of the generator, in MVA
- QMIN is the lower reactive power limit, in Mvar. It is used only if VIMP is nonzero
- QMAX is the upper reactive power limit, in Mvar. It is used only if VIMP is nonzero
- BR is the on/off status of the generator breaker. A zero value indicates that the breaker is open; any other value means that the breaker is closed.

For all generators with a nonzero value of VIMP, the connection bus is initially assumed of the PV type.

If this entails exceeding the generator upper reactive power limit QMAX, the bus switches to PQ type, the QMAX limit is enforced, and Newton iterations continue. If subsequently the bus voltage gets larger than VIMP, the bus switches back to PV type with the voltage imposed at the VIMP value.

Similarly, if the generator lower reactive power limit QMIN is exceeded, the bus switches to PQ type, the QMIN limit is enforced, and Newton iterations continue. If subsequently the bus voltage gets smaller than VIMP, the bus switches back to PV type with the voltage imposed at the VIMP value.

Only one generator is allowed per bus.

All generators are memorized, even those which are disconnected. A disconnected generator is not involved (it appears in the output results with a zero power output) but it can be put into service in the dynamic simulation.

Remark 1. For simplicity, a generator producing constant active and reactive powers can be modelled as a negative load using the BUS and no GENER record.

Remark 2. There exists a variant of the GENER record with the following syntax:

```
GENER NAME BUS P Q V SNOM QMIN QMAX PMIN PMAX BR ;
```

where PMIN (resp. PMAX) is the minimum (resp. maximum) active power that the generator can produce, in MW. If this record is present in the data, the PMIN and PMAX fields are ignored by STEPSS.

9.3. Slack-bus specification

The presence of a slack-bus is mandatory in power flow computations: indeed, not all buses can be of the PV or PQ type, since this would entail specifying the active power losses in the network, which are not known before performing the power flow calculation.

A generator of the PV type must be connected to the slack-bus. Its voltage magnitude, specified in its GENER record, is imposed at the bus, while the voltage phase angle is set to zero.

Helios can handle only one connected network (or island). If the network graph is not connected, *only the connected sub-network including the slack-bus is treated*; the rest of the network is ignored with a warning message.

The SLACK record allows specifying which bus is the slack-bus. The syntax is as follows:

SLACK NAME ;

where NAME is the name of the bus. This is a string of at most 8 characters.

There must be exactly one SLACK record in the whole set of data.

9.4. Static Var Compensators

9.4.1. Modelling

The Static Var Compensator (SVC) is modelled as shown in Fig. 9.1. j is the *monitored* bus, whose voltage is regulated, while i is the *controlled* bus, where the shunt susceptance B is varied. i and j can be any two buses but usually j is the transmission bus on the high-voltage side of the SVC step-up transformer, while the SVC is connected to bus i .

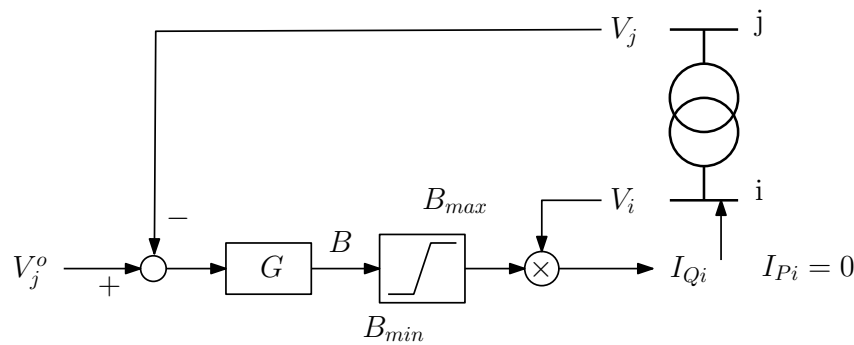


Figure 9.1: Block-diagram of the static var compensator

The SVC being assumed lossless, the active current injected at bus i is zero ($I_{Pi} = 0$), while the reactive current takes on one of the following forms:

$$I_{Qi} = BV_i = G(V_j^o - V_j)V_i \quad \text{under voltage control} \quad (9.1)$$

$$I_{Qi} = B_{max}V_i \quad \text{under upper susceptance limit} \quad (9.2)$$

$$I_{Qi} = B_{min}V_i \quad \text{under lower susceptance limit.} \quad (9.3)$$

Although reference is made to an SVC, the model can be used in general for a component controlling voltage with a droop.

9.4.2. Data format

The record, with keyword SVC, includes the following fields:

SVC NAME CON_BUS MON_BUS V0 Q0 SNOM BMAX BMIN G BR ;

where:

- NAME is the name of the compensator. This is a string of a most 20 characters

- CON_BUS is the name of the controlled bus. This is a string of a most 8 characters
- MON_BUS is the name of the monitored bus. This is a string of a most 8 characters
- V0 is the voltage setpoint V_j^o , in pu (see Fig. 9.1)
- Q0 is the reactive power setpoint, in Mvar. If V0 is set to zero, the SVC is treated under constant power, with $P = 0$ and $Q = Q0$, and no limit is tested. If V0 is nonzero, the Q0 field is ignored
- SNOM is the nominal reactive power of the SVC, in Mvar
- BMAX is the maximal nominal reactive power, in Mvar. This is the reactive power produced by the compensator under $V_i = 1$ pu, when B is at the maximal value B_{max} .
- BMIN is the minimal nominal reactive power, in Mvar. This is the reactive produced by the compensator under $V_i = 1$ pu, when B is at the minimal value B_{min} .
- G is the gain G , in pu on the $(V_B, SNOM)$ base, where V_B is the nominal voltage at the controlled bus, as given by its BUS record
- BR is the on/off status of the SVC breaker. A zero value indicates that the breaker is open, any other value means that it is closed.

It is common for BMAX to be positive and BMIN negative but other combinations are allowed. For all SVCs with a nonzero value of V0, Eq. (9.1) is solved initially.

If this entails exceeding the susceptance upper reactive power limit BMAX, Eq. (9.2) is substituted, the BMAX limit is enforced, and Newton iterations continue. If subsequently $G(V_j^o - V_j) < B_{max}$, the program reverts to Eq. 9.1 and keeps on iterating.

Similarly, if the SVC lower susceptance limit BMIN is exceeded, Eq. (9.3) is substituted, the BMIN limit is enforced, and Newton iterations continue. If subsequently $G(V_j^o - V_j) > B_{min}$, the program reverts to Eq. (9.1) and keeps on iterating.

Only one SVC is allowed per bus.

It is not allowed to connect both a generator and an SVC to the same bus.

All SVCs are memorized, even those which are disconnected. A disconnected SVC is not involved (it appears in the output results with a zero power output) but it can be put into service in the dynamic simulation.

9.5. Transformer ratio adjustment for voltage control

9.5.1. Modelling

Helios can adjust the ratio of a designated transformer with the objective of bringing a controlled voltage inside a deadband $[V_{des} - \epsilon, V_{des} + \epsilon]$, where V_{des} is the desired voltage and ϵ is a tolerance.

The ratio is changed in discrete steps (as in the real life), between a minimum and a maximum value. During the computation, the ratio is changed by one step at a time, after which Newton iterations are performed until convergence is achieved. The process is repeated until the controlled voltage falls in the deadband. When multiple transformers are adjusted, some may reach their deadbands before others.

9.5.2. First data format

The first way to specify ratio adjustment is through the TRFO record. The following refers to the material presented in Section 5.4.2.

The controlled bus is CONBUS. This must be one of the two ending buses of the transformer. An empty or blank string *enclosed within quotes* indicates that the transformer ratio is not to be adjusted. In this case, however, dummy values must be provided for each of the fields listed below.

The fields of concern are:

- NFIRST: the ratio in % corresponding to the first tap position. This is the lower bound on the transformer ratio
- NLAST: the ratio in % corresponding to the last tap position. This is the upper bound on the transformer ratio
- NBPOS: the total number of tap positions including the first and the last
- TOLV: the voltage tolerance ϵ , in pu
- VDES: the desired voltage V_{des} , in pu.

The transformer ratio n corresponding to position p ($1 \leq p \leq \text{NBPOS}$) of the tap changer is given by:

$$n = \frac{\text{NFIRST}}{100} + \frac{p - 1}{\text{NBPOS} - 1} \frac{\text{NLAST} - \text{NFIRST}}{100} \quad (9.4)$$

The value of p is increased or decreased by one at each adjustment iteration.

An initial value of the transformer ratio is given by the N field of the TRFO record. If the transformer is to be adjusted, before starting the power flow computation that value is adjusted to coincide with the nearest tap position, in accordance with Eq. (9.4).

9.5.3. Second data format

The second way to specify ratio adjustment is through a separate record with keyword LTC-V. The syntax is as follows:

LTC-V NAME CON_BUS NFIRST NLAST NBPOS TOLV VDES ;

where NAME is the name of the controlled transformer (a string of at most 20 characters) and all the other fields have the same meaning as for the TRFO record.

A transformer can be controlled by a single tap changer only. It is more natural to use the LTC-V record in association with a TRANSFO record. However, the LTC-V record can be associated with a TRFO record, provided that no adjustment is specified in the latter.

9.6. Phase-shifting transformer ratio adjustment for power control

9.6.1. Modelling

Helios can also adjust the phase angle of a transformer with the objective of bringing the active power flow in a branch inside a deadband $[P_{des} - \epsilon, P_{des} + \epsilon]$, where P_{des} is the desired power flow and ϵ is a tolerance.

The adjustment is very similar to that of in-phase transformers for voltage control detailed in Section 9.5.

9.6.2. Data format

The record, with keyword PSHIFT-P, includes the following fields:

PSHIFT-P CONTRFO MONBRANCH PHAFIRST PHALAST NBPOS SIGN PDES TOLP ;

where:

- CONTRFO is the name for the transformer whose phase angle is to be adjusted. This is a string of at most 20 characters, defined in either a TRFO or a TRANSFO record. If the transformer does not exist, the whole record is ignored with a warning message
- MONBRANCH is the name of the branch in which the active power flow P is monitored. This is a string of at most 20 characters, defined in either a LINE, a TRFO or a TRANSFO record. The sign convention is the following: P is the active power flow leaving the first bus specified in the LINE, TRFO or TRANSFO record and entering the branch of concern
- PHAFIRST is the phase angle ϕ , in degrees, corresponding to the first tap position. This is the lower bound on ϕ
- PHALAST is the phase angle ϕ , in degrees, corresponding to the last tap position. This is the upper bound on ϕ
- NBPOS is the number of tap positions
- SIGN is an indication of the direction in which the phase angle ϕ must be adjusted to reach the objective. A value of 1 indicates that ϕ must be increased to increase the active power flow in the monitored branch. A value of -1 indicates that it must be decreased. Any other value is invalid and causes the program to stop
- PDES is the desired active power flow, in MW
- TOLP is the tolerance ϵ , in MW.

The phase angle ϕ corresponding to position p ($1 \leq p \leq \text{NBPOS}$) of the tap changer is given by:

$$\phi = \text{PHAFIRST} + \frac{p - 1}{\text{NBPOS} - 1} (\text{PHALAST} - \text{PHAFIRST}) \quad (9.5)$$

The value of p is increased or decreased by one at each adjustment iteration.

Helios performs a sensitivity analysis to determine whether the phase angle should be increased or decreased. If this analysis indicates a direction opposite to what is specified in SIGN, a warning is issued and the value of SIGN is ignored. On output, when it produces a file with the records updated, Helios sets SIGN to the value corresponding to its sensitivity analysis.

An initial value of the phase angle is given by the PHI field of the TRANSFO record. If the transformer is to be adjusted, before starting the power flow computation, that value is adjusted to coincide with the nearest tap position, in accordance with Eq. (9.5).

A transformer cannot be controlled by both an LTC-V and a PSHIFT-P record.

Only one PSHIFT-P record per transformer is allowed. The PSHIFT-P record is intended to be used in association with a TRANSFO record. However, it is allowed to associate it with a TRFO record in spite of the fact that the latter assumes a zero phase angle. In this case, the angle will be initialized to zero and will be controlled as specified in the PSHIFT-P record.

9.7. Bus voltages: initial values and results

On output, Helios produces a file with the computed bus voltage magnitudes and phase angles. The latter are specified in records with keyword LFRESV. The syntax is as follows:

LFRESV BUS MODV PHASV ;

where:

- BUS is the name of the bus. This is a string of a most 8 characters defined in a BUS record
- MODV is the bus voltage magnitude, in pu
- PHAV is the bus voltage phase angle, in radian, the slack bus being the reference.

If LFRESV records are provided on input, they are used as initial voltages of the Newton iterations, at the specified buses.

If no LFRESV record is specified at a bus:

- the voltage magnitude is initialized to 1 pu if the bus is of the PQ type or to the value imposed by the generator in case of a PV bus
- the phase angle is initialized to 0 degree.

Thus, if the LFRESV records obtained from a first run of Helios are added to the data files, the other data being unchanged, no Newton iteration is going to be performed since we are already at the solution. This is an easy way to check that system data come with their corresponding voltages.

9.8. Share of records between Helios and RAMSES

A summary of the records used by respectively Helios and RAMSES is given in Table 9.1.

Table 9.1: records used by Helios and RAMSES, respectively

record	in Helios	in RAMSES
BUS	all 6 fields used	only first two fields used
LINE	used	used
SWITCH	used	used
NRTP	used	used
TRANSFO	used	used
TRFO	used	only fields 1 to 9 and 15 used
SHUNT	ignored	used
GENER	used	ignored
SVC	used	ignored
SLACK	used	used
LFRESV	used	used
LTC-V	used	ignored
PSHIFT-P	used	ignored

9.9. Computation control parameters

9.9.1. Parameters

Helios performs Newton(-Raphson) iterations to solve the power flow equations. At the k -th iteration, the following indices are computed:

$\epsilon_P = \max_i |f_i(\mathbf{v}^{(k)}, \boldsymbol{\theta}^{(k)}) - P_i^o|$ the largest absolute mismatch of the active power equations

$\epsilon_Q = \max_i |g_i(\mathbf{v}^{(k)}, \boldsymbol{\theta}^{(k)}) - Q_i^o|$ the largest absolute mismatch of the reactive power equations

$\epsilon_S = \max_i \sqrt{(f_i(\mathbf{v}^{(k)}, \boldsymbol{\theta}^{(k)}) - P_i^o)^2 + (g_i(\mathbf{v}^{(k)}, \boldsymbol{\theta}^{(k)}) - Q_i^o)^2}$ the largest apparent power mismatch.

Convergence is achieved once :

- both ϵ_P and ϵ_Q are below specified thresholds
- all controls on transformer ratios and phase shifts are satisfied, and
- all generators and SVCs are within their reactive limits.

ϵ_S is used to check that the solution is accurate enough to :

- "freeze" the Jacobian matrix (in order to save some computing time)
- check the generator and SVC reactive power limits and enforce the latter if needed
- adjust the transformer ratios and phase shifts to control voltage magnitudes and active power flows, if specified in the data.

Finally, the iterations stop as soon as divergence is detected. To this purpose the quadratic index:

$$\varphi(k) = \sum_i \sqrt{(f_i(\mathbf{v}^{(k)}, \boldsymbol{\theta}^{(k)}) - P_i^o)^2 + (g_i(\mathbf{v}^{(k)}, \boldsymbol{\theta}^{(k)}) - Q_i^o)^2}$$

is monitored. Under normal convergence conditions, φ decreases from one iteration to the next. Therefore, the increase in φ is used to detect divergence. The algorithm stops at the iteration k such that :

$$\varphi(k) > 1.1 \varphi(k-1)$$

However, this test is skipped at any iteration k that follows the switching of generators or SVCs under limit, or the adjustment of transformer ratios and phase shifts¹.

9.9.2. Records

The records detailed hereafter are used to control the computation. They all start with a \$ to distinguish them from the other records. They all have a single field, as detailed in Table 9.2. The default value assigned in the absence of the record is given in the fourth column.

¹indeed, these adjustments cause increases in φ that have nothing to do with divergence

Table 9.2: Computation control parameters

record	meaning of field	unit	default value
\$SBASE	base power (on which pu values are expressed)	MVA	100
\$TOLAC	ϵ_P	MW	0.1
\$TOLREAC	ϵ_Q	Mvar	0.1
\$NBITMA	max number of iterations	-	20
\$MISQLIM	value of ϵ_S below which the reactive power limits are checked and enforced. Set to 0 to skip the limit check	MVA	20
\$MISBLOC	value of ϵ_S below which the Jacobian is kept constant.	MVA	10
\$MISADJ	value of ϵ_S below which transformers are adjusted. Set to 0 to skip adjustment	MVA	10
\$DIVDET	set to 1 to activate divergence test during iterations. Set to 0 to skip test	-	0

Part IV

Dynamic Simulation with RAMSES

10

Reference frame and model initialization

10.1. Phasor approximation and reference frames

Under the phasor approximation, the network equations can be written in compact form as:

$$\bar{\mathbf{I}} = \mathbf{Y}\bar{\mathbf{V}} \quad (10.1)$$

where:

$\bar{\mathbf{I}}$ is the vector of complex currents injected into the network at the various buses

$\bar{\mathbf{V}}$ is the vector of complex voltages at the various buses

$\mathbf{Y}$ is the bus (or nodal) admittance matrix.

Which frequency consider for the phasors and the admittance matrix ? In dynamic regime, each synchronous machine defines a local frequency. In most cases, those various frequencies remain close to the nominal frequency f_N . For large deviations with respect to the nominal value f_N , it can be envisaged to update the entries of the $\mathbf{Y}$ matrix using the average system frequency, for instance. Otherwise, the admittances are simply computed at frequency f_N .

Under the phasor approximation, the voltage at the i -th bus takes on the form:

$$\begin{aligned} v_i(t) &= \sqrt{2}V_i(t) \cos(\omega_N t + \phi_i(t)) = \sqrt{2} \operatorname{re} [V_i(t)e^{j\phi_i(t)}e^{j\omega_N t}] \\ &= \sqrt{2} \operatorname{re} [(v_{xi}(t) + jv_{yi}(t))e^{j\omega_N t}] \end{aligned} \quad (10.2)$$

where $v_{xi}(t) + jv_{yi}(t)$ is the voltage phasor, in rectangular coordinates, expressed with respect to (x, y) axes rotating at the angular speed $\omega_N = 2\pi f_N$, as illustrated in Fig. 10.1.

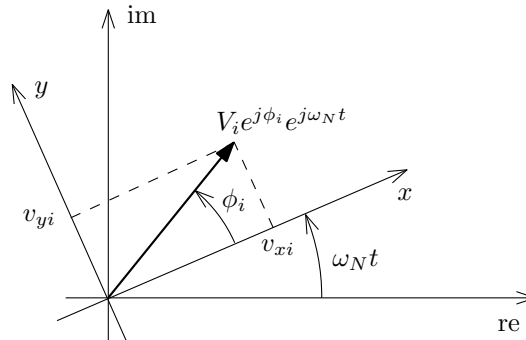


Figure 10.1: Reference axes and voltage phasor

Equations similar to (10.2) can be written for the currents injected into the network, the currents flowing into the network branches, etc.

Instead of determining the “full wave” evolution of voltages or currents, dynamic simulation in phasor mode aims at rendering the time evolution of v_{xi} and v_{yi} ¹.

The (x, y) axes rotating at angular speed ω_N make up a *synchronous reference*.

Although simple, this reference frame suffers from a significant limitation. Indeed, assuming that the power system initially operates at the nominal frequency f_N ², after a disturbance, it will settle at a different frequency f , unless its model includes an infinite bus imposing the frequency f_N . From there on, the various voltage and current phasors rotate at the angular speed $2\pi f \neq \omega_N$. Hence, phasor components such as v_{xi} and v_{yi} will oscillate at a frequency $|f - f_N|$, although the system is at equilibrium from a practical viewpoint. For that reason, the

¹In RAMSES the rectangular components have been preferred to the polar components V_i and ϕ_i

²This is a common assumption in power system simulation software

synchronous reference is not suitable for long-term simulations, since tracking the oscillations at frequency $|f - f_N|$ requires using a small enough time step size. The synchronous reference frame is suited to short-term simulations (where frequency has not yet returned to steady state) or when the model includes an infinite bus driving the frequency back to f_N .

In fact, any (more convenient) speed can be considered for the reference axes (x, y) . The only constraint is that all voltage and current phasors refer to the same axes.

In the *Center Of Inertia* (COI) reference frame, the (x, y) axes rotate at the angular frequency:

$$\omega_{coi} = \frac{\sum_{i=1}^m M_i \omega_i}{\sum_{i=1}^m M_i} \quad (10.3)$$

where:

m is the total number of synchronous machines

ω_i is the rotor speed of i -th synchronous machine ($i = 1, \dots, m$)

M_i is the inertia coefficient of the i -th machine ($i = 1, \dots, m$).

The M_i values relate to the inertia constants H_i (in s) of the individual machines, expressed on a common base power S_B (in MVA):

$$M_i = 2H_i \frac{S_{Ni}}{S_B}$$

where S_{Ni} is the nominal apparent power of the i -th machine (in MVA).

The COI reference frame does not suffer from the above mentioned drawback. Indeed, when the system settles at a frequency f , all synchronous machines rotate at the angular speed $2\pi f$, and so do the reference axes ($\omega_{coi} = 2\pi f$). Hence, phasor components such as v_{xi} and v_{yi} tend to constant values, a larger time step size can be used and the simulation is computationally less demanding. The COI reference frame is well suited long-term simulations.

The COI frequency can be used as an average system frequency in injector and two-port models: see Section 25.4.4.

10.1.1. Specifying the reference frame

The reference to be used is specified in the simulation settings: see Chapter 13.

The presence of a Thévenin equivalent (involving a constant frequency voltage source) among the components causes the simulation to use the synchronous reference frame.

10.1.2. Implementation of COI reference frame

To preserve the “sparsity” of the model in spite of Eq. (10.3), which embeds the rotor speeds of all synchronous machines, the value of ω_{coi} at the previous time step is used. See:

D. Fabozzi and T. Van Cutsem “On angle references in long-term time-domain simulations”, IEEE Transactions on Power Systems, Vol. 26, No 1, pp. 483-484, Feb. 2011

for a more detailed presentation of this technique.

10.2. Network equations

With all voltage and current phasors referred to the (x, y) axes, the network equations can be decomposed into:

$$\mathbf{i}_x + j \mathbf{i}_y = \mathbf{Y} (\mathbf{v}_x + j \mathbf{v}_y) = (\mathbf{G} + j \mathbf{B}) (\mathbf{v}_x + j \mathbf{v}_y) \quad (10.4)$$

with:

$$\mathbf{v}_x = \begin{bmatrix} v_{x1} \\ \vdots \\ v_{xN} \end{bmatrix} \quad \mathbf{v}_y = \begin{bmatrix} v_{y1} \\ \vdots \\ v_{yN} \end{bmatrix} \quad \mathbf{i}_x = \begin{bmatrix} i_{x1} \\ \vdots \\ i_{xN} \end{bmatrix} \quad \mathbf{i}_y = \begin{bmatrix} i_{y1} \\ \vdots \\ i_{yN} \end{bmatrix} \quad (10.5)$$

where $\mathbf{G}$ is the conductance matrix and $\mathbf{B}$ the susceptance matrix.

Decomposing into real and imaginary parts and assembling into a single equation yields:

$$\begin{bmatrix} \mathbf{i}_x \\ \mathbf{i}_y \end{bmatrix} = \begin{bmatrix} \mathbf{G} & -\mathbf{B} \\ \mathbf{B} & \mathbf{G} \end{bmatrix} \begin{bmatrix} \mathbf{v}_x \\ \mathbf{v}_y \end{bmatrix} \quad (10.6)$$

Thus, for a network with N buses, there are $2N$ equations (10.6) involving $4N$ variables.

10.3. Initialization procedure

The dynamic simulation is initialized according to the procedure detailed next.

1. The starting point is the vector of initial bus voltages, as detailed in Section 9.7
2. setting the bus voltages to those values, the active and reactive power flows in the network branches and shunts are computed
3. by summing the power flows in incident branches, the bus power injections are determined. This is illustrated in Fig. 10.2, where P_i^{inj} (resp. Q_i^{inj}) is the active (resp. reactive) power injected into the i -th bus. A *positive value* corresponds to power *entering the network*
4. at each bus, the bus power injection is shared among the various components (generators, loads, injectors, etc.) connected to that bus.

There are two ways of assigning power to the j -th component:

- (i) by specifying the active and reactive powers injected into the network by that component:

$$P_j^c = P_j^{c0} \quad Q_j^c = Q_j^{c0} \quad (10.7)$$

- (ii) by specifying the fraction of the bus power injection taken by that component :

$$P_j^c = f_{Pj} P_i^{inj} \quad Q_j^c = f_{Qj} Q_i^{inj} \quad (10.8)$$

where f_{Pj} (resp. f_{Qj}) denotes the fraction relative to active (resp. reactive) power.

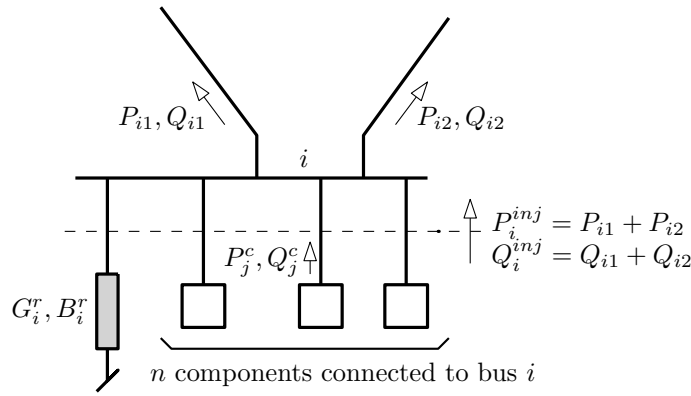
5. the remaining power, not taken by the components at bus i , i.e.

$$P_i^r = P_i^{inj} - \sum_{j=1}^n P_j^c \quad Q_i^r = Q_i^{inj} - \sum_{j=1}^n Q_j^c$$

is checked. If larger than an internal tolerance, the power is assigned to an impedance load³, as shown in Fig. 10.2. The load admittance is such that the power balance is satisfied at $t = 0$:

$$(G_i^r - j B_i^r) V_i(0)^2 = -(P_i^r + j Q_i^r) \quad (10.9)$$

where $V_i(0)$ is the initial bus voltage magnitude⁴. Note that the sign of G_i^r is opposite to that of P_i^r , while the sign of B_i^r is the same as that of Q_i^r .

Figure 10.2: initial powers at bus i

The two ways mentioned under items (i) and (ii) above are *mutually exclusive*, i.e. either a nonzero power or a nonzero fraction is specified, but not both. In mathematical terms:

$$f_{Pj} \times P_j^{c0} = 0 \quad f_{Qj} \times Q_j^{c0} = 0 \quad (10.10)$$

Although they are typically in the interval $[0 \ 1]$, the values of the f_{Pj} 's and f_{Qj} 's may be negative or larger than one.

In the quite common case when a single component is connected to a bus, and it takes the whole power consumed or produced at that bus, it is convenient to specify:

$$f_{Pj} = 1 \quad P_j^{c0} = 0 \quad f_{Qj} = 1 \quad Q_j^{c0} = 0$$

With fractions instead of powers, the data can be re-used at another initial operating point⁵.

The constant admittance (or impedance) loads corresponding to Eq. (10.9) are created automatically at system initialization. They are given names of the type M_bus , where M stands for "mismatch" and bus is the name of the bus to which the admittance is connected. Names starting with $M_$ are reserved and must not be used when defining constant admittance loads.

A large value of G_i^r (resp. B_i^r) may be intentional, when a load is to be modelled as a constant shunt admittance and the task is left to RAMSES. However, it may also result from a mistake in the initial power balance. It is recommended to check the values of the $M_$ loads through the menu "Load Initialization" in the STEPSS interface.

☛ Here is an example of output produced by RAMSES at initialization.

NUMBER OF IMPEDANCE LOADS : 3 (M_ type: 3)

load name	bus name	P	Q
M_2	2	90.002	17.997
M_3	3	0.013	-0.011
M_4	4	-0.017	-0.022

In this example, three $M_$ loads have been produced at initialization, respectively at buses 2, 3 and 4. The one at bus 3 consumes 0.013 MW and produces 0.011

³to be more precise a constant shunt impedance, or equivalently a constant shunt admittance load

⁴the left-hand side of Eq. (10.9) is the complex power consumed by the admittance, while $P_i^r + jQ_i^r$ is counted positive when the power is injected into the network. Hence, the minus sign in the right hand side of the equation

⁵i.e. when another set of initial voltages is used

Mvar. This load was created because it is larger than the internal tolerance but, for a transmission system, this is a negligible value. The same holds true for the M_ load at bus 4. On the other hand, the M_ load at bus 2 is non negligible. This could be due to forgetting to specify a load at bus 2.

10.4. Data format

The way to specify the values of f_{pj} , P_j^{c0} , f_{Qj} and Q_j^{c0} is detailed in the following subsequent sections of this documentation:

- for a synchronous machine: Section 11.2.9
- for a Thévenin equivalent: Section 11.3.2
- for a constant impedance load: Section 11.4.2
- for an injector with user-defined model: Section 25.6.1
- for a two-port with user-defined model: Section 25.6.2

11

Synchronous machines, Thévenin
equivalents and impedance loads

STEPSS is an open-source software in the sense explained in Section 1.4. Nevertheless, two power system components have their model hard-coded in the software: the synchronous machine and the Thévenin equivalent. The reason is that those components are used to define the phase angle references, as explained in Section 10.1; it would be impractical to leave this task to a user-defined model.

The constant impedance (or admittance) load is also hard-coded, since it is used at initialisation to automatically replace missing bus power injection, as explained in Section 10.3.

This chapter is devoted to presenting the models of those three basic components.

11.1. Nominal system frequency

The nominal system frequency is specified by an FNOM record as follows.

FNOM F ;

where F is the value of the nominal frequency, in Hz.

Note that the presence of this record is **mandatory**.

11.2. Synchronous machines

The synchronous machine has a detailed sixth-order model, including four rotor windings with saturation effects. Models with less rotor windings can be specified by skipping some data. The notation used for the windings is shown in Fig. 11.1.

To encompass the various combinations in a single model, “switches” are used. These are integer parameters defined as follows:

$S_{d1} = 1$ if there is a damper winding $d1$, $= 0$ otherwise

$S_{q1} = 1$ if there is a damper winding $q1$, $= 0$ otherwise

$S_{q2} = 1$ if there is an equivalent winding $q2$, $= 0$ otherwise

The allowed combinations of S_{d1} , S_{q1} and S_{q2} are shown in the Table 11.1, with the corresponding values of the switches. Note that the second and third combinations yield the same results.

Table 11.1: Variants of the synchronous machine model

model	switches
detailed, round rotor	$S_{d1} = 1, S_{q1} = 1, S_{q2} = 1$
detailed, salient-pole rotor	$S_{d1} = 1, S_{q1} = 1, S_{q2} = 0$
detailed, salient-pole rotor	$S_{d1} = 1, S_{q1} = 0, S_{q2} = 1$
simplified, field winding only	$S_{d1} = 0, S_{q1} = 0, S_{q2} = 0$

11.2.1. Park transformation

The well-known Park transformation is used to replace time-varying inductances and oscillatory stator currents and voltages with constant values. The equivalent windings are defined in Fig. 11.1.

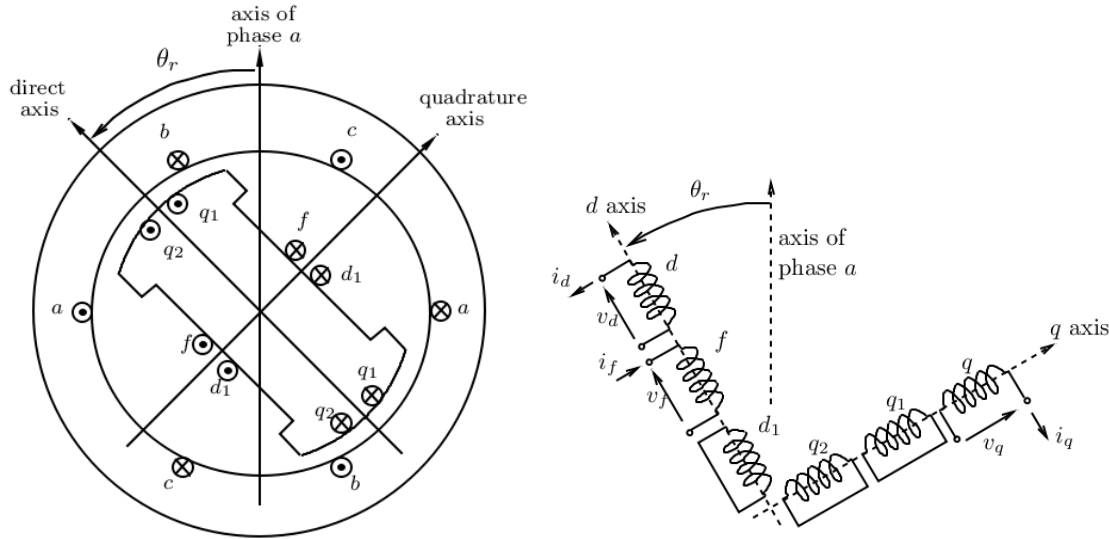


Figure 11.1: Equivalent windings of Park transformation

11.2.2. Flux-current relationships

Using the Equal-Mutual-Flux-Linkage (EMFL) per unit system, the relationship between magnetic flux linkages (denoted with ψ) and currents (denoted with i) can be written as:

$$\begin{bmatrix} \psi_d \\ \psi_f \\ \psi_{d1} \end{bmatrix} = \begin{bmatrix} L_\ell + M_d & M_d & S_{d1}M_d \\ M_d & L_{\ell f} + M_d & S_{d1}M_d \\ S_{d1}M_d & S_{d1}M_d & L_{\ell d1} + S_{d1}M_d \end{bmatrix} \begin{bmatrix} i_d \\ i_f \\ i_{d1} \end{bmatrix} \quad (11.1)$$

$$\begin{bmatrix} \psi_q \\ \psi_{q1} \\ \psi_{q2} \end{bmatrix} = \begin{bmatrix} L_\ell + M_q & S_{q1}M_q & S_{q2}M_q \\ S_{q1}M_q & L_{\ell q1} + S_{q1}M_q & S_{q2}M_q \\ S_{q2}M_q & S_{q2}M_q & L_{\ell q2} + S_{q2}M_q \end{bmatrix} \begin{bmatrix} i_q \\ i_{q1} \\ i_{q2} \end{bmatrix} \quad (11.2)$$

where L_ℓ is the stator leakage inductance, $L_{\ell f}$ the field winding leakage inductance, $L_{\ell d1}$, $L_{\ell q1}$, $L_{\ell q2}$ the rotor winding leakage inductances, and M_d (resp. M_q) the direct-axis (resp. quadrature-axis) mutual inductance.

The d and q components of the air-gap flux are given by:

$$\psi_{ad} = M_d(i_d + i_f + S_{d1}i_{d1}) \quad (11.3)$$

$$\psi_{aq} = M_q(i_q + S_{q1}i_{q1} + S_{q2}i_{q2}) \quad (11.4)$$

Substituting in Eqs. (11.1) and (11.2), one easily obtains:

$$\psi_d = L_\ell i_d + \psi_{ad} \quad (11.5)$$

$$\psi_f = L_{\ell f} i_f + \psi_{ad} \quad (11.6)$$

$$\psi_{d1} = L_{\ell d1} i_{d1} + S_{d1} \psi_{ad} \quad (11.7)$$

$$\psi_q = L_\ell i_q + \psi_{aq} \quad (11.8)$$

$$\psi_{q1} = L_{\ell q1} i_{q1} + S_{q1} \psi_{aq} \quad (11.9)$$

$$\psi_{q2} = L_{\ell q2} i_{q2} + S_{q2} \psi_{aq} \quad (11.10)$$

Using Eqs. (11.6, 11.7, 11.9, 11.10), the rotor currents are obtained from the flux linkages as:

$$i_f = \frac{\psi_f - \psi_{ad}}{L_{\ell f}} \quad (11.11)$$

$$i_{d1} = \frac{\psi_{d1} - S_{d1}\psi_{ad}}{L_{\ell d1}} \quad (11.12)$$

$$i_{q1} = \frac{\psi_{q1} - S_{q1}\psi_{aq}}{L_{\ell q1}} \quad (11.13)$$

$$i_{q2} = \frac{\psi_{q2} - S_{q2}\psi_{aq}}{L_{\ell q2}} \quad (11.14)$$

11.2.3. Saturation model

Let M_d^u and M_q^u be the *unsaturated* direct- and quadrature-axis mutual inductances, related to their corresponding saturated values M_d and M_q by:

$$M_d = \frac{M_d^u}{1 + m \left(\sqrt{\psi_{ad}^2 + \psi_{aq}^2} \right)^n} \quad (11.15)$$

$$M_q = \frac{M_q^u}{1 + m \left(\sqrt{\psi_{ad}^2 + \psi_{aq}^2} \right)^n} \quad (11.16)$$

Replacing M_d by the above expression and i_f and i_{d1} by Eqs. (11.11, 11.12), respectively, the expression (11.3) of the d -axis air-gap flux becomes:

$$\psi_{ad} = \frac{M_d^u}{1 + m \left(\sqrt{\psi_{ad}^2 + \psi_{aq}^2} \right)^n} \left(i_d + \frac{\psi_f - \psi_{ad}}{L_{\ell f}} + S_{d1} \frac{\psi_{d1} - S_{d1}\psi_{ad}}{L_{\ell d1}} \right)$$

Rearranging terms yields the algebraic equation:

$$\psi_{ad} \left(\frac{1 + m \left(\sqrt{\psi_{ad}^2 + \psi_{aq}^2} \right)^n}{M_d^u} + \frac{1}{L_{\ell f}} + \frac{S_{d1}}{L_{\ell d1}} \right) - i_d - \frac{1}{L_{\ell f}} \psi_f - \frac{S_{d1}}{L_{\ell d1}} \psi_{d1} = 0 \quad (11.17)$$

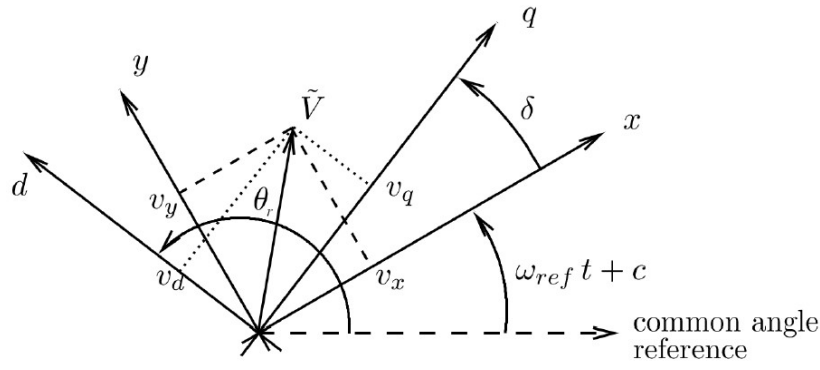
Similar operations in the q axis yield:

$$\psi_{aq} \left(\frac{1 + m \left(\sqrt{\psi_{ad}^2 + \psi_{aq}^2} \right)^n}{M_q^u} + \frac{S_{q1}}{L_{\ell q1}} + \frac{S_{q2}}{L_{\ell q2}} \right) - i_q - \frac{S_{q1}}{L_{\ell q1}} \psi_{q1} - \frac{S_{q2}}{L_{\ell q2}} \psi_{q2} = 0 \quad (11.18)$$

11.2.4. Reference frame

All synchronous machines have their rotor positions referred to the x axis (defined in the previous chapter).

The *rotor angle* δ of a machine is the angle difference between its q axis and the x reference axis, as shown in Fig. 11.2. In steady state, the machine internal emf (proportional to field current) is aligned along the q axis; δ is thus the phase angle of that emf with respect to the x axis.

Figure 11.2: Definition of the rotor angle δ

The d and q components of the stator voltage relate to their x and y components through:

$$\begin{pmatrix} v_d \\ v_q \end{pmatrix} = \begin{pmatrix} -\sin \delta & \cos \delta \\ \cos \delta & \sin \delta \end{pmatrix} \begin{pmatrix} v_x \\ v_y \end{pmatrix} \quad (11.19)$$

and similarly for the current:

$$\begin{pmatrix} i_d \\ i_q \end{pmatrix} = \begin{pmatrix} -\sin \delta & \cos \delta \\ \cos \delta & \sin \delta \end{pmatrix} \begin{pmatrix} i_x \\ i_y \end{pmatrix} \quad (11.20)$$

where δ is the rotor angle.

Replacing i_d and i_q with the above expressions in Eqs. (11.17, 11.18) gives:

$$\psi_{ad} \left(\frac{1 + m(\sqrt{\psi_{ad}^2 + \psi_{aq}^2})^n}{M_d^u} + \frac{1}{L_{\ell f}} + \frac{S_{d1}}{L_{\ell d1}} \right) + \sin \delta i_x - \cos \delta i_y - \frac{1}{L_{\ell f}} \psi_f - \frac{S_{d1}}{L_{\ell d1}} \psi_{d1} = 0 \quad (11.21)$$

and similarly for the q axis:

$$\psi_{aq} \left(\frac{1 + m(\sqrt{\psi_{ad}^2 + \psi_{aq}^2})^n}{M_q^u} + \frac{S_{q1}}{L_{\ell q1}} + \frac{S_{q2}}{L_{\ell q2}} \right) - \cos \delta i_x - \sin \delta i_y - \frac{S_{q1}}{L_{\ell q1}} \psi_{q1} - \frac{S_{q2}}{L_{\ell q2}} \psi_{q2} = 0 \quad (11.22)$$

11.2.5. Park equations

The original Park equations are written as:

$$v_d = -R_a i_d - \omega \psi_q \quad (11.23)$$

$$v_q = -R_a i_q + \omega \psi_d \quad (11.24)$$

$$\frac{d\psi_f}{dt} = \omega_N (K_f v_f - R_f i_f) \quad (11.25)$$

$$\frac{d\psi_{d1}}{dt} = -\omega_N R_{d1} i_{d1} \quad (11.26)$$

$$\frac{d\psi_{q1}}{dt} = -\omega_N R_{q1} i_{q1} \quad (11.27)$$

$$\frac{d\psi_{q2}}{dt} = -\omega_N R_{q2} i_{q2} \quad (11.28)$$

where R_a is the stator resistance, R_f the field winding resistance, R_{d1} , R_{q1} , R_{q2} the rotor winding resistances, ω the rotor speed, ω_N the nominal angular frequency (in rad/s), v_f the field voltage, and K_f a coefficient to pass from per unit values of the excitation system to per unit values of the machine.

The stator equations are transformed as follows. Eqs. (11.20) are used to express v_d , v_q , i_d and i_q as functions of v_x , v_y , i_x , i_y , while Eqs. (11.5, 11.8) are used to involve ψ_{ad} and ψ_{aq} . This yields for the d axis:

$$\begin{aligned} v_d &= -R_a(-\sin \delta i_x + \cos \delta i_y) - \omega(L_\ell i_q + \psi_{aq}) \\ &= -R_a(-\sin \delta i_x + \cos \delta i_y) - \omega L_\ell(\cos \delta i_x + \sin \delta i_y) - \omega \psi_{aq} \end{aligned}$$

and finally:

$$0 = v_x \sin \delta - v_y \cos \delta + (R_a \sin \delta - \omega L_\ell \cos \delta) i_x - (R_a \cos \delta + \omega L_\ell \sin \delta) i_y - \omega \psi_{aq} \quad (11.29)$$

The corresponding expressions in the q axis are:

$$\begin{aligned} v_q &= -R_a(\cos \delta i_x + \sin \delta i_y) + \omega(L_\ell i_d + \psi_{ad}) \\ &= -R_a(\cos \delta i_x + \sin \delta i_y) + \omega L_\ell(-\sin \delta i_x + \cos \delta i_y) + \omega \psi_{ad} \end{aligned} \quad (11.30)$$

and:

$$0 = -\cos \delta v_x - \sin \delta v_y - (R_a \cos \delta + \omega L_\ell \sin \delta) i_x - (R_a \sin \delta - \omega L_\ell \cos \delta) i_y + \omega \psi_{ad} \quad (11.31)$$

The rotor equations are transformed by substituting the expressions (11.11, 11.12, 11.13, 11.14) for the rotor currents, thereby obtaining:

$$\frac{d\psi_f}{dt} = \omega_N(K_f v_f - R_f \frac{\psi_f - \psi_{ad}}{L_{\ell f}}) \quad (11.32)$$

$$\frac{d\psi_{d1}}{dt} = -\omega_N R_{d1} \frac{\psi_{d1} - S_{d1} \psi_{ad}}{L_{\ell d1}} \quad (11.33)$$

$$\frac{d\psi_{q1}}{dt} = -\omega_N R_{q1} \frac{\psi_{q1} - S_{q1} \psi_{aq}}{L_{\ell q1}} \quad (11.34)$$

$$\frac{d\psi_{q2}}{dt} = -\omega_N R_{q2} \frac{\psi_{q2} - S_{q2} \psi_{aq}}{L_{\ell q2}} \quad (11.35)$$

11.2.6. Rotor motion

The equations of the rotor motion are:

$$\frac{1}{\omega_N} \frac{d\delta}{dt} = \omega - \omega_{ref} \quad (11.36)$$

$$2H \frac{d\omega}{dt} = K_m T_m - T_e - D(\omega - \omega_{ref}) \quad (11.37)$$

where H is the inertia constant (in s), ω_{ref} the angular speed of the reference axes, T_e is the electromagnetic torque, T_m is the mechanical torque produced by the turbine and K_m a coefficient to pass from per unit values of the turbine to per unit values of the machine.

The expression of T_e is obtained as follows:

$$\begin{aligned} T_e &= \psi_d i_q - \psi_q i_d = (L_\ell i_d + \psi_{ad}) i_q - (L_\ell i_q + \psi_{aq}) i_d \\ &= \psi_{ad} i_q - \psi_{aq} i_d = \psi_{ad}(\cos \delta i_x + \sin \delta i_y) - \psi_{aq}(-\sin \delta i_x + \cos \delta i_y) \end{aligned} \quad (11.38)$$

11.2.7. Summing up: unknowns vs. equations

The synchronous machine model involves 10 state variables, namely:

- 6 differential states: $\psi_f, \psi_{d1}, \psi_{q1}, \psi_{q2}, \delta, \omega$
- 4 algebraic states: $i_x, i_y, \psi_{ad}, \psi_{aq}$.

They are balanced by:

- 4 algebraic equations: (11.21, 11.22, 11.29, 11.31)
- 6 differential equations: (11.32, 11.33, 11.34, 11.35, 11.36, 11.37)

Well done!

11.2.8. Per unit systems

The above model relies on the EMFL per unit system, which allows writing the inductance matrices in the form shown in Eqs. (11.1, 11.2). On the other hand, it is more practical to have the excitation system and the turbine modelled in their own per unit systems. A change of base is thus necessary to interface those models.

Turbine

The parameters of the turbine are expressed in per unit on the turbine nominal power P_{nom} base (in MW), while those of the synchronous machine refer to the machine nominal apparent power S_{nom} (in MVA). Both P_{nom} and S_{nom} are specified in the machine model (see Section 11.2.9).

Excitation system

The case of the excitation system is a little more involved and requires specifying an additional parameter.

Let the base current of the field winding be I_{fB}^{mac} in the machine model and I_{fB}^{exc} in the excitation system model. Define the ratio:

$$\mathcal{R} = \frac{I_{fB}^{mac}}{I_{fB}^{exc}}. \quad (11.39)$$

A given field current i_f (in A) has the following value in per unit on the machine base :

$$i_{fpu}^{mac} = \frac{i_f}{I_{fB}^{mac}} \quad (11.40)$$

and the following value in per unit on the excitation system base :

$$i_{fpu}^{exc} = \frac{i_f}{I_{fB}^{exc}} \quad (11.41)$$

By combining Eqs. (11.39, 11.40, 11.41):

$$\mathcal{R} = \frac{i_{fpu}^{exc}}{i_{fpu}^{mac}}. \quad (11.42)$$

The following are possible choices of per unit systems:

1. Open-circuit unsaturated machine

In this most often used per unit system, I_{fB}^{exc} is the field current which produces the nominal stator voltage ($V = 1$ pu) when the machine rotates at its nominal speed ($\omega = 1$ pu) with its stator open ($i_d = i_q = 0$), saturation being neglected.

In these conditions, the machine Park equations give:

$$v_d = \psi_q = \psi_{aq} = 0 \quad v_q = \psi_d = \psi_{ad} = M_d^u i_{fpu}^{mac}.$$

Since:

$$V = \sqrt{v_d^2 + v_q^2} = 1$$

one has:

$$M_d^u i_{fpu}^{mac} = 1$$

while, on the excitation system base:

$$i_{fpu}^{exc} = 1$$

Introducing the last two relations in (11.42) yields :

$$\mathcal{R} = M_d^u = X_d - X_\ell \quad (11.43)$$

where X_d is the unsaturated direct-axis synchronous reactance and X_ℓ the stator leakage reactance¹.

2. Open-circuit saturated machine

In this per unit system, I_{fB}^{exc} is the field current which produces the nominal stator voltage ($V = 1$ pu) when the machine rotates at its nominal speed ($\omega = 1$ pu) with its stator open ($i_d = i_q = 0$), saturation being taken into account.

In these conditions, the machine Park equations give:

$$v_d = \psi_q = \psi_{aq} = 0 \quad v_q = \psi_d = \psi_{ad} = M_d i_{fpu}^{mac}$$

Since:

$$V = \sqrt{v_d^2 + v_q^2} = 1$$

one has:

$$M_d i_{fpu}^{mac} = 1$$

M_d can be obtained from the saturation equation (11.15):

$$M_d = \frac{M_d^u}{1 + m \psi_{ad}^n} = \frac{M_d^u}{1 + m (M_d i_{fpu}^{mac})^n} = \frac{M_d^u}{1 + m}$$

while, on the excitation system base:

$$i_{fpu}^{exc} = 1$$

Introducing the last three relations in (11.42) yields :

$$\mathcal{R} = \frac{M_d^u}{1 + m} = \frac{X_d - X_\ell}{1 + m} \quad (11.44)$$

¹in per unit $X_d^u = M_d^u$ and $X_\ell = L_\ell$

3. Saturated machine at nominal operating conditions

In this per unit system, I_{fB}^{exc} is the field current when the machine, rotating at its nominal speed ($\omega = 1$ pu), produces its nominal active and reactive powers under its nominal stator voltage ($V = 1$ pu, $P = \cos \phi_N$ and $Q = \sin \phi_N$), saturation being taken into account.

11.2.9. Data format

Synchronous machines are specified by SYNC_MACH records. There are two variants, depending on the value of the MOD_TYPE field:

- if MOD_TYPE is set to the (string value) XT, the data relate to reactances and time constants that are used internally to determine the resistances and inductances involved in Eqs. (11.1, 11.2, 11.23-11.28)
- if MOD_TYPE is set to RL, the data relate to those resistances and inductances directly.

The two sets of data are described next.

First variant: reactances and time constants are specified

The data format is as follows.

```
SYNC_MACH NAME BUS FP FQ P Q SNOM PNOM H D IBRATIO
MOD_TYPE XL XD XPD XSD XQ XPQ XSQ M N RA TPD0 TSD0 TPQ0 TSQ0
EXC E1 E2 ... EN TOR T1 T2 ... TN ;
```

where:

- NAME is the name of the machine. This is a string of at most 20 characters
- BUS is the name of the bus which the machine is connected to. This is a string of at most 8 characters
- FP is the fraction f_P of the initial bus active power injection, as defined in Eq. (10.8) (see Section 10.1)
- FQ is the fraction f_Q of the initial bus reactive power injection, as defined in Eq. (10.8)
- P is the initial active power injected by the machine, in MW, denoted by P_j^c in Eq. (10.7) (see Section 10.1)
- Q is the initial reactive power injected by the machine, in Mvar, denoted by Q_j^c in Eq. (10.7)
- SNOM is the nominal apparent power of the machine, in MVA. It is used as base power in the machine model
- PNOM is the nominal active power of the machine, in MW. It is used as base power for the turbine model
- H is the inertia time constant H in Eq. (11.37), in s
- D is the damping time constant D in Eq. (11.37), in pu. D is usually set to zero when the damper windings are modelled
- IBRATIO is the dimensionless current ratio $\mathcal{R}$ as defined in Eq. (11.42)
- MOD_TYPE is set to the string 'XT' in this variant (the quotes are not needed)

- XL is the armature leakage reactance, in pu
- XD is the unsaturated direct-axis synchronous reactance, in pu
- XPD is the direct-axis transient reactance, in pu
- XSD is the direct-axis subtransient reactance, in pu
- XQ is the unsaturated quadrature-axis synchronous reactance, in pu
- XPQ is the quadrature-axis transient reactance, in pu
- XSQ is the quadrature-axis subtransient reactance, in pu
- M is the dimensionless saturation factor m as defined in Eqs. (11.15, 11.16). M is set to zero if saturation is neglected
- N is the dimensionless saturation exponent n as defined in Eqs. (11.15, 11.16). The value is ignored if M is set to zero
- RA is the armature resistance, in pu
- TPD0 is the direct-axis open-circuit transient time constant, in s. It is usually denoted T'_{d0}
- TSD0 is the direct-axis open-circuit subtransient time constant, in s. It is usually denoted T''_{d0}
- TPQ0 is the quadrature-axis open-circuit transient time constant, in s. It is usually denoted T'_{q0}
- TSQ0 is the quadrature-axis open-circuit subtransient time constant, in s. It is usually denoted T''_{q0}
- EXC is a keyword (not a data !) indicating the beginning of the excitation system data section
- E1 E2 ... EN are the successive excitation system data. Please refer to Section 25.4.2 for more details
- TOR is a keyword (not a data !) indicating the end of the excitation system data and the beginning of the torque control data section
- T1 T2 ... TN are the successive torque control data. Please refer to Section 25.4.2 for more details

Second variant: resistances and inductances are specified

The data format is as follows.

```

SYNC_MACH NAME BUS FP FQ P Q SNOM PNOM H D IBRATIO
MOD_TYPE LL MDU LLF LLD1 MQU LLQ1 LLQ2 M N RA RF RD1 RQ1 RQ2
EXC E1 E2 ... EN TOR T1 T2 ... TN ;

```

where:

- NAME is the name of the machine. This is a string of at most 20 characters
- BUS is the name of the bus which the machine is connected to. This is a string of at most 8 characters

- FP is the fraction f_P of the initial bus active power injection, as defined in Eq. (10.8) (see Section 10.1)
- FQ is the fraction f_Q of the initial bus reactive power injection, as defined in Eq. (10.8)
- P is the initial active power injected by the machine, in MW, denoted by P_j^c in Eq. (10.7) (see Section 10.1)
- Q is the initial reactive power injected by the machine, in Mvar, denoted by Q_j^c in Eq. (10.7)
- SNOM is the nominal apparent power of the machine, in MVA. It is used as base power in the machine model
- PNOM is the nominal active power of the machine, in MW. It is used as base power for the turbine model
- H is the inertia time constant H in Eq. (11.37), in s
- D is the damping time constant D in Eq. (11.37), in pu. D is usually set to zero when the damper windings are modelled
- IBRATIO is the dimensionless current ratio $\mathcal{R}$ as defined in Eq. (11.42)
- MOD_TYPE is set to the string 'RL' in this variant (the quotes are not needed)
- LL is the leakage inductance L_ℓ of the armature, in pu
- MDU is the direct-axis unsaturated mutual inductance M_d , in pu
- LLF is the leakage inductance $L_{\ell f}$ of the field winding, in pu
- LLD1 is the leakage inductance $L_{\ell d1}$ of the $d1$ damper winding, in pu
- MQU is the quadrature-axis unsaturated mutual inductance M_q , in pu
- LLQ1 is the leakage inductance $L_{\ell q1}$ of the $q1$ damper winding, in pu
- LLQ2 is the leakage inductance $L_{\ell q2}$ of the $q2$ damper winding, in pu
- M is the dimensionless saturation factor m as defined in Eqs. (11.15, 11.16). M is set to zero if saturation is neglected
- N is the dimensionless saturation exponent n as defined in Eqs. (11.15, 11.16). The value is ignored if M is set to zero
- RA is the armature resistance, in pu
- RF is the resistance R_f of the field winding, in pu
- RD1 is the resistance R_{d1} of the $d1$ damper winding, in pu
- RQ1 is the resistance R_{q1} of the $q1$ damper winding, in pu
- RQ2 is the resistance R_{q2} of the $q2$ damper winding, in pu
- EXC is a keyword (not a data !) indicating the beginning of the excitation system data section

- E1 E2 ... EN are the successive excitation system data. Please refer to Section 25.4.2 for more details
- TOR is a keyword (not a data !) indicating the end of the excitation system data and the beginning of the torque control data section
- T1 T2 ... TN are the successive torque control data. Please refer to Section 25.4.2 for more details.

As explained in the preamble of this section, the full model can be simplified by ignoring some of the rotor windings². This is specified by replacing the values of some parameters with an asterisk '*' (the quotes are not needed). The parameters of concern are detailed in Table 11.2.

Table 11.2: Simplified model specification (the quotes are not needed)

simplification	MOD_TYPE = 'XT'	MOD_TYPE = 'RL'
no d1 winding ($S_{d1} = 0$)	replace values of XSD and TSD0 by '*'	replace values of LLD1 and RD1 by '*'
no q1 winding ($S_{q1} = 0$)	replace values of XPQ and TPQ0 by '*'	replace values of LLQ1 and RQ1 by '*'
no q2 winding ($S_{q2} = 0$)	replace values of XSQ and TSQ0 by '*'	replace values of LLQ2 and RQ2 by '*'

11.2.10. Initialization

👉 Here is an example of output produced by RAMSES at initialization.

NUMBER OF ISLANDS : 1

5 buses in island # 1 (includes a Thevenin equivalent)

NUMBER OF SHUNTS : 0 (B_type: 0)

NUMBER OF SYNCHRONOUS MACHINES : 1

machine	at bus excit model	V vf(pu)	P torque model	Q	delta Tm(pu)	sat	island	br
G5	5 exc_GENERIC3	1.0000 2.3680	450.00186 THERMAL_GENERIC1	68.49769	70.99 0.97826	1.0000	1	1

In this example, the model involves a single synchronous machine, named G5, connected to bus 5. The network is connected (single island, numbered 1). The machine is in service (br = 1) and injects around 450 MW and 68 Mvar into the grid, under a bus voltage of 1 pu.

delta is the initial value of the rotor angle δ , in degrees; see Fig. 11.2.

sat is the saturation factor defined by:

$$sat = 1 + m \left(\sqrt{\psi_{ad}^2 + \psi_{aq}^2} \right)^n \geq 1 \quad (11.45)$$

This is the ratio between the field current in the saturated machine and the corresponding field current when saturation is neglected, for the same operating conditions. Thus, it characterizes the extra excitation current needed in the presence of saturated material. In the above example, saturation was neglected (a zero value was specified for M) and, hence, $sat = 1$.

²let us recall that not all combinations of windings are allowed

The machine has an excitation controller whose model is named `exc_GENERIC3`. At the initial operating point the field voltage is $v_f = 2.3680$ pu on the exciter base, which is indirectly defined by the `IBRATIO` parameter of the machine.

The machine is driven by a turbine whose model is named `THERMAL_GENERIC1`. The initial mechanical torque is $T_m = 0.97826$ pu, on the turbine base, which is defined by the `PNOM` parameter of the machine.

11.3. Thévenin equivalents

11.3.1. Modelling

Thévenin equivalents are used to represent “external” systems in a simplified manner.

The equivalent includes a voltage source $\bar{E}_{th}$ in series with a reactance X_{th} , as shown in Fig. 11.3. The voltage source has a constant frequency, equal to the nominal value given by the `FNOM` record. It has constant magnitude and phase angle. The latter are determined at initialisation from the bus voltage and the injected powers.

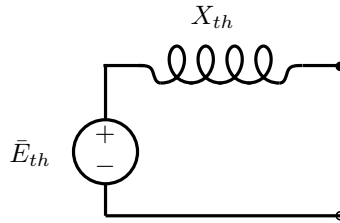


Figure 11.3: Thévenin equivalent

The Thévenin reactance is specified through the *short-circuit power*³:

$$S_{sc \text{ MVA}} = \frac{U_{nom}^2}{X_{th \ \Omega}} \quad (11.46)$$

where U_{nom} is the nominal voltage (in kV) of the bus to which the Thévenin equivalent is connected, and $X_{th \ \Omega}$ is the Thévenin reactance in Ohm.

Note that $S_{sc \text{ MVA}}$ is the short-circuit of the Thévenin equivalent *alone*; it is not the short-circuit at the bus where the equivalent is connected, which depends on the other components present in the system.

Taking U_{nom} as base voltage and S_B as base power, the Thévenin reactance in per unit is given by:

$$X_{th \text{ pu}} = \frac{X_{th \ \Omega}}{Z_B} = \frac{X_{th \ \Omega} S_B}{U_{nom}^2} = \frac{S_B}{S_{sc \text{ MVA}}} = \frac{1}{S_{sc \text{ pu}}} \quad (11.47)$$

where Z_B is the base impedance and $S_{sc \text{ pu}}$ the short-circuit power in per unit.

11.3.2. Data format

Thévenin equivalents are specified as injectors with model name `THEVEQ`. The format is as follows:

`INJEC THEVEQ NAME BUS FP FQ P Q SSC ;`

where:

- `NAME` is the name of the Thévenin equivalent. This is a string of at most 20 characters

³also known as fault level

- BUS is the name of the bus which the equivalent is connected to. This is a string of at most 8 characters
- FP is the fraction f_P of the initial bus active power injection, as defined in Eq. (10.8) (see Section 10.1)
- FQ is the fraction f_Q of the initial bus reactive power injection, as defined in Eq. (10.8)
- P is the initial active power injected by the machine, in MW, denoted by P_j^c in Eq. (10.7) (see Section 10.1)
- Q is the initial reactive power injected by the machine, in Mvar, denoted by Q_j^c in Eq. (10.7)
- SSC is the short-circuit power, in MVA, as defined in Eq. (11.46).

11.4. Impedance loads

11.4.1. Modelling

An impedance load⁴ is merely modelled by a shunt admittance. The active power P (resp. reactive power Q) consumed varies with the square of the bus voltage magnitude V according to:

$$P = G V^2 \quad Q = -B V^2 \quad (11.48)$$

where G (resp. B) is the shunt conductance (resp. susceptance). Both are determined at initialization and remain constant (unless they are changed by a user-defined action). Let us recall that B is negative (resp. positive) for an inductive (resp. capacitive) load.

11.4.2. Data format

Impedance loads are specified by IMPLOAD records. The format is as follows:

IMPLOAD NAME BUS FP FQ P Q ;

where:

- NAME is the name of the load. This is a string of at most 20 characters. It must not start with 'M_': this prefix is reserved to denote a mismatch load (see Section 10.3)
- BUS is the name of the bus which the load is connected to. This is a string of at most 8 characters
- FP is the fraction f_P of the initial bus active power injection, as defined in Eq. (10.8) (see Section 10.1)
- FQ is the fraction f_Q of the initial bus reactive power injection, as defined in Eq. (10.8)
- P is the initial active power injected by the machine, in MW, denoted by P_j^c in Eq. (10.7) (see Section 10.1)
- Q is the initial reactive power injected by the machine, in Mvar, denoted by Q_j^c in Eq. (10.7).

⁴to be more precise: a constant impedance load. The formulation in terms of admittance is preferred here

12

Disturbances

Disturbances need to have a continuity.

12.1. Continue Solver

time(s) CONTINUE SOLVER disc_meth max_h(s) min_h(s) latency(pu) upd_over

Discretization method (disc_meth):

- TR: Trapezoidal
- BE: Backward Euler
- BD: BDF2

Jacobian update override (upd_over):

- ALL: force a full Jacobian refactorisation
- NET: force a refactorisation of the network Jacobian
- ABL: force an update of the injector Jacobian at every bus
- IBL: force nothing in addition, but keep the bus Jacobian updates already requested by the other disturbances applied at this time instant
- NOT: override, discarding even those; the bus Jacobian flags are restored to their values on entry

It is mainly used to modify the settings of the solver and has to exist at the first line. For example:

```
0.000 CONTINUE SOLVER BD 0.0200 0.001 0. ALL
```

12.2. Continue Display

time(s) CONTINUE DISPLAY new_plot_step(s)

Changes the sampling time step used for the output of observed variables from that point of the simulation onwards. For example, reducing the output sampling to one point per second after $t = 20$ s:

```
20.000 CONTINUE DISPLAY 1.0
```

12.3. Set stopping criteria for voltage

Define in the data files the following record:

```
DCTL SIM_MINMAXVOLT CTRL_Name VMAX(pu) VMIN(pu) DEADTIME(s) Stop_Simulation(T/F) ;
```

12.4. Set stopping criteria for machine speed

Define in the data files the following record:

```
DCTL SIM_MINMAXSPEED CTRL_Name MAX_SPEED(pu) MIN_SPEED(pu) DEADTIME(s) Stop_Simulation(T/F) ;
```

12.5. Stop

time(s) STOP

It is used to signal the end of the simulation and has to exist at the last line.

For example:

```
100.000 STOP
```

12.6. Trip line

time(s) BREAKER BRANCH name_of_line orig_break(0/1) extrem_break(0/1)

Used to open/close the breakers of a line.

For example, opening both ends of a line:

10.000 BREAKER BRANCH 1044-4032 0 0

12.7. Trip synchronous machine / injector

time(s) BREAKER SYNC_MACH/INJ name_of_synch_mach/name_of_injector breaker(0/1)

Used to open/close the breaker (trip) of a synchronous machine or injector.

For example:

10.000 BREAKER INJ L_11 0

12.8. Trip two-port

time(s) BREAKER TWOP name_of_twoport 0

Used to disconnect a two-port component (e.g. an HVDC link) at both ends. Only opening is supported, so the breaker status must be 0.

For example:

10.000 BREAKER TWOP hvdc1 0

12.9. Three phase short-circuit (with use of resistance to ground)

This demands two commands:

time(s) FAULT BUS name_of_bus rfault [xfault]

time(s) CLEAR BUS name_of_bus

The first line is used to declare the starting of the short-circuit. The fault has a resistance of $rfault + j \cdot xfault$ to the ground where the values of rfault and xfault are in Ohm. If xfault is not defined, a fully resistive fault is assumed.

Example of a 100ms short-circuit directly to ground:

10.000 FAULT BUS 1044 0. 0.

10.100 CLEAR BUS 1044

12.10. Three phase short-circuit (with use of voltage reached after fault)

This demands two commands:

time(s) VFAULT BUS name_of_bus Voltage_reached_after_fault

time(s) CLEAR BUS name_of_bus

The first line is used to declare the starting of the short-circuit. Internally, RAMSES creates a temporary shunt admittance B_{fault} at the faulted bus and iteratively adjusts it using a secant method until the bus voltage matches the target value (declared in pu) to within $\pm 0.5\%$. The injected currents are $i_x = B_{fault} v_x$ and $i_y = -B_{fault} v_y$. Up to 10 correction steps are attempted; if convergence is not reached, the simulation halts with an error message.

Example of a 100ms short-circuit where the voltage at the faulted bus reached 0.5 pu after the fault:

10.000 VFAULT BUS 1044 0.5

10.100 CLEAR BUS 1044

Supported: Version 3.13 and above.

12.11. Change parameters

Used to change the parameters of a model during the simulation.

12.11.1. BRANCH

time(s) CHGPRM BRANCH name_of_line MAGN/PHAN $\pm$ increment

12.11.2. SHUNT

time(s) CHGPRM SHUNT name_of_shunt QNOM $\pm$ increment

The increment should be expressed in MVar and it is per unitized internally by RAMSES.

12.11.3. EXC

time(s) CHGPRM EXC name_of_equipment name_of_parameter $\pm$ increment (MVar/%) duration(s)

The units is not obligatory. If nothing is given then the parameter is modified in absolute value. If MVar is given, then the increment is per unitized using S_{nom} of the machine before applying the change. If % is given, then the parameter is changed as a percentage of the original value. If $duration = 0$ then a step change is applied, otherwise the change is applied as a ramp over the given duration (in seconds).

For example:

10.000 CHGPRM EXC g1 V0 +10 % 10

This means the parameter V0 of the exciter of synchronous machine g1 is ramped by +10% between 10 and 20 seconds.

12.11.4. TOR

time(s) CHGPRM TOR name_of_equipment name_of_parameter $\pm$ increment (MW/%) duration(s)

The units is not obligatory. If nothing is given then the parameter is modified in absolute value. If MW is given, then the increment is per unitized using P_{nom} of the machine before applying the change. If % is given, then the parameter is changed as a percentage of the original value. If $duration = 0$ then a step change is applied, otherwise the change is applied as a ramp over the given duration (in seconds).

For example:

10.000 CHGPRM TOR g1 P0 +1 MW 10

This means the parameter P0 of the torque controller of synchronous machine g1 is ramped by +1 MW between 10 and 20 seconds.

12.11.5. INJ/TWOP/DCTL

time(s) CHGPRM INJ/TWOP/DCTL name_of_equipment name_of_parameter $\pm$ increment (MW/MVar/%/SETP) duration(s)

The units is not obligatory. If nothing is given then the parameter is modified in absolute value. If MW or MVar is given, then the increment is per unitized using system's S_{base} before applying the change. If % is given, then the parameter is changed as a percentage of the original value. If SETP is given then the value increment is actually the new setpoint. If $duration = 0$ then a step change is applied, otherwise the change is applied as a ramp over the given duration (in seconds).

For example:

10.000 CHGPRM INJ L_11 P0 +50 % 60

10.000 CHGPRM INJ L_11 Q0 +30 % 60

This means the parameter P0 (resp. Q0) of the injector L_11 is ramped by +50% (resp. 30%) between 10 and 70 seconds. In this case, if L_11 is a load model, it can be used to simulate a load increase.

12.12. Export Jacobian matrix

time(s) JAC 'name_of_filename'

Also, make sure to add these to your settings:

\$OMEGA_REF SYN ;

\$SCHEME IN;

12.13. Small-signal stability analysis

time(s) EIG 'name_of_filename'

Reduces the linearised differential-algebraic model to a state matrix, solves the eigenproblem, and writes the modes, participation factors and mode shapes. The analysis runs inside RAMSES; no external tool is required.

Required settings:

\$OMEGA_REF SYN ;

\$SCHEME DE ;

Three files are written, named from the given basename:

- <name>_modes.dat: one line per mode, giving the index, the real and imaginary parts of the eigenvalue, the damping ratio, the frequency in Hz, a dominance flag, and a simplicity flag;
- <name>_pf.dat: participation factors, with the family, device and variable name of each state;
- <name>_ms.dat: mode shapes, as magnitude and angle of each machine's rotor speed, normalised so the largest magnitude in a mode is unity and angles are relative to it.

The damping ratio is $\zeta = -\sigma/|\lambda|$ for $\lambda = \sigma \pm j\omega$, and the frequency is $|\omega|/2\pi$. A mode with $\zeta < 0$ grows rather than decays.

The simplicity flag deserves attention. Identical device models with identical parameters produce identical eigenvalues, and the eigenvectors of a repeated eigenvalue are not unique. Participation factors and mode shapes of such a mode are therefore basis-dependent and carry no physical meaning. The flag is unity when the eigenvalue is well separated from every other and zero otherwise; do not interpret the rows of a mode flagged zero.

Unlike JAC, which logs a warning and skips, EIG refuses when it cannot produce a result it can justify: it writes nothing, states the reason in the log, and terminates with exit code 78. This occurs under the COI reference frame, under the integrated scheme, above the \$EIG_MAX_STATES limit, and when the algebraic block is singular.

12.14. Export load flow

Takes a snapshot of the system and exports the load flow at a specific time.

time(s) LFRESV 'name_of_filename'

13

Solver Settings

13.1. Sampling time for observed variables

\$PLOT_STEP time(s) ;

13.2. Display Profiling Results

\$DISP_PROF T/F ;

13.3. Run-time observables refresh interval

\$GP_REFRESH_RATE time_interval(s) ;

How often the run-time observable file is flushed, in seconds. A reader watching that file, such as the run-time curve window in STEPSS GUI, polls at this rate, so it is also how often those curves advance. Default: 1 s.

13.4. Observable buffer size

\$OBS_BUFFER_SIZE size(GB) ;

Internal memory reserved for storing observables during simulation. Default: 8 GB. Set this to less than half of the available RAM for large simulations.

13.5. Time constant of load restoration

\$T_LOAD_REST time(s) ;

13.6. Omega Reference

\$OMEGA_REF SYN/COI ;

Synchronous reference frame or center of inertia reference frame.

13.7. Maximum Fault Value

\$MAX_FAULT value ;

13.8. Base Power

Sets the global base power of the system.

\$S_BASE BASE(MVA) ;

13.9. Nominal Frequency

FNOM Frequency(Hz) ;

13.10. Newton Tolerance

\$NEWTON_TOLER NETWORK_TOLERANCE INJ_RELATIVE_TOLERANCE INJ_ABSOLUTE_TOLERANCE ;

Set's the solver Newton iteration tolerance for stopping. Default values are: 1e-03, 5e-04, 5e-04.

13.11. Finite Difference Values

\$FIN_DIFFER proportional_value absolute_value ;

Values used to calculate numerically Jacobian matrices of injectors.

13.12. Full Jacobian Update

\$FULL_UPDATE T/F ;

Disable partial Jacobian updates.

13.13. Skip Converged Blocks

\$SKIP_CONV T/F ;

Activate/Deactivate stopping to solve converged injectors.

13.14. Latency Tolerance

\$LATENCY OBS_TIME_WINDOW(s) EARLY_STOP(T/F) ;

13.15. Solution Scheme

\$SCHEME DE/IN ;

Integrated or Decomposed solution scheme.

13.16. Number of Threads for parallel computing

\$NB_THREADS Number ;

13.17. Way of injector distribution over parallel threads

\$OMP STA/DYN/GUI chunk ;

STA is for static assignment (better for NUMA architecture computers), DYN is for dynamic assignment (better for UMA architecture computers) and GUI is for guided. Chunk is the number of consecutive injectors assigned to each thread.

13.18. Update network elements with frequency

\$NET_FREQ_UPD T/F ;

Check Network update with frequency.

13.19. Call Gnuplot

\$CALL_GP T/F ;

Whether RAMSES drives Gnuplot itself through a pipe. With F it writes the observables and the .plt script beside them and calls nothing, which is what the STEPSS interfaces read. With T it pipes to Gnuplot, which must then be on the PATH; STEPSS does not ship it. Default: F.

13.20. Gnuplot output mode

\$GP_MODE term/png ;

Which terminal the exported .plt script names, for opening that script in Gnuplot: term an interactive terminal, png a PNG file. It has no effect on the curves STEPSS draws. Default: term.

13.21. Minimum branch impedance

\$ZMIN value(pu) ;

Lower bound imposed on the magnitude of a branch impedance. Branches with a smaller impedance are clamped to this value, which prevents the network admittance matrix from becoming singular. Default: 10^{-5} pu.

13.22. Size limit of the small-signal analysis

\$EIG_MAX_STATES value ;

Largest number of states accepted by the EIG small-signal analysis. The reduced state matrix is solved densely, so the peak workspace is of the order of $9N_x^2$ doubles, about 1.8 GB at the default. Above this limit the analysis refuses and terminates with exit code 78 rather than attempting the allocation. Default: 5000.

13.23. Latency on subnetworks

\$LAT_SUBNETS T/F ;

Applies the latency treatment at the level of the subnetworks, in addition to the injectors. Default: F.

13.24. Load a compiled model library

\$BASE_MDL filename ;

Loads a compiled user-model library (MDL) at start-up, so that user-defined models can be used without relinking the executable.

☛ This record is only acted upon by the Windows build made with the Intel compiler. On the other platforms it is accepted and silently ignored; there, user-defined models must be linked in with URAMSES instead.

13.25. License key

\$LICENSE email license_key ;

Unlocks the full version, removing the limits of the free version stated at the beginning of this document (1000 buses and 2 cores). The key is 64 characters long and is tied to the electronic mail address given in the same record. Both are provided by the authors; contact them as indicated in Section 1. Without this record the free-version limits apply.

14

Discrete controller models

Discrete controllers (DCTL) in RAMSES implement **event-driven logic** rather than continuous differential equations. They fire at specific simulation events (voltage crossing a threshold, a timer expiring, a tap position changing) and execute discrete actions such as tripping a generator, shedding load, or adjusting a transformer tap. They run inside RAMSES' event-driven loop and can call `disp_disc` to log switching actions.

The RAMSES data keyword is DCTL:

```
DCTL model_name controller_name field1 field2 ... ;
```

Usage in Dynamic Data Files

Discrete controllers are defined with the DCTL keyword as standalone records in the dynamic data file. The syntax is:

```
DCTL model_name ctl_name param1 param2 ... ;
```

Where `model_name` is the controller type, `ctl_name` is a unique instance name, and the remaining parameters are model-specific.

Recognised discrete-controller model names (case-sensitive):

- **Uppercase short names** (no prefix): PST, LTC, LTC2, OLTC2, LTCINV, MAIS, UVLS, RT, UVPROT, FRT, VOLT_VAR, SIM_MINMAXVOLT, SIM_MINMAXSPEED.
- **Prefixed name**: RAMSES adds the `dctl_` prefix automatically, so both `line_prot` and `dctl_line_prot` resolve to the line-protection model.

All of the above are built into every RAMSES distribution (standalone executable and shared library used by `stepss`). **MAIS** is an automatic shunt-reactor switching scheme (*Manoeuvre Automatique d'Inductances Shunt*) that switches susceptance at a monitored bus on under-voltage, voltage-drop and over-voltage thresholds; it does not have a dedicated documentation section below. HVDC_LIM, `injprot` and `LOSPROT`, documented below, have been callable since 3.79; before that they were compiled under no name.

14.1. Tap Changers

TRFO and DCTL LTC

The TRFO record combines the transformer model with load tap changer data (tap range, positions, controlled bus, voltage setpoint), but this data is used **only by the power flow** for ratio adjustment. To control a transformer's ratio during dynamic simulation with RAMSES, a DCTL LTC controller (below) must be associated with the transformer, whether it was defined with TRANSFO or TRFO.

14.1.1. LTC (dctl1_ltc): Load Tap Changer (Standard)

Description

Controls the ratio of a transformer to regulate voltage at a **monitored bus**. It monitors the voltage, compares it against a deadband around the setpoint (initialised to the voltage at $t = 0$), and initiates a tap change after a delay if the voltage remains outside the deadband.

The first tap change uses a longer delay (`delay_first`); subsequent changes use a shorter delay (`delay_next`), implementing the standard IEC/IEEE slow-start behaviour.

Logic Description

Let V be the monitored bus voltage and V_{set} the setpoint:

$$\text{Error} = V_{set} - V$$

Deadband check: No action if $|V - V_{set}| \leq \delta V$ (half-deadband).

Direction logic: If `direction` > 0, increasing the ratio increases the voltage.

Timer logic:

$$\text{if } |V - V_{set}| > \delta V \text{ and } (t - t_{last}) \geq \tau_{delay} : \quad \text{tap} \leftarrow \text{tap} \pm \Delta r, \quad t_{last} \leftarrow t$$

where $\tau_{delay} = \tau_{first}$ for the first operation, τ_{next} thereafter, and the tap is clamped to $[r_{min}, r_{max}]$.

State variable `w(13)`: - 0: voltage within deadband - +1: $V < V_{set} - \delta V$ (tap must increase) - -1: $V > V_{set} + \delta V$ (tap must decrease)

Parameters

Field	Working variable	Description
<code>branch_name</code>	<code>w(1)</code>	Transformer branch name (must be type <code>trfo</code>)
<code>bus_name</code>	<code>w(2)</code>	Monitored (controlled) bus name
<code>direction</code>	<code>w(3)</code>	Sign convention: >0 means increasing ratio raises voltage
<code>ratio_min</code>	<code>w(4)</code>	Minimum transformer ratio (% of nominal , e.g. 85. for 0.85 pu)
<code>ratio_max</code>	<code>w(5)</code>	Maximum transformer ratio (% of nominal , e.g. 115. for 1.15 pu)
<code>nb_positions</code>	<code>w(6)</code>	Number of tap positions (step = $(\text{max} - \text{min}) / (\text{nb} - 1)$)
<code>half_deadband</code>	<code>w(7)</code>	Voltage half-deadband δV (pu)
<code>delay_first</code>	<code>w(8)</code>	Delay before first tap change (s)
<code>delay_next</code>	<code>w(9)</code>	Delay between subsequent tap changes (s)

Caution

The ratio limits are given in **percent**, not per unit: the model divides `ratio_min` and `ratio_max` by 100 when reading the data. A record with 0.85 1.15 would silently produce

limits of 0.0085/0.0115 pu.

Usage Example

```
DCTL LTC LTC_TR1 TR1-HV-MV BUS_MV 1 85. 115. 33 0.01 30.0 10.0 ;
```

14.1.2. LTC2 (dctl_ltc2): Load Tap Changer (Variant 2)

Description

Identical logic to dctl_ltc but the **voltage setpoint is an explicit input parameter** rather than being taken from the initial operating point. This allows prescribing a setpoint different from the initial bus voltage, which is useful when the initial load-flow voltage differs from the desired operating voltage.

Logic Description

Same deadband, direction, and timer logic as dctl_ltc. The voltage setpoint V_{set} is read directly from the data field Vset instead of being captured at $t = 0$.

$$\text{if } |V - V_{set}| > \delta V \text{ and } (t - t_{last}) \geq \tau_{delay} : \text{ tap change}$$

Parameters

Field	Working variable	Description
branch_name	w(1)	Transformer branch (type trfo)
bus_name	w(2)	Monitored bus
direction	w(3)	>0: ratio increase raises voltage
ratio_min	w(4)	Minimum ratio (% of nominal)
ratio_max	w(5)	Maximum ratio (% of nominal)
nb_positions	w(6)	Number of tap positions
half_deadband	w(7)	Voltage half-deadband (pu)
Vset	w(8)	Voltage setpoint (pu)
delay_first	w(9)	First tap change delay (s)
delay_next	w(10)	Subsequent tap change delay (s)

Usage Example

```
DCTL LTC2 LTC2_TR1 TR1-HV-MV BUS_MV -1 88. 120. 33 0.01 1.0 30 8 ;
```

14.1.3. LTCINV (dctl_ltcinv): Inverse-Time Load Tap Changer

Description

An **inverse-time** tap changer in which the delay before a tap change is proportional to the integral of the voltage deviation rather than a fixed timer. Large voltage errors trigger faster operation; small but persistent deviations trigger slower operation.

This implements behaviour analogous to inverse-time overcurrent relays, applied to voltage control.

Logic Description

The controller integrates the signed voltage deviation over time:

$$\Sigma = \int_{t_0}^t (V_{set} - V) dt'$$

A tap change fires when the integrated error exceeds a threshold derived from the maximum delay and acceleration parameter:

$$\text{if } |\Sigma| \geq \frac{\tau_{max} \cdot \delta V}{\text{accel}} : \text{ tap change, } \Sigma \leftarrow 0$$

where τ_{max} is the maximum delay and `accel` is the acceleration factor (smaller value = faster response).

State variable `w(13)`: same coding as `dctl_ltc`.

Parameters

Field	Working variable	Description
<code>branch_name</code>	<code>w(1)</code>	Transformer branch
<code>bus_name</code>	<code>w(2)</code>	Monitored bus
<code>direction</code>	<code>w(3)</code>	Sign convention
<code>ratio_min</code>	<code>w(4)</code>	Minimum ratio (% of nominal)
<code>ratio_max</code>	<code>w(5)</code>	Maximum ratio (% of nominal)
<code>nb_positions</code>	<code>w(6)</code>	Number of tap positions
<code>half_deadband</code>	<code>w(7)</code>	Voltage half-deadband (pu)
<code>Vset</code>	<code>w(8)</code>	Voltage setpoint (pu)
<code>delay_max</code>	<code>w(9)</code>	Maximum delay before tap change (s)
<code>accel</code>	<code>w(10)</code>	Acceleration factor (s), smaller = faster

Usage Example

```
DCTL LTCINV LTCINV_TR1 TR1-HV-MV BUS_MV 1 85. 115. 33 0.01 1.025 60.0 0.1
↪ ;
```

14.1.4. OLTC2 (`dctl_oltc2`): On-Load Tap Changer Type 2

Description

An advanced on-load tap changer with **R/X line drop compensation**, end-stop flags. It includes separate flags to prevent tap movement when the transformer is at its mechanical end-stop, and applies voltage correction based on the current flowing through the transformer using the compensation resistances `R_comp` and `X_comp`.

The **Ratio2-factor** enables a secondary correction factor for the transformer ratio independent of the main tap position.

Logic Description

Compensated voltage seen by the controller:

$$V_{comp} = V_{meas} - R_{comp} I_{re} - X_{comp} I_{im}$$

where I_{re} , I_{im} are the real and imaginary components of the transformer current. The deadband check and timer logic then operate on V_{comp} .

End-stop flags `w(15)` and `w(16)` prevent further tap changes when the ratio is already at its limit and the requested change would move it beyond.

Parameters

Field	Working variable	Description
branch_name	w(1)	Transformer branch
bus_name	w(2)	Controlled bus
direction	w(3)	Sign convention
ratio_min	w(4)	Minimum ratio (pu)
ratio_max	w(5)	Maximum ratio (pu)
nb_positions	w(6)	Number of tap positions
half_deadband	w(7)	Voltage half-deadband (pu)
Vset	w(8)	Voltage setpoint (pu)
delay_first	w(9)	First tap change delay (s)
delay_next	w(10)	Subsequent tap change delay (s)
R_comp	w(17)	Line drop compensation resistance (pu)
X_comp	w(18)	Line drop compensation reactance (pu)
ratio2_factor	w(20)	Secondary ratio correction factor (pu)

14 fields required (includes from-bus, to-bus, circuit ID for PSS/E format).

Usage Example

```
DCTL  OLTC2  OLTC_TR2  FROM_BUS  TO_BUS  1  1  0.88  1.12  25  0.0075
      1.02  45.0  15.0  0.0  0.0 ;
```

14.2. Protection**14.2.1. UVPROT (dctl_uvprot): Under-Voltage Protection Relay****Description**

Trips a **generator or injector** when the voltage at a monitored bus falls below a threshold for a sustained time delay. This models definite-time under-voltage protection relays.

Logic Description

The relay monitors voltage V_{meas} at a specified bus:

if $V_{meas} < V_{min}$: start timer

if timer $\geq T_{delay}$: trip generator, state $\leftarrow -1$ (latched)

Once tripped ($w(8) = -1$), the controller is permanently latched and takes no further action.

Parameters

Field	Working variable	Description
bus_name	w(1)	Monitored bus name
gen_name	w(2)	Generator/injector to trip
Vmin	w(3)	Under-voltage pickup threshold (pu)
T_delay	w(4)	Time delay before tripping (s)

Internal variables: $w(5)$ = measured voltage, $w(6)$ = start time of violation, $w(7)$ = simulation time, $w(8)$ = controller state.

Usage Example

```
DCTL  UVPROT  UV_GEN1  BUS_HV  GEN1  0.85  0.5 ;
```

14.2.2. UVLS (dctl_uvls): Under-Voltage Load Shedding

Description

Sheds load in **multiple steps** in response to sustained voltage depression. The controller integrates the area between the measured voltage and a threshold (energy integral criterion) and triggers load shedding when the area exceeds a limit C . This prevents unnecessary shedding for short transient dips.

After each shedding step, the controller resets the integral and waits a minimum delay before the next step.

Logic Description

Area integral (trapezoidal):

$$A(t) = \int_{t_{start}}^t \max(V_{th} - V_{meas}, 0) dt'$$

Shedding condition:

$$\text{if } A \geq C \text{ and } (t - t_{last}) \geq T_{min} : \text{ shed load step}$$

Amount shed per step (bounded):

$$\Delta P_{shed} = \text{clip}(K \cdot A, P_{min,step}, P_{max,step})$$

Controller terminates when total shedding reaches the fraction `frac_load` of initial load or when `max_steps` is exhausted.

Parameters

Field	Working variable	Description
<code>bus_name</code>	<code>w(1)</code>	Monitored bus
<code>load_name</code>	<code>w(2)</code>	Controlled load (injector)
<code>V_th</code>	<code>w(3)</code>	Voltage threshold (pu)
<code>C</code>	<code>w(4)</code>	Maximum area $\int (V_{th} - V) dt$ before shedding (pu·s)
<code>T_min</code>	<code>w(5)</code>	Minimum delay between steps (s)
<code>T_max</code>	<code>w(6)</code>	Maximum delay between steps (s)
<code>frac_load</code>	<code>w(7)</code>	Maximum fraction of load that may be shed
<code>K</code>	<code>w(8)</code>	Gain: area $\rightarrow$ shed amount
<code>P_min_step</code>	<code>w(9)</code>	Minimum load shedding per step (MW)
<code>P_max_step</code>	<code>w(10)</code>	Maximum load shedding per step (MW)
<code>max_steps</code>	<code>w(11)</code>	Maximum number of shedding steps

Internal variables: `w(12)–w(18)` (previous voltage, timers, area accumulator, step counter, total shed).

Usage Example

```
DCTL UVLS UVLS1 BUS_MV LOAD1 0.90 0.5 5.0 20.0 0.3 2.0 5.0 50.0 5 ;
```

14.2.3. FRT (dctl_FRT): Fault Ride-Through Controller

Description

Implements a **piecewise-linear voltage–time FRT envelope** for an inverter-based generator (IBG). It monitors the bus voltage of the controlled injector and trips the unit if the voltage stays below the FRT envelope longer than allowed by the characteristic.

The FRT envelope is defined by four (voltage, time) breakpoints: (V_1, t_1) , (V_2, t_2) , (V_3, t_3) , (V_4) . If the voltage is above the envelope at all times, no action is taken. If it falls below the envelope for more than the corresponding time limit, the unit is tripped.

Logic Description

The FRT curve defines the minimum voltage $V(t)$ that must be sustained:

Segment	Condition
$V \geq V_1$	Always ride-through
$V_2 \leq V < V_1$	Must recover within t_1 s
$V_3 \leq V < V_2$	Must recover within t_2 s
$V_4 \leq V < V_3$	Must recover within t_3 s
$V < V_4$	Immediate trip

Two internal timers ($w(8)$, $w(9)$) track time spent in violation zones.

Parameters

Field	Working variable	Description
inj_name	$w(10)$	Controlled injector
Val1	$w(1)$	Voltage threshold 1 (pu), upper FRT boundary
t1	$w(2)$	Time limit for zone 1 (s)
Val2	$w(3)$	Voltage threshold 2 (pu)
t2	$w(4)$	Time limit for zone 2 (s)
Val3	$w(5)$	Voltage threshold 3 (pu)
t3	$w(6)$	Time limit for zone 3 (s)
Val4	$w(7)$	Voltage threshold 4 (pu), below this: trip immediately

8 fields required. Field 1 is the injector name, fields 2–8 map to Val1, t1, Val2, t2, Val3, t3, Val4.

Usage Example

```
DCTL FRT FRT_WTG1 WT1 0.90 0.15 0.70 0.5 0.20 1.0 0.05 ;
```

14.2.4. dctl_injprot: Injector Protection

Note

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected.

Description

A **generic injector protection** relay that monitors any observable variable of an injector (e.g., current magnitude, active power, reactive power) and trips the injector if the variable goes outside prescribed bounds for a sustained time.

The controller identifies the variable to monitor by name from the injector's observable list, making it flexible and applicable to any injector model.

Logic Description

if $x_{obs} < L_{lower}$ or $x_{obs} > L_{upper}$: start timer
 if timer $\geq T_{trip}$: trip injector, state $\leftarrow -1$

The controller latches once tripped and cannot reset. State variable w(7): - 1 = active (monitoring) - 0 = idle (within bounds) - -1 = already tripped

Parameters

Field	Working variable	Description
inj_name	w(1)	Monitored/controlled injector name
var_name	w(2)	Observable variable name (from injector's observable list)
lower_bound	w(3)	Lower bound for the variable
upper_bound	w(4)	Upper bound for the variable
T_trip	w(5)	Time out-of-bounds before tripping (s)

5 fields required.

Usage Example

```
DCTL injprot PROT_WTG1 WT1 I 0.0 1.2 0.1 ;
```

14.2.5. dctl_losprot: Loss-of-Synchronism Protection**Note**

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected.

Description

Trips a **synchronous generator** when its rotor speed exceeds a maximum threshold for a sustained time, detecting loss of synchronism (pole slipping). The controller monitors the machine's per-unit rotor speed ω and triggers disconnection when $\omega > \omega_{max}$ for longer than the specified time delay.

Logic Description

if $\omega > \omega_{max}$: start timer
 if timer $\geq T_{delay}$: trip generator, state $\leftarrow -1$

The timer resets if the speed returns within bounds. The controller latches permanently after tripping.

Parameters

Field	Working variable	Description
gen_name	w(1)	Controlled synchronous generator name
omega_max	w(2)	Maximum rotor speed threshold (pu)
T_delay	w(3)	Time delay before tripping (s)

3 fields required. Internal variables: w(4) = measured speed, w(5) = violation start time, w(6) = current time, w(7) = state.

Usage Example

```
DCTL  LOSPROT  LOS_GEN2  GEN2  1.05  0.2 ;
```

14.2.6. line_prot (dctl_line_prot): Line Overcurrent Protection

The data-file model name is `line_prot` (or `dctl_line_prot`). RAMSES adds the `dctl_` prefix automatically.

Description

Monitors the **apparent power flow on a set of lines** and trips them if they exceed their thermal rating (with a configurable oversize factor). It is designed to protect multiple lines simultaneously and disconnects any line whose flow exceeds $\text{oversize_factor} \times S_{\max}$.

The controller uses RAMSES' internal `smax_bra` array (maximum apparent power ratings from the network data) and reads real-time power flows via `pqbra`.

Logic Description

For each monitored branch k :

if $|S_k| > \text{oversize_factor} \times S_{\max,k}$: disconnect branch k

The action is immediate (no time delay); the oversize factor allows temporary overloads.

Parameters

Field	Working variable	Description
<code>oversize_factor</code>	<code>w(1)</code>	Allowed overload factor (e.g., 1.05 = 5% overload permitted)
<code>nb_lines</code>	<code>w(2)</code>	Number of protected lines
<code>line_1, ...</code>	<code>w(3-)</code>	Names of protected branches (space-separated)

Usage Example

```
DCTL  line_prot  LINEPROT1  1.05  2  L1-L2  L3-L4 ;
```

14.2.7. PST (dctl_pst): Phase-Shifting Transformer Control**Description**

Controls the phase angle (and thus the **active power flow**) of a phase-shifting transformer (PST) to regulate power flow on a monitored branch. It is structurally identical to `dctl_ltc` but acts on the PST angle rather than on voltage: the deadband is in MW (or pu power), and the controller moves the tap to keep active power within a band around the setpoint.

Logic Description

Let P_{meas} be the active power flow on the monitored branch and P_{set} the setpoint (initialised at $t = 0$):

if $|P_{meas} - P_{set}| > \delta P$: start timer

if timer $\geq \tau_{delay}$: advance/retard PST angle by $\Delta\phi$

The direction parameter defines whether increasing the tap increases or decreases power flow.

Parameters

Field	Working variable	Description
pst_branch	w(1)	PST branch name (must be type trfo)
mon_branch	w(2)	Monitored branch for power flow
direction	w(3)	Sign convention for power vs. Angle
ratio_min	w(4)	Minimum PST ratio (pu)
ratio_max	w(5)	Maximum PST ratio (pu)
nb_positions	w(6)	Number of positions
half_deadband	w(7)	Power half-deadband (pu)
delay_first	w(8)	First change delay (s)
delay_next	w(9)	Subsequent change delay (s)

9 fields required. Internal variables: w(10) = power setpoint, w(11) = time of last change, w(12) = active delay, w(13) = state.

Usage Example

```
DCTL PST PST_L1 PST-TR1 LINE-MON 1 -0.5 0.5 21 0.02 30.0 10.0 ;
```

14.2.8. RT (dctl_rt): Real-Time Simulation Control**Description**

Forces the RAMSES simulation to track **wall-clock time** at a configurable ratio, enabling hardware-in-the-loop (HIL) or real-time simulation testing. A ratio of 1.0 means the simulation runs in exact real time; a ratio of 2.0 allows the simulation to run up to twice as fast as real time (useful for digital twin applications).

An optional over-run tolerance parameter `allowed_overrun` specifies how far the simulation may lag behind real time before an error is triggered.

Parameters

Field	Working variable	Description
ratio	w(1)	Ratio to real time (1.0 = real time, >1 = faster)
allowed_overrun	w(2)	Allowed over-run before error (s), optional

1 or 2 fields. Only one DCTL RT controller may be active per simulation.

Observable: elapsed (wall-clock elapsed time in seconds).

Usage Example

```
DCTL RT RT_CTL 1.0 0.01 ;
```

14.2.9. dctl_hvdc_lim: HVDC LCC Limiter/Controller**Note**

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected.

Description

A discrete controller that **supervises and adjusts an LCC-HVDC link** (twop_HVDC_LCC) by: - Moving rectifier and inverter transformer tap changers to maintain firing and extinction angles within desired ranges - Triggering reduced-current or blocked operation when AC voltages are too low for normal commutation - Setting the α_{min} , γ_{min} , and I_{red} parameters of the HVDC model

It interacts directly with the HVDC two-port model's internal parameters.

Logic Description

Rectifier tap changer: if α is outside $[\alpha_{min,des}, \alpha_{max,des}]$ for longer than $delay_rect$, move rectifier transformer tap.

Inverter tap changer: if γ is outside $[\gamma_{min,des}, \gamma_{max,des}]$ for longer than $delay_inv$, move inverter transformer tap.

Reduced-current control: when V_{dc} falls below $V_{min,des}$ or $V_{max,des}$ the controller switches the inverter to reduced-current mode ($inv_ctl = -1$) and sets I_{red} .

Blocked operation: if conditions are unrecoverable (very low AC voltage), the HVDC is blocked ($rec_ctl = 3, inv_ctl = 3$).

Parameters

Field	Working variable	Description
hvdc_name	w(1)	Target HVDC two-port device name
nr_min	w(2)	Minimum rectifier transformer ratio
nr_max	w(3)	Maximum rectifier transformer ratio
nr_step	w(4)	Rectifier tap step size
ni_min	w(5)	Minimum inverter transformer ratio
ni_max	w(6)	Maximum inverter transformer ratio
ni_step	w(7)	Inverter tap step size
alpha_min_des	w(8)	Desired minimum firing angle (degrees)
alpha_max_des	w(9)	Desired maximum firing angle (degrees)
gamma_min_des	w(10)	Desired minimum extinction angle (degrees)
gamma_max_des	w(11)	Desired maximum extinction angle (degrees)
V_min_des	w(12)	Minimum desired DC voltage (kV)
V_max_des	w(13)	Maximum desired DC voltage (kV)
delay_rect	w(14)	Delay between rectifier tap changes (s)
delay_inv	w(15)	Delay between inverter tap changes (s)
alphamin	w(16)	Minimum firing angle limit (degrees)
Idred	w(17)	Reduced DC current (kA)
gammamin	w(18)	Minimum extinction angle (degrees)

18 fields required. Internal variables: w(19)–w(22) (operation modes and tap-change timers).

Usage Example

```
DCTL HVDC_LIM HVDCLIM1 HVDClink
  0.85 1.15 0.00625
  0.85 1.15 0.00625
 15.0 18.0 15.0 18.0
 400.0 600.0
 10.0 10.0
 5.0 0.0 15.0 ;
```

14.3. Monitoring

14.3.1. SIM_MINMAXSPEED (dctl_sim_minmaxspeed): Speed Monitoring (Min/-Max)

Description

A **simulation monitoring** controller that checks whether any synchronous machine speed violates upper or lower bounds. If a violation persists beyond a deadtime, the controller logs a warning or stops the simulation, depending on the `stop_sim` flag. It is used to automatically terminate simulations that have become numerically unstable or that have reached an unacceptable operating state.

Logic Description

At each evaluation, the controller scans all synchronous machines and checks:

if $\omega > \omega_{max}$ or $\omega < \omega_{min}$: record violation time
 if violation duration $> T_{deadtime}$: warn or stop simulation

Parameters

Field	Working variable	Description
<code>omega_max</code>	<code>w(1)</code>	Upper speed limit (pu)
<code>omega_min</code>	<code>w(2)</code>	Lower speed limit (pu)
<code>deadtime</code>	<code>w(3)</code>	Deadtime before action (s)
<code>stop_sim</code>	<code>w(4)</code>	1 = stop simulation on violation, 0 = warn only

4 fields required. Internal: `w(5)` = last violation time.

Parameter names exported: `SPEEDMIN`, `SPEEDMAX`, `DEADTIME`.

Usage Example

```
DCTL SIM_MINMAXSPEED SPEEDMON 1.2 0.5 0.5 1 ;
```

14.3.2. SIM_MINMAXVOLT (dctl_sim_minmaxvolt): Voltage Monitoring (Min/-Max)

Description

Analogous to `dctl_sim_minmaxspeed` but for **bus voltages**. The controller scans all buses and checks whether any voltage violates the specified bounds. If the violation persists for longer than the deadtime, a warning is logged or the simulation is terminated.

Logic Description

if $V > V_{max}$ or $V < V_{min}$: record violation time
 if violation duration $> T_{deadtime}$: warn or stop simulation

Parameters

Field	Working variable	Description
<code>V_max</code>	<code>w(1)</code>	Upper voltage limit (pu)
<code>V_min</code>	<code>w(2)</code>	Lower voltage limit (pu)
<code>deadtime</code>	<code>w(3)</code>	Deadtime before action (s)
<code>stop_sim</code>	<code>w(4)</code>	1 = stop on violation, 0 = warn only

Field	Working variable	Description
-------	------------------	-------------

4 fields required. Internal: $w(5)$ = last violation time.

Parameter names exported: VMIN, VMAX, DEADTIME.

Usage Example

```
DCTL SIM_MINMAXVOLT VOLTMON 1.15 0.80 1.0 0 ;
```

14.3.3. VOLT_VAR (dctl_voltage_variability): Voltage Variability Monitor

The data-file model name is VOLT_VAR.

Description

Computes a **running Exponential Moving Average (EMA)** of voltage variability (variance) across a list of monitored buses. It is used to assess power quality metrics such as voltage flicker or ramping caused by variable renewable energy sources.

The EMA uses two decay constants (λ_1 , λ_2) derived from the averaging time window, enabling a computationally efficient recursive estimation:

$$\text{EMA}(t) = \lambda_1 \cdot \text{EMA}(t - 1) + (1 - \lambda_1) \cdot V(t) + (\lambda_1 - \lambda_2)(V(t) - V(t - 1))$$

The controller accumulates per-bus variances and a system-wide sum.

Parameters

Field	Description
aver_time_window	Averaging time window (s) for EMA computation
nb_list	Number of buses to monitor (bus names follow in subsequent RAMSES data records)

2 fields required. Only one VOLT_VAR controller may exist per simulation. Parameters are stored in module-level variables (`volt_var_mod`) rather than $w(*)$.

Usage Example

```
DCTL VOLT_VAR VVAR 60.0 5 ;
```


15

Small-signal stability analysis

RAMSES computes small-signal stability analysis internally. It reduces the linearised differential-algebraic model to a state matrix, solves the dense eigenproblem, and writes eigenvalues, damping ratios, participation factors and mode shapes to file. No external tool is involved.

Requires a RAMSES newer than v3.60

An older engine accepts the EIG disturbance and writes no results files. The `stepss` package version's leading components name the bundled RAMSES, so `stepss.__version__` tells you directly.

15.1. What is computed

The linearised model is a set of differential-algebraic equations,

$$\begin{aligned}\Delta \dot{x} &= f_x \Delta x + f_y \Delta y \\ 0 &= g_x \Delta x + g_y \Delta y\end{aligned}$$

with states x and algebraic variables y . Eliminating Δy gives the state matrix

$$A_{sys} = f_x - f_y g_y^{-1} g_x$$

whose eigenvalues are the system modes. RAMSES assembles the unreduced Jacobian, factorises g_y once with KLU, forms A_{sys} , and solves it with LAPACK. The elimination exists only if g_y is nonsingular, which is what makes the model index-1; a singular g_y is reported rather than worked around.

For each eigenvalue $\lambda = \sigma \pm j\omega$:

Quantity	Definition
Frequency	$f = \omega /2\pi$ in Hz
Damping ratio	$\zeta = -\sigma/ \lambda $

A mode with $\zeta < 0$ grows rather than decays: the operating point is small-signal unstable.

15.2. Running an analysis

Add an EIG event to the disturbance file:

```
1.000 EIG 'ssa'
```

or inject it from Python:

```
import stepss

case = stepss.cfg()
case.addData('lf.dat')
case.addData('dyn.dat')
case.addData('solveroptions.dat')
case.addDst('nothing.dst')
case.addObs('obs.dat')
case.addTrj('out.trj')

ram = stepss.sim()
ram.execSim(case, 0.0)          # pause at the operating point
ram.addDisturb(0.001, "EIG 'ssa'") # schedule the analysis
ram.contSim(0.01)              # advance past it so the event fires
ram.endSim()
```

Results are computed at the instant the event fires, so where you pause determines what you get. Small-signal results are only meaningful at an operating point: pausing mid-swing linearises about a non-equilibrium.

15.2.1. Required settings

Setting	Why
\$OMEGA_REF SYN	Under the centre-of-inertia frame the COI equations are computed by finite differences and never enter the assembled Jacobian, so reducing under COI would silently hold COI speed constant and produce a plausible, wrong spectrum
\$SCHEME DE	Under the integrated scheme the pure differential-algebraic values exist only briefly inside the Newton loop

Both are refused rather than approximated. See Refusals below.

They are yours to set on the command line and from Python. The graphical interface's **Run small-signal stability analysis** writes both itself, into an extra data file read after the case's own, so a case configured for time-domain runs analyses without being edited. Settings are applied in the order they are read and the last of each kind wins, which is what makes that an override rather than a conflict.

15.3. Output files

Three files per analysis, named from the basename given to EIG. All are plain whitespace-separated text with # comment headers, so `numpy.loadtxt` reads them directly. Names are left-justified and contain no spaces, so splitting on whitespace is safe.

15.3.1. <name>_modes.dat

One line per mode, every mode written.

Column	Meaning
index	Mode number, 1-based
re, im	Real and imaginary parts of λ
zeta	Damping ratio
freq_hz	Frequency in Hz

Column	Meaning
dom	1 if $\text{Re}(\lambda)$ exceeded the <code>real_limit</code> filter
smp	1 if the eigenvalue is simple, 0 if degenerate

The header records `nstates`, `nalg`, the time, and the `real_limit`, `pf_threshold` and `gap_tol` values used.

15.3.2. <name>_pf.dat

Participation factors, one line per mode and state: `mode`, `state`, `pf`, `family`, `device`, `variable`. The `pf` column is the participation factor itself, which STEPSS GUI abbreviates to **PF**.

The participation of state k in mode i is $p_{ki} = |w_{ki} v_{ki}|$, built from the left and right eigenvectors and normalised so each mode's largest entry is 1. Written only for dominant modes, and only for entries above `pf_threshold`, so a state that is absent is below the threshold rather than exactly zero.

15.3.3. <name>_ms.dat

Mode shapes, one line per mode and machine: `mode`, `state`, `magnitude`, `angle_deg`, `device`.

Rotor-speed components, normalised so the largest magnitude in each mode is 1, with **angles relative to that largest entry**, because an eigenvector's absolute phase is arbitrary.

15.4. Saving a run as one archive

The three files above are only useful together, and a spectrum is only worth much beside the matrix it was reduced from. STEPSS GUI writes all of them, plus the Jacobian that the JAC disturbance dumped at the same instant, into a single `.zip` or `.tar.gz`: **Save dynamic Jacobian...** on the Analysis tab, with the format chosen in the dialog. **Load dynamic Jacobian...** beside it opens one back into a results window of its own, leaving any window already up alone, so an archived run and a fresh one can be read side by side.

In the archive

<code>stepss-ssa.txt</code>	The manifest: the basename, the engine version, <code>t</code> , <code>real_limit</code> and <code>pf_threshold</code>
<code><name>_modes.dat</code> , <code><name>_pf.dat</code> , <code><name>_ms.dat</code>	The results above
<code><name>_eqs.dat</code> , <code><name>_var.dat</code> , <code><name>_val.dat</code> , <code><name>_struc.dat</code>	The unreduced Jacobian

An archive is a record of a result, not an input that reproduces one. The data files, the solver settings and the disturbance that produced the run are not in it. It is something to hand to a colleague, attach to an issue, or come back to in a year and still be able to read; re-running the analysis needs the case, which is a separate thing to keep.

The manifest is what makes it readable that much later. None of the results files records which engine wrote them, so an archive analysed by an older build is otherwise indistinguishable from one made today: STEPSS names the recorded version when it opens one, and says so when it differs from the engine now in use. An archive carrying no manifest is refused with a reason rather than opened into an empty window.

Both formats are ordinary archives that `unzip` and `tar xzf` read, and everything sits under one directory named for the run. The archive is also the only way back into the interface: results sitting loose in a directory, from a run made at a terminal for instance, are read by the Python API rather than by STEPSS GUI, which opens a run either by producing it or by loading one of these.

15.5. Degenerate modes, and why the `smp` column matters

Identical machine models with identical parameters produce identical poles, so power system spectra are heavily degenerate. In the Kundur two-area system, 20 of 70 modes share an eigenvalue with another mode.

In a degenerate eigenspace the individual eigenvectors are not unique. The participation factors and mode shape of such a mode are therefore basis-dependent: real numbers that carry no physical meaning and that would come out differently on another LAPACK build.

The `smp` column is 1 when the gap from this eigenvalue to every other exceeds `gap_tol` (default $1e-6$, recorded in the header), and 0 otherwise. **Do not interpret participation factors or mode shapes for a mode flagged 0.**

15.6. Refusals

The analysis refuses rather than returning a result it cannot justify. Every refusal writes no results files, states the reason in the log and through `getLastErr()`, and exits with code 78.

Condition	Reason
<code>\$OMEGA_REF COI</code>	The assembled Jacobian does not carry the COI equations
<code>\$SCHEME IN</code>	The pure DAE values are not available at that point
States above <code>\$EIG_MAX_STATES</code>	The dense solve is not practical at that size
Singular g_y	The model is not index-1

A run that produced nothing and exited 0 is a different problem, most likely an engine older than v3.60.

The size ceiling is `$EIG_MAX_STATES`, default 5000. Systems beyond it need sparse shift-invert methods, which `scipy.sparse.linalg.eigs` can drive from the descriptor matrices `getJac()` returns.

15.7. Worked example

The Kundur two-area system is the standard inter-area oscillation benchmark, and the `stepss` package ships an annotated notebook for it under `examples/eigenanalysis/`. Analysed with and without its power system stabilisers:

	inter-area	area 1 local	area 2 local
without PSS	0.625 Hz, $\zeta = -0.0233$	1.085 Hz, $\zeta = 0.099$	1.116 Hz, $\zeta = 0.097$
with PSS	0.624 Hz, $\zeta = +0.1087$	1.242 Hz, $\zeta = 0.288$	1.295 Hz, $\zeta = 0.287$

The inter-area damping ratio flips sign with the stabilisers: without them the 0.62 Hz oscillation between the two areas grows and the operating point is small-signal unstable. This reproduces Kundur, *Power System Stability and Control*, Example 12.6.

Participation factors separate the two local modes without any prior knowledge of the topology: the 1.085 Hz mode lists only G1 and G2, the 1.116 Hz mode only G3 and G4, and the inter-area mode lists all four.

In STEPSS GUI the same run is read from the small-signal results window, which **Run small-signal stability analysis** on the Analysis tab opens on the run it just made:

Reading that window across is the whole method in one view. The table gives the frequency and damping of each mode; the s-plane shows how much margin each one has, with the boundary drawn at the imaginary axis, so a mode crossing it is the instability. Every mode is one circle there, crimson if it is unstable and filled if it is the one selected. The **Participation** panel answers which machines make up that mode, its last column **PF** being the participation factor, normalised so the largest in each mode is 1. The mode shape answers how those

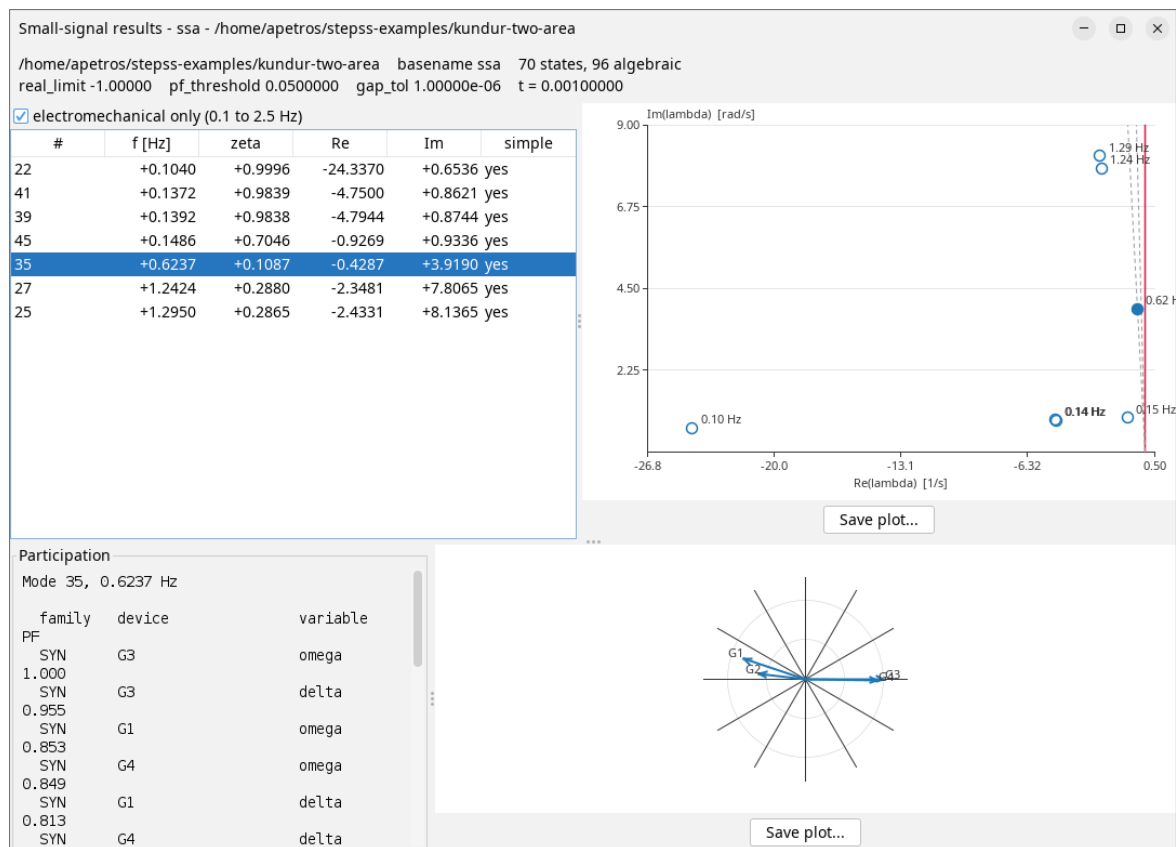


Figure 15.1: The small-signal results window for the Kundur case.

machines move relative to each other: here G1 and G2 swing against G3 and G4, which is what makes 0.62 Hz the inter-area mode rather than a local one.

Ticking **electromechanical only** restricts the table to the 0.1 to 2.5 Hz band, which is where rotor-angle modes live. The full spectrum for this case is 70 states, most of them fast controller and network dynamics that no rotor participates in.

Note

This screenshot is light in both site themes. The window itself follows the application's theme from v3.74.20, plot panels included; before that release the s-plane and the mode shape stayed white under the dark theme. A plot saved with **Save plot...** is drawn on white whatever theme is in use, since it is meant for a report rather than the screen.

15.8. See Also

- Export Jacobian Matrix, the JAC disturbance, for exporting the matrices instead of analysing them
- `getJac()`, the descriptor matrices as SciPy sparse objects, for driving your own solver
- Kundur Two-Area System

Part V

Model library

16

The model library

This is the catalogue of models RAMSES can load. Each family has its own page with equations, parameters in declaration order, and worked examples. For the record syntax that references these models, see Dynamic Data Records.

16.1. How a Model Name Resolves

RAMSES adds the family prefix (`exc_`, `tor_`, `inj_`, `twop_`) automatically, so `AC1A` and `exc_AC1A` are the same model. Names are **case sensitive**.

A model can be in one of three states, and the tables below say which:

State	Meaning
Built in	Handled directly by the family dispatcher, no prefix
Registered	Compiled and mapped to a name, callable from a data file
Not compiled	Present as source but excluded from the build, so not reachable at all

Every compiled model is reachable

As of 3.79 every model compiled into the library is mapped to a name and can be loaded by writing that name in a data file. Before 3.79 twenty-four of them shipped under no name and could only be reached by adding a case through URAMSES. The Hydro-Québec families (`models/*/hq/`) and `inj_norton` are excluded from the build entirely and are not reachable by any route.

16.2. Exciters

IEEE Exciters and Custom Exciters

Data-file name	State	Documented on
<code>CONSTANT</code> , <code>1ST_ORDER</code>	Built in	Custom Exciters
<code>GENERIC1</code> , <code>GENERIC2</code>	Built in	Custom Exciters
<code>GENERIC</code>	Registered, since 3.57	Custom Exciters
<code>kundur</code>	Registered	Custom Exciters
<code>AVR_DG</code>	Registered, since 3.76	Custom Exciters
<code>AC1A</code> , <code>AC4A</code> , <code>IEEE5</code>	Registered	IEEE Exciters
<code>ST1A</code> , <code>ST1A_IEEEST</code> , <code>ST1A_PSS2B</code> , <code>ST1A_PSS4B</code>	Registered	IEEE Exciters
<code>SEXS</code> , <code>SEXS_IEEEST</code>	Registered	IEEE Exciters

Data-file name	State	Documented on
EXPIC1_PSS2B	Registered	IEEE Exciters
ENTSOE_simp	Registered	IEEE Exciters
AC1A_MAXEX2, AC1A_RETRO, AC1A_RETRO_PSS4B, AC8B, AC8B_PSS3B_lim, DC3A, EXPIC1, EXPIC1_PSS2B_MAXEX2, SEXS_STAB3_lim, ST1A_IEEEST_MAXEX2, ST1A_lim, ST1A_PSS2B_MAXEX2, ST1A_PSS3B, ST1A_PSS4B_MAXEX2, ST2A	Registered, since 3.79	IEEE Exciters

16.3. Governors

IEEE Governors and Custom Governors

Data-file name	State	Documented on
CONSTANT, 1ST_ORDER	Built in	Custom Governors
HYDRO_GENERIC1, THERMAL_GENERIC1	Built in	Custom Governors
HYDRO_DG	Registered, since 3.76	Custom Governors
DEGOV1, ENTSOE_simp	Registered, since 3.40	IEEE Governors
GAST, TGOV1, HYGOV	Registered, since 3.50	IEEE Governors
GASTURBM, GOVCLASM, GOVHYDR, GOVNUC	Built in, since 3.79	Custom Governors

16.4. Injectors

Injector Models

Data-file name	State
LOAD, RESTLD	Built in
INDMACH1, INDMACH2	Built in
SVC_GENERIC1	Built in
THEVEQ	Built in
PQ, IBG	Registered
WT3, WT4	Registered
BESS, GFOL, GFOR	Registered
vfd_load, PMU	Registered
VFAULT	Registered, added automatically by RAMSES
INDM1, PV	Registered, since 3.79
inj_norton	Excluded from the build

16.5. Two-Port Models

Two-Port Models

Data-file name	State
HVDC_LCC	Registered
HVDC_VSC_SC	Registered
DCL_WCL	Registered
HVDC_VSC	Registered, since 3.79

16.6. Discrete Controllers

Discrete Controller Models

Discrete controllers take no prefix in the data file, though the dispatcher prepends `dctl_` internally.

Data-file name	State
LTC, LTC2, LTCINV, OLTC2	Built in
PST	Built in
UVLS, UVPROT, FRT	Built in
MAIS	Built in
RT	Built in
SIM_MINMAXVOLT, SIM_MINMAXSPEED	Built in
VOLT_VAR	Built in
line_prot	Registered as <code>dctl_line_prot</code>

16.7. The Synchronous Machine

The machine itself is not selected by name; every `SYNC_MACH` record uses the same model, parameterised by the record's own fields.

- Synchronous Machine Model, equations and parameters
- Parameter Conversion, converting between the XT and RL forms

16.8. Next Steps

- Dynamic Data Records, the record syntax that references these models
- User-Defined Models, write and register a model of your own

Synchronous machine models

This page documents the mathematical model of the synchronous machine implemented in RAMSES. The model is a detailed sixth-order model, including four rotor windings with saturation effects. It uses the Equal-Mutual-Flux-Linkage (EMFL) per unit system and supports detailed (round rotor, salient-pole) and simplified (field winding only) configurations through model switches.

17.1. Model Switches

To accommodate different rotor configurations within a single model, integer “model switches” are defined:

Switch	Meaning
S_{d1}	1 if there is a damper winding $d1$, 0 otherwise
S_{q1}	1 if there is a damper winding $q1$, 0 otherwise
S_{q2}	1 if there is an equivalent winding $q2$, 0 otherwise

Model	Switches
Detailed, round rotor	$S_{d1} = 1, S_{q1} = 1, S_{q2} = 1$
Detailed, salient-pole rotor	$S_{d1} = 1, S_{q1} = 1, S_{q2} = 0$
Detailed, salient-pole rotor	$S_{d1} = 1, S_{q1} = 0, S_{q2} = 1$
Simplified, field winding only	$S_{d1} = 0, S_{q1} = 0, S_{q2} = 0$

The second and third combinations yield the same results. Models with fewer rotor windings are specified by skipping the corresponding data in the SYNC_MACH record (see below).

17.2. Park Transformation

The well-known Park transformation is used to replace time-varying inductances and oscillatory stator currents and voltages with constant values. The machine is represented by equivalent windings along the direct (d) and quadrature (q) axes: a field winding f and damper winding $d1$ on the d axis, and windings $q1, q2$ on the q axis:

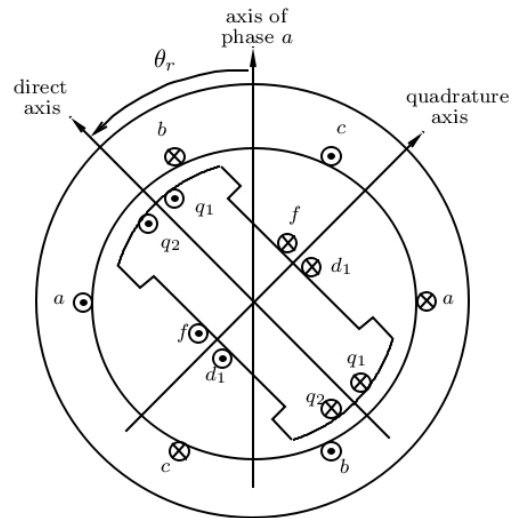


Figure 17.1: Synchronous machine windings

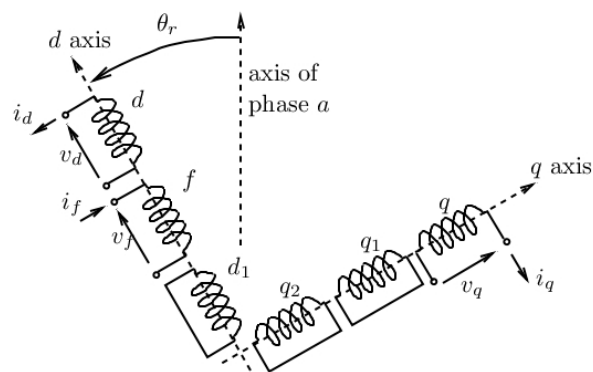


Figure 17.2: Equivalent windings of the Park transformation

17.3. Flux-Current Relationships

Using the EMFL per unit system, the relationship between magnetic flux linkages and currents is:

$$\begin{pmatrix} \psi_d \\ \psi_f \\ \psi_{d1} \end{pmatrix} = \begin{pmatrix} L_\ell + M_d & M_d & S_{d1}M_d \\ M_d & L_{\ell f} + M_d & S_{d1}M_d \\ S_{d1}M_d & S_{d1}M_d & L_{\ell d1} + S_{d1}M_d \end{pmatrix} \begin{pmatrix} i_d \\ i_f \\ i_{d1} \end{pmatrix}$$

$$\begin{pmatrix} \psi_q \\ \psi_{q1} \\ \psi_{q2} \end{pmatrix} = \begin{pmatrix} L_\ell + M_q & S_{q1}M_q & S_{q2}M_q \\ S_{q1}M_q & L_{\ell q1} + S_{q1}M_q & S_{q2}M_q \\ S_{q2}M_q & S_{q2}M_q & L_{\ell q2} + S_{q2}M_q \end{pmatrix} \begin{pmatrix} i_q \\ i_{q1} \\ i_{q2} \end{pmatrix}$$

The d and q components of the air-gap flux are:

$$\psi_{ad} = M_d(i_d + i_f + S_{d1}i_{d1})$$

$$\psi_{aq} = M_q(i_q + S_{q1}i_{q1} + S_{q2}i_{q2})$$

Individual flux linkages in terms of air-gap flux:

$$\psi_d = L_\ell i_d + \psi_{ad}$$

$$\psi_f = L_{\ell f} i_f + \psi_{ad}$$

$$\psi_{d1} = L_{\ell d1} i_{d1} + S_{d1} \psi_{ad}$$

$$\psi_q = L_\ell i_q + \psi_{aq}$$

$$\psi_{q1} = L_{\ell q1} i_{q1} + S_{q1} \psi_{aq}$$

$$\psi_{q2} = L_{\ell q2} i_{q2} + S_{q2} \psi_{aq}$$

Rotor currents from flux linkages:

$$i_f = \frac{\psi_f - \psi_{ad}}{L_{\ell f}}, \quad i_{d1} = \frac{\psi_{d1} - S_{d1} \psi_{ad}}{L_{\ell d1}}, \quad i_{q1} = \frac{\psi_{q1} - S_{q1} \psi_{aq}}{L_{\ell q1}}, \quad i_{q2} = \frac{\psi_{q2} - S_{q2} \psi_{aq}}{L_{\ell q2}}$$

17.4. Saturation Model

Let M_d^u and M_q^u be the unsaturated direct- and quadrature-axis mutual inductances. The saturated values M_d and M_q are:

$$M_d = \frac{M_d^u}{1 + m \left(\sqrt{\psi_{ad}^2 + \psi_{aq}^2} \right)^n}$$

$$M_q = \frac{M_q^u}{1 + m \left(\sqrt{\psi_{ad}^2 + \psi_{aq}^2} \right)^n}$$

where m and n are the saturation exponents specified in the SYNC_MACH record. Substituting into the air-gap flux expressions yields the algebraic equations:

$$\begin{aligned} \psi_{ad} \left(\frac{1 + m(\sqrt{\psi_{ad}^2 + \psi_{aq}^2})^n}{M_d^u} + \frac{1}{L_{\ell f}} + \frac{S_{d1}}{L_{\ell d1}} \right) - i_d - \frac{1}{L_{\ell f}} \psi_f - \frac{S_{d1}}{L_{\ell d1}} \psi_{d1} &= 0 \\ \psi_{aq} \left(\frac{1 + m(\sqrt{\psi_{ad}^2 + \psi_{aq}^2})^n}{M_q^u} + \frac{S_{q1}}{L_{\ell q1}} + \frac{S_{q2}}{L_{\ell q2}} \right) - i_q - \frac{S_{q1}}{L_{\ell q1}} \psi_{q1} - \frac{S_{q2}}{L_{\ell q2}} \psi_{q2} &= 0 \end{aligned}$$

17.5. Reference Frame

All synchronous machines have their rotor positions referred to the x axis of the network reference frame (see Reference Frames & Initialization). The **rotor angle** δ of a machine is the angle difference between its q axis and the x reference axis. In steady state, the machine internal emf (proportional to field current) is aligned along the q axis; δ is thus the phase angle of that emf with respect to the x axis.

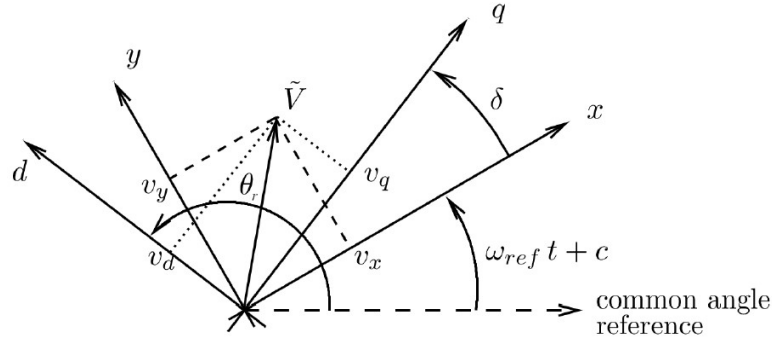


Figure 17.3: Definition of the rotor angle delta

The d and q components of the stator voltage and current relate to the network (x, y) components through the rotor angle δ :

$$\begin{aligned} \begin{pmatrix} v_d \\ v_q \end{pmatrix} &= \begin{pmatrix} -\sin \delta & \cos \delta \\ \cos \delta & \sin \delta \end{pmatrix} \begin{pmatrix} v_x \\ v_y \end{pmatrix} \\ \begin{pmatrix} i_d \\ i_q \end{pmatrix} &= \begin{pmatrix} -\sin \delta & \cos \delta \\ \cos \delta & \sin \delta \end{pmatrix} \begin{pmatrix} i_x \\ i_y \end{pmatrix} \end{aligned}$$

After transformation, the air-gap flux algebraic equations in (x, y) coordinates become:

$$\begin{aligned} \psi_{ad} \left(\frac{1 + m(\sqrt{\psi_{ad}^2 + \psi_{aq}^2})^n}{M_d^u} + \frac{1}{L_{\ell f}} + \frac{S_{d1}}{L_{\ell d1}} \right) + \sin \delta i_x - \cos \delta i_y - \frac{1}{L_{\ell f}} \psi_f - \frac{S_{d1}}{L_{\ell d1}} \psi_{d1} &= 0 \\ \psi_{aq} \left(\frac{1 + m(\sqrt{\psi_{ad}^2 + \psi_{aq}^2})^n}{M_q^u} + \frac{S_{q1}}{L_{\ell q1}} + \frac{S_{q2}}{L_{\ell q2}} \right) - \cos \delta i_x - \sin \delta i_y - \frac{S_{q1}}{L_{\ell q1}} \psi_{q1} - \frac{S_{q2}}{L_{\ell q2}} \psi_{q2} &= 0 \end{aligned}$$

17.6. Park Equations

17.6.1. Original form

$$v_d = -R_a i_d - \omega \psi_q \quad v_q = -R_a i_q + \omega \psi_d$$

$$\frac{d\psi_f}{dt} = \omega_N (K_f v_f - R_f i_f) \quad \frac{d\psi_{d1}}{dt} = -\omega_N R_{d1} i_{d1} \quad \frac{d\psi_{q1}}{dt} = -\omega_N R_{q1} i_{q1} \quad \frac{d\psi_{q2}}{dt} = -\omega_N R_{q2} i_{q2}$$

where R_a is the stator (armature) resistance, R_f the field winding resistance, R_{d1} , R_{q1} , R_{q2} the rotor winding resistances, ω the rotor speed (pu), $\omega_N = 2\pi f_{nom}$ the nominal angular frequency (rad/s), v_f the field voltage, and K_f a coefficient to pass from per unit values of the excitation system to per unit values of the machine.

The stator equations are transformed to the (x, y) frame using the rotation matrices above, and the rotor currents are eliminated using the flux-current relationships, yielding the equations actually solved by RAMSES:

17.6.2. Stator equations (algebraic, in x - y frame)

$$0 = \sin \delta v_x - \cos \delta v_y + (R_a \sin \delta - \omega L_\ell \cos \delta) i_x - (R_a \cos \delta + \omega L_\ell \sin \delta) i_y - \omega \psi_{aq}$$

$$0 = -\cos \delta v_x - \sin \delta v_y - (R_a \cos \delta + \omega L_\ell \sin \delta) i_x - (R_a \sin \delta - \omega L_\ell \cos \delta) i_y + \omega \psi_{ad}$$

17.6.3. Rotor equations (differential)

$$\frac{d\psi_f}{dt} = \omega_N \left(K_f v_f - R_f \frac{\psi_f - \psi_{ad}}{L_{\ell f}} \right)$$

$$\frac{d\psi_{d1}}{dt} = -\omega_N R_{d1} \frac{\psi_{d1} - S_{d1} \psi_{ad}}{L_{\ell d1}}$$

$$\frac{d\psi_{q1}}{dt} = -\omega_N R_{q1} \frac{\psi_{q1} - S_{q1} \psi_{aq}}{L_{\ell q1}}$$

$$\frac{d\psi_{q2}}{dt} = -\omega_N R_{q2} \frac{\psi_{q2} - S_{q2} \psi_{aq}}{L_{\ell q2}}$$

17.7. Rotor Motion

$$\frac{1}{\omega_N} \frac{d\delta}{dt} = \omega - \omega_{ref}$$

$$2H \frac{d\omega}{dt} = K_m T_m - T_e - D(\omega - \omega_{ref})$$

where H is the inertia constant (in s), T_m the mechanical torque produced by the turbine, K_m a coefficient to pass from per unit values of the turbine to per unit values of the machine, and ω_{ref} the angular speed of the reference axes: ω_{coi} in the COI reference frame, or 1 pu in the synchronous frame (selected by the \$OMEGA_REF solver setting).

The electromagnetic torque T_e is:

$$T_e = \psi_{ad} i_q - \psi_{aq} i_d = \psi_{ad} (\cos \delta i_x + \sin \delta i_y) - \psi_{aq} (-\sin \delta i_x + \cos \delta i_y)$$

17.8. State Variables and Equations Summary

The model has **10 state variables**: $i_x, i_y, \psi_{ad}, \psi_{aq}, \psi_f, \psi_{d1}, \psi_{q1}, \psi_{q2}, \delta, \omega$.

These are balanced by: - **4 algebraic equations**: air-gap flux (d and q), stator voltage (d and q) - **6 differential equations**: field flux, d1 damper flux, q1 damper flux, q2 damper flux, rotor angle, rotor speed

17.9. Per Unit System and IBRATIO

The synchronous machine model uses the EMFL per unit system, while the excitation system typically uses its own per unit system. The parameter IBRATIO bridges these two bases:

$$IBRATIO = \frac{I_{fB}^{mac}}{I_{fB}^{exc}}$$

where I_{fB}^{mac} is the field winding base current in the machine model and I_{fB}^{exc} is the base current in the excitation system model. The relationship between per-unit field currents in the two systems is:

$$IBRATIO = \frac{i_{f,pu}^{exc}}{i_{f,pu}^{mac}}$$

17.9.1. Common per unit conventions for IBRATIO

Open-circuit unsaturated machine (most common): I_{fB}^{exc} is the field current that produces nominal stator voltage ($V = 1$ pu) at nominal speed ($\omega = 1$ pu) with the stator open, neglecting saturation:

$$IBRATIO = M_d^u = X_d^u - X_\ell$$

Open-circuit saturated machine: Same conditions but with saturation:

$$IBRATIO = \frac{M_d^u}{1+m} = \frac{X_d^u - X_\ell}{1+m}$$

Saturated machine at nominal operating conditions: I_{fB}^{exc} is the field current when the machine produces nominal active and reactive powers ($P = \cos \phi_N, Q = \sin \phi_N$) at nominal voltage and speed, with saturation.

17.10. SYNC_MACH Record

The machine model requires the nominal system frequency, given by the **mandatory** FNOM record (see Solver Settings):

FNOM F ;

where F is the nominal frequency in Hz.

The synchronous machine itself is declared with the SYNC_MACH record:

```

SYNC_MACH name bus FP FQ P Q SNOM Pnom H D IBRATIO
          TYPE_MOD <14 machine parameters, see below>
          EXC exc_type param1 param2 ...
          TOR tor_type param1 param2 ... ;

```

TYPE_MOD is a keyword selecting which of two **equivalent parameter formats** the 14 machine parameters that follow are given in:

- **RL**: the inductances and resistances of the Park model are supplied directly:

RL L1 Mdu L1f L1d1 Mqu L1q1 L1q2 m n Ra Rf Rd1 Rq1 Rq2

- **XT**: characteristic reactances and open-circuit time constants are supplied; RAMSES converts them internally to the Park parameters (see Parameter Conversion):

XT X1 Xd X'd X''d Xq X'q X''q m n Ra T'do T''do T'qo T''qo

17.10.1. Common parameters

Parameter	Description	Unit
name	Machine name (max 20 characters)	
bus	Connection bus name (max 8 characters)	
FP	Active power participation fraction (0–1)	
FQ	Reactive power participation fraction (0–1)	
P	Initial active power (used when FP = 0)	MW
Q	Initial reactive power (used when FQ = 0)	Mvar
SNOM	Nominal apparent power, used as base power in the machine model	MVA
Pnom	Nominal active power of the turbine, used as base power for the turbine model	MW
H	Inertia constant	s
D	Damping coefficient (usually set to zero when the damper windings are modelled)	pu
IBRATIO	Field current base ratio $I_{fB}^{mac} / I_{fB}^{exc}$ (see above)	pu
TYPE_MOD	Parameter format keyword: RL or XT (case-insensitive)	

17.10.2. Machine parameters, XT format

Parameter	Description	Unit
X1	Leakage reactance L_ℓ	pu
Xd	d-axis synchronous reactance ($M_d^u = X_d - X_\ell$)	pu
X'd	d-axis transient reactance (must be smaller than Xd)	pu
X''d	d-axis subtransient reactance (* if no d1 damper winding)	pu
Xq	q-axis synchronous reactance ($M_q^u = X_q - X_\ell$)	pu
X'q	q-axis transient reactance (* if no q1 winding; must be smaller than Xq)	pu
X''q	q-axis subtransient reactance (* if no q2 winding)	pu
m	Saturation coefficient (set to 0 to neglect saturation)	
n	Saturation exponent (ignored when m = 0)	
Ra	Armature resistance	pu
T'do	d-axis open-circuit transient time constant	s
T''do	d-axis open-circuit subtransient time constant (* if no d1 damper winding)	s
T'qo	q-axis open-circuit transient time constant (* if no q1 winding)	s
T''qo	q-axis open-circuit subtransient time constant (* if no q2 winding)	s

17.10.3. Machine parameters, RL format

Parameter	Description	Unit
Ll	Stator leakage inductance L_ℓ	pu
Mdu	Unsaturated d-axis mutual inductance M_d^u	pu
Llf	Field winding leakage inductance $L_{\ell f}$	pu
Lld1	d1 damper leakage inductance $L_{\ell d1}$ (* if no d1 winding)	pu
Mqu	Unsaturated q-axis mutual inductance M_q^u	pu
Llq1	q1 winding leakage inductance $L_{\ell q1}$ (* if no q1 winding)	pu
Llq2	q2 winding leakage inductance $L_{\ell q2}$ (* if no q2 winding)	pu
m	Saturation coefficient (set to 0 to neglect saturation)	
n	Saturation exponent (ignored when m = 0)	
Ra	Armature resistance	pu
Rf	Field winding resistance R_f	pu
Rd1	d1 damper resistance R_{d1} (* if no d1 winding)	pu
Rq1	q1 winding resistance R_{q1} (* if no q1 winding)	pu
Rq2	q2 winding resistance R_{q2} (* if no q2 winding)	pu

17.10.4. Omitting rotor circuits

A rotor circuit the machine does not have is skipped by putting * in **both** of its fields, the inductance/reactance **and** the matching resistance/time-constant field (specifying only one is an error):

Circuit	RL fields	XT fields	Model switch set to 0
d1 damper	Lld1, Rd1	X"d, T"do	S_{d1}
q1 winding	Llq1, Rq1	X'q, T'qo	S_{q1}
q2 winding	Llq2, Rq2	X"q, T"qo	S_{q2}

The combination $S_{d1} = 0$ with $S_{q1} = S_{q2} = 1$ (field winding plus both q-axis windings but no d-axis damper) is rejected. In the XT format, if the fitted Park parameters come out negative, RAMSES logs an “unrealistic Park inductances or resistances” warning: the supplied reactances and time constants are physically inconsistent.

All reactances, inductances and resistances are in per unit on the machine base (S_{nom} , nominal voltage), using the EMFL per unit system for the rotor quantities. Time constants are entered in seconds and normalised internally by $t_b = 1/(2\pi f_{nom})$.

The EXC and TOR sub-records specify the excitation system and turbine-governor models. See the Model Reference for available models.

Note

The FP, FQ, P, Q fields control how the machine’s initial operating point is determined from the power flow solution. See Reference Frames & Initialization for details.

17.11. Initialization Output

At initialization RAMSES prints one block per synchronous machine. Example:

NUMBER OF SYNCHRONOUS MACHINES : 1

machine	at bus	V	P	Q	delta
↪ sat	island br				
	excit model	vf(pu)	torque model		Tm(pu)

G5		5	1.00000	450.00186	68.49769	70.99
↪	1.00000	1	1			
		exc_GENERIC	2.3680	THERMAL_GENERIC1		0.97826

Here the machine G5, connected to bus 5, is in service ($br = 1$) and injects about 450 MW and 68 Mvar into the grid under a bus voltage of 1 pu.

- δ is the initial value of the rotor angle δ , in **degrees**.
- sat is the saturation factor

$$sat = 1 + m \left(\sqrt{\psi_{ad}^2 + \psi_{aq}^2} \right)^n \geq 1$$

the ratio between the field current in the saturated machine and the corresponding field current when saturation is neglected, for the same operating conditions. It characterizes the extra excitation current needed in the presence of saturated material ($sat = 1$ when m is set to zero).

- v_f is the initial field voltage on the **exciter base**, which is indirectly defined by the `IBRATIO` parameter of the machine.
- T_m is the initial mechanical torque on the **turbine base**, which is defined by the `Pnom` parameter of the machine.

17.12. Parameter Conversion (XT ↔ RL)

For a detailed derivation of how STEPSS converts the XT standard parameters (reactances and open-circuit time constants) to the RL Park parameters (inductances and resistances), including the exact algorithm from the source, known conversion pitfalls when cross-checking against EMT simulators, and a reference Python implementation, see:

→ Synchronous Machine Parameter Conversion (XT ↔ RL)

18

Synchronous machine parameter conversion

A SYNC_MACH record can be entered using one of two equivalent parameter formats, selected by the TYPE_MOD keyword:

- **RL**: the Park-model inductances and resistances are supplied directly.
- **XT**: characteristic reactances and open-circuit time constants are supplied; STEPSS/RAMSES converts them internally.

Both formats describe the same machine. XT is convenient when data comes from manufacturer datasheets or Kundur-style standard parameters. This page documents exactly how that conversion is done, so it can be reproduced by hand or cross-checked against external simulators (e.g. Typhoon HIL EMT).

Common pitfalls

Two details that break hand calculations against STEPSS: 1. Open-circuit time constants are normalised to **radians** by $t_b = 1/(2\pi f_{nom})$ before use in the resistance formulas. 2. The field circuit base is tied to IBRATIO via $p_{uf} = R_f/IBRATIO$; a wrong assumption here corrupts M_d , $L_{\ell f}$, and R_f after per-unit $\rightarrow$ Henry conversion.

18.1. Authoritative Field Order

From the RAMSES source (sync.f90, get_sync_mach):

```
SYNC_MACH NAME BUS FP FQ P Q SNOM PNOM H D IBRATIO
RL  LL MDU  LLF  LLD1 MQU LLQ1 LLQ2 M N RA RF   RD1  RQ1  RQ2
XT  LL XD   XPD  XSD  XQ  XPQ  XSQ  M N RA TPD0 TSD0 TPQ0 TSQ0
EXC <model> ... TOR <model> ... ;
```

A rotor circuit the machine does not have is skipped with * in **both** its reactance/inductance and its resistance/time-constant field.

18.2. Conversion Algorithm (XT → RL)

All parameters are in per unit; time constants are entered in seconds and normalised internally.

Time base

$$t_b = \frac{1}{2\pi f_{nom}}$$

All time constants (TPD0, TSD0, TPQ0, TSQ0) are divided by t_b before entering the resistance formulas.

Unsaturated mutuals

$$M_d^u = X_d - L_\ell, \quad M_q^u = X_q - L_\ell$$

d axis, two rotor circuits (field + damper, XSD/TSD0 present)

$$\begin{aligned} T_d' &= T_{d0}' \frac{X_d'}{X_d}, & T_d'' &= \frac{X_d'' T_{d0}' T_{d0}''}{X_d T_d'} \\ a &= \frac{X_d T_d' T_d'' - L_\ell T_{d0}' T_{d0}''}{X_d - L_\ell}, & b &= \frac{X_d (T_d' + T_d'') - L_\ell (T_{d0}' + T_{d0}'')}{X_d - L_\ell} \\ c &= \frac{X_d (T_{d0}' T_{d0}'' - T_d' T_d'')}{(X_d - L_\ell)^2}, & d &= \frac{X_d (T_{d0}' + T_{d0}'' - T_d' - T_d'')}{(X_d - L_\ell)^2} \\ T_f &= \frac{b + \sqrt{b^2 - 4a}}{2}, & T_{d1} &= b - T_f \\ R_{d1} &= \frac{T_f - T_{d1}}{c - d T_{d1}}, & R_f &= \frac{-R_{d1}}{1 - d R_{d1}} \\ L_{\ell f} &= T_f R_f, & L_{\ell d1} &= T_{d1} R_{d1} \end{aligned}$$

d axis, single rotor circuit (XSD/TSD0 skipped)

$$L_{ff} = \frac{(M_d^u)^2}{X_d - X_d'}, \quad L_{\ell f} = L_{ff} - M_d^u, \quad R_f = \frac{L_{ff}}{T_{d0}'}$$

q axis is fully symmetric to the d axis. Two circuits (XPQ/TPQ0 and XSQ/TSQ0) use the same quadratic with $X_q, L_\ell, T_{q0}', T_{q0}'', X_q', X_q''$. Single-circuit fallbacks:

- Transient only (XPQ/TPQ0): $L_{\ell q1} = M_q^{u2}/(X_q - X_q') - M_q^u$, $R_{q1} = (M_q^{u2}/(X_q - X_q'))/T_{q0}'$
- Subtransient only (XSQ/TSQ0): same with X_q'', T_{q0}'' , result assigned to $L_{\ell q2}, R_{q2}$

Field base scaling

$$p_{uf} = R_f / \text{IBRATIO}$$

The field current is then reconstructed internally as $i_f = (\psi_f - \psi_{ad}) \cdot (R_f / p_{uf}) / L_{\ell f}$.

Sanity check

If any of $R_f, R_{d1}, L_{\ell f}, L_{\ell d1}$ (or q-axis equivalents) turns out negative, the supplied reactances/time constants are physically inconsistent. STEPSS emits an “unrealistic Park inductances or resistances” warning.

18.3. Reference Python Implementation

A standalone Python port of the XT branch is available at `Sync_mach_Octave` (Octave) and as `ramses_xt_to_park.py` (attached below). It reproduces the algorithm above including the t_b normalisation, the quadratic solve, the symmetric q axis, and $puf = RF/IBRATIO$. Pass `None` for any rotor circuit the machine does not have.

```
from ramses_xt_to_park import ramses_xt_to_park

p = ramses_xt_to_park(
    fnom=50.0, ibratio=1.0,
    ll=0.15, ra=0.003,
    xd=1.81, xpd=0.30, xsd=0.23, tpd=8.0, tsd=0.03,
    xq=1.76, xpq=0.65, xsq=0.25, tpq=1.0, tsq=0.07)
# -> llf=0.169902, lld1=0.166338, rf=0.000741, rd1=0.033390, ...
```

18.4. Worked Examples

Inputs (both cases, 50 Hz, IBRATIO = 1): $L_\ell = 0.15$, $R_a = 0.003$, $X_d = 1.81$, $X'_d = 0.30$, $X''_d = 0.23$, $T'_{d0} = 8.0$ s, $T''_{d0} = 0.03$ s, $X_q = 1.76$, $X'_q = 0.65$, $T'_{q0} = 1.0$ s. The round-rotor case adds $X''_q = 0.25$, $T''_{q0} = 0.07$ s.

Parameter	Round rotor (d2/q2)	Single q-damper (d2/q1)
M_d^u	1.660000	1.660000
$L_{\ell f}$	0.169902	0.169902
$L_{\ell d1}$	0.166338	0.166338
M_q^u	1.610000	1.610000
$L_{\ell q1}$	0.928153	0.725225
$L_{\ell q2}$	0.120461	n/a
R_f	0.000741	0.000741
R_{d1}	0.033390	0.033390
R_{q1}	0.009236	0.007433
R_{q2}	0.028210	n/a
p_{uf}	0.000741	0.000741

18.5. Comparing with an EMT Simulator

When cross-checking STEPSS (RMS/phasor) against an EMT tool such as Typhoon HIL:

1. **Per unit vs physical units.** STEPSS stays entirely in per unit. Converting to Henry/Ohm for the EMT tool requires a consistent per-unit base on the rotor side; use the same IBRATIO assumption on both sides.
2. **Radians vs seconds.** The t_b normalisation is internal to STEPSS. Your hand calculation must divide every time constant by $t_b = 1/(2\pi f_{nom})$ before computing resistances.
3. **Expected post-fault difference.** STEPSS uses the phasor approximation and neglects stator transformer voltages ($d\psi_d/dt$, $d\psi_q/dt$), so it omits the DC-offset and high-frequency

current components immediately after a short circuit. An EMT model retains them. Compare the **slow post-fault envelopes** first: envelope agreement with first-cycle differences indicates a modelling assumption, not a parameter error.

18.6. See Also

- Synchronous Machine Model, equations, per unit system, and SYNC_MACH record reference
- Octave reference implementation
- Phasor approximation
- Synchronous machine dynamics

IEEE excitation controllers

These excitation system models implement the IEEE Std 421.5-2016 “IEEE Recommended Practice for Excitation System Models for Power System Stability Studies” (and selected ENTSO-E variants) within RAMSES. Each model is defined using the CODEGEN domain-specific language (DSL) and compiled into Fortran for time-domain simulation. The DSL describes block diagrams as interconnected transfer function primitives (`tf1p`, `tf1p1z`, `tf1plim`, `tfder1p`, `inlim`, `pict1`, etc.), initial conditions, and algebraic constraints, forming a self-contained, portable model definition.

Usage in Dynamic Data Files

Exciter models in RAMSES are defined as sub-records within a `SYNC_MACH` block using the `EXC` keyword, followed by the model name and its parameters. They are not standalone records. The `EXC` line appears after the machine parameters and before the `TOR` (torque/governor) line. The entire block ends with a single `;` after the `TOR` record.

```

SYNC_MACH name bus FP FQ P Q SNOM Pnom H D IBRATIO
          XT Xl Xd X'd X''d Xq X'q X''q m n Ra T'do T''do T'qo
          ↪ T''qo
          EXC model_name param1 param2 ...
          TOR model_name param1 param2 ... ;

```

RAMSES adds the `exc_` prefix to the model name automatically, so both `AC1A` and `exc_AC1A` are accepted.

The **State** column of the Model Index says whether a model is compiled and callable, or only a DSL description:

- **Registered**, callable from a data file as it stands. Every compiled exciter has been registered since 3.79; before that, fifteen of them were mapped to no name and RAMSES rejected them in a data file.
- **DSL only**, present as a CODEGEN model description but not as compiled Fortran. `ENTSOE_lim` ships as a ready-made example in the `URAMES custom_models/` directory; `AC8B_lim` is listed for reference only and is not distributed.

The Model Reference index carries the same information across all model families.

19.1. Model Index

Model Name	State	Base Type	PSS	OEL / Limiter	IEEE Reference
AC1A	Registered	AC type 1			IEEE 421.5 Type AC1A
AC1A_MAXEX2	Registered	AC type 1		MAXEX2 field current limiter	IEEE 421.5 Type AC1A
AC1A_RETRO	Registered	AC type 1 (retrofit)	PSS4B (inter- nal)		
AC1A_RETRO_PSS4B	Registered	AC type 1 (retrofit)	PSS4B		
AC4A	Registered	AC type 4			IEEE 421.5 Type AC4A
AC8B	Registered	AC type 8			IEEE 421.5 Type AC8B
AC8B_PSS3B_lim	Registered	AC type 8	PSS3B	Integral OEL + SCL	IEEE 421.5 Type AC8B
AC8B_lim	DSL only	AC type 8		Integral OEL + SCL	IEEE 421.5 Type AC8B
DC3A	Registered	DC type 3			IEEE 421.5 Type DC3A
ENTSOE_simp	Registered	ENTSO-E simplified	IEEEEST (inter- nal)		ENTSO-E
ENTSOE_lim	DSL only	ENTSO-E simplified	IEEEEST (inter- nal)	Integral OEL	ENTSO-E
EXPIC1	Registered	AC/ST (PIC type)			
EXPIC1_PSS2B	Registered	AC/ST (PIC type)	PSS2B		
EXPIC1_PSS2B_- MAXEX2	Registered	AC/ST (PIC type)	PSS2B	MAXEX2	
IEEET5	Registered	DC type 5			IEEE 421.5 Type DC5A (legacy)
SEXS	Registered	ST simplified			CIGRÉ simplified
SEXS_IEEEST	Registered	ST simplified	IEEEEST		CIGRÉ simplified
SEXS_STAB3_lim	Registered	ST simplified	STAB3	Integral OEL	
ST1A	Registered	ST type 1			IEEE 421.5 Type ST1A
ST1A_IEEEST	Registered	ST type 1	IEEEEST		IEEE 421.5 Type ST1A
ST1A_IEEEST_- MAXEX2	Registered	ST type 1	IEEEEST	MAXEX2	IEEE 421.5 Type ST1A
ST1A_PSS2B	Registered	ST type 1	PSS2B		IEEE 421.5 Type ST1A

Model Name	State	Base Type	PSS	OEL / Limiter	IEEE Reference
ST1A_PSS2B_MAXEX2	Registered	ST type 1	PSS2B	MAXEX2	IEEE 421.5 Type ST1A
ST1A_PSS3B	Registered	ST type 1	PSS3B		IEEE 421.5 Type ST1A
ST1A_PSS4B	Registered	ST type 1	PSS4B		IEEE 421.5 Type ST1A
ST1A_PSS4B_MAXEX2	Registered	ST type 1	PSS4B	MAXEX2	IEEE 421.5 Type ST1A
ST1A_lim	Registered	ST type 1		Integral OEL + SCL	IEEE 421.5 Type ST1A
ST2A	Registered	ST type 2			IEEE 421.5 Type ST2A (via EXPIC1)

19.2. AC Exciter Models

19.2.1. AC1A

The AC1A is a rotating AC exciter with a self-excited field winding and non-linear saturation. It corresponds to IEEE Std 421.5-2016 Type AC1A. The generator terminal voltage V_t is compensated for line drop and filtered before being compared with the reference. An AC alternator supplies the field through a rectifier whose output depends on the commutation voltage V_E and field current I_{fd} .

Mathematical Description

The voltage regulator processes the error signal through a lead-lag network and a limited amplifier:

$$\Delta V = V_{REF} - V_c - V_F - \Delta V_{PSS}$$

$$V_1(s) = \Delta V \cdot \frac{1 + sT_C}{1 + sT_B}$$

$$V_A(s) = V_1 \cdot \frac{K_A}{1 + sT_A}, \quad V_{AMIN} \leq V_A \leq V_{AMAX}$$

The AC exciter integrator with saturation and demagnetization:

$$V_{FE} = K_D \cdot I_{fd} + (K_E + S_E(V_E)) V_E$$

$$\dot{V}_E = \frac{1}{T_E} (V_R - V_{FE}), \quad 0 \leq V_E$$

$$E_{fd} = V_E \cdot \left(1 - \frac{K_C \cdot I_{fd}}{V_E} \right)$$

The derivative feedback (rate feedback) path:

$$V_F(s) = E_{fd} \cdot \frac{sK_F/T_F}{1 + sT_F}$$

where $S_E(V_E)$ is the saturation function interpolated from the two-point saturation characteristic (V_{E1}, SE_1) and (V_{E2}, SE_2) .

Signal Flow

1. **Line drop compensation** (algeq): $V_{c1} = vcomp(V_t, P, Q, K_v, R_c, X_c)$
2. **Voltage measurement filter** (tf1p): $V_c \leftarrow V_{c1}$ with time constant T_R
3. **AVR summation** (algeq): $\Delta V = V_{REF} - V_c - V_F$
4. **Lead-lag compensator** (tf1p1z): $V_1 \leftarrow \Delta V$ with (T_C, T_B)
5. **Amplifier with limits** (tf1plim): $V_2 \leftarrow V_1$ with gain K_A , time constant T_A , limits $[V_{AMIN}, V_{AMAX}]$
6. **UEL high-value gate** (max1v1c): $V_3 = \max(V_2, V_{UEL})$
7. **OEL low-value gate** (min1v1c): $V_4 = \min(V_3, V_{OEL})$
8. **Output limiter** (lim): $V_R = \text{clamp}(V_4, V_{RMIN}, V_{RMAX})$
9. **Exciter integrator** (inlim): $V_E \leftarrow (V_R - V_{FE})/T_E$
10. **Saturation computation** (algeq): V_{FE} from $K_D, K_E, S_E(V_E)$
11. **Rectifier output** (algeq): $E_{fd} = vrectif(I_{fd}, V_E, K_C)$
12. **Derivative feedback** (tfder1p): $V_F \leftarrow V_{FE}$ with gain K_F/T_F , time constant T_F

Parameters

Parameter	Description	Unit/Type
Kv	Voltage compensation gain	pu
Rc	Resistance for line drop compensation	pu
Xc	Reactance for line drop compensation	pu
TR	Voltage transducer time constant	s
KA	Voltage regulator gain	pu
TA	Voltage regulator time constant	s
TB	Lead-lag denominator time constant	s
TC	Lead-lag numerator time constant	s
VAMAX	Maximum regulator output	pu
VAMIN	Minimum regulator output	pu
KE	Exciter constant (self-excitation term)	pu
TE	Exciter time constant	s
KF	Rate feedback gain	pu
TF	Rate feedback time constant	s
VRMAX	Maximum field voltage	pu
VRMIN	Minimum field voltage	pu
VE1	Saturation factor voltage point 1	pu
SE1	Saturation factor at V_{E1}	pu
VE2	Saturation factor voltage point 2	pu
SE2	Saturation factor at V_{E2}	pu
KC	Commutation factor (rectifier regulation)	pu
KD	Demagnetization factor	pu
VUEL	UEL input signal	pu

Variants

Model	Description
AC1A	Base model, IEEE Type AC1A
AC1A_MAXEX2	Adds MAXEX2 field current limiter; VOEL moves to reference summation point (LV gate removed)
AC1A_RETRO	Retrofit variant with internal PSS4B; uses first-order AVR without separate exciter block

Model	Description
AC1A_RETRO_PSS4B	Retrofit variant with external PSS4B speed input

Usage Example

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
                XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
                ↪ 0.05 * 0.1
EXC AC1A 0. 0. 0. 0.02 200.0 0.02 10.0 1.0 7.32 -7.32 1.0 1.0 0.03
↪ 1.0 6.03 -5.43 4.18 0.1 3.14 0.033 0.2 0.38 99999
TOR CONSTANT ;

```

19.2.2. AC4A

The AC4A is a simplified static representation of an AC exciter system with an inner-loop voltage-dependent output limit. It corresponds to IEEE Std 421.5-2016 Type AC4A. The upper output limit is a function of field current: $V_{RMAX,eff} = V_{RMAX} - K_C \cdot I_{fd}$.

Mathematical Description

$$\Delta V = V_{REF} - V_c$$

$$V_I = \text{clamp}(\Delta V, V_{IMIN}, V_{IMAX})$$

$$V_1(s) = V_I \cdot \frac{1 + sT_C}{1 + sT_B}$$

$$E_{fd}(s) = V_1 \cdot \frac{K_A}{1 + sT_A}, \quad V_{RMIN} \leq E_{fd} \leq V_{RMAX} - K_C \cdot I_{fd}$$

The variable upper limit is updated algebraically at each time step, making AC4A a time-varying limited integrator system.

Signal Flow

1. **Line drop compensation** (algeq): V_{c1} from terminal measurements
2. **Voltage measurement filter** (tf1p): V_c with T_R
3. **AVR summation** (algeq): $\Delta V = V_{REF} - V_c$
4. **Input limiter** (lim): $V_I = \text{clamp}(\Delta V, V_{IMIN}, V_{IMAX})$
5. **Lead-lag** (tf1p1z): $V_1 \leftarrow V_I$ with (T_C, T_B)
6. **UEL high-value gate** (max1v1c): $V_2 = \max(V_1, V_{UEL})$
7. **Variable upper limit** (algeq): $uplim = V_{RMAX} - K_C \cdot I_{fd}$
8. **Variable-limit amplifier** (tf1pvlim): $E_{fd} \leftarrow V_2$ with K_A, T_A , limits $[V_{RMIN}, uplim]$

Parameters

Parameter	Description	Unit/Type
Kv	Voltage compensation gain	pu
Rc	Line drop compensation resistance	pu
Xc	Line drop compensation reactance	pu
TR	Measurement filter time constant	s
VIMAX	Maximum input error signal	pu
VIMIN	Minimum input error signal	pu

Parameter	Description	Unit/Type
TC	Lead numerator time constant	s
TB	Lag denominator time constant	s
VUEL	UEL signal value	pu
KA	Regulator gain	pu
TA	Regulator time constant	s
VRMAX	Maximum output (at zero field current)	pu
VRMIN	Minimum output	pu
KC	Field current derating factor	pu

Variants

Model	Description
AC4A	Base model, IEEE Type AC4A, no PSS or OEL

Usage Example

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
               XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
               ↪ 0.05 * 0.1
EXC AC4A 0. 0. 0. 0.02 8.0 -8.0 1.0 10.0 0.0 200.0 0.015 4.44 -4.44
          ↪ 0.0
TOR CONSTANT ;

```

19.2.3. AC8B

The AC8B is a rotating AC exciter with a PID voltage regulator (rather than the proportional-integral amplifier of earlier types). It corresponds to IEEE Std 421.5-2016 Type AC8B. The PID structure, proportional gain K_{PR} , integral gain K_{IR} , and derivative gain K_{DR} , provides improved transient response compared to Type AC1A.

Mathematical Description

The PID voltage regulator:

$$\Delta V = V_c - V_{REF}$$

$$V_{PID}(s) = K_{PR} \cdot \Delta V + \frac{K_{IR}}{s} \cdot \Delta V + \frac{sK_{DR}/T_{DR}}{1 + sT_{DR}} \cdot \Delta V$$

$$V_R(s) = V_{PID} \cdot \frac{K_A}{1 + sT_A}, \quad V_{RMIN} \leq V_R \leq V_{RMAX}$$

The AC exciter with variable limits based on maximum field excitation voltage V_{FEMAX} :

$$V_{FE} = K_D \cdot I_{fd} + (K_E + S_E(V_E)) V_E$$

$$\dot{V}_E = \frac{1}{T_E} (V_R - V_{FE}), \quad V_{EMIN} \leq V_E \leq \frac{V_{FEMAX} - K_D I_{fd}}{K_E + S_E(V_E)}$$

Signal Flow

1. **Voltage measurement** (tf1p): V_c with T_R
2. **Error signal** (algeq): $\Delta V = V_c - V_{REF}$
3. **PID derivative part** (tfder1p): $x_{der} \leftarrow \Delta V$ with gain K_{DR}/T_{DR}
4. **PID proportional-integral** (pictl): $x_{propint} \leftarrow \Delta V$ with K_{IR}, K_{PR}
5. **PID sum** (algeq): $x_{pid} = x_{propint} + x_{der}$, limited to $[V_{PIDMIN}, V_{PIDMAX}]$
6. **Amplifier** (tf1plim): $V_R \leftarrow x_{pid}$ with K_A, T_A
7. **Exciter integrator** (invlim): V_E with variable limits from V_{FEMAX}
8. **Saturation and rectification** (algeq): V_{FE}, E_{fd}

Parameters

Parameter	Description	Unit/Type
Kv	Voltage compensation gain	pu
Rc	Line drop resistance	pu
Xc	Line drop reactance	pu
TR	Measurement time constant	s
KPR	PID proportional gain	pu
KIR	PID integral gain	pu/s
KDR	PID derivative gain	pu·s
TDR	PID derivative filter time constant	s
VPIDMAX	PID output upper limit	pu
VPIDMIN	PID output lower limit	pu
KA	Amplifier gain	pu
TA	Amplifier time constant	s
VRMAX	Maximum regulator output	pu
VRMIN	Minimum regulator output	pu
KC	Rectifier commutation factor	pu
KD	Demagnetization factor	pu
KE	Exciter self-excitation constant	pu
TE	Exciter time constant	s
VFEMAX	Maximum field excitation voltage	pu
VEMIN	Minimum exciter output	pu
VE1	Saturation voltage point 1	pu
SE1	Saturation factor at V_{E1}	pu
VE2	Saturation voltage point 2	pu
SE2	Saturation factor at V_{E2}	pu

Variants

Model	Description
AC8B	Base model, IEEE Type AC8B
AC8B_PSS3B_lim	Adds PSS3B dual-input stabilizer and integral OEL + SCL (stator current limiter)
AC8B_lim	Adds integral OEL and SCL without PSS

Usage Example

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
          XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
          ↪ 0.05 * 0.1
EXC AC8B 0. 0. 0. 0.02 1.0 5.0 0.1 0.1 8.0 -8.0 1.0 0.1 6.5 -6.5 0.2
          ↪ 1.0 1.0 0.5 6.5 0.0 4.0 0.3 3.0 0.33
TOR CONSTANT ;

```

19.3. DC Exciter Models

19.3.1. DC3A

The DC3A represents a non-continuously acting (rheostat-type) DC excitation system. It corresponds to IEEE Std 421.5-2016 Type DC3A. Rather than a proportional amplifier, it uses a three-position switch (VRMIN / VRH / VRMAX) driven by an integrator that holds the rheostat position V_{RH} . The switch position is determined by whether the error signal V_{ERR} falls inside or outside a deadband $\pm K_V$.

Mathematical Description

$$V_{ERR} = V_{REF} - V_c$$

The rheostat integrator:

$$\dot{V}_{RH} = \frac{K_V \cdot T_{RH}}{V_{RMAX} - V_{RMIN}} \cdot \text{clamp}(V_{ERR}, -K_V, K_V), \quad V_{RMIN} \leq V_{RH} \leq V_{RMAX}$$

The three-position switch:

$$V_R = \begin{cases} V_{RMIN} & V_{ERR} < -K_V \\ V_{RH} & |V_{ERR}| \leq K_V \\ V_{RMAX} & V_{ERR} > K_V \end{cases}$$

The DC exciter with saturation:

$$\dot{E}_{fd} = \frac{1}{T_E} [V_R - (K_E + S_E(E_{fd})) E_{fd}]$$

Parameters

Parameter	Description	Unit/Type
Kvcomp	Voltage compensation gain	pu
Rc	Line drop resistance	pu
Xc	Line drop reactance	pu
TR	Measurement time constant	s
TRH	Rheostat traversal time	s
KV	Sensitivity of non-continuously acting controller	pu
VRMAX	Maximum field voltage	pu
VRMIN	Minimum field voltage	pu
TE	Exciter time constant	s
KE	Exciter self-excitation constant	pu
E1	Saturation voltage point 1	pu
SE1	Saturation factor at E_1	pu
E2	Saturation voltage point 2	pu
SE2	Saturation factor at E_2	pu

Variants

Model	Description
DC3A	Base model, IEEE Type DC3A, non-continuously acting rheostat controller

Usage Example

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
           XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
           ↪ 0.05 * 0.1
EXC DC3A 0. 0. 0. 0.0 1.0 1.0 1.0 -1.0 0.8 1.0 3.1 0.33 2.3 0.1
TOR CONSTANT ;

```

19.4. ST (Static) Exciter Models**19.4.1. ST1A**

The ST1A is a static (thyristor) excitation system with two cascaded lead-lag networks in the voltage regulator path and an inner field current limiter (KLR/ILR). It corresponds to IEEE Std 421.5-2016 Type ST1A. The output limits are proportional to terminal voltage V_t , and field current derating applies through K_C : $uplim = V_t \cdot V_{RMAX} - K_C \cdot I_{fd}$.

Mathematical Description

The voltage regulator with dual lead-lag and instantaneous field current limiting:

$$\Delta V = V_{REF} - V_c + V_{PSS} + V_{UEL,path} + V_F$$

$$V_1 = \text{clamp}(\Delta V, V_{IMIN}, V_{IMAX})$$

$$V_2(s) = V_1 \cdot \frac{1 + sT_C}{1 + sT_B}$$

$$V_3(s) = V_2 \cdot \frac{1 + sT_{C1}}{1 + sT_{B1}}$$

$$V_A(s) = V_3 \cdot \frac{K_A}{1 + sT_A}, \quad V_{AMIN} \leq V_A \leq V_{AMAX}$$

Instantaneous field current limiter:

$$V_{LR} = K_{LR}(I_{fd} - I_{LR})$$

$$V_5 = V_A - \max(V_{LR}, 0)$$

Final output with terminal-voltage-dependent limits:

$$E_{fd} = \text{clamp}(V_5, V_t \cdot V_{RMIN}, V_t \cdot V_{RMAX} - K_C \cdot I_{fd})$$

Derivative feedback:

$$V_F(s) = E_{fd} \cdot \frac{sK_F/T_F}{1 + sT_F}$$

Signal Flow

1. **Line drop** (algeq): V_{c1}
2. **Measurement filter** (tf1p): V_c with T_R
3. **AVR summation** (algeq): ΔV , including UEL (mode 1), PSS, and VF
4. **Error limiter** (lim): $V_1 \in [V_{IMIN}, V_{IMAX}]$
5. **UEL gate (mode 2)** (max1v1c): HV gate V_2
6. **First lead-lag** (tf1p1z): V_3 with (T_C, T_B)
7. **Second lead-lag** (tf1p1z): V_4 with (T_{C1}, T_{B1})
8. **Amplifier** (tf1plim): V_A with $K_A, T_A, [V_{AMIN}, V_{AMAX}]$
9. **Field current limiter** (algeq, lim): V_{LR}, V_{LRlim}
10. **Limiter subtraction** (algeq): $V_5 = V_A - V_{LRlim}$
11. **UEL gate (mode 3)** (max1v1c): V_6
12. **OEL gate** (min2v): $V_7 = \min(V_6, V_{OEL})$
13. **Variable-limit output** (limvb): E_{fd} with terminal-voltage-scaled limits
14. **Derivative feedback** (tfder1p): V_F

Parameters

Parameter	Description	Unit/Type
Kv	Voltage compensation gain	pu
Rc	Line drop resistance	pu
Xc	Line drop reactance	pu
TR	Measurement filter time constant	s
UEL	UEL connection mode (1=summation, 2=HV gate after V1, 3=HV gate after VA)	integer
VIMIN	Input error lower limit	pu
VIMAX	Input error upper limit	pu
VUEL	UEL signal value	pu
TC	First lead-lag numerator	s
TB	First lead-lag denominator	s
TC1	Second lead-lag numerator	s
TB1	Second lead-lag denominator	s
KA	Regulator gain	pu
TA	Regulator time constant	s
VAMIN	Regulator lower limit	pu
VAMAX	Regulator upper limit	pu
VRMIN	Output lower limit scaling factor	pu
VRMAX	Output upper limit scaling factor	pu
KC	Field current derating factor	pu
KF	Rate feedback gain	pu
TF	Rate feedback time constant	s
KLR	Field current limiter gain	pu
ILR	Field current limiter reference	pu

Variants

Model	Description
ST1A	Base model, IEEE Type ST1A
ST1A_IEEEEST	Adds IEEEEST single-input PSS (speed or power input, selectable)
ST1A_IEEEEST_MAXEX2	Adds IEEEEST PSS + MAXEX2 field current limiter
ST1A_PSS2B	Adds PSS2B dual-input stabilizer
ST1A_PSS2B_MAXEX2	Adds PSS2B + MAXEX2

Model	Description
ST1A_PSS3B	Adds PSS3B two-channel second-order filter stabilizer
ST1A_PSS4B	Adds PSS4B three-band stabilizer
ST1A_PSS4B_MAXEX2	Adds PSS4B + MAXEX2
ST1A_lim	Integral OEL + stator current limiter (SCL), simplified structure without Kv/Rc/Xc

Usage Example

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
          XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
          ↪ 0.05 * 0.1
EXC ST1A 0. 0. 0. 0.02 1 -0.87 0.87 0.0 1.0 10.0 0.0 1.0 200.0 0.001
↪ 7.8 -6.7 0.038 0.0 999.0 0.0 99.0 0.0 0.0
TOR CONSTANT ;

```

19.4.2. ST2A (via EXPIC1)

The EXPIC1 model implements a static exciter with an AC rotating bus-fed rectifier and an integrating (PI-type) amplifier path: $K_A(1 + sT_A)/s$. The output V_B is proportional to the AC voltage phasor magnitude, creating a bus-fed topology analogous to IEEE Type ST2A. When $K_P = K_I = 0$, the rectifier gain is unity.

Mathematical Description

The AC exciter voltage and rectifier:

$$V_E = |K_P V_t + jK_I V_t|, \quad V_B = V_E \cdot \left(1 - \frac{K_C I_{fd}}{V_E}\right)^+$$

The integrating amplifier:

$$G_{amp}(s) = K_A \cdot \frac{1 + sT_A}{s}$$

The field voltage integrator:

$$\dot{E}_{fd} = \frac{1}{T_E} [V_R \cdot V_B - K_E \cdot E_{fd}], \quad E_{FDMIN} \leq E_{fd} \leq E_{FDMAX}$$

Signal Flow

1. **Voltage measurement** (tf1p): V_c with T_R
2. **Summation** (algeq): $\Delta V = V_{REF} - V_c - V_F - \Delta V_{PSS}$
3. **UEL gate** (max1v1c): clamped to V_{UEL}
4. **PI amplifier** (pict1): V_A with $K_{gain} = K_A \cdot T_A$ and K_A
5. **Output limiter** (lim): $V_R \in [V_{RMIN}, V_{RMAX}]$
6. **Rectifier voltage** (algeq): V_E, V_B
7. **Field integrator** (inlim): E_{fd} from $V_R \cdot V_B - K_E \cdot E_{fd}$
8. **Derivative feedback** (tfder1p): V_F

Parameters

Parameter	Description	Unit/Type
Kvcomp	Voltage compensation gain	pu
Rc	Line drop resistance	pu
Xc	Line drop reactance	pu
TR	Measurement time constant	s
VUEL	UEL signal	pu
KA	Amplifier gain	pu
TA	Amplifier time constant	s
VRMAX	Maximum regulator output	pu
VRMIN	Minimum regulator output	pu
KF	Rate feedback gain	pu
TF	Rate feedback time constant	s
KE	Exciter self-excitation constant	pu
EFDMAX	Maximum field voltage	pu
EFDMIN	Minimum field voltage	pu
TE	Field integrator time constant	s
KP	AC source voltage proportional factor	pu
KI	AC source voltage quadrature factor	pu
KC	Commutation voltage drop factor	pu

Variants

Model	Description
EXPIC1	Base bus-fed static exciter
EXPIC1_PSS2B	Adds PSS2B dual-input stabilizer
EXPIC1_PSS2B_MAXEX2	Adds PSS2B + MAXEX2 field current limiter

Usage Example

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
          XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
          ↪ 0.05 * 0.1
EXC EXPIC1 0. 0. 0. 0.02 0.0 200.0 0.02 99.0 -99.0 0.0 1.0 1.0 8.0
↪ -8.0 0.5 0.0 1.0 0.0
TOR CONSTANT ;

```

19.5. Other IEEE Models

19.5.1. IEEE T5

The IEEE T5 (also referenced as DC5A in some conventions) is a non-continuously acting exciter similar to DC3A but uses an inner limiter block `lim_civ` that gates the integrator holding signal within a deadband $\pm K_V$. This creates the three-state (hold/raise/lower) behavior of rheostatic controllers.

Mathematical Description

$$V_{ERR} = V_{REF} - V_c$$

The integrator for rheostat position:

$$\dot{V}_{RH} = \frac{1}{T_{RH}} \cdot V_{ERR}, \quad V_{RMIN} \leq V_{RH} \leq V_{RMAX}$$

The `lim_civ` block implements the deadband gating so that integration is active only when $|V_{ERR}| > K_V$:

$$V_R = \begin{cases} V_{RMIN} & V_{ERR} < -K_V \\ V_{RH} & |V_{ERR}| \leq K_V \\ V_{RMAX} & V_{ERR} > K_V \end{cases}$$

$$\dot{E}_{fd} = \frac{1}{T_E} [V_R - (K_E + S_E(E_{fd})) E_{fd}]$$

Parameters

Parameter	Description	Unit/Type
Kvcomp	Voltage compensation gain	pu
Rc	Line drop resistance	pu
Xc	Line drop reactance	pu
TRH	Rheostat time	s
KV	Controller sensitivity deadband	pu
VRMAX	Maximum field voltage	pu
VRMIN	Minimum field voltage	pu
TE	Exciter integrator time constant	s
KE	Exciter self-excitation constant	pu
E1	Saturation point 1	pu
SE1	Saturation factor at E_1	pu
E2	Saturation point 2	pu
SE2	Saturation factor at E_2	pu

Variants

Model	Description
IEEE5	Base model, non-continuously acting DC exciter

19.5.2. SEXS

The SEXS (Simplified Excitation System) is a minimal two-parameter static exciter model from CIGRÉ publications, widely used for cases where detailed exciter data is unavailable. It has no measurement filter, no line drop compensation, and no rate feedback, just a lead-lag network and a limited first-order block.

Mathematical Description

$$\Delta V_{AVR} = V_t - V_o + V_{PSS}$$

$$G_{lag}(s) = \frac{1 + sT_A}{1 + sT_B}$$

$$E_{fd}(s) = G_{lag}(\Delta V_{AVR}) \cdot \frac{K_E}{1 + sT_E}, \quad E_{MIN} \leq E_{fd} \leq E_{MAX}$$

where $V_o = V_t + E_{fd,0}/K_E$ is computed from initial conditions to achieve zero initial error.

Parameters

Parameter	Description	Unit/Type
TA	Lead-lag numerator time constant	s
TB	Lead-lag denominator time constant	s
KE	Exciter gain	pu
TE	Exciter time constant	s
EMIN	Minimum field voltage	pu
EMAX	Maximum field voltage	pu

Variants

Model	Description
SEXS	Base simplified exciter
SEXS_IEEEST	Adds IEEEST PSS (multi-mode: speed, power, or accelerating power input)
SEXS_STAB3_lim	Adds STAB3 PSS + integral OEL

Usage Example

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
              XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
              ↪ 0.05 * 0.1
EXC SEXS 0.1 10.0 25.0 0.5 -5.0 5.0
TOR CONSTANT ;

```

19.6. ENTSO-E Models**19.6.1. ENTSGE_simp**

The ENTSO-E simplified exciter combines a built-in speed-signal PSS with a simple lead-lag AVR and first-order exciter block. It is the standard ENTSO-E model for dynamic studies where detailed exciter data is unavailable.

Mathematical Description

The PSS (speed input $\omega - 1$) uses two washout stages and two lead-lag stages:

$$V_{PSS}(s) = (\omega - 1) \cdot \frac{sT_{W1}}{1 + sT_{W1}} \cdot \frac{sT_{W2}}{1 + sT_{W2}} \cdot K_{S1} \cdot \frac{1 + sT_1}{1 + sT_2} \cdot \frac{1 + sT_3}{1 + sT_4}$$

The AVR:

$$\Delta V_{AVR} = V_t - V_o + V_{PSS}$$

$$G_{lag}(s) = \frac{1 + sT_A}{1 + sT_B}$$

$$E_{fd}(s) = G_{lag}(\Delta V_{AVR}) \cdot \frac{K_E}{1 + sT_E}, \quad E_{MIN} \leq E_{fd} \leq E_{MAX}$$

Parameters

Parameter	Description	Unit/Type
TW1	First washout time constant	s

Parameter	Description	Unit/Type
TW2	Second washout time constant	s
KS1	PSS gain	pu
T1	First lead-lag numerator	s
T2	First lead-lag denominator	s
T3	Second lead-lag numerator	s
T4	Second lead-lag denominator	s
VSTMIN	PSS output lower limit	pu
VSTMAX	PSS output upper limit	pu
TA	AVR lead-lag numerator	s
TB	AVR lead-lag denominator	s
KE	Exciter gain	pu
TE	Exciter time constant	s
EMIN	Minimum field voltage	pu
EMAX	Maximum field voltage	pu

Variants

Model	Description
ENTSOE_simp	Base ENTSO-E simplified model with integrated PSS
ENTSOE_lim	Adds integral OEL (same structure as AC8B_lim OEL)

Usage Example

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
          XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
          ↪ 0.05 * 0.1
EXC ENTSOE_simp 10.0 3.0 10.0 0.2 0.1 0.1 0.1 -0.05 0.05 0.05 0.1
↪ 25.0 1.0 -10.0 10.0
TOR CONSTANT ;

```

19.7. PSS Models

Power System Stabilizers (PSS) inject an additional signal V_{PSS} at the AVR summation point to damp electromechanical oscillations. All variants listed in the model index table include one of the PSS types described here.

19.7.1. PSS2B: Dual-Input Stabilizer

PSS2B accepts two input signals (typically speed ω and electrical power P_e) and processes them through independent washout chains before combining. It corresponds to IEEE Std 421.5-2016 Type PSS2B.

Differences from other vendors' PSS2B

Implementations of PSS2B vary, so parameter lists do not transfer between tools unchanged. Three differences are worth knowing when porting data in:

- RAMSES names the two transducer lags T_6 and T_7 , following IEEE 421.5. Some tools name the same two constants T_{W6} and T_{W7} , which is easy to misread as a third and fourth washout.
- RAMSES has no K_{S4} and no ramp-tracking filter (AA , T_A , T_B). Data sets carrying those come from an extended PSS2B and cannot be transferred field by field.

- RAMSES adds a leading speedinput selector that chooses the speed signal source. It has no counterpart elsewhere and must be supplied as the first stabiliser parameter.

$$V_{S1} = \text{clamp}(V_{SI1}, V_{S1MIN}, V_{S1MAX})$$

Channel 1 (input 1 washout):

$$V_{S1} \xrightarrow{sT_{W1}/(1+sT_{W1})} \xrightarrow{sT_{W2}/(1+sT_{W2})} \xrightarrow{1/(1+sT_6)} pss_3$$

Channel 2 (input 2 washout):

$$V_{S2} \xrightarrow{sT_{W3}/(1+sT_{W3})} \xrightarrow{sT_{W4}/(1+sT_{W4})} \xrightarrow{K_{S2}/(1+sT_7)} pss_6$$

Combination and lead-lag chain (variant M=5, N=1 implemented):

$$pss_7 = pss_3 - K_{S3} \cdot pss_6$$

$$pss_7 \xrightarrow{(1+sT_8)/(1+sT_9)} \xrightarrow{1/(1+sT_9)^4} pss_{12}$$

$$pss_{13} = pss_{12} + pss_6$$

$$pss_{13} \xrightarrow{K_{S1}(1+sT_1)/(1+sT_2)} \xrightarrow{(1+sT_3)/(1+sT_4)} \xrightarrow{(1+sT_{10})/(1+sT_{11})} V_{PSS}$$

$$V_{PSS} = \text{clamp}(V_{PSS,unlim}, V_{STMIN}, V_{STMAX})$$

PSS2B Parameters (additional to base exciter):

Parameter	Description
speedinput	Input selection (1=speed+power, 2=power+speed)
TW1, TW2	Channel 1 washout time constants
T6	Channel 1 filter
TW3, TW4	Channel 2 washout time constants
T7, KS2	Channel 2 filter gain and time constant
KS3	Channel coupling gain
T8, T9	Ramp-tracking filter
KS1	Overall PSS gain
T1–T4, T10, T11	Lead-lag time constants
VS1MIN, VS1MAX	Input 1 limits
VS2MIN, VS2MAX	Input 2 limits
VSTMIN, VSTMAX	Output limits

19.7.2. PSS3B: Two-Input Second-Order Filter Stabilizer

PSS3B processes two input channels through measurement filters and washout stages, then combines them and applies two cascaded second-order lead-lag filters. It corresponds to IEEE Std 421.5-2016 Type PSS3B.

$$V_{SI1} \xrightarrow{1/(1+sT_1)} \xrightarrow{K_{S1}sT_{W1}/(1+sT_{W1})} pss_2$$

$$V_{SI2} \xrightarrow{1/(1+sT_2)} \xrightarrow{K_{S2} s T_{W2}/(1+sT_{W2})} pss_4$$

$$pss_5 = pss_2 - pss_4$$

$$pss_5 \xrightarrow{K_S s T_{W3}/(sT_{W4}+1)} pss_6$$

$$pss_6 \xrightarrow{\text{TF1: } \frac{A_2+sA_1+s^2/A_4}{A_4+sA_3+s^2/A_4}} \xrightarrow{\text{TF2: } \frac{A_6+sA_5+s^2/A_8}{A_8+sA_7+s^2/A_8}} V_{PSS}$$

$$V_{PSS} = \text{clamp}(V_{PSS,unlim}, V_{STMIN}, V_{STMAX})$$

PSS3B Parameters (additional):

Parameter	Description
speedinput	Input mode selection
T1, KS1, TW1	Channel 1 filter, gain, washout
T2, KS2, TW2	Channel 2 filter, gain, washout
KS, TW3, TW4	Combined washout gain and time constants
A1–A8	Second-order filter coefficients
VSTMIN, VSTMAX	Output limits

19.7.3. PSS4B: Three-Band Multi-Input Stabilizer

PSS4B is a three-band stabilizer that separates the speed deviation signal into low-, intermediate-, and high-frequency components. Each band uses a pair of lead-lag filters. This model corresponds to IEEE Std 421.5-2016 Type PSS4B.

The speed signal $\Delta\omega = \omega - 1$ passes through a digital transducer (tf2p2z) to obtain the low/intermediate input. Electrical power P_e passes through two washouts and a low-pass filter to derive the high-frequency component $\Delta\omega_H$.

Low-frequency band ($\Delta\omega_{L-I}$ input):

$$pss_4 = \Delta\omega_{L-I} \cdot \frac{K_{L1}(K_{L11} + sT_{L1})}{1 + sT_{L2}}$$

$$pss_5 = \Delta\omega_{L-I} \cdot \frac{K_{L2}(K_{L17} + sT_{L7})}{1 + sT_{L8}}$$

$$V_{PSSL} = \text{clamp}(K_L(pss_4 - pss_5), V_{LMin}, V_{LMax})$$

Intermediate-frequency band (same $\Delta\omega_{L-I}$ input):

$$V_{PSSI} = \text{clamp}(K_I(pss_7 - pss_8), V_{IMin}, V_{IMax})$$

High-frequency band ($\Delta\omega_H$ input):

$$V_{PSSH} = \text{clamp}(K_H(pss_{10} - pss_{11}), V_{HMin}, V_{HMax})$$

$$V_{PSS} = \text{clamp}(V_{PSSL} + V_{PSSI} + V_{PSSH}, V_{STMin}, V_{STMax})$$

PSS4B Parameters (additional):

Parameter	Description
H	Generator inertia constant (for high-freq path)
KL1, KL11, TL1, TL2	Low-band filter 1
KL2, KL17, TL7, TL8	Low-band filter 2
KL, VLMax, VLMin	Low-band gain and limits
KI1, KI11, TI1, TI2	Intermediate-band filter 1
KI2, KI17, TI7, TI8	Intermediate-band filter 2
KI, VIMax, VIMin	Intermediate-band gain and limits
KH1, KH11, TH1, TH2	High-band filter 1
KH2, KH17, TH7, TH8	High-band filter 2
KH, VHMax, VHMin	High-band gain and limits
VSTMax, VSTMin	Total output limits

19.7.4. IEEEEST: Single-Input Stabilizer

The IEEEEST is a general-purpose single-input PSS with selectable input signal (speed deviation, electrical power, or computed accelerating power). It corresponds to IEEE Std 421.5-2016 Type PSS1A. Two second-order lead-lag filters provide frequency shaping before two first-order lead-lags and a washout stage.

$$\begin{aligned}
 VS_1 &\xrightarrow{\frac{1}{A_2+sA_1+s^2/(A_1A_2)}} \xrightarrow{\frac{A_6+sA_5+s^2/(A_3A_4)}{A_4+sA_3+s^2/(A_3A_4)}} VS_3 \\
 VS_3 &\xrightarrow{\frac{1+sT_1}{1+sT_2}} \xrightarrow{\frac{1+sT_3}{1+sT_4}} \xrightarrow{\frac{K_S s T_5 / T_6}{1+sT_6}} V_{PSS}
 \end{aligned}$$

$$V_{PSS} = \text{clamp}(V_{PSS,unlim}, V_{LSMIN}, V_{LSMAX})$$

Input selection (speedinput): 1 = rotor speed deviation, 3 = electrical power, 4 = computed accelerating power ($2H d\omega/dt$).

IEEEEST Parameters (additional):

Parameter	Description
speedinput	Input mode (1, 3, or 4)
A1–A6	Second-order filter coefficients
T1–T6	Lead-lag and washout time constants
KS	Overall PSS gain
LSMIN, LSMAX	Output limits
KS4	Inertia constant for accelerating power (= 2H)
TW4	Derivative filter time constant

19.7.5. STAB3: Power-Input Stabilizer

The STAB3 is a simplified PSS that takes electrical power P_e as input, applies two sequential first-order filters and a washout-type derivative block, producing a stabilizing signal.

$$P_e \xrightarrow{1/(1+sT_i)} stab_1 \xrightarrow{1/(1+sT_{X1})} stab_2 \xrightarrow{-K_X s / (1+sT_{X2})} stab_3$$

$$V_{PSS} = \text{clamp}(\text{stab}_3, -V_{LIM}, V_{LIM})$$

STAB3 Parameters:

Parameter	Description
Tt	Input measurement time constant
TX1	First filter time constant
KX	Stabilizer gain
TX2	Washout time constant
VLIM	Output limit

19.8. OEL and Limiter Models

Over-Excitation Limiters (OEL) prevent sustained field current overloads that would thermally damage the field winding. They generate a signal V_{OEL} that reduces the AVR output when field current exceeds thermal limits.

19.8.1. MAXEX2: Field Current Limiter with Timer and Hysteresis

The MAXEX2 is a field current limiter derived from IEEE work, using a simplified three-point piece-wise linear timer. It is designed for the EXST1/ST1A topology where the OEL output is injected at the reference summation point (not the LV gate).

Timer phase: The measured current $I_{fd,mes}$ (selected as either field current or E_{fd} via parameter IC) drives a timer3 block with three (I_{fd}, T) operating points. The hyst block latches when timer ≥ 1 .

Integral phase: Once latched, the error drives a limited integrator:

$$e_{IFD} = I_{fd,ref} - I_{fd,mes}$$

$$\dot{V}_{OEL} = \frac{e_{IFD}}{K_{MX}}, \quad V_{LOW} \leq V_{OEL} \leq 0$$

V_{OEL} is subtracted from the reference ($V_{REF} - V_{OEL}$, i.e. Negative reduces reference).

MAXEX2 Parameters:

Parameter	Description
IC	Input selection: 0 = E_{fd} , 1 = I_{fd}
IFDrated	Rated field current (reference)
IFD1, TIME1	Timer point 1 (current level, allowable duration)
IFD2, TIME2	Timer point 2
IFD3, TIME3	Timer point 3
IFDDES	Desired steady-state field current after limiting
KMX	Integrator time constant inverse ($1/KMX$ = integration rate)
VLOW	Lower limit of OEL output (e.g. -1.0 pu)

19.8.2. Integral OEL (_lim variants)

The simple integral OEL used in AC8B_lim, ST1A_lim, SEXS_STAB3_lim, and ENTISOE_lim variants computes the over-excitation signal through a clamped integrator acting on the difference between measured field current and $1.05 \cdot I_{FDN}$, with an input clamp at $0.35 \cdot I_{FDN}$.

$$\Delta I_{fd,1} = I_{fd} - 1.05 \cdot I_{FDN}$$

$$\Delta I_{fd,2} = \min(\Delta I_{fd,1}, 0.35 \cdot I_{FDN})$$

$$\dot{x}_{OEL} = \frac{\Delta I_{fd,2}}{T_{OEL}}, \quad L_{OEL} \leq x_{OEL} \leq U_{OEL}$$

$$V_{OEL} = \text{clamp}(K_{OEL} \cdot x_{OEL}, OEL_{LI}, 0)$$

A stator current limiter (SCL) with analogous structure acts on $I_{st} = \sqrt{P^2 + Q^2}/V_t$ relative to $1.025 \cdot I_{GN}$.

Integral OEL Parameters (in `_lim` variants):

Parameter	Description
IFDN	Nominal field current (OEL trigger at 1.05×)
TOEL	OEL integrator time constant
LOEL	OEL integrator lower limit
UOEL	OEL integrator upper limit
KOEL	OEL output gain
OELLI	OEL output lower limit
IGN	Nominal stator current (SCL trigger at 1.025×)
TSCL	SCL integrator time constant
LSCL	SCL integrator lower limit
USCL	SCL integrator upper limit
KSCL	SCL output gain
SCLLI	SCL output lower limit

For full documentation of the CODEGEN DSL primitives used in these models (`tf1p`, `tf1p1z`, `tf1plim`, `inlim`, `pictl`, etc.), see the CODEGEN Blocks reference.

20

Other excitation controllers

This page documents the custom excitation system models available in RAMSES. These models range from a trivial constant-voltage source to full-featured AVR with PSS and OEL. Models without a .txt definition file are implemented directly in Fortran and are described from their source headers.

Usage in Dynamic Data Files

Exciter models in RAMSES are defined as sub-records within a SYNC_MACH block using the EXC keyword, followed by the model name and its parameters. They are not standalone records. The EXC line appears after the machine parameters and before the TOR (torque/governor) line. The entire block ends with a single ; after the TOR record.

```
SYNC_MACH name bus FP FQ P Q SNOM Pnom H D IBRATIO
          XT Xl Xd X'd X''d Xq X'q X''q m n Ra T'do T''do T'qo
          ↪ T''qo
          EXC model_name param1 param2 ...
          TOR model_name param1 param2 ... ;
```

This page documents the exciters that are not IEEE-standard models: CONSTANT, 1ST-ORDER, GENERIC1 and GENERIC2 (built in, uppercase, no prefix), plus kundur, GENERIC and AVR_DG (registered, and RAMSES adds the exc_ prefix automatically).

The IEEE-standard exciters are on the IEEE Exciter Models page. For the whole catalogue at a glance, including which models are compiled but not callable by name, see the Model Reference index.

20.1. CONSTANT (exc_constant)

The data-file model name is CONSTANT.

20.1.1. Scientific Description

The exc_constant model provides a **constant field voltage** source. The field voltage V_f is fixed at its load-flow initialisation value and does not respond to any input:

$$V_f(t) = V_{f0} \quad \forall t$$

The single algebraic state satisfies:

$$f(1) = x(1) - V_{f0} = 0$$

This model is useful for open-loop tests, for machines that are not equipped with a voltage regulator, or as a placeholder during model development.

Implementation details (from Fortran source): - nbxvar = 1, nbzvar = 0, nbdata = 0 - One additional parameter: prm(1) = Vf0 (set at initialisation from the load-flow field voltage) - Observables: vf, if

20.1.2. Parameters

Parameter	Description
(none)	No user-supplied data parameters. The field voltage is taken from the initialisation (load-flow) value automatically.

20.1.3. Usage Example

RAMSES Data File

```

SYNC_MACH GEN1  BUS1  1.  1.  0.  0.  600. 570. 3. 0. 0.95
                XT   0.15  1.1  0.25  0.2  0.7  *  0.2  0.1  6.0257  0.  5.00
                ↪   0.05  *  0.1
                EXC CONSTANT
                TOR CONSTANT ;

```

Notes

- No parameters need to be supplied; the model self-initialises to the load-flow V_f value.
- Useful for simulation of machines operating in manual excitation mode.
- The model name in the data file is CONSTANT (not exc_constant).

20.2. 1ST_ORDER (exc_1storder)

The data-file model name is 1ST_ORDER (uppercase, with the underscore).

20.2.1. Scientific Description

The exc_1storder model is a **first-order automatic voltage regulator** (AVR). It represents the simplest dynamic exciter: a proportional gain K_A acting on the terminal voltage error, filtered through a single first-order lag T_A , with output limits.

The AVR computes the voltage error and drives the field voltage:

$$\frac{dV_f}{dt} = \frac{1}{T_A} [K_A (V_{ref} - V) - V_f], \quad V_f \in [V_{f,min}, V_{f,max}]$$

The reference voltage V_{ref} is initialised from:

$$V_{ref} = V_0 + \frac{V_{f0}}{K_A}$$

Implementation details (from Fortran source): - nbxvar = 1, nbzvar = 1, nbdata = 4 - Parameters: prm(1) = K_A , prm(2) = T_A , prm(3) = V_{fmin} , prm(4) = V_{fmax} - Additional parameter: prm(5) = V_0 (voltage setpoint, computed at initialisation) - The ODE in the continuous mode is:

$$f(1) = \frac{-V_f + K_A(V_{ref} - V)}{T_A}$$

- With anti-windup: if the limiter is active, $f(1) = V_f - V_{fmin}$ (or V_{fmax}).

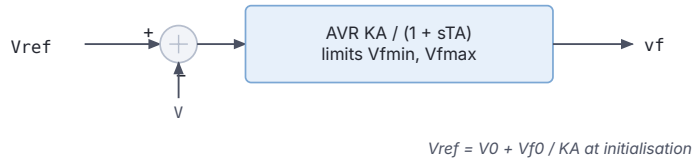


Figure 20.1: 1ST_ORDER exciter block diagram.

20.2.2. Parameters

Parameter	Description
KA	AVR gain (pu/pu)
TA	AVR time constant (s)
Vfmin	Minimum field voltage (pu)
Vfmax	Maximum field voltage (pu)

20.2.3. Usage Example

RAMSES Data File

```

SYNC_MACH GEN1  BUS1  1.  1.  0.  0.  600. 570. 3.  0.  0.95
      XT  0.15  1.1  0.25  0.2  0.7  *  0.2  0.1  6.0257  0.  5.00
      ↪  0.05  *  0.1
EXC 1ST_ORDER  200.0  0.05  0.0  6.0
      ! KA  TA  Vfmin Vfmax
TOR CONSTANT ;

```

Notes

- This model is the Fortran-coded equivalent of a simple `tf1plim` block with input $V_{ref} - V$.
- KA and TA together set the closed-loop bandwidth.
- The model name in the data file is 1ST_ORDER (uppercase, with the underscore).

20.3. kundur (exc_kundur)

The data-file model name is `kundur` or `exc_kundur`. The `exc_` prefix is added automatically.

20.3.1. Scientific Description

The `exc_kundur` model implements the **excitation system from Kundur's textbook** (*Power System Stability and Control*, 1994). It includes a first-order voltage transducer, a proportional AVR with transient gain reduction (TGR), and a two-stage PSS acting on rotor speed deviation.

Voltage transducer:

$$\frac{dV_{fil}}{dt} = \frac{1}{T_R}(V - V_{fil})$$

AVR with transient gain reduction (lead-lag):

$$V_f(s) = K_A \cdot \frac{1 + sT_A}{1 + sT_B} \cdot (V_{ref} - V_{fil} + V_s)$$

where $V_{ref} = V + V_f/K_A$ is computed at initialisation and V_s is the PSS output.

PSS (two-stage lead-lag with washout):

$$x_{wash}(s) = \frac{K_{STAB} \cdot sT_W}{1 + sT_W} \cdot \omega(s)$$

$$V_s(s) = \frac{1 + sT_1}{1 + sT_2} \cdot \frac{1 + sT_3}{1 + sT_4} \cdot x_{wash}(s)$$

This is the standard two-stage phase-compensating PSS structure. The CODEGEN blocks in the .txt file are:

Block	CODEGEN type	Function
Voltage filter	tf1p	$1/(1 + sT_R)$
Main summing junction	algeq	$V_{ref} - V_{fil} + V_s - dv$
AVR (TGR)	tf1p1z	$K_A(1 + sT_A)/(1 + sT_B)$
PSS washout	tfder1p	$K_{STAB} \cdot sT_W/(1 + sT_W)$
PSS lead-lag 1	tf1p1z	$(1 + sT_1)/(1 + sT_2)$
PSS lead-lag 2	tf1p1z	$(1 + sT_3)/(1 + sT_4)$

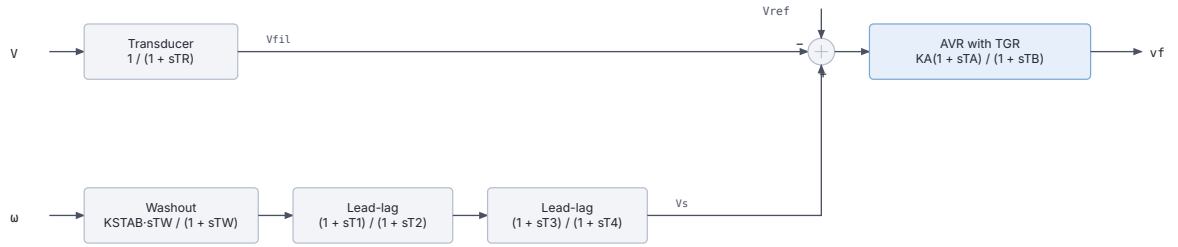


Figure 20.2: exc_kundur block diagram.

20.3.2. Parameters

Parameter	Description
TR	Voltage measurement filter time constant (s)
KA	AVR gain (pu/pu)
TA	TGR lead time constant (s)
TB	TGR lag time constant (s)
KSTAB	PSS gain
TW	PSS washout time constant (s)
T1	PSS first lead-lag zero time constant (s)
T2	PSS first lead-lag pole time constant (s)
T3	PSS second lead-lag zero time constant (s)
T4	PSS second lead-lag pole time constant (s)

Computed at initialisation: $V_{ref} = V + V_f / K_A$

20.3.3. Usage Example

RAMSES Data File

```

SYNC_MACH GEN1  BUS1  1.  1.  0.  0.  600. 570. 3. 0. 0.95
                XT  0.15  1.1  0.25  0.2  0.7  *  0.2  0.1  6.0257  0.  5.00
                ↪  0.05  *  0.1
EXC kundur  0.02  200.0  0.02  10.0  9.5  1.41  0.154  0.033  0.154
↪  0.033
                ! TR  KA      TA      TB      KSTAB TW      T1      T2      T3
                ↪  T4
TOR CONSTANT ;

```

Notes

- Parameters match the Example 12.6 from Kundur (1994), applicable to a medium-speed generator.
- The PSS stabilises local modes in the 0.5–2 Hz range.
- No output limiter is defined in the base model; add clipping externally if required.
- The model name in the data file is `kundur` (not `exc_kundur`).

20.4. GENERIC1 (exc_generic1)

The data-file model name is `GENERIC1`.

20.4.1. Scientific Description

The `exc_generic1` model is a **generic AVR with OEL (Over-Excitation Limiter) and PSS**, implemented in Fortran. It includes a proportional–integral AVR, a PSS acting on rotor speed or active power, and an inverse-time OEL.

AVR:

The excitation system uses a first-order integrating controller (washout or PI-type, set by the gain G and time constants T_A, T_B) with output limits $[L_3, L_4]$:

$$V_f(s) = G \cdot \frac{1 + sT_A}{1 + sT_B} \cdot (V_{ref} - V) \cdot \frac{1}{1 + sT_E}$$

PSS (speed or power input, selectable):

Input signal is selected by `SPEEDIN`: 1 = rotor speed ω , 0 = electrical power P .

$$V_{PSS}(s) = K_{PSS} \cdot \frac{sT_W}{1 + sT_W} \cdot \frac{1 + sT_1}{1 + sT_2} \cdot \frac{1 + sT_3}{1 + sT_4} \cdot u(s)$$

Output is clamped to $[DV_{MIN}, DV_{MAX}]$.

OEL (inverse-time limiter):

The OEL monitors field current I_f . When $I_f > I_{FLIM}$, a timer begins accumulating via:

- Dead zone d and phase f parameters control OEL activation thresholds.
- The OEL integrates the overcurrent error and reduces V_{ref} once the timer reaches threshold L_1 , with output range $[L_1, L_2]$.

Parameters from Fortran source (`prm` array):

Index	Name	Description
1	IFLIM	Field current OEL threshold (pu)
2	d	OEL dead zone (pu)
3	f	OEL phase characteristic
4	S	AVR setpoint input (usually 0 = V)
5	K1	AVR lead numerator gain
6	K2	AVR lag denominator gain
7	L1	OEL integrator lower limit (pu)
8	L2	OEL integrator upper limit (pu)
9	G	AVR overall gain
10	TA	AVR lead time constant (s)
11	TB	AVR lag time constant (s)
12	TE	Exciter time constant (s)
13	L3	AVR output lower limit (pu)
14	L4	AVR output upper limit (pu)
15	SPEEDIN	PSS input signal: 1 = speed, 0 = power
16	KPSS	PSS gain
17	Tw	PSS washout time constant (s)
18	T1	PSS lead-lag 1 zero (s)
19	T2	PSS lead-lag 1 pole (s)
20	T3	PSS lead-lag 2 zero (s)
21	T4	PSS lead-lag 2 pole (s)
22	DVMIN	PSS output lower limit (pu)
23	DVMAX	PSS output upper limit (pu)

20.4.2. Usage Example

RAMSES Data File

```

SYNC_MACH GEN1 BUS1 1. 1. 0. 0. 600. 570. 3. 0. 0.95
      XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
      ↪ 0.05 * 0.1
EXC GENERIC1 1.05 -0.1 0.5 0.0 1.0 1.0 0.0 0.5 200.0 0.02
↪ 0.10 0.05 0.0 6.0
      ! IFLIM d f S K1 K2 L1 L2 G TA
      ↪ TB TE L3 L4
      1 9.5 1.40 0.15 0.03 0.15 0.03 -0.10 0.10
      ! SPEEDIN KPSS Tw T1 T2 T3 T4 DVMIN
      ↪ DVMAX
TOR CONSTANT ;

```

20.5. AVR_DG (exc_AVR_DG)

The data-file model name is AVR_DG. Registered since 3.76.

20.5.1. Scientific Description

An AVR for **dispersed generation**: the GENERIC1 excitation system, OEL and PSS, plus a **reactive-power PI controller** that trims the voltage reference so the unit holds a reactive-power setpoint.

Parameters 1 to 23 are those of GENERIC1, in the same positions and with the same meanings, and the AVR, PSS and OEL behave identically. Only the summing junction changes, gaining a third contribution:

$$e = V_{ref} - V + \Delta V_{PSS} + \Delta V_Q$$

Reactive-power regulator:

The error is the deviation from the setpoint captured at initialisation:

$$\dot{x}_Q = K_{QI} (Q^* - Q), \quad \Delta V_Q = K_{QP} (Q^* - Q) + x_Q$$

clamped to $[L_5, L_6]$. Setting $K_{QP} = K_{QI} = 0$ leaves the loop dormant and the model reduces to GENERIC1.

Parameters:

Indices 1 to 23 are exactly those of GENERIC1. The four that follow are new:

Index	Name	Description
24	KQP	Reactive-power regulator proportional gain
25	KQI	Reactive-power regulator integral gain
26	L5	Regulator output lower limit (pu)
27	L6	Regulator output upper limit (pu)

Implementation details: - nbxvar = 8, nbzvar = 7, nbdata = 27, nbddpar = 2 - Additional: prm(28) = V0 (voltage reference) and prm(29) = Qset, both captured at initialisation - Observables: those of GENERIC1 plus deltavq and Qset

20.5.2. Usage Example

RAMSES Data File

```

SYNC_MACH G4 N4 1. 1. 0. 0. 5 4.5 3. 0. 1.9333
XT 0.0937 2.027 0.210 0.173 2.027 * 0.173 0.1 6 0.00 3.1
↪ 0.0387 * 0.0387
EXC AVR_DG 2.865 -0.1 0. 1. 100. -1. -20 10 50 4 20 0.1 0 5
! IFLIM d fc S K1 K2 L1 L2 G TA TB TE L3
↪ L4
1 0. 5. 1. 1. 1. 1. 0. 0.
! SPEEDIN KPSS Tw T1 T2 T3 T4 DVMIN DVMAX
0.04 0.06 -0.2 0.33
! KQP KQI L5 L6
TOR HYDRO_DG 0.0 0.1 0.4 0.2 0.1 1.0 ;

```

Notes

- The setpoint is **not** a parameter. Qset is captured from the power-flow solution at initialisation, so the unit holds whatever reactive power it was dispatched at.
- With KQP and KQI non-zero the unit regulates reactive power rather than terminal voltage, which is the usual arrangement for dispersed generation connected to a distribution network under a connection agreement.
- KPSS = 0 disables the stabiliser. Note that this drives the PSS branch to its lower limit, so DVMIN should be 0 as well, as in the example above.
- Pair it with HYDRO_DG for a unit that regulates both its active and reactive power and provides no primary frequency response.

20.6. GENERIC2 (exc_generic2)

The data-file model name is GENERIC2.

20.6.1. Scientific Description

The `exc_generic2` model is a more detailed **generic AVR**. It includes a voltage compensator, a combined PI–proportional regulator, an AC exciter with saturation, a PSS, and an OEL.

Voltage measurement with line-drop compensation:

$$V_c = V + X_c \cdot I_q - R_c \cdot I_d \quad (\text{or simplified})$$

PI–proportional AVR with transient gain reduction:

$$V_{AVR}(s) = \left(K_P + \frac{K_I}{s} + C \cdot \frac{1 + sT_C}{1 + sT_B} \right) \cdot K_A \cdot \frac{1}{1 + sT_A} (V_{ref} - V_c)$$

AC exciter with saturation:

$$\frac{dV_E}{dt} = \frac{1}{T_E} [V_R - K_E \cdot V_E - S_E(V_E)]$$

where $S_E(V_E)$ is the piecewise-linear saturation function defined by (E_1, S_{E1}) and (E_2, S_{E2}) .

PSS (type selectable by `typss`):

$$V_s(s) = K_{PSS} \cdot \frac{sT_W}{1 + sT_W} \cdot \frac{1 + sT_1}{1 + sT_2} \cdot \frac{1 + sT_3}{1 + sT_4} \cdot u(s)$$

OEL (inverse-time, three-segment):

The OEL uses a three-level piecewise-linear characteristic (s_1, T_{oel1}) , (s_2, T_{oel2}) , (s_3, T_{oel3}) to compute the allowable overcurrent time.

Parameters from Fortran `associate` block (39 data parameters + 4 additional):

Index	Name	Description
1	Xc	Line-drop compensation reactance (pu)
2	Tr	Voltage measurement time constant (s)
3	C	TGR numerator gain
4	Tc	TGR lead time constant (s)
5	Tb	TGR lag time constant (s)
6	Ka	AVR proportional gain
7	Ta	AVR time constant (s)
8	Ki	AVR integral gain
9	Kp	AVR proportional path gain
10	VrmaxD	Regulator max (dynamic, pu)
11	VrminD	Regulator min (dynamic, pu)
12	G	Exciter overall gain
13	Ke	Exciter self-excitation coefficient
14	Te	Exciter time constant (s)
15	E1	Saturation breakpoint 1 (pu)
16	SeE1	Saturation factor at E_1
17	E2	Saturation breakpoint 2 (pu)
18	SeE2	Saturation factor at E_2
19	Vrmax	Field voltage upper limit (pu)
20	Vrmin	Field voltage lower limit (pu)
21	typss	PSS input type (1 = speed, 2 = power)
22	W	PSS input gain
23	Tf	PSS derivative filter time constant (s)
24	Kpss	PSS gain
25	Tw	PSS washout time constant (s)
26	T1	PSS lead–lag 1 zero (s)
27	T2	PSS lead–lag 1 pole (s)

Index	Name	Description
28	T3	PSS lead-lag 2 zero (s)
29	T4	PSS lead-lag 2 pole (s)
30	Lim	PSS output symmetric limit (pu)
31	LimSup	PSS upper asymmetric limit (pu)
32	s1	OEL segment 1 (pu I_f)
33	toel1	OEL time at segment 1 (s)
34	s2	OEL segment 2 (pu I_f)
35	toel2	OEL time at segment 2 (s)
36	s3	OEL segment 3 (pu I_f)
37	toel3	OEL time at segment 3 (s)
38	perm	OEL permanent current limit (pu)
39	LimInf	PSS lower asymmetric limit (pu)

20.6.2. Usage Example

RAMSES Data File

```

SYNC_MACH GEN1  BUS1  1.  1.  0.  0.  600. 570. 3.  0.  0.95
      XT  0.15  1.1  0.25  0.2  0.7  *  0.2  0.1  6.0257  0.  5.00
      ↪  0.05  *  0.1
EXC GENERIC2  0.0  0.02  1.0  0.10  1.00  200.0  0.05  5.0  1.0  6.0
↪ -6.0  1.0  0.0  0.50
      ! Xc  Tr  C  Tc  Tb  Ka  Ta  Ki  Kp  VrmaxD
      ↪ VrminD G  Ke  Te
      3.10  0.066  4.18  0.91  6.0  -6.0  1.0  1.0  0.05  9.5
      ↪  1.40  0.154  0.033  0.154  0.033
      ! E1  SeE1  E2  SeE2  Vrmax  Vrmin  typss  W  Tf  Kpss
      ↪ Tw  T1  T2  T3  T4
      0.10  0.10  1.1  200.  1.2  60.  1.5  10.  1.05  -0.10
      ! Lim  LimSup  s1  toel1  s2  toel2  s3  toel3  perm  LimInf
TOR CONSTANT ;

```

20.7. GENERIC (exc_GENERIC)

The data-file model name is exc_GENERIC (or GENERIC. The exc_ prefix is added automatically).

Renamed in RAMSES 3.57

This model was called exc_GENERIC3 up to and including 3.56, and a second registered name, exc_GENERIC4, resolved to a byte-for-byte copy of it. Both names were removed in 3.57 in favour of the single name exc_GENERIC.

On 3.57 and later, a record naming exc_GENERIC3 or exc_GENERIC4 stops with Excitation model not found in definition. Change it to exc_GENERIC; the 20 data parameters and their order are unchanged, so nothing else in the record moves.

20.7.1. Scientific Description

The exc_GENERIC model (note capital letters) is a **generic AVR with integrated PSS and OEL**, defined via the CODEGEN .txt description. It provides a clean, modular three-block structure: AVR, PSS, and OEL.

AVR (transient gain reduction + exciter):

$$\Delta V = V_0 - V + V_{PSS} - V_{OEL}$$

$$V_1(s) = G \cdot \frac{1 + sT_a}{1 + sT_b} \cdot \Delta V(s)$$

$$V_f(s) = \frac{1}{1 + sT_e} \cdot V_1(s), \quad V_f \in [V_{f,min}, V_{f,max}]$$

where $V_0 = V + V_f/G$ is computed at initialisation.

PSS (washout + two lead-lag stages):

$$V_{PSS}(s) = \frac{K_{PSS} \cdot sT_W}{1 + sT_W} \cdot \frac{1 + sT_1}{1 + sT_2} \cdot \frac{1 + sT_3}{1 + sT_4} \cdot \omega(s)$$

Output is limited to $[-C, +C]$.

OEL (inverse-time integrator):

$$x_{oel1} = I_f - if_{lim}$$

$$\frac{dx_{oel2}}{dt} = \frac{1}{T_{oel}} \cdot \min(x_{oel1}, if_{lim}), \quad x_{oel2} \in [L_1, L_2]$$

$$V_{OEL} = \text{clip}(K_{oel} \cdot x_{oel2}, 0, L_3)$$

The CODEGEN block structure from the .txt file:

Block	CODEGEN type	Signal
AVR summing junction	algeq	$V_0 - V + dv_{PSS} - dv_{OEL} - \Delta V$
Transient gain reduction	tf1p1z	$G(1 + sT_a)/(1 + sT_b)$
Exciter	tf1p1lim	$1/(1 + sT_e)$ with $[V_{f,min}, V_{f,max}]$
PSS washout	tfder1p	$K_{PSS} \cdot sT_W/(1 + sT_W)$
PSS lead-lag 1	tf1p1z	$(1 + sT_1)/(1 + sT_2)$
PSS lead-lag 2	tf1p1z	$(1 + sT_3)/(1 + sT_4)$
PSS limiter	lim	$[-C, +C]$
OEL error	algeq	$I_f - if_{lim}$
OEL upper bound	min1v1c	$\min(\cdot, if_{lim})$
OEL integrator	inlim	$1/T_{oel}$ with $[L_1, L_2]$
OEL multiplier	algeq	$K_{oel} \cdot x_{oel2}$
OEL output limiter	lim	$[0, L_3]$

20.7.2. Parameters

Parameter	Description
G	AVR gain (pu/pu)
Ta	AVR lead time constant / TGR zero (s)
Tb	AVR lag time constant / TGR pole (s)
Te	Exciter time constant (s)
vfmin	Minimum field voltage (pu)
vfmax	Maximum field voltage (pu)
Kpss	PSS gain
Tw	PSS washout time constant (s)
T1	PSS first lead-lag zero (s)
T2	PSS first lead-lag pole (s)
T3	PSS second lead-lag zero (s)
T4	PSS second lead-lag pole (s)
C	PSS output symmetric limit (pu)

Parameter	Description
if1lim	OEL field current threshold (pu)
if2lim	OEL maximum overcurrent error (pu)
Toel	OEL integrator time constant (s)
Koel	OEL output gain
L1	OEL integrator lower limit
L2	OEL integrator upper limit
L3	OEL output upper limit (pu)

Computed at initialisation: $V_o = V + V_f / G$

20.7.3. Usage Example

RAMSES Data File

```

SYNC_MACH GEN1  BUS1  1. 1.  0.  0.  600. 570. 3. 0. 0.95
                XT  0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
                ⇐ 0.05 * 0.1
EXC exc_GENERIC 200.0 0.05 10.0 0.04 0.0 5.0
                ! G    Ta    Tb    Te    vfmín vfmáx
                9.5 1.40 0.154 0.033 0.154 0.033 0.10
                ! Kpss Tw   T1    T2    T3    T4    C
                1.05 0.30 30.0 1.0 0.0 1.0 2.0
                ! if1lim if2lim Toel Koel L1  L2  L3
TOR CONSTANT ;

```

20.8. See Also

- Model Reference, the full model catalogue
- IEEE Exciters, the IEEE-standard excitation models
- Dynamic Data Records, the SYNC_MACH record these models sit in

IEEE turbine-governor models

This page documents the IEEE-standard turbine-governor models available in RAMSES, all using the `tor_` prefix. These models implement standardised transfer-function blocks for diesel, gas-turbine, steam-turbine, hydraulic, and simplified ENTSO-E governors.

Usage in Dynamic Data Files

Governor/torque models in RAMSES are defined as sub-records within a `SYNC_MACH` block using the `TOR` keyword, followed by the model name and its parameters. They are not standalone records. The `TOR` line appears after the `EXC` (exciter) line, and the `;` at the end closes the entire `SYNC_MACH` block.

```

SYNC_MACH name bus FP FQ P Q SNOM Pnom H D IBRATIO
          XT Xl Xd X'd X''d Xq X'q X''q m n Ra T'do T''do T'qo
          ↪ T''qo
      EXC model_name param1 param2 ...
      TOR model_name param1 param2 ... ;

```

RAMSES adds the `tor_` prefix to the model name automatically, so both `DEGOV1` and `tor_DEGOV1` resolve to the same model. The IEEE governor names are:

Data-file name	With explicit prefix	Implemented by	Available since
DEGOV1	<code>tor_DEGOV1</code>	<code>tor_DEGOV1</code>	3.40
ENTSOE_simp	<code>tor_ENTSOE_simp</code>	<code>tor_ENTSOE_simp</code>	3.40
GAST	<code>tor_GAST</code>	<code>tor_GAST</code>	3.50
TGOV1	<code>tor_TGOV1</code>	<code>tor_TGOV1D</code>	3.50
HYGOV	<code>tor_HYGOV</code>	<code>tor_hygov</code>	3.50

Only the first two columns are valid in a data file. The third column is the name of the Fortran subroutine, which you need only when registering the model in URAMSES.

‘GAST’, ‘TGOV1’ and ‘HYGOV’ require RAMSES 3.50 or later

Those three were added after the 3.40 release; `DEGOV1` and `ENTSOE_simp` were already registered in 3.40. A distribution that does not have them rejects the name with:

```

STOP CALL FROM tor_TGOV1:
Torque model not found in definition

```

If you see this, check the banner RAMSES prints at startup. Versions 3.50 and later show a **Version:** line; if yours does not, upgrade before investigating anything else.

‘TGOV1D’ is not a valid data-file name

The TGOV1 model is implemented by a subroutine called `tor_TGOV1D`, but the name registered for data files is TGOV1 (or `tor_TGOV1`), **without** the D. Writing TGOV1D or `tor_TGOV1D` after TOR produces the same “Torque model not found in definition” error, on every version.

21.1. DEGOV1 (tor_DEGOV1)

21.1.1. Scientific Description

The DEGOV1 model represents the **diesel engine governor** as defined in the IEEE Committee Report on governors. It consists of three main stages:

1. Speed governor (electrical control / lead-lag)

The speed error is formed from the measured speed deviation and a reference signal derived from either mechanical power (P) or mechanical torque (T_m), selected by the switch SWM:

$$V_{60} = (1 - \text{SWM}) \cdot T_m + \text{SWM} \cdot P$$

$$\text{REF} = V_{60} \cdot R$$

The governor speed error drives a second-order lead-lag controller (transfer function approximating a two-pole, two-zero system) with time constants T_1, T_2, T_3 :

$$G_{gov}(s) \approx \frac{1 + sT_3}{(1 + sT_1)(1 + sT_2)}$$

2. Actuator (fuel injection system)

The governor output passes through a lead-lag block with gain K and time constants T_4, T_6 :

$$G_{act}(s) = K \cdot \frac{1 + sT_4}{1 + sT_6}$$

followed by a first-order lag T_5 , and an integrator with limits $[T_{MIN}, T_{MAX}]$.

3. Engine dead time and torque output

The actuator output is subject to engine dead time T_D , approximated by a second-order Padé:

$$e^{-sT_D} \approx \frac{1 - 0.5sT_D + 0.0833s^2T_D^2}{1 + 0.5sT_D + 0.0833s^2T_D^2}$$

The final mechanical torque is computed as:

$$T_m = \frac{1}{1 + sT_E} \cdot V_{engine}$$

$$P_m = T_m \cdot \omega$$

The droop characteristic is:

$$\Delta\omega + R \cdot (V_{60} - REF) = 0 \implies \text{error} = \Delta\omega + R \cdot V_{act}$$

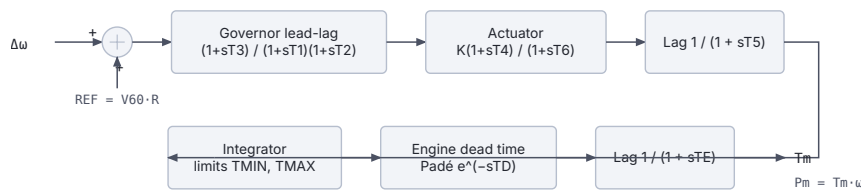


Figure 21.1: DEGOV1 block diagram.

21.1.2. Parameters

Parameter	Description
SWM	Input switch: 0 = mechanical torque (T_m), 1 = electrical power (P)
T1	Governor time constant, lead numerator (s)
T2	Governor time constant, lag denominator (s)
T3	Governor time constant, second-order denominator (s)
K	Actuator gain
T4	Actuator lead time constant (s)
T5	First-order actuator lag (s)
T6	Actuator lag time constant (s)
TMIN	Minimum fuel/torque limit (pu)
TMAX	Maximum fuel/torque limit (pu)
TD	Engine dead time (s)
R	Speed droop (pu/pu)
TE	Engine time constant (s)

Internal parameters: V60 (selected reference input, mechanical torque or electrical power according to SWM) and REF ($V60 \cdot R$), both set at initialisation

21.1.3. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 600. 570. 6. 0. 2.05
          XT 0.15 2.2 0.3 0.2 2. 0.4 0.2 0.1 6.0257 0. 7.00
          ↪ 0.05 1.5 0.05
EXC GENERIC1 3.0618 -0.1 1. 0. 100. -1. -20.0 10. 120. 5. 12.5
          ↪ 0.1 0. 5.
          1 75. 15. 0.22 0.012 0.22 0.012
          ↪ -0.1 0.1
TOR DEGOV1 0 ! SWM: 0 = torque input, 1 = power input
          0.02 ! T1: governor lead time constant (s)
          0.02 ! T2: governor lag time constant (s)
          0.20 ! T3: governor second-order denominator (s)
          1.0 ! K: actuator gain
          0.25 ! T4: actuator lead time constant (s)
          0.009 ! T5: first-order actuator lag (s)
          0.038 ! T6: actuator lag time constant (s)
          0.0 ! TMIN: minimum fuel/torque limit (pu)
          1.05 ! TMAX: maximum fuel/torque limit (pu)

```

```

0.01    ! TD: engine dead time (s)
0.05    ! R: speed droop (pu/pu)
0.05 ; ! TE: engine time constant (s)

```

Notes

- Set $SWM = 0$ to use mechanical torque as the reference input, $SWM = 1$ for electrical power.
- $TMIN$ and $TMAX$ are per-unit limits on the integrator output (fuel demand / valve position).
- R is the permanent droop; typical values are 0.04–0.05 pu/pu.
- The Padé approximation of the dead time T_D is a second-order filter.
- Parameters are listed in order: SWM $T1$ $T2$ $T3$ K $T4$ $T5$ $T6$ $TMIN$ $TMAX$ TD R TE .

21.2. ENTSOE_simp (tor_ENTSOE_simp)

21.2.1. Scientific Description

The `ENTSOE_simp` model is a simplified **ENTSO-E speed governor** suitable for primary frequency response studies. It is derived from the ENTSO-E recommendations for equivalent turbine-governor representation.

The governor equation forms a speed error with permanent droop R :

$$R \cdot \dot{p}_1 - C + (\omega - 1) = 0, \quad C = T_m \cdot R$$

This error drives a first-order lag (representing the valve or steam chest) with limits $[V_{MIN}, V_{MAX}]$:

$$\frac{dp_2}{dt} = \frac{1}{T_1}(p_1 - p_2), \quad p_2 \in [V_{MIN}, V_{MAX}]$$

The turbine mechanical power is modelled by a lead-lag transfer function with time constants T_2 (zero) and T_3 (pole):

$$P_m(s) = \frac{1 + sT_2}{1 + sT_3} \cdot p_2(s)$$

The final mechanical torque output accounting for shaft speed is:

$$T_m \cdot \omega = P_m$$

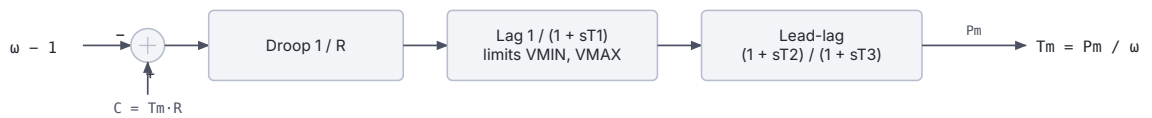


Figure 21.2: ENTSOE_simp block diagram.

21.2.2. Parameters

Parameter	Description
R	Permanent speed droop (pu/pu)
T1	Governor/valve time constant (s)
VMIN	Minimum valve position / lower output limit (pu)
VMAX	Maximum valve position / upper output limit (pu)
T2	Turbine lead time constant, zero (s)
T3	Turbine lag time constant, pole (s)

Internal parameter: $C = T_m \cdot R$ (initialised from the load-flow mechanical torque)

21.2.3. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 400. 360. 6. 0. 2.05
          XT 0.15 2.2 0.3 0.2 2. 0.4 0.2 0.1 6.0257 0. 7.00
          ↪ 0.05 1.5 0.05
EXC GENERIC1 3.0618 -0.1 1. 0. 100. -1. -20.0 10. 120. 5. 12.5
↪ 0.1 0. 5.
          1 75. 15. 0.22 0.012 0.22 0.012
          ↪ -0.1 0.1
TOR ENTISOE_simp 0.05 ! R: permanent droop (pu/pu)
                  0.50 ! T1: governor/valve time constant (s)
                  0.0 ! VMIN: minimum valve position (pu)
                  1.1 ! VMAX: maximum valve position (pu)
                  0.0 ! T2: turbine lead time constant (s)
                  10.0 ; ! T3: turbine lag time constant (s)

```

Notes

- Typical primary-frequency response studies use $R = 0.04$ – 0.05 .
- Setting $T_2 = 0$ simplifies the turbine to a pure first-order lag $1/(1 + sT_3)$.
- This model is commonly used as a simplified equivalent for aggregated generation.
- Parameters are listed in order: R T1 VMIN VMAX T2 T3.

21.3. GAST (tor_GAST)

21.3.1. Scientific Description

The GAST model represents a **gas turbine** unit, following the IEEE Committee Report (1973) and the convention common to industry tools. It captures the speed governor, compressor discharge lag, and fuel/combustion dynamics.

Speed governor with droop and load reference:

$$\text{error} = R \cdot (V_{act} - LR) + (\omega - 1) = 0$$

where $LR = T_m$ (load reference, initialised from mechanical torque) and R is the droop.

The speed error drives a valve position through a limiter $[V_{MIN}, V_{MAX}]$ and a first-order lag T_1 :

$$\frac{dp_2}{dt} = \frac{1}{T_1}(p_1 - p_2)$$

The compressor discharge is modelled by lag T_2 :

$$\frac{dp_3}{dt} = \frac{1}{T_2}(p_2 - p_3)$$

The mechanical power (including speed-dependent damping D_{TURB}) is:

$$P_m = p_3 + D_{TURB} \cdot (\omega - 1)$$

The exhaust temperature is tracked by a lag T_3 :

$$\frac{dp_4}{dt} = \frac{1}{T_3}(p_3 - p_4)$$

A temperature limit is applied via:

$$p_5 = A_T + K_T \cdot (A_T - p_4)$$

A minimum gate blocks the valve position: $p_1 = \min(p_{gov}, p_5)$.

The torque-power conversion:

$$T_m \cdot \omega = P_m$$

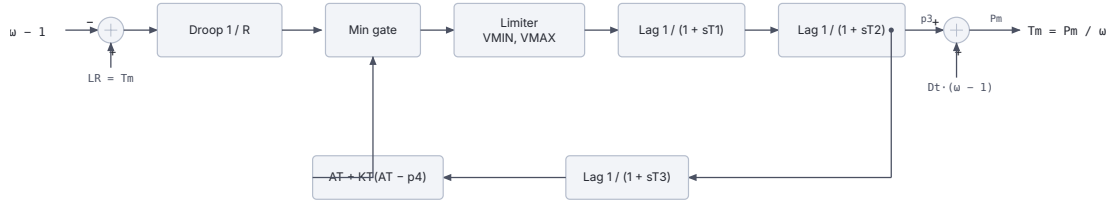


Figure 21.3: GAST block diagram.

21.3.2. Parameters

Parameter	Description
R	Permanent speed droop (pu/pu)
T1	Governor/valve time constant (s)
T2	Compressor discharge time constant (s)
T3	Radiation shield / exhaust temperature lag (s)
AT	Ambient temperature load limit (pu)
KT	Temperature control loop gain
VMAX	Maximum valve position (pu)
VMIN	Minimum valve position (pu)
DTURB	Turbine damping factor (pu torque / pu speed)

Internal parameter: LR = Tm (load reference, set at initialisation)

21.3.3. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 400. 360. 6. 0. 2.05
              XT 0.15 2.2 0.3 0.2 2. 0.4 0.2 0.1 6.0257 0. 7.00
              ↪ 0.05 1.5 0.05
EXC GENERIC1 3.0618 -0.1 1. 0. 100. -1. -20.0 10. 120. 5. 12.5
              ↪ 0.1 0. 5.
              1 75. 15. 0.22 0.012 0.22 0.012
              ↪ -0.1 0.1
TOR GAST 0.05 ! R: permanent droop (pu/pu)
          0.40 ! T1: governor/valve time constant (s)
          0.10 ! T2: compressor discharge time constant (s)
          3.00 ! T3: exhaust temperature lag (s)
          1.00 ! AT: ambient temperature load limit (pu)
          2.50 ! KT: temperature control loop gain
          1.05 ! VMAX: maximum valve position (pu)
          0.0 ! VMIN: minimum valve position (pu)
          0.0 ; ! DTURB: turbine damping factor

```

Notes

- Based on the IEEE Committee Report on “Dynamic Models for Steam and Hydro Turbines in Power System Studies” (1973), later adapted for gas turbines.
- AT and KT define the exhaust-temperature protection characteristic.
- DTURB = 0 disables speed-dependent damping.
- Parameters are listed in order: R T1 T2 T3 AT KT VMAX VMIN DTURB.

21.4. TGOV1 (tor_TGOV1D)

21.4.1. Scientific Description

The TGOV1 model represents a **simple steam turbine governor**, based on the IEEE TGOV1 structure with an added damping term. It is one of the most widely used simple governor models in power-system stability studies.

Speed governor with permanent droop:

The speed error (with droop R) drives a first-order lag T_1 with output limits $[V_{MIN}, V_{MAX}]$:

$$R \cdot p_1 - C + (\omega - 1) = 0, \quad C = T_m \cdot R$$

$$\frac{dp_2}{dt} = \frac{1}{T_1}(p_1 - p_2), \quad p_2 \in [V_{MIN}, V_{MAX}]$$

Turbine with reheater and damping:

The valve position passes through a lead-lag block representing the steam chest and reheater:

$$P_m(s) = \frac{1 + sT_2}{1 + sT_3} \cdot p_2(s)$$

The total mechanical power includes a speed-dependent damping term D_t :

$$P_m = p_{turbine} + D_t \cdot (\omega - 1)$$

Hence the mechanical torque:

$$T_m = \frac{P_m}{\omega}$$

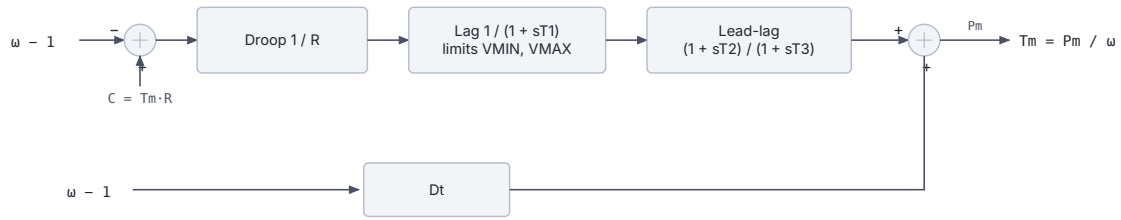


Figure 21.4: TGOV1 block diagram.

21.4.2. Parameters

Parameter	Description
R	Permanent speed droop (pu/pu)
T1	Steam chest / valve actuator time constant (s)
VMAX	Maximum valve position (pu)
VMIN	Minimum valve position (pu)
T2	Lead time constant, reheater zero (s)
T3	Lag time constant, reheater pole (s)
Dt	Turbine damping coefficient (pu torque / pu speed deviation)

Internal parameter: $C = T_m \cdot R$ (initialised from load-flow mechanical torque)

21.4.3. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 400. 360. 6. 0. 2.05
           XT 0.15 2.2 0.3 0.2 2. 0.4 0.2 0.1 6.0257 0. 7.00
           ↪ 0.05 1.5 0.05
EXC GENERIC1 3.0618 -0.1 1. 0. 100. -1. -20.0 10. 120. 5. 12.5
↪ 0.1 0. 5.
           1 75. 15. 0.22 0.012 0.22 0.012
           ↪ -0.1 0.1
TOR TGOV1 0.05 ! R: permanent droop (pu/pu)
           0.50 ! T1: steam chest/valve actuator time constant (s)
           1.05 ! VMAX: maximum valve position (pu)
           0.0 ! VMIN: minimum valve position (pu)
           2.10 ! T2: lead time constant, reheater zero (s)
           7.00 ! T3: lag time constant, reheater pole (s)
           0.0 ; ! Dt: turbine damping coefficient

```

Notes

- This is essentially the IEEE TGOV1 model with an additional damping term D_t .
- Setting $T_2 = 0$ gives a pure first-order turbine $1/(1 + sT_3)$.
- $D_t > 0$ adds a stabilising speed-dependent power correction (useful for islanded operation).
- Parameters are listed in order: R T1 VMAX VMIN T2 T3 Dt.

21.5. HYG0V (tor_hygov)

21.5.1. Scientific Description

The HYG0V model represents a **hydraulic turbine and governor** following a simplified penstock/water-column formulation, consistent with the IEEE representation for hydro governors (IEEE Std 1110).

Governor with droop and PI controller:

The speed error (with permanent droop R and transient droop r) enters a lag T_f to produce the error signal e :

$$\text{input} = 1 - \omega - R \cdot (c - P_0)$$

$$\frac{de}{dt} = \frac{1}{T_f} (\text{input} - e)$$

The rate of change of gate demand is computed via the transient droop:

$$\dot{x}_2 = \frac{e}{r \cdot T_r}$$

This is rate-limited to $[-V_{ELM}, +V_{ELM}]$, then integrated with gate limits $[G_{min}, G_{max}]$:

$$\frac{dx_4}{dt} = x_3, \quad x_4 \in [G_{min}, G_{max}]$$

The PI controller output (desired gate) is:

$$x_5 = x_4 + \frac{e}{r}, \quad c = \text{clip}(x_5, G_{min}, G_{max})$$

The actual gate position follows through the servo:

$$\frac{dg}{dt} = \frac{1}{T_g} (c - g)$$

Penstock / water column:

The head deviation dH is determined by the non-linear water flow equation and water-starting time T_W :

$$1 - \left(\frac{Q}{g}\right)^2 = dH$$

$$T_W \cdot \frac{dQ}{dt} = dH$$

Turbine mechanical torque:

$$T_m \cdot \omega = A_t \cdot (1 - dH)(Q - Q_{nl}) - D_{turb} \cdot (1 - \omega) \cdot g$$

$$P_m = T_m \cdot \omega$$

Parameter	Description
-----------	-------------

21.5.2. Parameters

Parameter	Description
R	Permanent droop (pu/pu)
r	Transient droop (pu/pu)
Tr	Transient droop reset time (s)
Tf	Filter time constant for speed error (s)
Tg	Gate servo time constant (s)
VELM	Gate velocity limit (pu/s)
Gmax	Maximum gate opening (pu)
Gmin	Minimum gate opening (pu)
Tw	Water starting time (s)
At	Turbine gain (pu power / pu flow)
Dturb	Turbine damping factor
Qnl	No-load water flow (pu)

Internal parameter: $P_o = (P / At) + Qnl$ (initial gate setpoint from load-flow)

21.5.3. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
          XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
          ↪ 0.05 * 0.1
EXC GENERIC1 1.8991 -0.1 0. 1. 100. -1. -11 10. 70. 10. 20.
          ↪ 0.1 0. 4.
          1 75. 15. 0.2 0.01 0.2 0.01
          ↪ -0.1 0.1
TOR HYGOV 0.05 ! R: permanent droop (pu/pu)
          0.30 ! r: transient droop (pu/pu)
          5.00 ! Tr: transient droop reset time (s)
          0.05 ! Tf: filter time constant for speed error (s)
          0.20 ! Tg: gate servo time constant (s)
          0.20 ! VELM: gate velocity limit (pu/s)
          1.00 ! Gmax: maximum gate opening (pu)
          0.0 ! Gmin: minimum gate opening (pu)
          1.50 ! Tw: water starting time (s)
          1.10 ! At: turbine gain (pu power / pu flow)
          0.50 ! Dturb: turbine damping factor
          0.05 ; ! Qnl: no-load water flow (pu)

```

Notes

- Tw (water starting time) is the dominant parameter controlling frequency response; typical values are 0.5–2 s.
- $r > R$ is required for stable regulation; set $r \approx 2\text{--}5 \times R$.
- Tr is the reset time for transient droop; values of 5–30 s are typical.
- Non-linear head-flow equation makes this model suitable for large transients.
- Parameters are listed in order: R r Tr Tf Tg VELM Gmax Gmin Tw At Dturb Qnl.

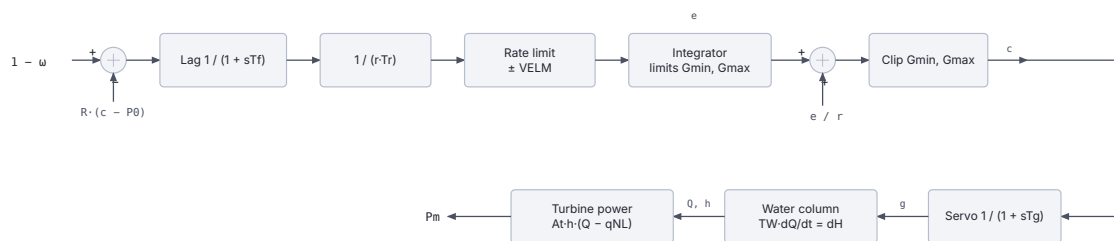


Figure 21.5: HYGOV block diagram.

Other turbine-governor models

This page documents the custom turbine-governor (`tor_`) models in RAMSES. These models are implemented in Fortran using the legacy multi-subroutine API and cover a wide range of prime-mover types: thermal (steam), gas turbine, hydraulic, nuclear, and simplified equivalents.

Usage in Dynamic Data Files

Governor/torque models in RAMSES are defined as sub-records within a `SYNC_MACH` block using the `TOR` keyword, followed by the model name and its parameters. They are not standalone records. The `TOR` line appears after the `EXC` (exciter) line, and the `;` at the end closes the entire `SYNC_MACH` block.

```
SYNC_MACH name bus FP FQ P Q SNOM Pnom H D IBRATIO
          XT Xl Xd X'd X''d Xq X'q X''q m n Ra T'do T''do T'qo
          ↪ T''qo
          EXC model_name param1 param2 ...
          TOR model_name param1 param2 ... ;
```

This page documents the four governors built into every RAMSES distribution, `CONSTANT`, `1ST_ORDER`, `HYDRO_GENERIC1` and `THERMAL_GENERIC1`, which take no prefix and are written in uppercase, together with `HYDRO_DG`, registered since 3.76.

It also documents `GASTURBM`, `GOVCLASM`, `GOVHYDR` and `GOVNUC`, which the family dispatcher has handled directly since 3.79. Before 3.79 they were compiled under no name and RAMSES rejected them in a data file.

The IEEE governors, `DEGOV1`, `ENTSOE_simp`, `GAST`, `TGOV1` and `HYGOV`, are on the IEEE Governors page. For the whole catalogue at a glance, see the Model Reference index.

22.1. `CONSTANT` (`tor_constant`)

The data-file model name is `CONSTANT`.

22.1.1. Scientific Description

The `tor_constant` model provides a **constant mechanical torque** source. The mechanical torque is fixed at its initialisation value and does not respond to speed deviations or load changes:

$$T_m(t) = T_{m0} \quad \forall t$$

The single algebraic state satisfies:

$$f(1) = x(1) - T_{m0} = 0$$

The observable output is mechanical power:

$$P_m = T_m \cdot \omega$$

Implementation details: - nbxvar = 1, nbzvar = 0, nbdata = 0 - One additional parameter: prm(1) = Tm0 (set at initialisation from load-flow) - Observable: Pm = x(1) * omega

22.1.2. Parameters

Parameter	Description
(none)	No user-supplied parameters. T_{m0} is taken from the load-flow initialisation.

22.1.3. Usage Example

RAMSES Data File

```

SYNC_MACH g6 g6 1. 1. 0. 0. 400. 360. 6. 0. 2.05
                XT 0.15 2.2 0.3 0.2 2. 0.4 0.2 0.1 6.0257 0. 7.00
                ↪ 0.05 1.5 0.05
EXC GENERIC1 3.0618 -0.1 1. 0. 100. -1. -20.0 10. 120. 5. 12.5
↪ 0.1 0. 5.
                1 75. 15. 0.22 0.012 0.22 0.012
                ↪ -0.1 0.1
TOR CONSTANT ;

```

Notes

- Useful for machines operating at fixed setpoint, for testing generator models in isolation, or as a placeholder.
- The mechanical power observable $P_m = T_m \cdot \omega$ will vary slightly with speed changes.
- TOR CONSTANT takes no parameters, T_{m0} is set automatically from the load-flow initialisation.

22.2. 1ST_ORDER (tor_1storder)

The data-file model name is 1ST_ORDER (uppercase, with the underscore).

22.2.1. Scientific Description

The tor_1storder model is a **first-order speed governor** with droop and a two-mass (HP/LP) turbine representation. It captures the primary frequency response through a proportional-droop speed controller with a first-order lag.

Speed controller with droop R :

The mechanical torque setpoint is:

$$T_{set} = T_{m0} - \frac{\omega - 1}{R}$$

The HP torque tracks T_{set} through a first-order lag with time constant T_2 :

$$\frac{dx_1}{dt} = \frac{1}{T_2} (T_{set} - x_1)$$

The LP (shaft) torque is a weighted combination:

$$x_2 = (1 - F_{HP}) \cdot x_1 + F_{HP} \cdot T_{set}$$

where F_{HP} is the fraction of power generated in the HP stage.

From the Fortran equations:

f(1) = (-x(1) + (prm(4) - (omega-1.d0)/prm(3))) / prm(2)

f(2) = (1.d0-prm(1))*x(1) + prm(1)*(prm(4)-(omega-1.d0)/prm(3)) - x(2)

i.e.: - prm(1) = FHP (HP fraction), prm(2) = T2 (time constant), prm(3) = R (droop) -
prm(4) = Tm0 (setpoint, set at init)

Implementation details: - nbxvar = 2, nbzvar = 0, nbdata = 3 - State 1 = HP torque x(1), State 2 = LP torque x(2) - Observables: HP torque, LP torque

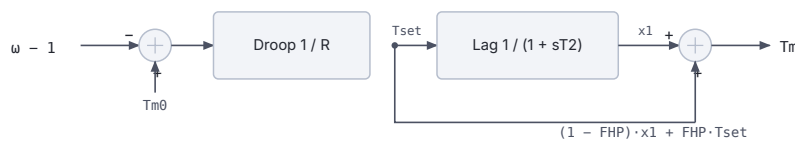


Figure 22.1: 1ST_ORDER governor block diagram.

22.2.2. Parameters

Parameter	Description
FHP	Fraction of torque produced by the HP turbine stage (0–1)
T2	Governor/HP turbine time constant (s)
R	Speed droop (pu/pu)

22.2.3. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
          XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
          ↪ 0.05 * 0.1
EXC GENERIC1 1.8991 -0.1 0. 1. 100. -1. -11 10. 70. 10. 20.
↪ 0.1 0. 4.
          1 75. 15. 0.2 0.01 0.2 0.01
          ↪ -0.1 0.1
TOR 1ST_ORDER 0.30 0.50 0.05 ; ! FHP T2 R

```

Notes

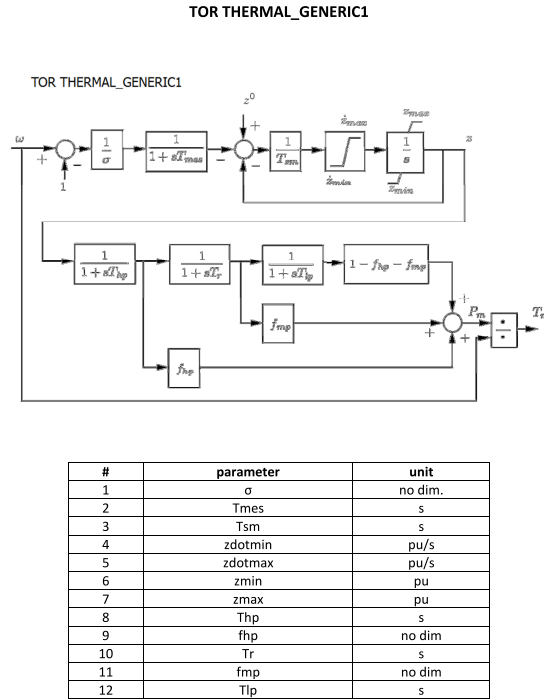
- FHP = 0 makes the model a pure first-order lag on the total torque.
- FHP = 1 bypasses the HP lag entirely; the torque instantly tracks the droop setpoint.
- Typical primary-frequency response studies use R = 0.04–0.05.

22.3. THERMAL_GENERIC1 (tor_thermal_generic1)

The data-file model name is THERMAL_GENERIC1.

22.3.1. Scientific Description

The tor_thermal_generic1 model is a **generic multi-stage steam turbine governor**, suitable for both single-shaft and cross-compound units. It includes a speed controller with rate-limiting, a three-section steam chest/reheater, and separate HP, MP, and LP turbine stages.



1

Figure 22.2: Thermal generic governor block diagram

Speed governor:

A speed measurement with time constant T_{mes} feeds a droop-based error:

$$e_{gov} = P_0 - P_{meas} \cdot \sigma + (1 - \omega)$$

where σ is the droop coefficient.

The gate/valve demand is computed by an integrator with rate limits $[\dot{z}_{min}, \dot{z}_{dotmax}]$ and position limits $[z_{min}, z_{max}]$, filtered through a servo time constant T_{sm} .

Turbine power stages:

The steam chest dynamics drive three stages with power fractions $f_{hp}, f_{mp}, (1 - f_{hp} - f_{mp})$:

$$\frac{dT_{mHP}}{dt} = \frac{1}{T_{hp}}(F_{hp} \cdot z - T_{mHP})$$

$$\frac{dT_{mMP}}{dt} = \frac{1}{T_r}(F_{mp} \cdot T_{mHP}/F_{hp} - T_{mMP})$$

$$\frac{dT_{mLP}}{dt} = \frac{1}{T_{lp}}((1 - F_{hp} - F_{mp}) \cdot T_{mMP}/F_{mp} - T_{mLP})$$

Total mechanical power:

$$P_m = T_{mHP} + T_{mMP} + T_{mLP}$$

Parameters from Fortran **associate** block:

Parameter	Index	Description
sigma	1	Speed droop (pu/pu)
Tmes	2	Speed measurement time constant (s)
Tsm	3	Servo/gate time constant (s)
zdotmin	4	Minimum gate rate (pu/s), also additional parameter
zdotmax	5	Maximum gate rate (pu/s)
zmin	6	Minimum gate opening (pu)
zmax	7	Maximum gate opening (pu)
Thp	8	HP turbine time constant (s)
Fhp	9	HP power fraction
Tr	10	Reheater / MP time constant (s)
Fmp	11	MP power fraction
Tlp	12	LP turbine time constant (s)

Implementation details: - nbxvar = 10, nbzvar = 2, nbdata = 12, nbddpar = 2 - Additional: prn(13) = P0 (initial power), prn(14) = ivo (initial valve opening) - Observables: z, PmHP, PmMP, PmLP, Pm

22.3.2. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
          XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
          ↪ 0.05 * 0.1
EXC GENERIC1 1.8991 -0.1 0. 1. 100. -1. -11 10. 70. 10. 20.
↪ 0.1 0. 4.
          1 75. 15. 0.2 0.01 0.2 0.01
          ↪ -0.1 0.1
TOR THERMAL_GENERIC1 0.05 0.02 0.20 -0.10 0.10 0.0 1.05 0.30
↪ 0.30 7.0 0.40 0.50 ;
!          sigma Tmes Tsm zdotmin zdotmax zmin zmax Thp
↪ Fhp Tr Fmp Tlp

```

Notes

- The LP fraction is $1 - F_{hp} - F_{mp}$; ensure $F_{hp} + F_{mp} \leq 1$.
- Set $T_{mes} = 0$ to bypass the speed measurement filter (instantaneous speed sensing).
- Rate limits $zdotmin/zdotmax$ are critical for AGC studies.

22.4. HYDRO_GENERIC1 (tor_hydro_generic1)

The data-file model name is HYDRO_GENERIC1.

22.4.1. Scientific Description

The tor_hydro_generic1 model is a **generic hydraulic turbine governor** with a penstock (water column) and PI controller. It implements the standard hydraulic governor structure following IEEE Std 1110 conventions.

PI speed governor:

The speed error with droop σ :

$$e = (P_0 - P_{meas}) \cdot \sigma + (1 - \omega)$$

A PI controller with gains K_P and K_I drives the gate demand:

$$\dot{x}_I = K_I \cdot e$$

$$z_{demand} = K_P \cdot e + x_I$$

The gate is rate-limited by LIM_z and position-limited by $[0, 1]$, with a servo time constant T_{sm} .

Water column (penstock):

The flow Q and head H are related by the water-starting time T_W :

$$T_W \cdot \frac{dQ}{dt} = 1 - H$$

The head is:

$$H = \left(\frac{Q_v + T_m(1 - Q_v)}{z} \right)^2$$

where Q_v is the no-load flow fraction.

Turbine mechanical torque:

$$T_m = \sqrt{H} \cdot (Q - Q_v \cdot \sqrt{H})$$

$$P_m = T_m \cdot \omega$$

Parameters from Fortran associate block:

Parameter	Index	Description
SIGMA	1	Speed droop (pu/pu)
Tmes	2	Speed measurement time constant (s)
Qv	3	No-load water flow fraction (pu)
KP	4	PI proportional gain
KI	5	PI integral gain
TSM	6	Gate servo time constant (s)
LIMZDOT	7	Gate velocity limit (pu/s)
TW	8	Water starting time (s)

Implementation details: - nbxvar = 6, nbzvar = 2, nbdata = 8, nbaddpar = 1 - Additional: prm(9) = P0 (initial electrical power) - Observables: z, Q, H, Pm

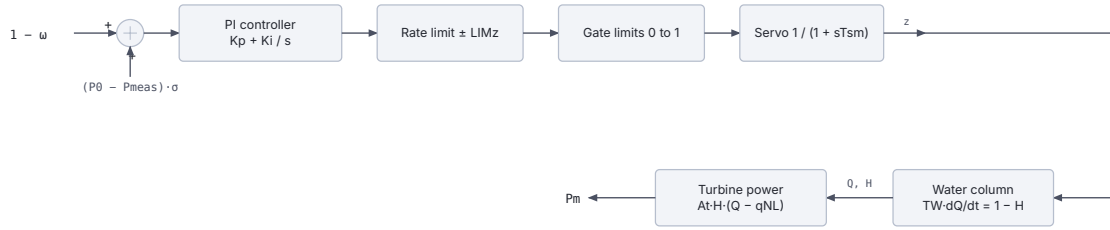


Figure 22.3: HYDRO_GENERIC1 block diagram.

22.4.2. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
            XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
            ↪ 0.05 * 0.1
EXC GENERIC1 1.8991 -0.1 0. 1. 100. -1. -11 10. 70. 10. 20.
            ↪ 0.1 0. 4.
            1 75. 15. 0.2 0.01 0.2 0.01
            ↪ -0.1 0.1
TOR HYDRO_GENERIC1 0.04 2.0 0. 2.00 0.40 0.2 0.1 1.0 ;
!                SIGMA TP Qv KP KI TSM LIMZDOT TW

```

Notes

- TW (water-starting time) is the most influential parameter; typical values are 0.5–3 s.
- Increase KP / KI carefully: large values can cause governor instability with high TW.
- $Q_v = 0$ assumes no no-load losses.
- Parameters follow the order in dyn_A.txt: SIGMA TP Qv KP KI TSM LIMZDOT TW.

22.5. HYDRO_DG (tor_HYDRO_DG)

The data-file model name is HYDRO_DG. Registered since 3.76.

22.5.1. Scientific Description

A hydraulic turbine governor for a **dispersed generation unit that holds an active-power setpoint and does not provide primary frequency response**.

The penstock and turbine are exactly those of HYDRO_GENERIC1. What differs is the governor control law, and the difference is the point of the model. HYDRO_GENERIC1 is a **speed governor**: its error signal carries an unconditional frequency term, so the unit responds to system frequency whether or not that is wanted. HYDRO_DG regulates power alone.

PI power regulator:

The error is the deviation from the setpoint captured at initialisation, with no droop term and no speed term:

$$e = P^* - P$$

$$\dot{x}_I = K_i \cdot e, \quad z_{demand} = K_p \cdot e + x_I$$

There is no measurement lag on P , so the regulator acts on the instantaneous electrical power. The gate follows through a servo of time constant T_{sm} , rate-limited to $\pm LIM_z$ and position-limited to $[0, 1]$.

Water column and turbine, identical to HYDRO_GENERIC1:

$$T_W \cdot \frac{dQ}{dt} = 1 - H, \quad H = \left(\frac{Q}{z}\right)^2$$

$$T_m \cdot \omega = \frac{(Q - Q_v)H}{1 - Q_v}$$

Parameters:

Parameter	Index	Description
Qv	1	No-load water flow fraction (pu)
Kp	2	PI proportional gain
Ki	3	PI integral gain
TSM	4	Gate servo time constant (s)
LIMZDOT	5	Gate velocity limit (pu/s)
TW	6	Water starting time (s)

These are exactly parameters 3 to 8 of HYDRO_GENERIC1, in the same order. HYDRO_GENERIC1 precedes them with SIGMA (droop) and Tmes (speed measurement lag), and those two are what HYDRO_DG does not have. Setting SIGMA to zero in HYDRO_GENERIC1 does **not** reproduce HYDRO_DG, because the frequency term survives.

Implementation details: - nbxvar = 5, nbzvar = 2, nbdata = 6, nbaddpar = 1 - Additional: prm(7) = Pset, the initial electrical power, captured at initialisation - Observables: z, Q, H, Pm, Pset



Penstock and turbine exactly as HYDRO_GENERIC1.

Figure 22.4: HYDRO_DG block diagram.

22.5.2. Usage Example

RAMSES Data File

```

SYNC_MACH G4 N4 1. 1. 0. 0. 5 4.5 3. 0. 1.9333
XT 0.0937 2.027 0.210 0.173 2.027 * 0.173 0.1 6 0.00 3.1
↪ 0.0387 * 0.0387
EXC AVR_DG 2.865 -0.1 0. 1. 100. -1. -20 10 50 4 20 0.1 0 5
          1 0. 5. 1. 1. 1. 1. 0. 0.
          0.04 0.06 -0.2 0.33
TOR HYDRO_DG 0.0 0.1 0.4 0.2 0.1 1.0 ;
!           Qv Kp Ki TSM LIMZDOT TW

```

Notes

- The setpoint is **not** a parameter. *Pset* is captured from the power-flow solution at initialisation, so the unit holds whatever it was dispatched to.
- Because there is no frequency term, a system built only of these units has no primary frequency control. That is deliberate for dispersed generation studies, and it is worth being explicit about when interpreting a frequency response.
- $Q_v = 0$ assumes no no-load losses.
- The head relation divides by the gate position, which is floored at 1e-3 in the implementation so a fully closed gate cannot produce a division by zero.

22.6. tor_gasturbm

Note

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected.

22.6.1. Scientific Description

The *tor_gasturbm* model is a **gas turbine** governor following a multi-stage combustion model. It captures the compressor, fuel system, combustion chamber delay, and turbine torque characteristic. The name suffix *m* indicates a modified version.

Speed governor:

The governor error drives a PI controller producing a fuel demand signal. The speed governor uses:

$$e_{gov} = (\omega_{ref} - \omega)/TDSP - STAT \cdot P_{elec}$$

with saturation limits $[MIN, MAX]$.

Fuel system and combustion:

The fuel valve position *g1* is driven through: - A speed governor with gain *ZZ* and lead-lag time constants determined by *XX*, *YY* - A fuel flow path through first-order lag *TVALVE* - Combustion chamber lag *TGAZ*

Turbine output:

The turbine mechanical power follows:

$$P_m = BF2 \cdot g_{comb} + AF2$$

where g_{comb} is the combustion output through lag *TCD*.

Additional thermodynamic correction terms involve *TF* (flame lag) and *ECR* (exhaust correction ratio).

Parameters from Fortran source (prm array):

Index	Name	Description
1	TDSP	Speed droop time constant (s)
2	STAT	Steady-state gain
3	ZZ	Governor proportional gain
4	XX	Lead numerator factor
5	YY	Lag denominator factor
6	SACC	Acceleration constant
7	MIN	Minimum fuel valve position (pu)
8	MAX	Maximum fuel valve position (pu)

Index	Name	Description
9	TVALVE	Fuel valve time constant (s)
10	TGAZ	Combustion chamber time constant (s)
11	TF	Flame lag time constant (s)
12	ECR	Exhaust correction ratio
13	TCD	Compressor discharge time constant (s)
14	BF2	Turbine power output slope
15	AF2	Turbine power output intercept (pu)

Implementation details: - nbxvar = 10, nbzvar = 4, nbdata = 15, nbaddpar = 1 - Additional: prm(16) = SETPOINT (initialised from load-flow power) - Observables: g2, x4, g1

22.6.2. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
          XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
          ↪ 0.05 * 0.1
EXC GENERIC1 1.8991 -0.1 0. 1. 100. -1. -11 10. 70. 10. 20.
↪ 0.1 0. 4.
          1 75. 15. 0.2 0.01 0.2 0.01
          ↪ -0.1 0.1
TOR GASTURBM 0.05 1.0 25.0 0.50 1.0 1.0 0.0 1.0 0.05 0.40
↪ 0.01 0.0 0.20 1.0 0.0 ;
!          TDSP STAT ZZ XX YY SACC MIN MAX TVALVE TGAZ TF
↪ ECR TCD BF2 AF2

```

22.7. tor_govclasm

Note

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected.

22.7.1. Scientific Description

The `tor_govclasm` model is a **classical thermal governor** with a composite control law including deadband, droop, integral control, and combustion dynamics. The name stands for “governor classical model”.

Speed governor with deadband and droop:

The speed error passes through a deadband [*DBON*, *DEADB*] before entering a proportional–integral controller. The integral path has saturation at *MAXINT* and gain *ALPHA*. The reset time *TRES* determines the integration rate.

Valve and combustion dynamics:

The valve demand passes through: 1. A governor integrator with time constant *TSM*, its rate held inside [*VFN*, *V0*] 2. A first-order lag *TFV* (valve actuator) 3. A rate limiter [*PVARMIN*, *PVARMAX*] (with gain *GPVAR*) representing fuel response 4. A combustion/steam chest lag *KCR*, *TCR* producing fuel flow 5. Lower bound: *MINFUEL* 6. Transport delay *TDELAY* (approximated) 7. Turbine output through lag *TCH* with limits [*PMIN*, *PMAX*]

Parameters from Fortran source (prm array):

Index	Name	Description
1	STAT	Governor static characteristic selector
2	DEADB	Deadband half-width (pu)
3	DBON	Deadband onset (pu)
4	TSM	Speed measurement / filter time constant (s)
5	V0	Maximum valve opening rate, upper limit on the governor integrator rate (pu/s)
6	VFN	Minimum valve rate, applied directly as the lower bound, so normally negative (pu/s)
7	TFV	Valve actuator time constant (s)
8	VFR	Fuel flow rate limit (pu/s)
9	MAXINT	Maximum integral output (pu)
10	ALPHA	Integral controller gain
11	TRES	Integral reset time (s)
12	GOMP	Governor proportional gain
13	PVARMAX	Maximum power variation rate (pu)
14	PVARMIN	Minimum power variation rate (pu)
15	GPVAR	Power variation gain
16	KCR	Combustion gain
17	TCR	Combustion time constant (s)
18	MINFUEL	Minimum fuel flow (pu)
19	TDELAY	Combustion transport delay (s)
20	PMAX	Maximum turbine output (pu)
21	TCH	Steam chest / turbine lag (s)
22	PMIN	Minimum turbine output (pu)

Implementation details: - nbxvar = 6, nbzvar = 10, nbdata = 22, nbaddpar = 1 - Additional: prm(23) = P0 (initialised from load-flow power) - Observables: x2, b3, b4

22.7.2. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
          XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
          ↪ 0.05 * 0.1
EXC GENERIC1 1.8991 -0.1 0. 1. 100. -1. -11 10. 70. 10. 20.
↪ 0.1 0. 4.
          1 75. 15. 0.2 0.01 0.2 0.01
          ↪ -0.1 0.1
TOR GOVCLASM 1.0 0.003 0.001 0.10 1.0 1.0 0.30 0.5 1.0 1.0
↪ 5.0 25.0 0.10 -0.10 0.05 1.0 0.20 0.05 0.30 1.05 0.30
↪ 0.05 ;
!          STAT DEADB DBON TSM V0 VFN TFV VFR MAXINT ALPHA
↪ TRES GOMP PVARMAX PVARMIN GPVAR KCR TCR MINFUEL TDELAY PMAX TCH
↪ PMIN

```

22.8. tor_govhydr

Note

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected.

22.8.1. Scientific Description

The `tor_govhydr` model is a **simplified hydraulic governor** with PI control and a non-linear penstock. It differs from `tor_hydro_generic1` in its simplified turbine characteristic and simpler water-column formulation.

PI governor:

$$\dot{x}_2 = K_I \cdot e, \quad x_2 \in [0, 1]$$

$$z_{demand} = K_P \cdot e + x_2$$

where the speed error $e = P_0/\omega - T_m - \omega \cdot STAT + \dots$ accounts for the droop and steady-state bias.

Penstock (two-lag water column):

$$T_{CE1} \cdot \frac{dx_3}{dt} = P - x_3(1 + T_{CE2}/T_{CE1})$$

The turbine mechanical power is computed via an empirical quadratic:

$$P_m = 1.35 \cdot z - 0.7z^2$$

derived from the non-linear head-flow curve approximated at initialisation.

Parameters from Fortran source:

Index	Name	Description
1	STAT	Steady-state gain selector
2	KP	Proportional gain
3	KI	Integral gain
4	TC	Governor/servo time constant (s)
5	V0	Maximum gate opening rate, upper limit on gate velocity (pu/s)
6	VF	Maximum gate closing rate, entered as a positive magnitude and applied as $-V_F$ (pu/s)
7	TCE2	Second penstock time constant (s)
8	TCE1	First penstock time constant (s)

Implementation details: - `nbxvar` = 4, `nbzvar` = 4, `nbdata` = 8, `nbddpar` = 1 - Additional: `prm(9)` = `P0` (initialised from load-flow power) - No observables defined (`nbobs` = 0)

22.8.2. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
           XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
           ↪ 0.05 * 0.1
EXC GENERIC1 1.8991 -0.1 0. 1. 100. -1. -11 10. 70. 10. 20.
↪ 0.1 0. 4.
           1 75. 15. 0.2 0.01 0.2 0.01
           ↪ -0.1 0.1
TOR GOVHYDR 1.0 2.0 0.5 0.10 1.0 1.0 1.0 5.0 ;
!           STAT KP KI TC V0 VF TCE2 TCE1

```

Notes

- TCE1 corresponds roughly to the water-starting time T_W .
- TCE2/TCE1 sets the ratio of the two penstock lags.

22.9. tor_govnuc

Note

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected.

22.9.1. Scientific Description

The `tor_govnuc` model is a **nuclear plant governor** with deadband, droop, integral control, and a non-linear turbine characteristic via `GOMP` (the gain of the output multiplier). It is structurally similar to `tor_govclasm` but tailored for the slower, tightly-regulated dynamics of nuclear steam supply systems.

Speed governor:

The speed error passes through a deadband [$DBON$, $DEADB$]:

$$e = \text{deadband}(\omega_{ref} - \omega) - STAT \cdot (P - P_0)$$

The error enters an integrator with rate limits [$GRADMIN$, $GRADMAX$] (pu/s) filtered through a ramp time $TGRAD$. The integrator output x_I passes through a servo TSM .

Turbine characteristic with `GOMP`:

The turbine output is non-linearly mapped through a gain $GOMP$:

$$P_m = \min\left(\frac{x_I}{GOMP}, 1\right) \cdot f(x_I)$$

where the limiter prevents over-power and f is a piecewise-linear characteristic.

The steam-valve rate is held inside [VFN , $V0$], then passes the valve actuator lag TFV , the reset lag $TRES$ and $GOMP$ to produce the final torque. VFR overrides the rate during fast valving, a path the shipped model never takes: the code fixes its selector at 1 and leaves the fast-valving logic unwritten.

Parameters from Fortran source:

Index	Name	Description
1	STAT	Governor static characteristic selector
2	DEADB	Deadband half-width (pu)
3	DBON	Deadband onset (pu)
4	GRADMAX	Maximum load gradient (pu/s)
5	GRADMIN	Minimum load gradient / unloading rate (pu/s)
6	TGRAD	Ramp filter time constant (s)
7	TSM	Servo time constant (s)
8	V0	Maximum valve opening rate, upper limit on the steam-valve rate (pu/s)
9	VFN	Minimum valve rate, applied directly as the lower bound, so normally negative (pu/s)
10	TFV	Valve actuator time constant (s)
11	VFR	Valve rate limit (pu/s)
12	ALPHA	Integral gain
13	TRES	Reset time constant (s)
14	GOMP	Output power gain

Implementation details: - nbxvar = 4, nbzvar = 8, nbdata = 14, nbaddpar = 1 - Additional: prm(15) = P0 (initialised from load-flow power) - Observable: TM

22.9.2. Usage Example

RAMSES Data File

```

SYNC_MACH g1 g1 1. 1. 0. 0. 800. 760. 3. 0. 0.95
                XT 0.15 1.1 0.25 0.2 0.7 * 0.2 0.1 6.0257 0. 5.00
                ↪ 0.05 * 0.1
EXC GENERIC1 1.8991 -0.1 0. 1. 100. -1. -11 10. 70. 10. 20.
↪ 0.1 0. 4.
                1 75. 15. 0.2 0.01 0.2 0.01
                ↪ -0.1 0.1
TOR GOVNUC 1.0 0.005 0.002 0.02 -0.02 10.0 5.0 1.0 1.0 0.50
↪ 0.10 1.0 20.0 1.0 ;
!          STAT DEADB DBON GRADMAX GRADMIN TGRAD TSM V0 VFN TFV
↪ VFR ALPHA TRES GOMP

```

Notes

- Nuclear plants typically have slow load-following characteristics: GRADMAX is typically 0.005–0.02 pu/s.
- The DEADB is important to prevent hunting around the setpoint.
- GOMP scales the turbine output; set to 1.0 for standard normalisation.

23

Injector models

RAMSES custom injector models represent loads, induction machines, inverter-based resources (IBR), and battery energy storage systems (BESS) connected to network buses.

Usage in Dynamic Data Files

Injector models are defined with the INJEC keyword as standalone records in the dynamic data file. The syntax is:

```
INJEC model_name inj_name bus FP FQ P Q param1 param2 ... ;
```

Where `model_name` is the model identifier, `inj_name` is a unique instance name, `bus` is the connection bus, `FP/FQ` are participation factors, and `P/Q` are initial values in MW/Mvar (one of each pair must be zero, either the fraction or the absolute power).

Recognised injector model names (case-sensitive):

- **Uppercase short names** (no prefix): `LOAD`, `RESTLD`, `THEVEQ`, `INDMACH1`, `INDMACH2`, `SVC_GENERIC1`.
- **Prefixed names**: RAMSES adds the `inj_` prefix automatically: `VFAULT/inj_VFAULT`, `vfd_load/inj_vfd_load`, `PQ/inj_PQ`, `IBG/inj_IBG`, `WT3/inj_WT3`, `WT4/inj_WT4`, `BESS/inj_BESS`, `GFOL/inj_GFOL`, `GFOR/inj_GFOR`, `PMU/inj_PMU`.

All of the above are built into every RAMSES distribution (standalone executable and shared library used by stepss). `INDM1` and `PV`, documented below, have been callable since 3.79; before that they were compiled under no name. `inj_norton` is excluded from the build entirely and is not available: it calls a Python bridge (`initialize_model_instance`, `call_python_evaluate`) that is not part of this source tree.

Note on case sensitivity. Match the case used above exactly. The uppercase short names must be uppercase. For the prefixed family, `vfd_load` and `VFAULT` use those exact cases. Most others follow the convention `inj_<UPPERCASE>` (e.g. `inj_PQ`, `inj_GFOR`).

23.1. Load Models

23.1.1. LOAD (inj_load): Exponential Recovery Load

Description

The exponential recovery load model captures the transient and steady-state voltage and frequency dependency of aggregated loads. Immediately after a voltage disturbance, the load behaves according to a transient voltage exponent; it then recovers exponentially to a steady-state behaviour described by a different exponent. The model supports separate active (P) and reactive (Q) power recovery dynamics, each with individual minimum/maximum limiters on the recovery variable.

Scientific Description

The model is parameterized in terms of initial active and reactive conductance/susceptance, $G_0 = P_0/V_0^2$ and $B_0 = -Q_0/V_0^2$. Two recovery state variables x_P and x_Q evolve according to:

$$\dot{x}_P = \frac{1}{T_r} \left[\left(\frac{V}{V_0} \right)^{\alpha_s} (1 + D_P \Delta\omega) - x_P \left(\frac{V}{V_0} \right)^{\alpha_t} \right]$$

$$\dot{x}_Q = \frac{1}{T_r} \left[\left(\frac{V}{V_0} \right)^{\beta_s} (1 + D_Q \Delta\omega) - x_Q \left(\frac{V}{V_0} \right)^{\beta_t} \right]$$

where $\Delta\omega = \omega_{\text{COI}} - 1$ is the per-unit speed deviation, V is the bus voltage magnitude, V_0 is the initial voltage, α_t, β_t are transient exponents, α_s, β_s are steady-state exponents, and T_r is the load recovery time constant. The injected currents are then:

$$i_x = G_0 x_P v_x (1 + D_P \Delta\omega) - B_0 x_Q v_y (1 + D_Q \Delta\omega)$$

$$i_y = G_0 x_P v_y (1 + D_P \Delta\omega) + B_0 x_Q v_x (1 + D_Q \Delta\omega)$$

When the recovery variable hits its limit (due to e.g. Stalling or complete voltage collapse), the limit is held until the direction of the derivative reverses.

Parameters

#	Name	Description	Unit
1	DP	Frequency sensitivity of active power	pu/pu
2	A1	Proportion of type-1 component in P	
3	alpha1	Transient voltage exponent for P (type 1)	
4	A2	Proportion of type-2 component in P	
5	alpha2	Transient voltage exponent for P (type 2)	
6	alpha3	Transient voltage exponent for P (type 3, proportion = $1 - A_1 - A_2$)	
7	DQ	Frequency sensitivity of reactive power	pu/pu
8	B1	Proportion of type-1 component in Q	
9	beta1	Transient voltage exponent for Q (type 1)	
10	B2	Proportion of type-2 component in Q	
11	beta2	Transient voltage exponent for Q (type 2)	
12	beta3	Transient voltage exponent for Q (type 3)	

Internal computed parameters include the steady-state exponents for each component (derived from the transient exponents), initial conductance G_0 , susceptance B_0 , and initial voltage V_0 .

State Variables

Variable	Description
iy	y-component of injected current (pu on system base)
ix	x-component of injected current (pu on system base)
xp	Active power recovery variable (dimensionless)
xq	Reactive power recovery variable (dimensionless)

Observables: P, Q, xp, xq

Usage Example

```
INJEC LOAD LOAD1 BUS1 1. 1. 0. 0. 1.5 0.3 1.0 0.2 2.0 0.5 1.8 0.4 2.5
↪ 0.1 3.0 2.0 ;
```

Parameters: DP A1 alpha1 A2 alpha2 alpha3 DQ B1 beta1 B2 beta2 beta3

23.1.2. vfd_load (inj_vfd_load): Variable Frequency Drive Load

Description

The VFD load model represents aggregate industrial loads driven by variable-frequency drives, where the power consumption exhibits a composite voltage-dependent characteristic with multiple exponential components and frequency sensitivity. It also includes low-voltage protection: below a configurable threshold $V_{\min}$, the load switches to a constant-admittance representation, preventing numerical difficulties during deep voltage sags.

Scientific Description

The active and reactive powers depend on bus voltage V and frequency deviation $\Delta f = f/f_0 - 1$:

$$P = (1 + D_P \Delta f) P_0 \cdot \frac{a_1 V^{\alpha_1} + a_2 V^{\alpha_2} + (1 - a_1 - a_2) V^{\alpha_3}}{a_1 V_0^{\alpha_1} + a_2 V_0^{\alpha_2} + (1 - a_1 - a_2) V_0^{\alpha_3}}$$

$$Q = (1 + D_Q \Delta f) Q_0 \cdot \frac{b_1 V^{\beta_1} + b_2 V^{\beta_2} + (1 - b_1 - b_2) V^{\beta_3}}{b_1 V_0^{\beta_1} + b_2 V_0^{\beta_2} + (1 - b_1 - b_2) V_0^{\beta_3}}$$

Below $V_{\min}$, an equivalent constant admittance is used:

$$P_{\text{low}} = G_{\text{eq}} V^2, \quad Q_{\text{low}} = -B_{\text{eq}} V^2$$

where G_{eq} and B_{eq} are computed at initialization to ensure continuity at $V = V_{\min}$. The switch between the two regimes is governed by a piecewise-linear function of V :

$$u = \begin{cases} 0 & V < V_{\min} \\ 1 & V \geq V_{\min} \end{cases}$$

A small voltage filter (time constant 0.003 s) smooths the transition. The frequency deviation can optionally be computed from the local bus frequency (measured via an f_inj block with time constant T_{mes}) or from the system centre-of-inertia speed.

The initial voltage V_0 can differ from the transmission bus voltage if a distribution transformer ratio is implied (set $V_{\text{init}} \neq 0$ to specify the distribution-side voltage; otherwise the transmission voltage is used directly).

Parameters

#	Name	Description	Unit
1	Dp	Frequency sensitivity of active power	pu/pu
2	a1	Fraction of type-1 component in P	
3	alpha1	Voltage exponent of type-1 P component	
4	a2	Fraction of type-2 component in P	
5	alpha2	Voltage exponent of type-2 P component	
6	alpha3	Voltage exponent of type-3 P component (fraction = $1 - a_1 - a_2$)	
7	Dq	Frequency sensitivity of reactive power	pu/pu
8	b1	Fraction of type-1 component in Q	
9	beta1	Voltage exponent of type-1 Q component	
10	b2	Fraction of type-2 component in Q	
11	beta2	Voltage exponent of type-2 Q component	
12	beta3	Voltage exponent of type-3 Q component	
13	Vinit	Initial distribution-bus voltage (0 = use transmission voltage)	pu
14	Vlow	Voltage threshold for constant-admittance regime (recommended 0.5–0.7)	pu
15	foption	1 = use local bus frequency; 0 = use COI speed	flag
16	Tmes	Frequency measurement time constant	s

State Variables

Variable	Description
Vreal	Voltage at equivalent distribution bus
V	Filtered distribution-bus voltage (3 ms filter)
P	Active power consumed
Q	Reactive power consumed
fbus	Local bus frequency (used if foption=1)
df	Frequency deviation Δf
u	Regime switch (1 = above $V_{\min}$, 0 = constant admittance)

Observables: P, Q, df, u

Usage Example

```
INJEC vfd_load VFD1 BUS_IND 1. 1. 0. 0. 1.5 0.7 2.0 0.2 1.0 0.5
                                1.2 0.5 2.5 0.1 1.5 0.8
                                0.0 0.6 0 0.1 ;
```

23.1.3. RESTLD (inj_restld): Restorative Load**Description**

The restorative load model represents loads that self-restore toward a nominal characteristic after a voltage disturbance. The load's active and reactive powers are governed by two internal recovery variables that evolve dynamically, allowing the simulation to capture the slow restoration of thermostatically controlled loads (heating, cooling) and similar self-restoring demand.

Scientific Description

The recovery variable x_P for active power satisfies:

$$\dot{x}_P = \frac{1}{T_r} [V^{\alpha_s} (1 + D_P \Delta\omega) - x_P V^{\alpha_t}]$$

with analogous equation for x_Q . Here α_s and α_t are the steady-state and transient voltage exponents respectively. Limiters $[x_{P,\min}, x_{P,\max}]$ are applied. The injected currents are expressed as:

$$i_y = -B_0 (1 + D_Q \Delta\omega) x_Q v_x + G_0 (1 + D_P \Delta\omega) x_P v_y$$

$$i_x = B_0 (1 + D_Q \Delta\omega) x_Q v_y + G_0 (1 + D_P \Delta\omega) x_P v_x$$

where the ratio V/V_0 governs the voltage dependence through v_{rat} .

Parameters

#	Name	Description	Unit
1	DP	Frequency sensitivity of active power	pu/pu
2	alphat	Transient active power voltage exponent	
3	alphas	Steady-state active power voltage exponent	
4	xP_min	Minimum limit for x_P	
5	xP_max	Maximum limit for x_P	
6	DQ	Frequency sensitivity of reactive power	pu/pu
7	betat	Transient reactive power voltage exponent	
8	betas	Steady-state reactive power voltage exponent	
9	xQ_min	Minimum limit for x_Q	
10	xQ_max	Maximum limit for x_Q	
11	Tr	Load recovery time constant	

State Variables

Variable	Description
iy	y-component of injected current
ix	x-component of injected current
xp	Active power recovery variable
xq	Reactive power recovery variable

Observables: P, Q, xp, xq

Usage Example

```
INJEC RESTLD RESTLD1 BUS2 1. 1. 0. 0. 1.5 0.5 2.0 0.0 2.0 1.2 0.5 2.5
↪ 0.0 2.0 60.0 ;
```

23.1.4. PQ (inj_PQ): Constant PQ Load

Description

The simplest injector model: maintains constant active and reactive power consumption regardless of bus voltage or frequency. The power is fixed at its initial operating-point value. A small first-order filter (time constant T_{out}) drives the injected currents smoothly to their target values, preventing algebraic loops.

Scientific Description

The current references are set to deliver the initial powers P_0 and Q_0 at the measured voltage V :

$$I_{x,\text{set}} = \frac{P_0 v_x + Q_0 v_y}{V^2}, \quad I_{y,\text{set}} = \frac{P_0 v_y - Q_0 v_x}{V^2}$$

These references pass through a first-order filter with time constant T_{out} to produce the actual injected currents:

$$T_{\text{out}} \dot{i}_x + i_x = I_{x,\text{set}}, \quad T_{\text{out}} \dot{i}_y + i_y = I_{y,\text{set}}$$

Parameters

#	Name	Description	Unit
1	Tout	Output filter time constant (recommended: 0.01 s)	s

Initial conditions P_0 , Q_0 , V_0 are computed automatically at initialization.

State Variables

Variable	Description
ix	Injected x -current
iy	Injected y -current
Ixset	Current reference x
Iyset	Current reference y
P	Observed active power
Q	Observed reactive power
V	Bus voltage magnitude

Observables: P, Q

Usage Example

```
INJEC inj_PQ LOAD_PQ BUS3 1. 1. 0. 0. 0.01 ;
```

23.1.5. THEVEQ (inj_theveq): Thévenin Equivalent**Description**

Models an external network or generator cluster as a Thévenin equivalent: an ideal voltage source $\bar{E}_{\text{th}}$ behind a pure reactance X_{th} . The model computes the internal voltage magnitude and phase angle at initialization from the initial bus conditions and holds them constant during the simulation. It is useful for representing neighbouring system equivalents or simplified machine representations.

Scientific Description

The Thévenin reactance is obtained from the specified short-circuit power S_{sc} (MVA):

$$X_{\text{th}} = \frac{S_{\text{base}}}{S_{\text{sc}}}$$

The internal voltage phasor is computed at $t = 0$:

$$E_{\text{th},x} = v_x - X_{\text{th}} i_y, \quad E_{\text{th},y} = v_y + X_{\text{th}} i_x$$

$$E_{th} = \sqrt{E_{th,x}^2 + E_{th,y}^2}, \quad \phi = \arctan\left(\frac{E_{th,y}}{E_{th,x}}\right)$$

During simulation the injected currents satisfy:

$$i_y = -\frac{E_{th} \cos \phi - v_x}{X_{th}}, \quad i_x = \frac{E_{th} \sin \phi - v_y}{X_{th}}$$

Parameters

#	Name	Description	Unit
1	XTH	Short-circuit power of the equivalent (converted to X_{th} at initialization)	MVA

Internal parameters ETH (Thévenin voltage magnitude, pu) and phase (internal angle, rad) are computed automatically.

State Variables

Variable	Description
iy	Injected y -current
ix	Injected x -current

Observables: P, Q (in MW and Mvar at system base)

Usage Example

```
INJEC THEVEQ EQUIV1 SLACK_BUS 1. 1. 0. 0. 2000.0 ;
```

23.2. Induction Machine Models

23.2.1. INDMACH1 (inj_indmach1): Single-Cage Induction Machine

Description

A single-cage (single-rotor-circuit) induction machine model for motor loads. The machine is represented on its own MVA base (or inferred from load factor LF) with a shunt capacitor B_{sh} to represent power factor correction. The mechanical torque is a quadratic function of rotor speed. At initialization, the model solves nonlinear algebraic equations to find the operating-point slip and flux linkages.

Scientific Description

The machine uses the standard d - q reference-frame formulation with $L_{ss} = L_{sr} + L_{ls}$ and $L_{rr} = L_{sr} + L_{lr}$. The rotor flux-linkage equations are:

$$\begin{aligned} \frac{d\psi_{dr}}{dt} &= \omega_0 \left(-\frac{R_r}{L_{rr}} \psi_{dr} + \frac{L_{sr} R_r}{L_{rr}} i_{ym} - (\omega - \omega_m) \psi_{qr} \right) \\ \frac{d\psi_{qr}}{dt} &= \omega_0 \left(-\frac{R_r}{L_{rr}} \psi_{qr} + \frac{L_{sr} R_r}{L_{rr}} i_{xm} + (\omega - \omega_m) \psi_{dr} \right) \end{aligned}$$

where $\omega_0 = 2\pi f_{nom}$ and ω_m is the rotor mechanical speed. The equations for the stator (with shunt susceptance B_{sh}) are:

$$R_s i_{ym} + \left(L_{ss} - \frac{L_{sr}^2}{L_{rr}} \right) \omega i_{xm} + \frac{L_{sr}}{L_{rr}} \omega \psi_{qr} = v_y$$

$$R_s i_{xm} - \left(L_{ss} - \frac{L_{sr}^2}{L_{rr}} \right) \omega i_{ym} - \frac{L_{sr}}{L_{rr}} \omega \psi_{dr} = v_x$$

The rotor speed dynamics follow the swing equation:

$$\frac{d\omega_m}{dt} = \frac{T_e - T_m}{2H}$$

where the electromagnetic torque is:

$$T_e = \frac{L_{sr}}{L_{rr}} (-\psi_{dr} i_{xm} + \psi_{qr} i_{ym})$$

and the mechanical torque is the quadratic load curve:

$$T_m = T_{m0} (A \omega_m^2 + B \omega_m + 1 - A - B)$$

Parameters

#	Name	Description	Unit
1	SNOM	Machine nominal apparent power (0 = infer from LF)	MVA
2	RS	Stator resistance	pu
3	Lls	Stator leakage inductance	pu
4	LSR	Magnetizing inductance	pu
5	RR	Rotor resistance	pu
6	Llr	Rotor leakage inductance	pu
7	H	Machine inertia constant	s
8	A	Quadratic torque-speed coefficient	
9	B	Linear torque-speed coefficient	
10	LF	Load factor (for SNOM inference)	

State Variables

Variable	Description
iy	y-component of stator current
ix	x-component of stator current
psidr	d-axis rotor flux linkage
psiqr	q-axis rotor flux linkage
omegam	Rotor mechanical speed

Observables: P, Qmot+comp, Qmot, omega, Tm

Usage Example

```
INJEC INDMACH1 MTR1 BUS_MV 1. 1. 0. 0. 10.0 0.01 0.10 2.50 0.015 0.10
↪ 1.5 0.8 0.1 0.0 ;
```


23.2.2. INDMACH2 (inj_indmach2): Double-Cage Induction Machine

Description

A double-cage (double-rotor-circuit) induction machine model following the Eurostag formulation. Two parallel rotor cages allow more accurate representation of the machine's impedance-vs-frequency characteristic, which is especially important for the starting transient. The model structure mirrors inj_indmach1 but includes a second set of rotor flux states.

Scientific Description

The machine has parameters: stator resistance R_1 , stator leakage L_1 , magnetizing inductance L_m , cage-1 resistance R_2 and leakage L_2 , cage-2 resistance R_3 and leakage L_3 . The state vector is $(\psi_{dr1}, \psi_{qr1}, \psi_{dr2}, \psi_{qr2}, \omega_m)$ with flux-linkage equations for each cage:

$$\frac{d\psi_{dr1}}{dt} = \omega_0 \left(-\frac{R_2}{L_A} \psi_{dr1} + \frac{L_m R_2}{L_A} i_{ym} - (\omega - \omega_m) \psi_{qr1} \right)$$

$$\frac{d\psi_{dr2}}{dt} = \omega_0 \left(-\frac{R_3}{L_B} \psi_{dr2} + \frac{L_m R_3}{L_B} i_{ym} - (\omega - \omega_m) \psi_{qr2} \right)$$

where $L_A = L_m + L_2$ and $L_B = L_m + L_3$. The total electromagnetic torque combines contributions from both cages:

$$T_e = \frac{L_m}{L_A} (-\psi_{dr1} i_{xm} + \psi_{qr1} i_{ym}) + \frac{L_m}{L_B} (-\psi_{dr2} i_{xm} + \psi_{qr2} i_{ym})$$

The swing equation is identical to inj_indmach1.

Parameters

#	Name	Description	Unit
1	SNOM	Machine MVA rating (0 = infer from LF)	MVA
2	R1	Stator resistance	pu
3	L1	Stator leakage inductance	pu
4	Lm	Magnetizing inductance	pu
5	R2	First cage resistance	pu
6	L2	First cage leakage inductance	pu
7	R3	Second cage resistance	pu
8	L3	Second cage leakage inductance	pu
9	H	Inertia constant	s
10	A	Quadratic torque-speed coefficient	
11	B	Linear torque-speed coefficient	
12	LF	Load factor (for SNOM inference)	

State Variables

Variable	Description
iy	y -component of stator current
ix	x -component of stator current
psidr1	d -axis flux of first rotor cage
psiqr1	q -axis flux of first rotor cage
psidr2	d -axis flux of second rotor cage
psiqr2	q -axis flux of second rotor cage
omegam	Rotor mechanical speed

Observables: P, Qmot+comp, Qmot, omega

Usage Example

```
INJEC INDMACH2 MTR2 BUS_MV 1. 1. 0. 0. 10.0 0.01 0.08 2.00 0.02 0.06
↪ 0.04 0.10 1.5 0.8 0.1 0.0 ;
```

23.2.3. inj_INDM1: Alternative Induction Machine**Note**

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected. For built-in single-cage motors, use INDMACH1 instead.

Description

An alternative single-cage induction machine model that uses the INI_indmach1 helper function for initialization. It is equivalent in physics to inj_indmach1 but implements the equations using RAMSES .txt-style model syntax with explicit state initialization calls. This can simplify parameterization when the helper function's output is directly used.

Scientific Description

The model equations match those of inj_indmach1. The key distinction is the use of the INI_indmach1 function at parameter-evaluation time to pre-compute B_{sh} , T_{m0} , and the initial flux linkages and rotor speed, rather than solving the initialization system in Fortran. With $L_{ss} = L_{sr} + L_{ls}$ and $L_{rr} = L_{sr} + L_{lr}$, the rotor flux equations are:

$$\frac{d\psi_{dr}}{dt} = 2\pi f_0 \left[-\frac{R_R}{L_{rr}}\psi_{dr} + \frac{L_{SR}R_R}{L_{rr}}i_{ym} - (\omega - \omega_m)\psi_{qr} \right]$$

$$\frac{d\psi_{qr}}{dt} = 2\pi f_0 \left[-\frac{R_R}{L_{rr}}\psi_{qr} + \frac{L_{SR}R_R}{L_{rr}}i_{xm} + (\omega - \omega_m)\psi_{dr} \right]$$

The rotor speed integral uses:

$$\frac{d\omega_m}{dt} = \frac{1}{2H} \left[\frac{L_{SR}}{L_{rr}}(-\psi_{dr}i_{xm} + \psi_{qr}i_{ym}) - T_{m0}(A\omega_m^2 + B\omega_m + 1 - A - B) \right]$$

with a lower limit of $\omega_m \geq 0$.

Parameters

#	Name	Description	Unit
1	SNOM	Machine MVA rating	MVA
2	RS	Stator resistance	pu
3	Lls	Stator leakage inductance	pu
4	LSR	Magnetizing inductance	pu
5	RR	Rotor resistance	pu
6	Llr	Rotor leakage inductance	pu
7	H	Inertia constant	s
8	A	Quadratic torque-speed coefficient	
9	B	Linear torque-speed coefficient	
10	LF	Load factor	

Computed internally: BSH (shunt susceptance), TM0 (initial mechanical torque), LSS, LRR.

State Variables

Variable	Description
Psidr	d -axis rotor flux linkage
Psiqr	q -axis rotor flux linkage
omegam	Rotor mechanical speed
iy _m , ix _m	Stator current components (on machine base)
dPsidr, dPsiqr, domegam	Time-derivative auxiliary states

Observables: omegam

Usage Example

```
INJEC INDM1 MTR3 BUS_MV 1. 1. 0. 0. 10.0 0.01 0.10 2.50 0.015 0.10 1.5
↪ 0.8 0.1 0.0 ;
```

23.3. Renewable Generation / Inverter-Based Resources**23.3.1. IBG (inj_IBG): Inverter-Based Generator (Generic IBR)****Description**

A generic inverter-based generation model suitable for representing aggregated distributed generation or any grid-following IBR. The model includes a Phase-Locked Loop (PLL), inner current control with active (I_p) and reactive (I_q) current commands, LVRT/HVRT logic with voltage-dependent reactive current injection, frequency-responsive active power modulation, and reconnection logic after disconnection events.

Scientific Description

The PLL tracks the terminal voltage angle θ via a second-order PI controller with a freeze option below $V_{\min, \text{pll}}$:

$$\frac{d\theta_{\text{PLL}}}{dt} = \Delta\omega_{\text{PLL}}, \quad \frac{d\Delta\omega_{\text{PLL}}}{dt} = k_{\text{PLL}} v_q$$

where v_q is the q -axis terminal voltage in the PLL frame. The current commands $I_{p, \text{cmd}}$ and $I_{q, \text{cmd}}$ are derived from outer controls:

- Active power is modulated by frequency according to frequency deadband fdbd :

$$P = P_{\text{ext}} (1 + b \text{sat}(\Delta f - f_{\text{start}}, f_{\text{min}}, f_{\text{max}}))$$

- During voltage dips (LVRT), reactive current is boosted:

$$I_{q, \text{boost}} = k_{\text{RCI}} (V_{\text{ref}} - V_t)$$

- Current magnitude is limited to I_{max} with priority to reactive current during LVRT.

The injected currents in x - y frame are:

$$i_x = I_p \cos \theta_{\text{PLL}} - I_q \sin \theta_{\text{PLL}}$$

$$i_y = I_p \sin \theta_{\text{PLL}} + I_q \cos \theta_{\text{PLL}}$$

Parameters

#	Name	Description	Unit
1	I _{max}	Maximum current magnitude	pu
2	I _N	Nominal current	pu
3	I _{prate}	Active current ramp rate	pu/s
4	T _g	Generator time constant	s
5	T _m	Measurement filter time constant	s
6	t _{LVRT1}	LVRT ride-through time threshold 1	s
7	t _{LVRT2}	LVRT ride-through time threshold 2	s
8	t _{LVRTint}	LVRT integration time	s
9	V _{max}	Maximum voltage for operation	pu
10	tau	PLL response time	ms
11	V _{minpll}	Voltage below which PLL is frozen	pu
12	a	LVRT voltage-current curve slope	
13	V _{min}	Minimum LVRT voltage	pu
14	V _{int}	Intermediate LVRT voltage	pu
15	f _{min}	Minimum frequency for operation	pu
16	f _{max}	Maximum frequency for operation	pu
17	f _{start}	Frequency threshold for P modulation	pu
18	b	Frequency droop gain	
19	f _r	Reference frequency	pu
20	T _r	Reconnection delay after trip	s
21	R _e	Grid equivalent resistance	pu
22	X _e	Grid equivalent reactance	pu
23	CM1	LVRT control mode flag	
24	k _{RCI}	Reactive current injection gain (LVRT)	
25	k _{RCA}	Reactive current absorption gain (HVRT)	
26	m	Active-reactive current priority parameter	
27	n	Active-reactive current priority parameter	
28	dbmin	Frequency deadband lower limit	pu
29	dbmax	Frequency deadband upper limit	pu
30	HVRT	HVRT flag	
31	LVRT	LVRT flag	
32	CM2	Control mode 2 flag	
33	V _{trip}	Trip voltage threshold	pu

State Variables

Variable	Description
v _{x1} , v _{y1}	Filtered terminal voltage components
V _t	Terminal voltage magnitude
PLLPhaseAngle	PLL phase angle
V _m	Voltage magnitude measurement
I _p , I _q	Active and reactive current outputs
I _p cmd, I _q cmd	Current commands
I _q max, I _q min	Reactive current limits
I _p max, I _p min	Active current limits
DeltaW, DeltaWf	Frequency deviation and filtered value
P _{gen} , Q _{gen}	Generated active and reactive power

Usage Example

```

INJEC  IBG  IBG1  BUS_GEN  1.  1.  0.  0.  1.2  1.0  0.5  0.02  0.01  0.5  1.0  0.05
                                1.1  20.0  0.1  2.0  0.1  0.5  0.95  1.05

```

```

0.0  2.0  1.0  1.0  0.0  0.05  1  2.0  1.5
2.0  2.0  -0.02  0.02  1  1  1  0.85 ;

```

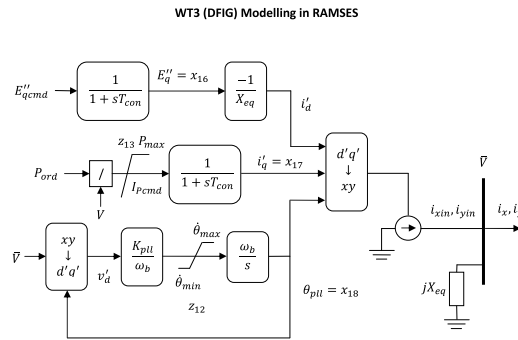
23.3.2. WT3 (inj_WT3): Type 3 Wind Turbine (DFIG)

The data-file model name is WT3 (or inj_WT3).

Description

A Type 3 wind turbine model implementing the WECC composite structure with four coupled sub-models: - **REPC_A**: Plant-level controller (reactive power / voltage regulation, frequency response) - **REEC_A**: Electrical controller (inner d - q current control, LVRT/HVRT logic) - **WTGTRQ_A**: Generator torque controller (rotor speed regulation via electrical torque command) - **WTGPT_A**: Pitch controller (aerodynamic power limitation) - **WTGAR_A**: Aerodynamic rotor model - **WTGT_A**: Two-mass mechanical drivetrain (turbine inertia H_t , generator inertia H_g , shaft stiffness K_{shaft} , damping D_{shaft})

The doubly-fed induction generator (DFIG) topology allows decoupled control of active and reactive power via rotor-side converter injection.



Network and injector currents relationship in DFIG base:

$$i_{xin} = i_q' \cos(\theta_{pll}) + \frac{E_q''}{X_{eq}} \sin(\theta_{pll})$$

$$i_{yin} = i_q' \sin(\theta_{pll}) - \frac{E_q''}{X_{eq}} \cos(\theta_{pll})$$

$$0 = -i_{yin} + i_y \frac{S_{base}}{S_{nom}} - \frac{v_x}{X_{eq}}$$

$$0 = -i_{xin} + i_x \frac{S_{base}}{S_{nom}} + \frac{v_y}{X_{eq}}$$

$$v_d = -v_x \sin(\theta_{pll}) + v_y \cos(\theta_{pll})$$

with $E_q'' = x_{16}$, $i_q' = x_{17}$, and $\theta_{pll} = x_{18}$

Figure 23.1: WT3 (DFIG) wind turbine model block diagram

Scientific Description

The two-mass drivetrain model governs rotor dynamics:

$$2H_t \frac{d\omega_t}{dt} = T_m - K_{\text{shaft}}(\theta_t - \theta_g) - D_{\text{shaft}}(\omega_t - \omega_g)$$

$$2H_g \frac{d\omega_g}{dt} = K_{\text{shaft}}(\theta_t - \theta_g) + D_{\text{shaft}}(\omega_t - \omega_g) - T_e$$

The torque controller (WTGTRQ_A) uses a piecewise power-speed characteristic:

$$T_{e,\text{ref}} = \text{interp}(\omega_g; [(\text{spd}_1, p_1), (\text{spd}_2, p_2), (\text{spd}_3, p_3), (\text{spd}_4, p_4)])$$

The electrical controller (REEC_A) provides reactive current injection during voltage dips:

$$I_{q,\text{vdip}} = k_{qv} (V_{\text{ref0}} - V_t) \quad \text{when } V_t < V_{\text{dip}}$$

with limiter $[I_{ql1}, I_{qh1}]$.

The plant controller (REPC_A) optionally provides frequency response:

$$\Delta P_{\text{ref}} = D_{\text{dn}} \text{sat}(\Delta f, f_{\text{dbd1}}, 0) + D_{\text{up}} \text{sat}(\Delta f, 0, f_{\text{dbd2}})$$

Parameters (selected key parameters)

#	Name	Sub-model	Description	Unit
1	SNOM		Nominal power	MW
2–32	REPC_A	Plant controller	Reactive/voltage/frequency control	various
33–46	WTGTRQ_A	Torque ctrl	Speed-torque lookup table, rate limits	various
47–56	WTGPT_A	Pitch ctrl	Pitch PI gains, angle limits, rate limits	various
57–58	WTGAR_A	Aero	Aerodynamic gain, initial pitch angle	
59–62	WTGT_A	Drivetrain	$H_t, H_g, D_{\text{shaft}}, K_{\text{shaft}}$	s, pu
63–97	REEC_A	Elec ctrl	LVRT/HVRT, current limits, inner PI	various
98+	REGC_A	Generator	Generator electrical conversion	various

Full parameter list has 80+ entries; refer to a working example data file for the complete ordering.

Usage Example

```
INJEC WT3 WT3_1 BUS_WIND 1. 1. 0. 0.
100.0 0.0 0.02 0 0.0 0.05 0 0.0 0.3 -0.3 2.0 0.4 0.3 -0.3 0.0 0.15
↪ 0.9 0.05
0.0 -0.06 0.06 -0.05 0.05 0.05 -0.05 2.0 1.0 1.0 -1.0 0.02 0 0 0.8
1.5 1.5 0.01 0.5 ... ;
```

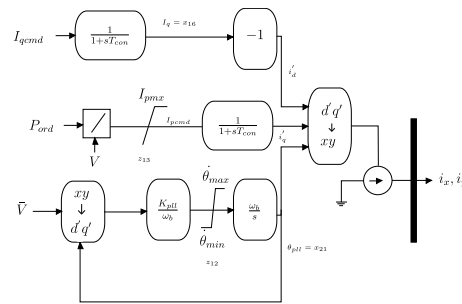
23.3.3. WT4 (inj_WT4): Type 4 Wind Turbine (Full Converter)

The data-file model name is WT4 (or inj_WT4).

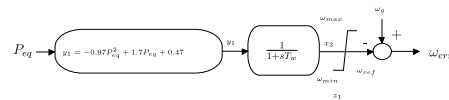
Description

A Type 4 wind turbine with full-rated converter. Unlike Type 3, the generator is fully decoupled from the grid through a back-to-back converter. The model implements the same WECC framework as WT3 but without the doubly-fed rotor circuit: the mechanical sub-model is a single-mass (or two-mass) drive train, and all electrical power passes through the converter. Sub-models include REPC_A (plant controller), REEC_A (electrical controller), WTGT_A (drivetrain), and REGC_A (generator/converter).

WT4 Modelling in RAMSES



Speed and Pitch Angle Control [1]:



1

Figure 23.2: WT4 full-converter wind turbine model block diagram

Scientific Description

The two-mass drivetrain is identical to WT3:

$$2H_t \frac{d\omega_t}{dt} = T_m - K_{\text{shaft}} \delta_{\text{shaft}} - D_{\text{shaft}} (\omega_t - \omega_g)$$

$$2H_g \frac{d\omega_g}{dt} = K_{\text{shaft}} \delta_{\text{shaft}} + D_{\text{shaft}} (\omega_t - \omega_g) - T_e$$

$$\frac{d\delta_{\text{shaft}}}{dt} = \omega_0 (\omega_t - \omega_g)$$

There is no pitch controller or aerodynamic rotor in the basic Type 4 configuration; the active power reference is supplied directly (or from REPC_A), and rate limits dPmax/dPmin apply to the power order ramp.

The REEC_A electrical controller supplies current commands with the same LVRT/HVRT logic as WT3, with current limits through the converter lookup table (piecewise linear in voltage: $(V_{p1}, I_{p1}), \dots, (V_{p4}, I_{p4})$ for active current and $(V_{q1}, I_{q1}), \dots, (V_{q4}, I_{q4})$ for reactive).

Parameters (selected key parameters)

#	Name	Sub-model	Description	Unit
1	SNOM		Nominal power	MW
2–32	REPC_A	Plant ctrl	Same as WT3	various
33–37	WTGT_A	Drivetrain	$H_t, H_g, \omega_0, D_{\text{shaft}}, K_{\text{shaft}}$	s, pu
38–80	REEC_A	Elec ctrl	LVRT, current limits, PI gains	various
81+	REGC_A	Generator	Converter electrical model	various

Usage Example

```
INJEC WT4 WT4_1 BUS_WIND 1. 1. 0. 0.
  100.0 0.0 0.02 0 0.0 0.05 0 0.0 0.3 -0.3 2.0 0.4 0.3 -0.3 0.0 0.15
  ↪ 0.9 0.05
  0.0 -0.06 0.06 -0.05 0.05 0.05 -0.05 2.0 1.0 1.0 -1.0 0.02 0 0 5.0
  ↪ 1.5 1.0 1.5 20.0 ... ;
```

23.3.4. inj_PVG: Photovoltaic Generator

Note

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected. For built-in IBR modelling, use IBG, WT3, or WT4. Note that the subroutine inside inj_PVG.f90 is named inj_PV; both refer to the same single model.

Description

A photovoltaic generator model with similar WECC-derived structure to the wind turbine models. The model includes a plant controller (voltage/reactive power regulation), an electrical controller (current limits, LVRT), and generator/converter representation. Unlike wind turbines, there is no mechanical drive train; the active power set-point follows an irradiance input or a fixed reference. LVRT/HVRT logic and current limiting are identical to the Type 4 wind model.

Scientific Description

The PV generator delivers active and reactive power through current commands:

$$I_p = \frac{P_{\text{ref}}}{V_t}, \quad I_q = f_{\text{QV}}(V_t, Q_{\text{ref}})$$

subject to the constraint $\sqrt{I_p^2 + I_q^2} \leq I_{\text{max}}$. The inner current loop is a first-order filter:

$$T_g \frac{dI_p}{dt} = I_{p,\text{cmd}} - I_p, \quad T_m \frac{dI_q}{dt} = I_{q,\text{cmd}} - I_q$$

Low-voltage power-logic (LVPL) limits active current during deep voltage sags via a piecewise-linear function of V_t with breakpoints lvpnt0, lvpnt1, Lvp11. HVRT and LVRT timers control disconnection and reconnection.

Parameters (selected)

#	Name	Description	Unit
1	Imax	Maximum converter current	pu
2	IN	Nominal current	pu
3	ratemax	Reconnection ramp rate	pu/s
4	Tg	Active current filter time constant	s
5	Tm	Reactive current filter time constant	s
6–9	LVRT timers	tLVRT1, tLVRT2, tLVRTint, Vmax	s, pu
10	kp11	PLL gain	
11	Vminp11	PLL freeze voltage	pu
12–14	LVRT curve	a, Vmin, Vint	
21–22	Re, Xe	Grid equivalent impedance	pu
23–29	Current control	CM1, kRCI, kRCA, m, n, dbmin, dbmax	
34	BM	Battery module flag	

Usage Example

```

INJEC PV PV1 BUS_PV 1. 1. 0. 0. 1.2 1.0 0.5 0.02 0.01 0.5 1.0 0.05
↪ 1.1
                20.0 0.1 2.0 0.1 0.5 0.95 1.05 0.0 2.0 1.0
                1.0 0.0 0.05 1 2.0 1.5 2.0 2.0 -0.02 0.02 1 1 1
↪ 0.85 0 ;

```

23.3.5. GFOR (inj_GFOR): Grid-Forming Converter (VSM)

The data-file model name is GFOR (or inj_GFOR).

Description

A grid-forming voltage source converter implementing Virtual Synchronous Machine (VSM) dynamics, modelled under the phasor approximation. The converter is an MMC-type VSC connected to the grid through its transformer (no LC filter); the DC side is not modelled (the DC voltage is assumed constant) so the focus is on the AC grid dynamics.

The modulated voltage magnitude is held at its setpoint V_m^0 in normal operation, while the phase angle δ_m is driven by a synthetic swing equation with inertia emulation (H) and oscillation damping (D) against a PLL estimate of the grid frequency. Participation in primary frequency control is optional, through the droop R_{droop} . Overcurrents are limited by proportionally scaling the d - q current vector back to the I_{max} circle with a small time constant.

Scientific Description

Reference frame. The d axis is attached to the internal phase angle δ_m of the modulated voltage. Voltages and currents at the point of common coupling are transformed as:

$$v_d = v_x \cos \delta_m + v_y \sin \delta_m, \quad v_q = -v_x \sin \delta_m + v_y \cos \delta_m$$

with the analogous transformation for the converter-side currents $i_{xt} = k i_x$, $i_{yt} = k i_y$, where $k = r S_{\text{base}}/S_{\text{nom}}$ converts to per-unit on the converter base.

Transformer. The voltage-current relationship across the connection transformer (resistance R , inductance L , ideal ratio r) in the d - q frame is:

$$v_{md} = \frac{v_d}{r} + R i_d - \omega_m L i_q, \quad v_{mq} = \frac{v_q}{r} + R i_q + \omega_m L i_d$$

PLL. The PLL only serves to estimate the grid angular frequency $\tilde{\omega}_g$ for the damping term. It is the same model as in the grid-following converter (see the PLL diagram there), with PI gains derived from the time constant T_{pll} :

$$K_{p\omega} = \frac{10}{\omega_N T_{pll}}, \quad K_{i\omega} = \frac{25}{\omega_N T_{pll}^2}, \quad \omega_N = 2\pi f_N$$

The PLL is frozen (hysteresis) when the PCC voltage falls below 0.4 pu and reactivated when it recovers above 0.5 pu (fixed thresholds in this model).

Active power and phase angle control (VSM).

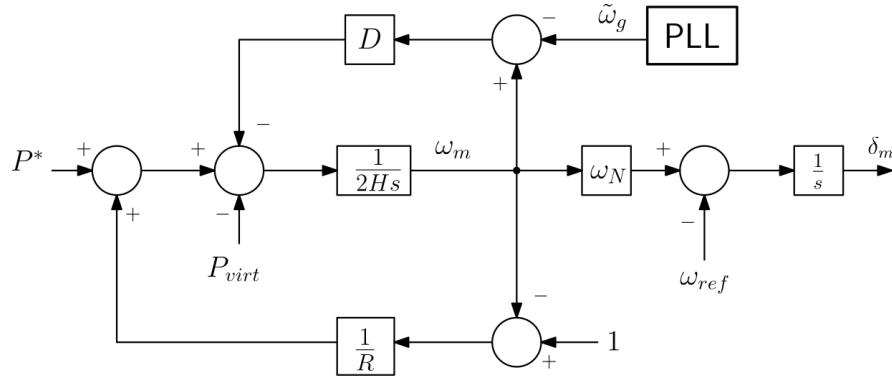


Figure 23.3: GFOR virtual synchronous machine active power and phase angle control block diagram

$$2H \frac{d\omega_m}{dt} = P^* - P_{\text{virt}} - D (\omega_m - \tilde{\omega}_g) + \frac{1 - \omega_m}{R_{\text{droop}}}$$

$$\frac{d\delta_m}{dt} = \omega_N (\omega_m - \omega_{\text{ref}})$$

where P^* is the active power setpoint and P_{virt} is the *virtual power* computed from the **unsaturated** currents:

$$P_{\text{virt}} = \frac{v_d}{r} i_d^* + \frac{v_q}{r} i_q^*$$

P_{virt} equals the actual active power P when the current is not limited, and yields improved large-disturbance stability while the current is limited (the angle dynamics do not wind up).

Current limitation. The currents (i_d^*, i_q^*) that would result from normal voltage control (i.e. $v_{md} = V_m^0, v_{mq} = 0$) are first determined from:

$$V_m^0 = \frac{v_d}{r} + R i_d^* - \omega_m L i_q^*, \quad 0 = \frac{v_q}{r} + R i_q^* + \omega_m L i_d^*$$

The current overload ratio $\rho = \sqrt{(i_d^*)^2 + (i_q^*)^2} / I_{\text{max}}$ is passed through a first-order lag with time constant T_e (typically 1 ms) and a non-windup **lower limit of 1**, giving ρ_s . Both current components are then decreased in the same proportion:

$$i_{ds}^* = \frac{i_d^*}{\rho_s}, \quad i_{qs}^* = \frac{i_q^*}{\rho_s}$$

and the modulated voltage is set to the value that yields the saturated currents:

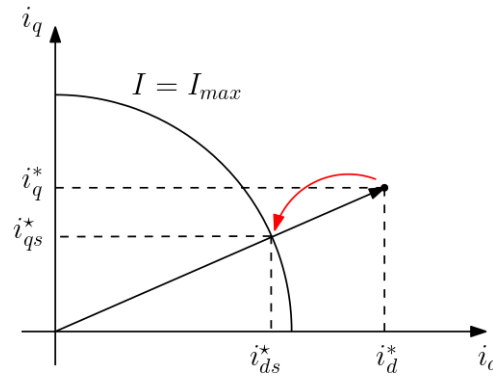


Figure 23.4: GFOR current limitation: proportional scaling of the dq current vector back to the I_{max} circle

$$v_{md} = \frac{v_d}{r} + R i_{ds}^* - \omega_m L i_{qs}^*, \quad v_{mq} = \frac{v_q}{r} + R i_{qs}^* + \omega_m L i_{ds}^*$$

When $I < I_{max}$: $\rho_s = 1$, the currents are unsaturated, and $v_{md} = V_m^0$, $v_{mq} = 0$.

Parameters

Default values correspond to a 1100 MVA converter. Per-unit values refer to the nominal apparent power of the converter.

#	Name	Description	Unit	Default
1	R	Resistance of connection transformer	pu	0.005
2	L	Inductance of connection transformer	pu	0.15
3	r	Ratio of ideal transformer		1.02
4	Snom	Nominal apparent power	MVA	1100
5	H	Inertia constant	s	5.0
6	D	Damping constant of virtual synchronous machine		300
7	Tpll	Time constant of PLL	s	0.1
8	Rdroop	Droop of primary frequency control	pu	999
9	Imax	Maximum current (typically 1–1.2)	pu	1.0
10	Te	Time constant of current saturation	s	0.001

A very large *Rdroop* (e.g. 999) effectively disables participation in primary frequency control.

Five additional parameters are computed at initialization from the power-flow solution: *k* (current conversion factor), *vmx0*, *vmy0* (components of the initial modulated voltage), *Vmo* (its magnitude V_m^0 , i.e. the voltage setpoint) and *Po* (the active power setpoint P^* , pu on *Snom* base). *Vmo* and *Po* can be modified during the simulation with CHGPRM disturbances to change the voltage and active power setpoints.

State Variables

The model has 33 states (2 output currents + 31 internal). Key internal states:

Variable	Description	Unit
wref	Angular speed of reference axes	pu
ixt, iyt	Converter-side currents (on <i>Snom</i> base)	pu
Vm, deltam	Modulated voltage magnitude and phase angle	pu, rad
V	Voltage magnitude at PCC	pu
mult_pll	PLL freezing factor (hysteresis output)	
w_pll, theta_pll	PLL frequency and angle estimates	pu, rad

Variable	Description	Unit
P, Pvirt	Active power and virtual power (on S _{nom} base)	pu
omegam	Angular frequency of modulated voltage	pu
vd, vq, id, iq	PCC voltage and current in <i>d-q</i> frame	pu
id*, iq*	Unsaturated currents	pu
id*s, iq*s	Saturated currents	pu
rho, rhos	Current overload ratio and its saturated value	
Q	Reactive power (on S _{nom} base)	pu

Observables: P, Pvirt, Q, Vm, vmd, vmq, deltam, omegam, w_pll, I, id, id*, id*s, iq, iq*, iq*s, rho, rhos.

Usage Example

A 1100 MVA grid-forming converter:

```
#          name  bus  FP  FQ  P  Q  R  L  r  Snom  H  D
↪ Tpll  Rdroop Imax  Te
INJEC  GFOR  HVDC  A  1.  1.  0.  0.  0.005  0.15  1.02  1100.  5.0  300.
↪ 0.100  999.  1.0  0.001 ;
```

Note

At operating points where the initial current is below $I_{\max}$, RAMSES reports a small initial mismatch on the ρ_s lag equation (“INJECTORS NOT IN STEADY STATE”). This is benign: ρ_s starts exactly at its lower limit of 1 and is clamped there by the discrete update within the first time step.

23.3.6. GFOL (inj_GFOL): Grid-Following Converter

The data-file model name is GFOL (or inj_GFOL).

Description

A grid-following voltage source converter modelled under the phasor approximation. As for the grid-forming model, the converter is an MMC-type VSC connected through its transformer (no LC filter) and the DC side is not modelled. The model is rather detailed: it includes measurement low-pass filters, a PLL with blocking/unblocking hysteresis, outer active power and voltage/reactive power control loops, inner *d-q* current control loops, a rate-limited *d*-axis current limit with priority to the reactive current, and a piecewise-linear dynamic voltage support characteristic.

Scientific Description

Reference frame. The VSC control frame (*d, q*) tracks the PCC voltage phasor through the PLL angle $\tilde{\delta}_g$ (state theta_pll):

$$v_d = v_x \cos \tilde{\delta}_g + v_y \sin \tilde{\delta}_g, \quad v_q = -v_x \sin \tilde{\delta}_g + v_y \cos \tilde{\delta}_g$$

with the analogous transformation for the converter-side currents ($i_{xt} = k i_x$, $k = r S_{\text{base}}/S_{\text{nom}}$). In steady state $v_q = 0$ and $v_d = V$.

Phase Locked Loop.

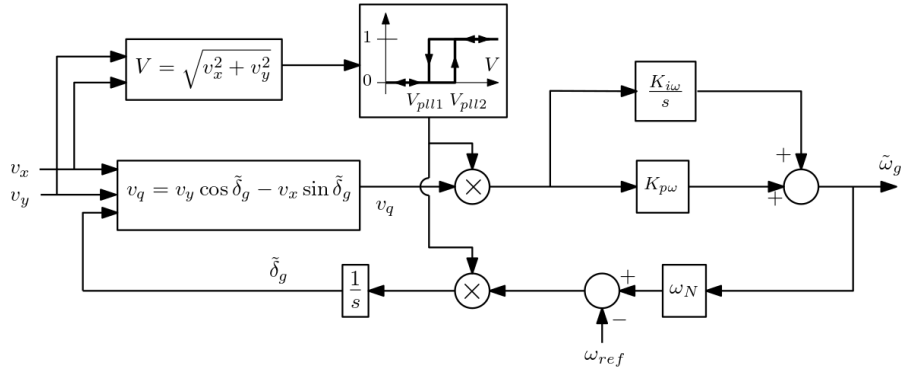


Figure 23.5: GFOL phase locked loop block diagram with blocking hysteresis

The q -axis voltage error drives a PI controller whose output is the grid frequency estimate $\tilde{\omega}_g$ (state `w_pll`); integrating $\omega_N(\tilde{\omega}_g - \omega_{ref})$ gives the PLL angle. The PI gains follow from the PLL response time τ :

$$K_{p\omega} = \frac{10}{\omega_N \tau}, \quad K_{i\omega} = \frac{25}{\omega_N \tau^2}, \quad \omega_N = 2\pi f_N$$

The PLL is blocked once the PCC voltage V falls below V_{pll1b} and reactivated once V recovers above V_{pll1u} (hysteresis).

Inner current control.

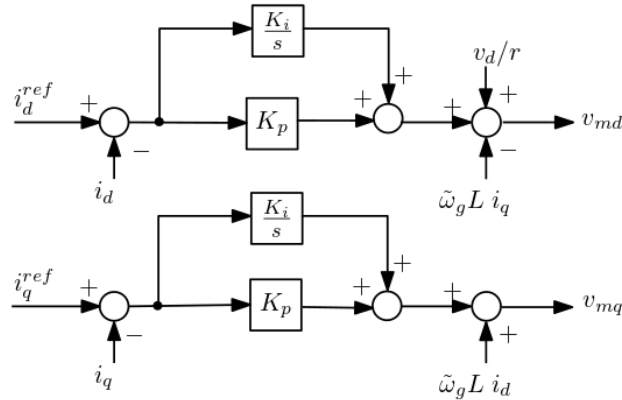


Figure 23.6: GFOL dq current control loops block diagram

$$v_{md} = \frac{v_d}{r} - \tilde{\omega}_g L i_q + \left(K_p + \frac{K_i}{s} \right) (i_d^{ref} - i_d)$$

$$v_{mq} = \tilde{\omega}_g L i_d + \left(K_p + \frac{K_i}{s} \right) (i_q^{ref} - i_q)$$

combined with the transformer voltage-current relationship in the d - q frame:

$$v_{md} = \frac{v_d}{r} + R i_d - \tilde{\omega}_g L i_q, \quad v_{mq} = \frac{v_q}{r} + R i_q + \tilde{\omega}_g L i_d$$

Active power control.

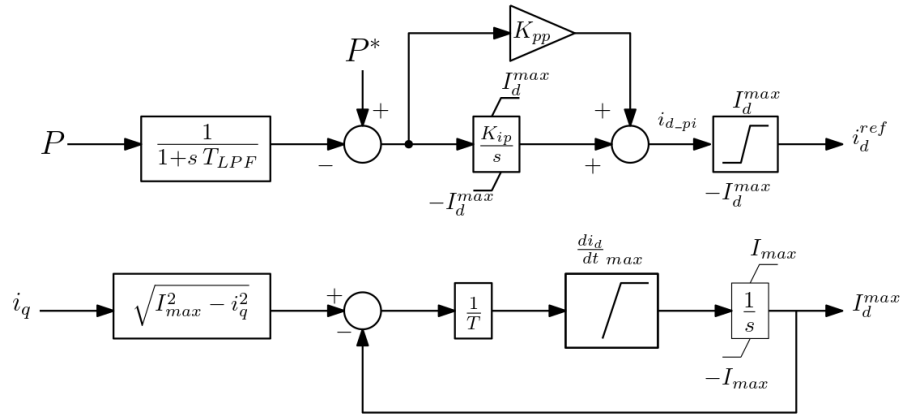


Figure 23.7: GFOL active power control and rate-limited d-axis current limit block diagram

The measured active power is filtered with time constant T_{lpf} and compared with the setpoint P^0 ; a PI controller (K_{pp} , K_{ip}) with non-windup limits $\pm I_d^{\max}$ produces i_d^{ref} . The limit gives **priority to the reactive current**:

$$I_d^{\max, \text{stat}} = \sqrt{\max(I_{\max}^2 - i_q^2, 0)}$$

and $I_d^{\max}$ tracks this static value through a first-order lag (T_{rlim} , typically 0.002 s) with rate limits $dPdt_{\min}/dPdt_{\max}$, limiting in particular the rate of recovery of the active current after it has been decreased by an increase of i_q .

Voltage / reactive power control.

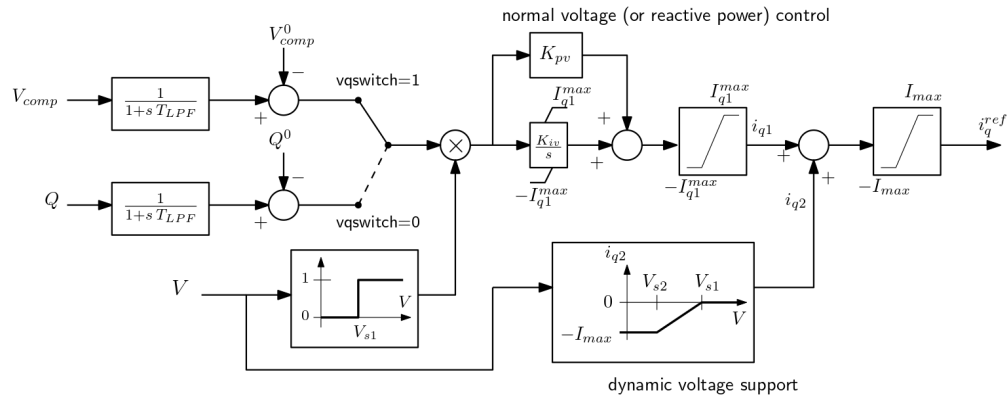


Figure 23.8: GFOL voltage/reactive power control and dynamic voltage support block diagram

Depending on `vqswitch`, either the compensated voltage or the reactive power is controlled:

$$V_c = \left| \frac{\bar{v}}{r} + (R_c + jX_c) \bar{i} \right| \quad (\text{vqswitch} = 1), \quad Q = \frac{v_q}{r} i_d - \frac{v_d}{r} i_q \quad (\text{vqswitch} = 0)$$

The controlled quantity is filtered (T_{lpf}) and its deviation from setpoint drives a PI controller (K_{pv} , K_{iv}) with non-windup limits $\pm I_{q1}^{\max}$, giving the component i_{q1} . The error is zeroed (controller frozen) while $V < V_{s2}$. A second component i_{q2} implements dynamic voltage support as a piecewise-linear function of V : zero above V_{s1} , decreasing linearly to $-I_{\max}$ at V_{s2} , constant below. The total $i_{q1} + i_{q2}$, limited to $\pm I_{\max}$, is the reactive current reference i_q^{ref} .

With $R_c = R$, $X_c = \omega L$ and $r = 1$, controlling V_c amounts to controlling the magnitude of the modulated voltage.

Parameters

Default values correspond to a 1200 MVA converter. Per-unit values refer to the nominal apparent power of the converter.

#	Name	Description	Unit	Default
1	R	Phase reactor resistance	pu	0.005
2	L	Phase reactor inductance	pu	0.15
3	r	Ratio of ideal transformer		1.02
4	Snom	Nominal apparent power	MVA	1200
5	Rc	Resistance used in compensated voltage	pu	0.005
6	Xc	Reactance used in compensated voltage	pu	0.15
7	Kp	Current control: proportional gain		0.573
8	Ki	Current control: integral gain		6.0
9	Tlpf	Measurement time constant (low-pass filter)	s	0.0033
10	Kpp	Active power control: proportional gain		0.0333
11	Kip	Active power control: integral gain		10.0
12	Trlim	Time constant of the $I_d^{\max}$ rate limiter	s	0.002
13	dPdt_min	Min rate of change of the d -current limit	pu/s	-999
14	dPdt_max	Max rate of change of the d -current limit	pu/s	10.0
15	Kpv	Reactive power control: proportional gain		0.1667
16	Kiv	Reactive power control: integral gain		50.0
17	tau	Response time of PLL	s	0.10
18	Vpll_b	Voltage below which the PLL is blocked	pu	0.4
19	Vpll_u	Voltage above which the PLL is unblocked	pu	0.5
20	I_max	Maximum current (typically 1–1.2)	pu	1.0
21	Vs1	Voltage below which dynamic reactive support starts	pu	-1000
22	Vs2	Voltage at which reactive support is maximum ($V_{s2} < V_{s1}$)	pu	-2000
23	iqmax	Limit ($\pm$) on quadrature current component i_{q1}	pu	1.001
24	vqswitch	1 = voltage control, 0 = reactive power control	flag	1

Setting Vs1 and Vs2 to large negative values (as in the defaults above) disables the dynamic voltage support; a more elaborate control is recommended for that function. Typical values of Kiv are 50 pu/s for voltage control and 10 pu/s for reactive power control.

Six additional parameters are computed at initialization from the power-flow solution: k (current conversion factor), $P0$, $Q0$ (active/reactive power setpoints, pu on Snom base), $Vc0$ (compensated voltage setpoint), $I_{\min}$ and $i_{q1\max}$ ($= \min(i_{q\max}, I_{\max})$). $P0$, $Q0$ and $Vc0$ can be modified during the simulation with CHGPRM disturbances.

State Variables

The model has 52 states (2 output currents + 50 internal). Key internal states:

Variable	Description	Unit
wref	Angular speed of reference axes	pu
ixt, iyt	Converter-side currents (on Snom base)	pu
mult_pll	PLL freezing factor (hysteresis output)	
w_pll, theta_pll	PLL frequency and angle estimates	pu, rad

Variable	Description	Unit
V, vd, vq	PCC voltage magnitude and d - q components	pu
id, iq	Converter currents in d - q frame	pu
vmd, vmq	Modulated voltage components	pu
Md, Mq	Current-control PI outputs	pu
Idmax_stat, Idmax, Idmin	Static / rate-limited / minimum d -current limits	pu
P, Pfil	Active power and its filtered value	pu
id_pi, idref	Active-power PI output and d -current reference	pu
Vc, Vcfil	Compensated voltage and its filtered value	pu
Q, Qfil	Reactive power and its filtered value	pu
mult_V	Voltage/reactive controller freezing factor	
iq_1, iq_2, iqref	Reactive current components and reference	pu

Observables: P_MW, Q_Mvar, Vc, Vm, vmd, vmq, I, w_pll, theta_pll_deg, mult_pll, Idmax, Md, idref, id, id_pi, iq_1, iq_2, iqref, iq.

Usage Example

A 1200 MVA grid-following converter (record spanning multiple lines):

```
#          name  bus  FP  FQ  P  Q  R      L      r  Snom  Rc  Xc
↪ Kp      Ki  Tlpf  Kpp  Kip  Trlim dPdt_min dPdt_max Kpv  Kiv
INJEC GFOL  HVDC1  A  1.  1.  0.  0.  0.005 0.15  1.02  1200.  0.005
↪ 0.15  0.5730  6.  0.0033 0.0333  10.  0.002  -999.  10.  0.1667  50.
# tau  Vpll  Vpllu  Imax  Vs1  Vs2  iqlmax  vqswitch
  0.10  0.4  0.5  1.000  -1000 -2000  1.001      1      ;
```

23.4. Energy Storage

23.4.1. BESS (inj_BESS): Battery Energy Storage System

The data-file model name is BESS (or inj_BESS).

Description

A comprehensive BESS model implementing the full WECC framework (REPC_A + REEC_C + REGC_A) plus battery state of charge (SOC) tracking. The model supports both grid-following operation (bidirectional active power dispatch) and reactive power / voltage regulation. It is built on the same REPC_A plant controller as the wind turbine models, and uses the REEC_C converter electrical controller which handles a piecewise-linear I_q -vs- V and I_p -vs- V characteristic for fault ride-through.

The battery SOC evolves through integration of injected power:

$$\frac{dSOC}{dt} = -\frac{P_{bess}}{C_{bat} \cdot S_{nom}}$$

with hard clamps at SOCmin and SOCmax that modify the active power limits Pmax/Pmin dynamically:

$$P_{max} = \begin{cases} 0 & SOC \geq SOC_{max} \\ P_{max,rated} & \text{otherwise} \end{cases}$$

$$P_{min} = \begin{cases} 0 & SOC \leq SOC_{min} \\ P_{min,rated} & \text{otherwise} \end{cases}$$

Parameters (key parameters)

#	Name	Sub-model	Description	Unit
1	SNOM		Nominal power	MW
2–32		REPC_A	Plant controller (same as WT3/WT4)	various
33–76		REEC_C	Electrical controller with piecewise $I_q(V)$, $I_p(V)$	various
68	CapBat	BESS	Battery energy capacity	MWh
69	SOCini	BESS	Initial state of charge	pu
70	SOCmax	BESS	Maximum SOC limit	pu
71	SOCmin	BESS	Minimum SOC limit	pu
72–73	dPmax, dPmin	BESS	Active power ramp rate limits	pu/s
74–75	pmax, pmin	REEC_C	Active current limits (pu on SNOM)	pu
76	Tpord	REEC_C	Active power order time constant	s

Full parameter listing has approximately 125 parameters; refer to a working example data file for the complete ordering.

State Variables

The BESS model uses over 40 internal states reflecting the three sub-models. Key states include:

Variable	Description
SOC	State of charge
Pref	Active power reference
Qext	Reactive power reference from REPC_A
Ip, Iq	Active/reactive current outputs
PLLPhaseAngle	PLL phase angle
Pgen, Qgen	Generated powers

Usage Example

```
INJEC BESS BESS1 BUS_ST 1. 1. 0. 0.
100.0 0.0 0.02 0 0.0 0.05 0 0.0 0.3 -0.3 2.0 0.4
0.3 -0.3 0.0 0.15 0.9 0.05 0.0 -0.06 0.06 -0.05 0.05
0.05 -0.05 2.0 1.0 1.0 -1.0 0.02 0 0
0.1 0.1 0.02 1.0 0.0 -0.2 0.2 0.2 -0.2 0.6 0.4 0.6 -0.6 1.1 0.9
1.0 0.5 0.01 0.01 0.05 0.9 0.2 0.9 0.1 0.9 -0.1 0.5 -0.5 0.5 -0.5
↪ 1.0 -1.0 0.5 -0.5
50.0 0.7 0.9 0.1 5.0 -5.0 1.0 -1.0 0.05 1.1 0.0 2 0.01 ;
```

23.5. Reactive Compensation**23.5.1. SVC_GENERIC1 (inj_svc_generic1): Generic Static Var Compensator**

The data-file model name is SVC_GENERIC1. It takes no prefix.

Description

A dynamic SVC injecting a purely reactive current at its bus. A PI voltage regulator with droop drives a susceptance B_{svc} between Bmin and Bmax, and two lead-lag stabiliser channels can add a supplementary signal to the voltage reference. This is the dynamic counterpart of the static SVC record used by the power flow: the static record fixes the operating point, this model governs how the device behaves during the simulation.

The voltage reference is initialised from the power flow solution, so that the model starts in equilibrium:

$$V_{ref} = V_0 + B_p \cdot B_{svc,0}$$

where B_p is the droop and $B_{svc,0}$ the initial susceptance implied by the reactive current at the bus.

Parameters

The record carries 17 data parameters, in order:

#	Name	Description
1	G1	Gain of the first stabiliser channel
2	T1	Time constant of the first stabiliser channel (s)
3	a	Lead-lag coefficient of the first channel
4	K1	Output gain of the first channel
5	L1	Output limit of the first channel (pu)
6	G2	Gain of the second stabiliser channel
7	T2	Time constant of the second stabiliser channel (s)
8	b	Lead-lag coefficient of the second channel
9	K2	Output gain of the second channel
10	L2	Output limit of the second channel (pu)
11	Ltot	Limit on the summed stabiliser output (pu)
12	Kp	Proportional gain of the voltage regulator
13	Ki	Integral gain of the voltage regulator
14	Bp	Droop, in pu on the SVC base
15	Bmax	Maximum susceptance (Mvar at 1 pu voltage)
16	Bmin	Minimum susceptance (Mvar at 1 pu voltage)
17	Bnom	Susceptance base used for the per-unit conversion (Mvar)

One additional parameter is computed at initialisation:

#	Name	Description
18	Vref	Voltage reference, set from the power flow solution

Observables

Q , $dvpss$, B_{svc} , V_{ref} , V_b

23.6. Measurement

23.6.1. PMU (inj_PMU): Phasor Measurement Unit

The data-file model name is PMU (or inj_PMU).

Description

A measurement-only injector: it injects **zero current** at its bus and exists purely to expose bus quantities as observables. It reports a filtered local frequency estimate computed the same way as the `f_inj` block, together with the bus voltage magnitude and angle and the speed and angle of the moving DQ reference frame. Attach one to any bus whose frequency you want to record without perturbing the solution.

The frequency estimate is a first-order filter on the bus voltage phasor. The measurement time constant is clamped to a floor of 0.05 s, so values below that have no effect.

Parameters

#	Name	Description	Unit
1	Tf	Frequency measurement time constant, recommended 0.05 to 0.10; values below 0.05 are clamped	s

Three additional parameters are set at initialisation: `wnom` (nominal angular frequency), `vm0` and `va0` (the initial bus voltage magnitude and angle).

Observables

Name	Description	Unit
f	Estimated bus frequency	pu of f_{nom}
vm	Bus voltage magnitude	pu
va	Bus voltage phase angle in the moving DQ frame, wrapping at $\pm\pi$	rad
thref	Accumulated angle of the DQ reference frame with respect to the nominally rotating frame, zero at $t = 0$	rad
dwref	Speed deviation of the DQ reference frame from nominal	rad/s

Caution

`dwref` and `thref` are only meaningful under `$OMEGA_REF COI`. With `$OMEGA_REF SYN` the reference frame rotates at nominal speed by construction, so both stay at zero.

Usage Example

```
INJEC PMU PMU_1041 1041 0. 0. 0. 0. 0.05 ;
```

All four participation and power fields are zero: the model neither consumes nor produces power.

24

Two-port models

Two-port models in RAMSES connect **two AC (or AC/DC) buses** and inject currents at both ends simultaneously. Each model receives the voltage and current phasors at bus 1 ($v_{x1}, v_{y1}, i_{x1}, i_{y1}$) and bus 2 ($v_{x2}, v_{y2}, i_{x2}, i_{y2}$) and computes the algebraic or differential equations governing the device.

The RAMSES data keyword is TWOP:

```
TWOP model_name twop_name bus_orig bus_extr S_or_A
      FP_ORIG FQ_ORIG P_ORIG Q_ORIG
      FP_EXTR FQ_EXTR P_EXTR Q_EXTR
      param1 param2 ... ;
```

Usage in Dynamic Data Files

The record fields, in order, are:

- **model_name**: two-port model type (e.g., HVDC_LCC, DCL_WCL, HVDC_VSC_SC). The **twop_** prefix is added automatically if omitted, so both HVDC_LCC and twop_HVDC_LCC are accepted.
- **twop_name**: unique instance name.
- **bus_orig, bus_extr**: origin and extremity bus names.
- **S_or_A**: synchronizing mode. S keeps the two buses in the same electrical island, appropriate for an AC link; A makes the link asynchronous, so island detection stops at it and the two ends can run at different frequencies, as an HVDC link must. Any other value is a hard error.
- **FP_ORIG/FQ_ORIG/P_ORIG/Q_ORIG**: initial active/reactive power injection at the origin bus. Either the fraction (FP_ORIG/FQ_ORIG, of the load at that bus) or the absolute power (P_ORIG/Q_ORIG, in MW/Mvar) must be zero.
- **FP_EXTR/FQ_EXTR/P_EXTR/Q_EXTR**: same pattern for the extremity bus.
- Remaining fields are the model-specific parameter list.

Four two-port models are available in every RAMSES distribution (both the standalone executable and the shared library used by stepss): HVDC_LCC / twop_HVDC_LCC, DCL_WCL / twop_DCL_WCL, and HVDC_VSC_SC / twop_HVDC_VSC_SC. HVDC_VSC / twop_HVDC_VSC, also documented below, has been callable since 3.79; before that it was compiled under no name.

24.1. HVDC Models

24.1.1. HVDC_LCC (twop_HVDC_LCC): Line-Commutated Converter HVDC

Description

Models a **point-to-point Line-Commutated Converter (LCC) HVDC link** consisting of a rectifier station (bus 1) and an inverter station (bus 2). LCC-HVDC uses thyristor valves that require an external AC voltage for commutation. The rectifier controls DC current (or power), while the inverter controls DC voltage (or extinction angle). Reactive power is always consumed at both stations and must be supplied locally.

The model includes: - Configurable rectifier control modes: constant current, constant power, minimum firing angle ($\alpha_{\min}$), or blocked - Configurable inverter control modes: constant voltage, constant extinction angle (γ), minimum γ , reduced current, or blocked - Transformer tap changers on both sides (handled via `dctl_hvdc_lim`) - Reactive power compensation shunts on both sides

Scientific Description

Rectifier DC voltage (bridge equation):

$$V_{dr} = B_r \left(\frac{1.35 E_r V_1}{n_r} \cos \alpha - (0.9549 X_{cr} + 2R_{cr}) I_d \right)$$

where B_r is the number of rectifier bridges, E_r is the open-circuit secondary voltage, n_r is the transformer ratio, V_1 is the AC voltage magnitude, and I_d is the DC current.

Rectifier overlap angle (commutation equation):

$$\cos(\alpha + \mu_r) = \cos \alpha - \frac{\sqrt{2} X_{cr} n_r I_d}{V_1 E_r}$$

Inverter DC voltage:

$$V_{di} = B_i \left(\frac{1.35 E_i V_2}{n_i} \cos \gamma - (0.9549 X_{ci} + 2R_{ci}) I_d \right)$$

Inverter overlap angle:

$$\cos(\gamma + \mu_i) = \cos \gamma - \frac{\sqrt{2} X_{ci} n_i I_d}{V_2 E_i}$$

DC link voltage balance (series resistance R_L):

$$V_{dr} - V_{di} = R_L I_d$$

Reactive power consumption at the rectifier:

$$Q_1 = P_1 \cdot \frac{2\mu_r + \sin 2\alpha - \sin 2(\mu_r + \alpha)}{\cos 2\alpha - \cos 2(\mu_r + \alpha)}$$

Active power injections (in per unit):

$$P_1 = -\frac{p V_{dr} I_d}{S_{\text{base1}}}, \quad P_2 = \frac{p V_{di} I_d}{S_{\text{base2}}}$$

where p is the number of poles (monopolar or bipolar).

Rectifier control equation (mode-dependent):

rec_ctl	Control mode	Constraint
1	Constant current	$I_{ord} - I_d = 0$
2	Constant power	$P_{set}/V_{dr} - I_d = 0$
-1	Minimum α	$\alpha_{\min} - \alpha = 0$
3	Blocked	$I_d = 0$

Inverter control equation (mode-dependent):

inv_ctl	Control mode	Constraint
1	Constant voltage	$V_{di} + R_{comp}I_d - V_{set} = 0$
2	Constant γ	$\gamma_0 - \gamma = 0$
-2	Minimum γ	$\gamma_{\min} - \gamma = 0$
-1	Reduced current	$I_{red} - I_d = 0$
3	Blocked	$\gamma = \pi/2$

The current order I_{ord} is obtained from the desired current I_{des} through a first-order filter with time constant T_p .

Parameters Table

Parameter	Description	Unit
p	Number of poles (1 = monopolar, 2 = bipolar)	
rec_ctl	Rectifier control mode (1=current, 2=power, -1= α min, 3=blocked)	
inv_ctl	Inverter control mode (1=voltage, 2= γ , -2= γ min, -1=reduced I_d , 3=blocked)	
Br	Number of bridges in series at rectifier	
Bi	Number of bridges in series at inverter	
Er	Rectifier open-circuit secondary voltage (at 1 pu AC)	kV
Ei	Inverter open-circuit secondary voltage (at 1 pu AC)	kV
Rcr	Rectifier commutating resistance per bridge	Ω
Xcr	Rectifier commutating reactance per bridge	Ω
Rci	Inverter commutating resistance per bridge	Ω
Xci	Inverter commutating reactance per bridge	Ω
RL	DC line series resistance	Ω
Rcomp	Compounding resistance for voltage control	Ω
Tp	Time constant for power/current order filter	s
Idset	Initial (and setpoint) DC current	kA
nr	Initial rectifier transformer ratio	pu/pu
ni	Initial inverter transformer ratio	pu/pu

Control Structure

The rectifier minimises α (advances firing) to maintain the current order. The inverter maintains DC voltage via the compounding characteristic $V_{set} = V_{di} + R_{comp}I_d$.

Usage Example

```

TWOP HVDC_LCC HVDClink BUS_RECT BUS_INV S
    0. 0. 0. 0.      ! FP/FQ/P/Q at origin (rectifier) bus
    0. 0. 0. 0.      ! FP/FQ/P/Q at extremity (inverter) bus
    2                ! p: bipolar
    1                ! rec_ctl: current control
    1                ! inv_ctl: voltage control
    2                ! Br

```

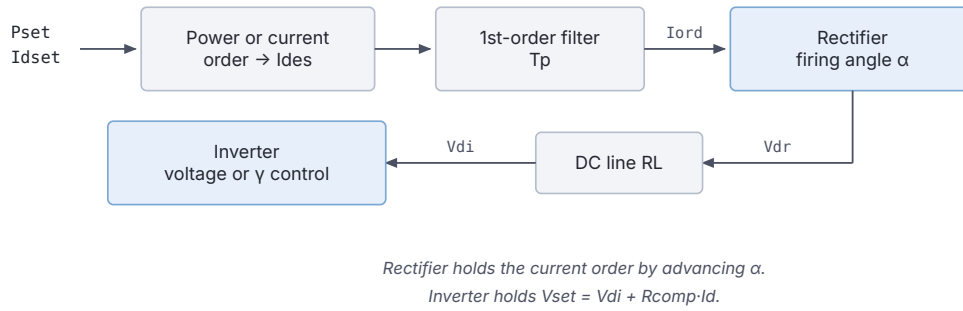


Figure 24.1: The HVDC_LCC control chain.

```

2          ! Bi
200.0      ! Er (kV)
200.0      ! Ei (kV)
0.5        ! Rcr (ohm)
10.0       ! Xcr (ohm)
0.5        ! Rci (ohm)
10.0       ! Xci (ohm)
5.0        ! RL (ohm)
20.0       ! Rcomp (ohm)
0.01       ! Tp (s)
1.5        ! Idset (kA)
1.0        ! nr
1.0        ! ni
;

```

24.1.2. twop_HVDC_VSC: Voltage Source Converter HVDC

Note

Callable since 3.79. Earlier releases compiled this model under no name, so a data file referring to it was rejected. `twop_HVDC_VSC_SC` is a separate, simplified VSC-HVDC model and remains available.

Description

Models a **point-to-point VSC-HVDC link** in which bus 1 is the AC-side connection and bus 2 is the DC side (DC voltage appears as v_{x2}). The converter uses IGBT-based switches that are self-commutating, enabling independent control of active and reactive power on the AC side and DC voltage on the DC side.

The model implements: - Phase reactor dynamics (R, L) - Phase-Locked Loop (PLL) for grid synchronisation - Inner current control loops (d - and q -axis PI controllers) - Outer loop for active power (P or V_{dc}) and reactive power/AC voltage control - Low-voltage reactive current injection (grid-code FRT support via piecewise-linear characteristic) - DC capacitor dynamics

Scientific Description

PLL dynamics (tracks $v_q = 0$):

$$\Delta\omega_{pll} = (\omega_{pll} - \omega_{ref}) \cdot 2\pi f_0$$

$$\dot{\theta}_{pll} = \Delta\omega_{pll}, \quad \omega_{pll} = K_{pw} v_q + K_{iw} \int v_q dt$$

Park transformation (bus-1 reference frame):

$$i_d = i_x \cos \theta_{pll} + i_y \sin \theta_{pll}, \quad i_q = -i_x \sin \theta_{pll} + i_y \cos \theta_{pll}$$

Phase reactor dynamics (in x - y frame):

$$L \frac{di_x}{dt} = -R i_x + \omega_{ref} L i_y + v_{md} \cos \theta_{pll} - v_{mq} \sin \theta_{pll} - v_{x1}$$

$$L \frac{di_y}{dt} = -R i_y - \omega_{ref} L i_x + v_{mq} \cos \theta_{pll} + v_{md} \sin \theta_{pll} - v_{y1}$$

Inner current controller (d -axis, decoupling):

$$v_{md} = v_d - \omega_{pll} L i_q + K_p(i_{d,ref} - i_d) + K_i \int (i_{d,ref} - i_d) dt$$

$$v_{mq} = v_q + \omega_{pll} L i_d + K_p(i_{q,ref} - i_q) + K_i \int (i_{q,ref} - i_q) dt$$

Outer active power / DC voltage loop (combined P-V droop):

$$N_1 = \alpha_1 \left(\frac{P_{ref} - P}{S_{base}} \right) + \beta_1 (V_{dc,ref} - V_{dc})$$

The integrator state N_{1a} feeds through gain K_{pd} to produce $i_{d,ref,unlim}$, which is then limited to $[I_{d,min}, I_{d,max}]$.

Outer reactive power / AC voltage loop:

$$N_2 = \alpha_2 \left(\frac{Q_{ref} - Q}{S_{base}} \right) + \beta_2 (V_{ac,ref} - V)$$

The integrator state N_{2a} feeds through gain K_{pq} to produce $i_{q,ref1}$.

Low-voltage reactive injection (piecewise-linear):

$$i_{q,ref2} = \begin{cases} -1 & V < V_{s2} \\ \frac{V - V_{s2}}{V_{s1} - V_{s2}} \cdot (-1) & V_{s2} \leq V < V_{s1} \\ 0 & V \geq V_{s1} \end{cases}$$

DC capacitor dynamics (electrostatic inertia H_{dc}):

$$2H_{dc} \frac{dV_{dc}}{dt} = i_{m,dc}$$

AC-DC power balance (lossless converter):

$$i_d v_{md} + i_q v_{mq} + V_{dc} i_{dc} \cdot \frac{P_{nom}}{S_{nom}} = 0$$

Current magnitude limit:

$$I_{d,max} = \frac{P_{nom}}{S_{nom}}, \quad I_{q,max} = \sqrt{1 - i_{d,ref}^2}$$

Parameters Table

Parameter	Description	Unit
R	Phase reactor resistance	pu
L	Phase reactor inductance	pu
Hdc	DC capacitor electrostatic inertia	s
Snom	Nominal apparent power	MVA
Pnom	Nominal active power (DC side rating)	MW
Kp	Inner current controller proportional gain	
Ki	Inner current controller integral gain	
Kpd	Outer d -axis loop proportional gain	
Kid	Outer d -axis loop integral gain	
Kpq	Outer q -axis loop proportional gain	
Kiq	Outer q -axis loop integral gain	
Kpw	PLL proportional gain	
Kiw	PLL integral gain	
alpha1	P-V outer loop: active power coefficient	
beta1	P-V outer loop: DC voltage coefficient	
alpha2	Q- V_{ac} outer loop: reactive power coefficient	
beta2	Q- V_{ac} outer loop: AC voltage coefficient	
Vs1	AC voltage threshold for reactive support activation	pu
Vs2	AC voltage for full reactive support	pu

Internal (auto-initialised) parameters: Pref, Vdcref, Qref, Vacref, switch.

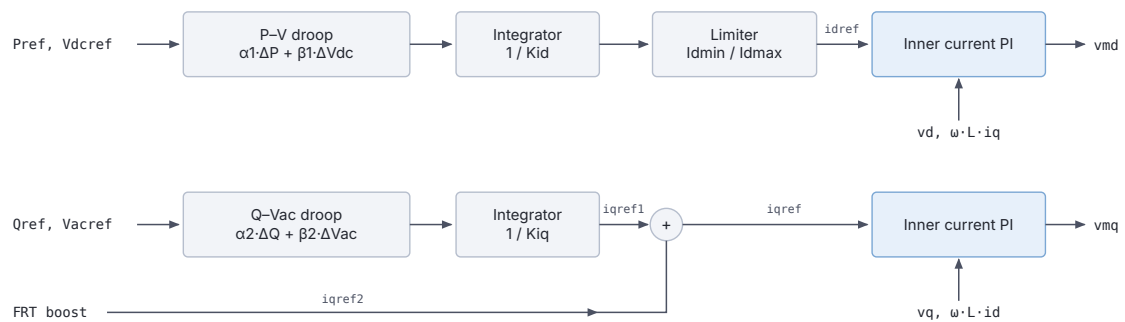
Control Structure

Figure 24.2: The HVDC_VSC control, one strip per axis.

Usage Example

```

TWOP HVDC_VSC VSC1 AC_BUS DC_BUS S
0. 0. 0. 0.      ! FP/FQ/P/Q at AC bus
0. 0. 0. 0.      ! FP/FQ/P/Q at DC bus
0.003 ! R (pu)
0.15  ! L (pu)
3.5   ! Hdc (s)
1000.0 ! Snom (MVA)
900.0  ! Pnom (MW)
1.0    ! Kp
50.0   ! Ki
0.0    ! Kpd
10.0   ! Kid

```

```

0.0      ! Kpq
20.0     ! Kiq
2.0      ! Kpw
40.0     ! Kiw
1.0      ! alpha1
0.0      ! beta1
1.0      ! alpha2
0.0      ! beta2
0.9      ! Vs1
0.5      ! Vs2
;

```

24.1.3. HVDC_VSC_SC (twop_HVDC_VSC_SC): VSC-HVDC with Self-Commutation (Grid-Forming)

Description

An enhanced VSC-HVDC model with **self-commutation capability and synchronous machine emulation**. Unlike the standard twop_HVDC_VSC, this variant includes a frequency droop loop (via K_{wi} , the virtual synchronous machine inertia gain) in the outer active power control, enabling the converter to participate in primary frequency regulation as if it were a synchronous machine. It is suited for modelling converters operating as **grid-forming** or **frequency-supporting** devices.

Key differences from twop_HVDC_VSC: - Outer active power loop uses **frequency droop**: $N_1 = (1 - \omega_1)K_{wi} + (P_0 + \Delta P - P)(S_{base}/S_{nom})$ - The integrator output is accumulated (not driven to zero), producing a continuous $i_{d,ref}$ - External power correction signal ΔP allows participation in hub-level coordination

Scientific Description

Outer active power control with frequency droop:

$$N_1 = (1 - \omega_1)K_{wi} + \frac{(P_0 + \Delta P - P) S_{base}}{S_{nom}}$$

$$\dot{N}_{1a} = \frac{N_1}{K_{ip}}, \quad i_{d,ref,unlim} = N_{1a} + K_{pp} N_1$$

where K_{wi} introduces inertia-like damping: when grid frequency deviates from 1 pu, the converter adjusts its active current reference to oppose the deviation.

Outer reactive power control (droop on Q and AC voltage):

$$N_2 = (Q_0 - Q) \cdot \frac{S_{nom}}{\dots} + N_{1a}$$

The inner current loop equations, PLL, phase reactor, and DC capacitor dynamics are identical to twop_HVDC_VSC.

Current limits (priority to active current):

$$I_{q,max} = \sqrt{\max(1 - i_{d,ref}^2, 0)}$$

Parameters Table

Parameter	Description	Unit
R	Phase reactor resistance	pu
L	Phase reactor inductance	pu
Hdc	DC capacitor electrostatic inertia	s
Snom	Nominal apparent power	MVA
Pnom	Nominal active power	MW
Kp	Inner current controller proportional gain	
Ki	Inner current controller integral gain	
Kpp	Outer active power loop proportional gain	
Kip	Outer active power loop integral gain	
Kwi	Frequency droop / virtual inertia gain	
Kpq	Outer reactive power loop proportional gain	
Kiq	Outer reactive power loop integral gain	
Kpw	PLL proportional gain	
Kiw	PLL integral gain	
Vs1	AC voltage threshold for reactive support	pu
Vs2	AC voltage for full reactive support	pu

Internal (auto-initialised): DeltaP, P0, Q0, Vac, switch.

Control Structure

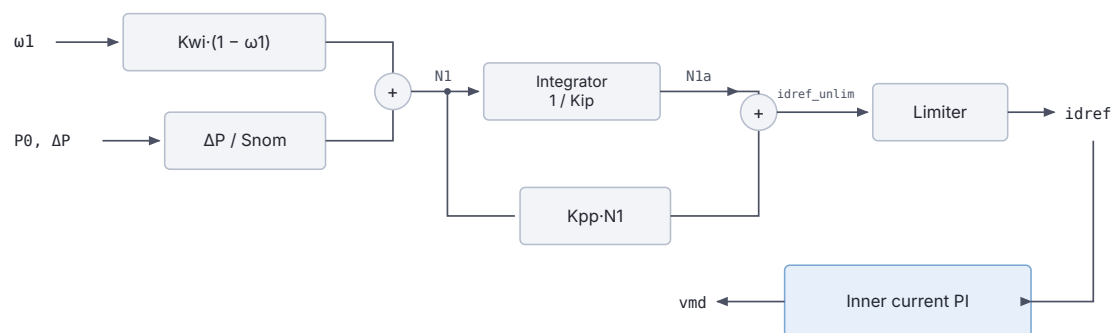


Figure 24.3: The grid-forming HVDC_VSC_SC control.

Usage Example

```

TWOP HVDC_VSC_SC VSC_SC1 AC_BUS DC_BUS S
0. 0. 0. 0.      ! FP/FQ/P/Q at AC bus
0. 0. 0. 0.      ! FP/FQ/P/Q at DC bus
0.003 ! R (pu)
0.15  ! L (pu)
3.5    ! Hdc (s)
1000.0 ! Snom (MVA)
900.0  ! Pnom (MW)
1.0    ! Kp
50.0   ! Ki
2.0    ! Kpp (outer P loop P gain)
5.0    ! Kip (outer P loop I gain)
10.0   ! Kwi (frequency droop gain)
0.0    ! Kpq
20.0   ! Kiq
2.0    ! Kpw

```

```

40.0    ! Kiw
0.9     ! Vs1
0.5     ! Vs2
;

```

24.1.4. DCL_WCL (twop_DCL_WCL): DC Link with Wind Converter Link (Offshore Wind HVDC)

Description

A comprehensive model of an **offshore wind HVDC connection** consisting of:

- **Converter 1 (offshore, bus 1):** Grid-forming VSC station connected to an offshore AC network (wind farm collector bus). It controls AC voltage and frequency at the offshore bus (island operation) and exports active power via the DC link.
- **DC cable:** A lumped-parameter π -model with series resistance R_{dc} and shunt capacitances at each end (represented as DC capacitors with inertia constants $H_{dc,1}$ and $H_{dc,2}$).
- **Converter 2 (onshore, bus 2):** Grid-following VSC station connected to the main AC grid. It controls DC voltage (ensuring power balance) and reactive power/AC voltage support.

The model fully captures: - PLL dynamics at both converters - Inner d - q current control loops at both converters - Offshore power/speed droop outer loop - Onshore DC voltage control with Q/V_{ac} support - DC cable current dynamics - Current limiters at both ends

Scientific Description

Converter 1, Offshore (Grid-forming, active power + frequency droop)

Outer active power loop with speed droop:

$$PB_1 = (1 - \omega_1)K_{wi,1} + \frac{(P_{0,1} + \Delta P - P_1) S_{base}}{S_{nom,1}}$$

$$i_{d,ref,1} = \frac{\int PB_1 dt}{K_{ip,1}} + K_{pp,1} PB_1$$

Outer reactive power loop with QV droop (slope β_V):

$$QB_1 = V_1 - V_{0,1} + \beta_V(Q_1 - Q_{0,1}) \frac{S_{base}}{S_{nom,1}}$$

$$i_{q,ref,1} = \frac{\int QB_1 dt}{K_{iq,1}} + K_{pq,1} QB_1$$

Converter 2, Onshore (Grid-following, DC voltage control)

Outer DC voltage loop:

$$\Delta V = V_{dc,ref} - v_{dc,2}$$

$$i_{d,ref,2} = \frac{\int \Delta V dt}{K_{id,2}} + K_{pd,2} \Delta V$$

Outer Q/V_{ac} control with combined droop:

$$QB_2 = \alpha_{2,2}(Q_{ref,2} - Q_2)/S_{base} + \beta_{2,2}(V_{0,2} - V_2)$$

$$i_{q,ref,2} = \frac{\int QB_2 dt}{K_{iq,2}} + K_{pq,2} QB_2$$

Phase reactor dynamics (both converters, in x - y frame):

$$\begin{aligned} -\frac{R_k}{L_k} i_{x,k} + \omega_{ref} i_{y,k} + \frac{1}{L_k} (v_{md,k} \cos \theta_k - v_{mq,k} \sin \theta_k - v_{x,k}) &= 0 \\ -\frac{R_k}{L_k} i_{y,k} - \omega_{ref} i_{x,k} + \frac{1}{L_k} (v_{mq,k} \cos \theta_k + v_{md,k} \sin \theta_k - v_{y,k}) &= 0 \end{aligned}$$

DC link (Ohmic drop + shunt capacitors):

$$v_{dc,1} - R_{dc} i_{dc} - v_{dc,2} = 0$$

$$2H_{dc,1} \frac{dv_{dc,1}}{dt} = i_{dc,1} - i_{m,dc,1} - i_{dc}$$

$$2H_{dc,2} \frac{dv_{dc,2}}{dt} = i_{dc} - i_{m,dc,2} - G v_{dc,2} - i_{dc,2}$$

where G is the shunt conductance computed at initialisation to balance active power at the onshore DC terminal.

AC-DC power balance (both converters, lossless):

$$i_{d,k} v_{md,k} + i_{q,k} v_{mq,k} \pm v_{dc,k} i_{dc,k} \frac{P_{nom}}{S_{nom,k}} = 0$$

PLL (both stations):

$$\dot{\theta}_{pll,k} = \Delta \omega_{pll,k} = (\omega_{pll,k} - \omega_{ref}) 2\pi f_0$$

$$\omega_{pll,k} = K_{pw,k} v_{q,k} + K_{iw,k} \int v_{q,k} dt$$

Current limits (both converters):

$$I_{q,max,k} = \sqrt{\max(1 - i_{d,k}^2, 0)}, \quad I_{d,max,k} = \frac{P_{nom}}{S_{nom,k}}$$

Parameters Table

Offshore Converter (1)

Parameter	Description	Unit
R_1	Phase reactor resistance	pu
L_1	Phase reactor inductance	pu
Hdc_1	DC capacitor electrostatic inertia	s
Snom_1	Nominal apparent power	MVA
Kpp_1	Outer active power loop proportional gain	
Kip_1	Outer active power loop integral gain	
Kwi_1	Speed/frequency droop gain	
Kpq_1	Outer reactive power loop proportional gain	
Kiq_1	Outer reactive power loop integral gain	
Kpw_1	PLL proportional gain	
Kiw_1	PLL integral gain	

Parameter	Description	Unit
betaV_1	QV droop slope ($\Delta V/\Delta Q$)	pu/pu

Onshore Converter (2)

Parameter	Description	Unit
R_2	Phase reactor resistance	pu
L_2	Phase reactor inductance	pu
Hdc_2	DC capacitor electrostatic inertia	s
Snom_2	Nominal apparent power	MVA
Kpd_2	Outer d -axis (DC voltage) loop proportional gain	
Kid_2	Outer d -axis (DC voltage) loop integral gain	
Kpq_2	Outer q -axis (Q/Vac) loop proportional gain	
Kiq_2	Outer q -axis (Q/Vac) loop integral gain	
Kpw_2	PLL proportional gain	
Kiw_2	PLL integral gain	
alpha2_2	$Q-V_{ac}$ droop: reactive power coefficient	
beta2_2	$Q-V_{ac}$ droop: AC voltage coefficient	

DC Link

Parameter	Description	Unit
Pnom	Nominal active power (DC link rating)	MW
R_dc	DC line series resistance	pu
Vdc0	Initial offshore DC voltage	pu

Arrangement

The offshore converter establishes the AC voltage and frequency for the wind farm. The onshore converter maintains DC voltage. An optional hub coordinator can inject a power correction ΔP to the offshore converter via the parameter `DeltaP`.

Usage Example

```

TWOP DCL_WCL WF_HVDC OFFSHORE_BUS ONSHORE_BUS S
0. 0. 0. 0.      ! FP/FQ/P/Q at offshore bus
0. 0. 0. 0.      ! FP/FQ/P/Q at onshore bus
0.003      ! R_1 (pu)
0.15       ! L_1 (pu)
3.5        ! Hdc_1 (s)
900.0      ! Snom_1 (MVA)
2.0        ! Kpp_1
5.0        ! Kip_1
10.0       ! Kwi_1
0.0        ! Kpq_1
20.0       ! Kiq_1
2.0        ! Kpw_1
40.0       ! Kiw_1
0.05       ! betaV_1
0.003      ! R_2 (pu)
0.15       ! L_2 (pu)
3.5        ! Hdc_2 (s)
900.0      ! Snom_2 (MVA)
0.0        ! Kpd_2
10.0       ! Kid_2
0.0        ! Kpq_2

```

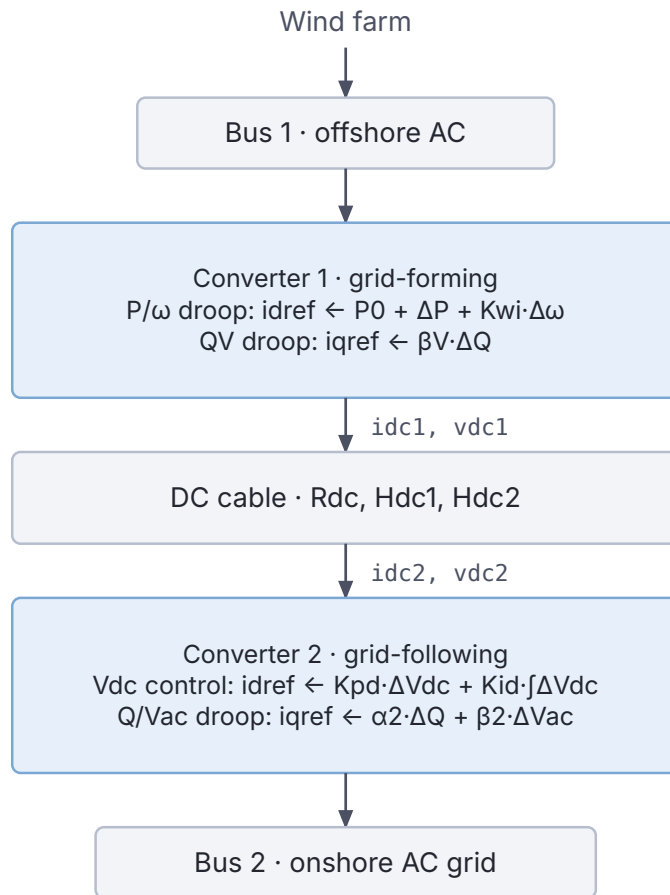


Figure 24.4: The DCL_VSC arrangement.


```
20.0      ! Kiq_2
2.0       ! Kpw_2
40.0      ! Kiw_2
1.0       ! alpha2_2
0.0       ! beta2_2
800.0     ! Pnom (MW)
0.05      ! R_dc (pu)
1.0       ! Vdc0 (pu)
;
```


Part VI

Adding user-defined models with CODEGEN

25

User-defined models: mathematical
formulation and syntax of description

25.1. States and equations

25.1.1. States

The four categories of user-defined models and their acronyms are as follows:

EXC excitation controller of synchronous machine: typically the excitation system and the Automatic Voltage Regulator (AVR), including the Power System Stabilizer (PSS)

TOR torque controller of synchronous machine: typically the turbine and the speed governor

INJ injector: a component connected to a single AC bus

TWOP two-port: a component connecting two buses.

Whichever its type, any model has input states (grouped in $\mathbf{x}_{IN}$), internal states (in $\mathbf{x}_{ITL}$) and output states (in $\mathbf{x}_{OUT}$) as sketched in Fig. 25.1. Note that states can be indifferently differential or algebraic states.

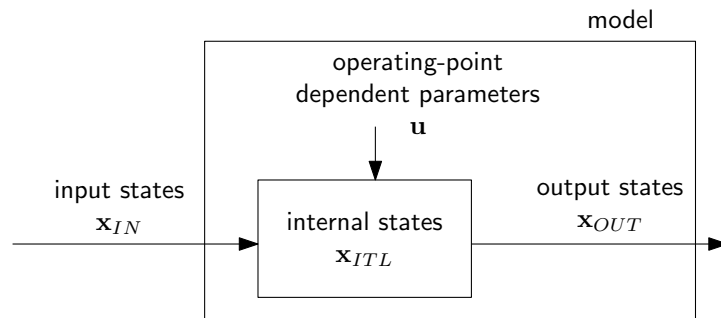


Figure 25.1: Outline of a model

The input and output states pertaining to each category of model are detailed in Table 25.1. The notation is as follows:

For an excitation controller:

V terminal voltage of the machine, in pu

P active power produced, in pu on the machine rated apparent power (MVA)

Q reactive power produced, in pu on the machine rated apparent power (MVA)

ω rotor speed, in pu

i_f field current, in pu on the exciter base current

v_f field voltage, in pu on the exciter base voltage.

For a torque controller:

ω rotor speed, in pu

P active power produced, in pu on the turbine rated power (MW)

T_m mechanical torque applied to rotor, in pu on the turbine rated torque.

For an injector:

v_x, v_y rectangular components of phasor of voltage at the connection bus (the (x, y) reference axes have been defined in Section 10.1)

Table 25.1: Input and output states

type of model	input states	output states
excitation controller of synchronous machine	V, P, Q, ω, i_f of machine	v_f of machine
torque controller of synchronous machine	P, ω of machine	T_m of machine
injector	v_x, v_y at bus ω_{coi}	i_x, i_y into bus
two-port	$v_{x1}, v_{y1}, v_{x2}, v_{y2}$ at buses ω_{coi}	$i_{x1}, i_{y1}, i_{x2}, i_{y2}$ into buses

i_x, i_y rectangular components of phasor of current injected into the connection bus

ω_{coi} angular frequency of center of inertia, in pu¹.

For a two-port:

v_{x1}, v_{y1} rectangular components of phasor of voltage at bus 1

v_{x2}, v_{y2} rectangular components of phasor of voltage at bus 2

i_{x1}, i_{y1} rectangular components of phasor of current injected into bus 1

i_{x2}, i_{y2} rectangular components of phasor of current injected into bus 2

ω_{coi} angular frequency of center of inertia, in pu.

The models also include operating-point dependent parameters (grouped in **u**).

The three categories of states are treated as follows:

- **at the initialization** of the model:
 - the input states are given. They are obtained from the synchronous machine states (see Section 11.2.9) or the initial power injected into buses (see Section 10.1)
 - the output states are also given, using the same information²
 - the internal states are initialized either explicitly by the user or automatically by RAMSES
 - the operating-point dependent parameters are initialized by the user, in agreement with the initial values of the states
- **during the simulation:**
 - the input, the internal and the output states are computed together with the other system states
 - the operating-point dependent parameters remain constant, unless they are modified by a user action (see Section 12.11).

¹This can be used as an average system frequency. One of the modelling blocks (see next Chapter) offers the possibility to estimate the “local” frequency at the connection bus

²at first glance, specifying both the input and the output states would lead to an overdetermined system with more equations than states; this is not the case since the excess equations are used to initialize the operating-point dependent parameters **u**. The number of the latter is equal to the number of output states

Note that the model must have a number of equations at least equal to the number of output states, otherwise it is not correctly formulated.

Note also that not all input states need be used.

✚ Consider the very simple excitation system shown in Fig. 25.2. The following equations are easily derived:

$$0 = V^o - V - dV \quad (25.1)$$

$$T \frac{dv_f}{dt} = -v_f + G dV \quad (25.2)$$

The model has a single input (V), a single internal state (dV) and the requested output state (v_f). dV is an algebraic state, while v_f is a differential one. V^o is the single component of the $\mathbf{u}$ vector.

At initialization, the system is assumed in steady state: $\frac{dv_f}{dt} = 0$. Hence, $dV(0) = \frac{v_f(0)}{G}$, where (0) denotes values at $t = 0$. V^o is obtained from Eq. (25.1) as: $V^o = V(0) + dV(0)$.

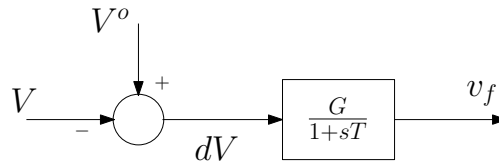


Figure 25.2: A very simple model of excitation system

25.1.2. Equations

As they are determined from the machine and the network equations, the input states are not part of the user model state vector, which thus takes on the form:

$$\mathbf{x} = \begin{bmatrix} \mathbf{x}_{ITL} \\ \mathbf{x}_{OUT} \end{bmatrix} \quad (25.3)$$

The differential-algebraic equations can be written in compact form as:

$$\mathbf{T}\dot{\mathbf{x}} = \mathbf{f}(\mathbf{x}, \mathbf{x}_{IN}, \mathbf{u}, \mathbf{z}) \quad (25.4)$$

where $\mathbf{x}$ and $\mathbf{f}$ have the same dimension. The i -th row of matrix $\mathbf{T}$ includes:

- only zeros if the i -th equation is algebraic
- a single nonzero term T_{ij} if the i -th equation is differential with:

$$T_{ij}\dot{x}_j = f_i(\mathbf{x}, \mathbf{x}_{IN}, \mathbf{u}, \mathbf{z})$$

where i and j may be different.

The nonzero components of $\mathbf{T}$ are time constants, typically.

$\mathbf{z}$ is a vector of discrete variables whose role is detailed in the next section.

25.2. Discrete transitions

The material of this section is provided for information in so far as the corresponding treatment is performed automatically by STEPSS. Nevertheless, it may help interpreting the output curves and/or some execution messages.

25.2.1. Formulation

✚ Consider now a little more detailed version of the model in Fig. 25.2, including non-windup limits on the integrator embedded in the first-order transfer function, as shown in Fig. 25.3.

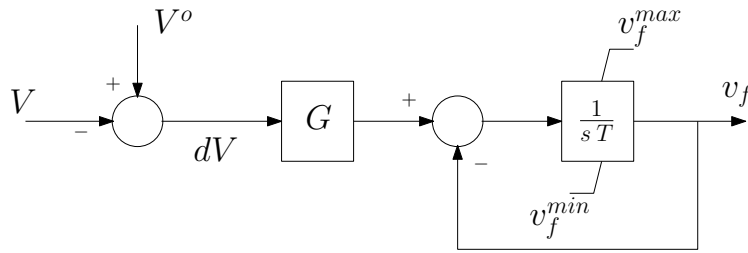


Figure 25.3: A little more detailed version of the system in Fig 25.2

The system is modelled by one of the following three sets of equations:

- if the integrator is not at any of its limits ($z = 0$):

$$0 = V^o - V - dV \quad (25.5)$$

$$T \frac{dv_f}{dt} = -v_f + G dV \quad (25.6)$$

- if the integrator is at its upper limit ($z = 1$):

$$0 = V^o - V - dV \quad (25.7)$$

$$0 = v_f^{max} - v_f \quad (25.8)$$

- if the integrator is at its lower limit ($z = -1$):

$$0 = V^o - V - dV \quad (25.9)$$

$$0 = v_f^{min} - v_f \quad (25.10)$$

As shown in the above example, a discrete variable z is used to identify which set of equations must be solved at any given time of the simulation. The values assigned to z are completely arbitrary. Changing z from one value to another corresponds to substituting one set of equations to another. Such “discrete transitions” take place when some inequality constraints are violated. A typical example is when a state exceeds its prescribed limit.

Note incidentally that differential equations can be changed into algebraic ones, and conversely. This is a feature offered by RAMSES.

✚ In the example of Eqs. (25.5-25.10), here is the pseudo-code performing the discrete transitions relative to the non-windup integrator:

if $z = 0$ then

```

if  $v_f > v_f^{max}$  then
   $z \leftarrow 1$ 
else if  $v_f < v_f^{min}$  then
   $z \leftarrow -1$ 
end if
else if  $z = 1$  then
  if  $(-v_f + G dV)/T < 0$  then
     $z \leftarrow 0$ 
  end if
else if  $z = -1$  then
  if  $(-v_f + G dV)/T > 0$  then
     $z \leftarrow 0$ 
  end if
end if

```

The z variables are initialized at the beginning of the simulation, as for the states.

✚ In the example of Eqs. (25.5-25.10), here is the pseudo-code initializing the z variable relative to the non-windup integrator:

```

if  $v_f > v_f^{max}$  then
   $z \leftarrow 1$ 
else if  $v_f < v_f^{min}$  then
   $z \leftarrow -1$ 
else
   $z \leftarrow 0$ 
end if

```

In the above example, at $t = 0$, if v_f violates one of its limits, it is brought back to that limit. This means that a discrete transition will take place right away at $t = 0$.

25.2.2. Discrete transition identification and treatment scheme

This section deals with the handling of discrete transitions when passing from time t to time $t + h$, where h is the time step size. It is assumed that the system equations have been irrevocably solved until time t .

When the solver detects that the condition for a discrete transition has been satisfied in the time interval $[t, t + h]$, it does not attempt to identify the exact time t' ($t < t' \leq t + h$) when the condition became satisfied. Instead, the following *ex post* solution scheme is used:

1. for the value of the discrete variables $\mathbf{z}$ prevailing at time t , Eqs. (25.4) are integrated from t to $t + h$ with full accuracy³
2. at the resulting point, the inequality constraints associated with discrete transitions are checked. If needed, $\mathbf{z}$ is changed, i.e. Eqs. (25.4) are changed
3. the integration step from t to $t + h$ is canceled, all states are reset to their values at time t , and the new equations (25.4) are integrated from t to $t + h$ with full accuracy

³solving them with less accuracy, to the purpose of gaining time, might lead to wrong identification of the discrete transitions

4. if needed, steps 2 and 3 are repeated until no change in $\mathbf{z}$ takes place. As the solver could be trapped in a limit cycle of discrete transitions, a maximum number of $\mathbf{z}$ changes at the same time is allowed. If that number is reached, the integration time step size is temporarily decreased from h to its minimum value h_{min} (see Section 12.1). If the limit cycle problem persists with the minimum step size, the simulation stops; the model should be adjusted with respect to the sequence of discrete transitions.

The solution scheme is presented in greater detail in:

D. Fabozzi, A. S. Chieh, P. Panciatici and T. Van Cutsem “On simplified handling of state events in time-domain simulation”, Proc. of the 17th Power System Computation Conference (PSCC), 2011

It is illustrated graphically in Fig. 25.4, for a single state x and a single discrete variable z . The numbers refer to the above steps and times are shown in parentheses.

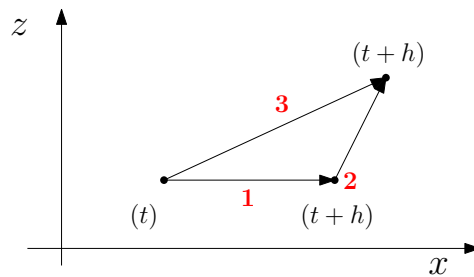


Figure 25.4: Graphical representation of the solution scheme to handle discrete transitions

25.3. Model assembly

In order for CODEGEN to generate the differential-algebraic equations of an arbitrarily complex user model, the latter is decomposed into a set of simple, interconnected *modelling blocks*. The latter correspond to time constants, integrators, PID controllers, non-linearities, etc. The library of available modelling blocks is documented in Chapter 26.

The EXC, TOR, INJ or TWOP model is thus handled as a set of interconnected modelling blocks, as illustrated in Fig. 25.5.

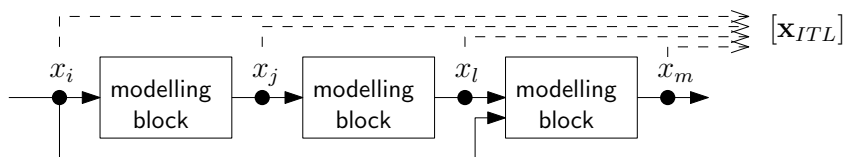


Figure 25.5: A user model made up of interconnected modelling blocks

Each block contributes with its algebraic and/or differential equations. Equations (25.4) are obtained by gathering the equations of all the blocks.

The blocks are interconnected through links. A distinct internal state (contributing to $\mathbf{x}_{ITL}$) is associated with each link. It can be differential or algebraic. Each of these states must be given by the user a unique name as well as an initial value.

Most of the modelling blocks also involve internal states (contributing to $\mathbf{x}_{ITL}$). Unlike the states associated with links between the blocks, the user does not need to name them, nor to initialize them; this is done automatically by the code generated by CODEGEN.

Finally, some modelling blocks involve discrete variables $\mathbf{z}$.

✚ Example. The modelling block `tf1p1z` is an example of block with one internal state. It implements the transfer function with one pole and one zero shown in Fig. 25.6.

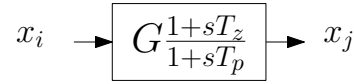


Figure 25.6: The `tf1p1z` modelling block

x_i is either an input or an internal state, while x_j is either an internal or an output state. The model requires three data: G , T_z and T_p .

CODEGEN generates the following equations:

$$\dot{x}_1 = G x_i - x_j \quad (25.11)$$

$$0 = T_p x_j - G T_z x_i - x_1 \quad (25.12)$$

where x_1 is an internal state. The latter is automatically initialized to :

$$x_1(0) = G (T_p - T_z) x_i(0)$$

which is obtained by setting the derivative of x_1 to zero in the above equations.

25.4. Syntax of the model description

A model is specified in a text file with the contents and structure shown in Fig. 25.7. The name of the file can be freely chosen.

The file is made up of six sections, which are detailed hereafter. All sections must be present and in the order shown in Fig. 25.7.

All keywords must be written exactly as specified in this documentation. In particular the upper/lowercase must be the same and no blank must be inserted or skipped.

25.4.1. Header

The header includes two lines in which:

<type of model> specifies the type of model. It can be `exc`, `tor`, `inj` or `twop`, as explained in Section 25.1.1

<name of model> specifies the name of the model. This is a string of at most 16 characters

If a model type `abc` and a model name `xyz` are specified, the complete name of the model will be `abc_xyz.f90`. This name has to be used in the data files (see Sections 25.6.1 and 25.6.2). CODEGEN will correspondingly produce a file named `abc_xyz.f90` with the FORTRAN code of the model.

25.4.2. Data section

The data section starts with the keyword `%data`.

Each data must be given a unique name. That name, enclosed with braces {}, is used in the rest of the model description.

The braces must not be used when a data is first defined, i.e. before the `=` symbol⁴. If used, the brace(s) will be treated as part of the data name, which most likely will lead to an error and the failure to compile the model.

Each name can be followed by a comment, on the same line, starting with an exclamation mark !.

⁴since CODEGEN expects to find a data, there is no ambiguity

```

<type of model>
<name of model>
%data
<name of data 1>
<name of data 2>
    :
%parameters
<name of parameter 1>=<mathematical expression 1>
<name of parameter 2>=<mathematical expression 2>
    :
%states
<name of internal state 1>=<mathematical expression of initial value 1>
<name of internal state 2>=<mathematical expression of initial value 2>
    :
%observables
<name of state, data or parameter 1>
<name of state, data or parameter 2>
    :
%models
& <name of modelling block 1>
<name of state 1>
<name of state 2>
    :
<name of data 1, name of parameter 1 or mathematical expression 1>
<name of data 2, name of parameter 2 or mathematical expression 2>
    :
& <name of modelling block 2>
<name of state 1>
<name of state 2>
    :
<name of data 1, name of parameter 1 or mathematical expression 1>
<name of data 2, name of parameter 2 or mathematical expression 2>
    :

```

Figure 25.7: Syntax of model files

At the initialization of a simulation, RAMSES maps the data present in the input file with those declared in the data section of the model. The data are read from the data file in the order specified in the data section.

The base power used for per unit values in the network is a data available by default in the `inj` and `twop` models, as detailed in Table 25.2. The names are enclosed with braces, as for other data. The names are reserved and must not be used for another data.

25.4.3. Parameter section

The parameter section starts with the keyword `%parameters`.

Table 25.2: Base powers that can be used in models (reserved names)

model type	name	meaning
exc	-	-
tor	-	-
inj	{sbase}	base power used for per unit values in the network (MVA)
twop	{sbase1}	base power used for per unit values in the subnetwork including bus 1 (MVA)
	{sbase2}	base power used for per unit values in the subnetwork including bus 2 (MVA)

Each parameter must be given a unique name. That name, enclosed with braces { }, is used in the rest of the model description.

The braces must not be used when a parameter is first defined, i.e. before the = symbol⁵. If used, the brace(s) will be treated as part of the parameter name, which most likely will lead to an error and the failure to compile the model.

Each name can be followed by a comment, on the same line, starting with an exclamation mark !.

A data and a parameter cannot be given the same name.

At initialization of the simulation, each parameter is given the value specified by the mathematical expression after the = symbol. This expression usually involves data but it can also involve parameters which have been previously defined⁶. It may involve standard mathematical functions such as *cos*, ****, *sqr*t, etc. The syntax is that of the FORTRAN language. It can be found for instance at:

<https://www.intel.com/content/www/us/en/docs/fortran-compiler/developer-guide-reference/2023-0/categories-of-intrinsic-functions.html>

Attention is drawn on the following syntactic items:

- the exponent is denoted with ****, not *^* (as with MATLAB)
- the operators in boolean expressions are denoted as follows: *.lt.* (smaller than), *.le.* (smaller or equal to), *.gt.* (greater than), *.ge.* (greater or equal to), *.eq.* (equal to), *.ne.* (not equal to).

25.4.4. State section

The state section starts with the keyword *%states*.

Each internal state must be given a unique name. That name, enclosed with brackets [], is used in most places of the model description.

The brackets must not be used when a state is first defined, i.e. before the = symbol⁷. If used, the bracket(s) will be treated as part of the state name, which most likely will lead to an error and the failure to compile the model.

Similarly, brackets must not be used when specifying the input and/or output states of a block.

Each state declaration can be followed by a comment, on the same line, starting with an exclamation mark !.

A state cannot have the same name as a data or a parameter.

Input and output states have their own, reserved names that cannot be used for internal states. They are listed in Table 25.3. All values are in pu on the bases detailed in Section 25.1.1. As for other states, their names must be enclosed with brackets.

⁵since CODEGEN expects to find a parameter, there is no ambiguity

⁶the names of those parameters must be enclosed in braces

⁷since CODEGEN expects to find a state, there is no ambiguity

Table 25.3: Reserved names that must not be used for internal states

model type	reserved names	meaning of state
exc	[v] [p] [q] [omega] [if] [vf]	terminal voltage of machine active power produced by machine reactive power produced by machine rotor speed of machine field current of machine field voltage of machine
tor	[p] [omega] [tm]	mechanical power produced by turbine rotor speed of machine mechanical torque of turbine
inj	[vx] [vy] [omega] [ix] [iy]	x component of bus voltage y component of bus voltage angular speed of center of inertia x component of current injected into network y component of current injected into network
twop	[vx1] [vy1] [vx2] [vy2] [omega1] [omega2] [ix1] [iy1] [ix2] [iy2]	x component of voltage at bus 1 y component of voltage at bus 1 x component of voltage at bus 2 y component of voltage at bus 2 angular speed of center of inertia of subsystem including bus 1 angular speed of center of inertia of subsystem including bus 2 x component of current injected into network at bus 1 y component of current injected into network at bus 1 x component of current injected into network at bus 2 y component of current injected into network at bus 2

Each internal state is initialized at the value specified by the mathematical expression after the = symbol. This expression may involve data, parameters or states (the initial values of those states, to be precise) that have been previously defined or are listed in Table 25.3. The mathematical expression may involve standard mathematical functions: see previous section.

Only internal states are declared, input and output states are not. Let us recall that their initial values are known.

25.4.5. Observable section

The observable section starts with the keyword `%observables`.

Observables are quantities candidate to be plotted as functions of time at the end of the simulation. They are usually (input, output or internal) states but data or parameters are also allowed⁸.

The data and the parameter names must not be enclosed with braces. Similarly, the state names must not be enclosed with brackets.

25.4.6. Model section

The model section starts with the keyword `%models`.

The modelling blocks and their data are enumerated sequentially. The order does not matter. Each modelling block is identified by the `&` symbol, **followed by a space**, followed by the name of the block. The information required by each block is detailed in the next chapter.

⁸this allows displaying the time evolution of a controller setpoint, for instance

Each name can be followed by a comment, on the same line, starting with an exclamation mark !.

The end of the section coincides with the end of the file.

25.4.7. An example

Here is the code of the simple excitation system shown in Fig. 25.3. The name of the model is "simple_avr". There are three observables: one parameter, one internal state and one output state

```
exc
simple_avr
%data
G                ! gain of exciter
T                ! time constant of the exciter
vfmin            ! lower fielf voltage
vfmax            ! upper field voltage
%parameters
Vo = [v]+ [vf]/{G} ! voltage setpoint of AVR
%states
dv = [vf]/{G}      ! voltage error
%observables
Vo
dv
vf
%models
& algeq          ! calculation of voltage error
{Vo}-[v]-[dv]
& tf1plim        ! exciter transfer function
dv
vf
{G}
{T}
{vfmin}
{vfmax}
```

Here is the trace of execution, showing the counts of equations and states, respectively, as the modelling blocks are assembled.

```
WELCOME TO CODEGEN    v5
the model generator of STEPSS
```

Input file with model description: simple_avr.txt

MODEL NAME : exc_simple_avr

Processing data...

```
prm( 1)= G          ! gain of exciter
prm( 2)= T          ! time constant of the exciter
prm( 3)= vfmin      ! lower fielf voltage
prm( 4)= vfmax      ! upper field voltage
```

Processing parameters...


```
prm( 5)= Vo    voltage setpoint of AVR
```

Processing states...

```
Output states
x( 1)= vf          field voltage
Internal states defined by user
x( 2)= dv          voltage error
```

Processing observables...

```
Vo
dv
vf
```

Number of user-defined and output d/a states : 2

Processing models...

```
& algeq          ! calculation of voltage error
                  1 d/a eqs      2 d/a states      0 disc states
& tf1plim        ! exciter transfer function
                  2 d/a eqs      2 d/a states      1 disc states
```

Merging temporary files...

```
com.tmp
head.tmp
obs.tmp
init.tmp
evalf.tmp
updz.tmp
```

25.4.8. What about time ?

Time is neither a data, nor a parameter, nor a state. Although this is infrequent, time may appear explicitly in some models. It is denoted t (without brackets).

25.5. Error detection

CODEGEN performs some “sanity checks” to detect mistakes such as:

- unbalanced braces or brackets
- missing keyword %data, %parameters, %states, %observables or %models
- multiply defined data, states or parameters
- usage of a reserved name for an internal state
- typo leading to an unknown name of state or modelling block
- output variable not appearing in any equation of the model
- number of states different from number of equations (differential or algebraic).

However, not all mistakes are flagged by CODEGEN. Typos or syntax errors may not be detected, leading to error messages by the compiler when the .f90 file is compiled. Some examples are:

- typos in the name of a mathematical function. For instance, “cus([delta])” instead of “cos([delta])”, exponent denoted by ^ instead of **
- forgotten braces { } enclosing the name of a data or a parameter
- forgotten brackets [] enclosing the name of a state.

25.6. Data format

25.6.1. Injectors

The data of an injector are defined in the Data section of the user-defined model, see Section 25.4.2. The corresponding values are to be provided in an INJEC record with the following syntax:

```
INJEC MODEL_NAME INJ_NAME BUS FP FQ P Q DATA1 DATA2 ...;
```

where:

- MODEL_NAME is the name of injector model, as defined in the header of the model description, see Section 25.4.1. This is a string of at most 20 characters
- INJ_NAME is the name of the injector (a particular instance of the model). This is a string of at most 20 characters
- BUS is the name of the bus to which the injector is connected. This is a string of at most 8 characters
- FP is the fraction f_{Pj} defined by Eq. (10.8)
- FQ is the fraction f_{Qj} defined by Eq. (10.8)
- P is the initial active power defined by Eq. (10.7). Let us recall that FP and P must obey Eq. (10.10)
- Q is the initial reactive power defined by Eq. 10.7. Let us recall that FQ and Q must obey Eq. 10.10
- DATA1 DATA2 ... are the successive values of the data, as defined in the Data section of the user-defined model. In particular they appear in the order defined in that section.

25.6.2. Two-ports

The data of a two-port are defined in the Data section of the user-defined model, see Section 25.4.2. The corresponding values are to be provided in an TWOP record with the following syntax:

```
TWOP MODEL_NAME TWOP_NAME BUS1 BUS2 IND FP1 FQ1 P1 Q1 FP2 FQ2 P2 Q2
```

```
DATA1 DATA2 ...;
```

where:

- MODEL_NAME is the name of two-port model, as defined in the header of the model description, see Section 25.4.1. This is a string of at most 20 characters
- INJ_NAME is the name of the two-port (a particular instance of the model). This is a string of at most 20 characters
- BUS1 is the name of the first bus to which the two-port is connected. This is a string of at most 8 characters
- BUS2 is the name of the second bus to which the two-port is connected. This is a string of at most 8 characters

- IND is a “synchronization” indicator:
 - if it is set to “S”, the two-port causes the subnetworks of BUS1 and BUS2 to be “synchronous”, i.e. to operate at the same frequency, as for an AC transmission line, for instance
 - if it is set to “A”, the two sub-networks are left “asynchronous”, i.e. they operate at different frequencies, as for an HDVC link, for instance.

Other values of IND are incorrect.

- FP1 is the fraction f_{Pj} defined by Eq. (10.8), relative to BUS1
- FQ1 is the fraction f_{Qj} defined by Eq. (10.8), relative to BUS1
- P1 is the initial active power defined by Eq. (10.7) relative to BUS1. Let us recall that FP1 and P1 must obey Eq. (10.10)
- Q1 is the initial reactive power defined by Eq. 10.7 relative to BUS1. Let us recall that FQ1 and Q1 must obey Eq. 10.10
- FP2 is the fraction f_{Pj} defined by Eq. (10.8), relative to BUS2
- FQ2 is the fraction f_{Qj} defined by Eq. (10.8), relative to BUS2
- P2 is the initial active power defined by Eq. (10.7) relative to BUS2. Let us recall that FP2 and P2 must obey Eq. (10.10)
- Q2 is the initial reactive power defined by Eq. 10.7 relative to BUS2. Let us recall that FQ2 and Q2 must obey Eq. 10.10
- DATA1 DATA2 . . . are the successive values of the data, as defined in the Data section of the user-defined model. In particular they appear in the order defined in that section.

26

Library of modelling blocks

26.1. List of modelling blocks

The list of modelling blocks is given in the table below, together with a brief description.

Table 26.1: List of modelling blocks

abs	Absolute value of input
algeq	Algebraic equation
db	Deadband
f_inj	Estimate of frequency at bus of injector
ftwop_bus1	Estimate of frequency at first bus of two-port
ftwop_bus2	Estimate of frequency at second bus of two-port
fsa	Finite State Automaton
hyst	Hysteresis
int	Integrator with time constant
inlim	Integrator with time constant and non-windup limits on output
invlim	Integrator with time constant and non-windup variable limits on output
lim	Limiter with constant bounds
limvb	Limiter with variable bounds
max1v1c	Maximum between a state and a constant
max2v	Maximum between two states
min1v1c	Minimum between a state and a constant
min2v	Minimum between two states
nint	Integer nearest to the input shifted by a constant
pictl	Proportional-Integral (PI) controller
pictllim	Proportional-Integral (PI) controller with non-windup limit on integral term
pictl2lim	PI controller with non-windup limit on integral term and limit on proportional term
pictlieee	PI controller with non-windup limit on integral term, compliant with IEEE standards
pwlin	Piece-wise linear function of input
switch	Set output to one among n inputs, based on value of a controlling state
swnsign	Switch between two input states, based on sign of a third input state
tf1p	Transfer function between input and output: one time constant
tf1plim	same as tf1p with non-windup limits on output
tf1pvlm	same as tf1p with variable non-windup limits on output
tf1p2lim	same as tf1p with limits on rate of change of output and non-windup limits on output
tfder1p	Transfer function: derivative with one time constant
tf1p1z	Transfer function between input and output: one zero and one pole
tf2p2z	Transfer function between input and output: two real zeros and two real poles
timer	Timer with delay varying piecewise linearly with monitored variable
timersc	Timer with delay varying as a staircase function of monitored variable
tsa	Two-state automaton with transitions based on signs of two inputs

26.2. Information provided for each block

Most blocks have input and output states. **The names of those states must not be enclosed with brackets¹.** If used, the brackets will be treated as part of the state name, which most likely will to a wrong reference and the failure to compile the model.

Most (but not all) blocks require parameters. Each of them is either a data (enclosed with braces) or a parameter in the sense defined in the previous chapter (also enclosed with braces)

¹since CODEGEN expects to find a state name, there is no ambiguity

or a mathematical expression involving data and/or parameters.

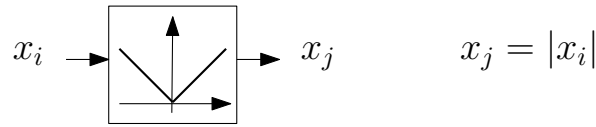
☛ Here are three examples of information that can be specified for a time constant:

2.

{T}

$1/\{\omega_{\text{c}}\}$

26.3. Library

abs*Absolute value of input.*

Syntax:

```

& abs
name of state  $x_i$ 
name of state  $x_j$ 

```

Internal states : none

Discrete variable : $z \in \{-1, 1\}$

Equations :

$$0 = \begin{cases} x_j - x_i & \text{if } z = 1 \\ x_j + x_i & \text{if } z = -1 \end{cases}$$

Discrete transitions :

```

if  $z = 1$  then
  if  $x_i < 0$  then
     $z \leftarrow -1$ 
  end if
else
  if  $x_i > 0$  then
     $z \leftarrow 1$ 
  end if
end if

```

Initialization of discrete variables:

```

if  $x_i > 0$  then
   $z \leftarrow 1$ 
else
   $z \leftarrow 0$ 
end if

```

The model having a discontinuous derivative at $x_i = 0$, it is implemented internally with a small hysteresis; see model **hyst** with $x_I = \epsilon$, $y_{IB} = -\epsilon$, $y_{IA} = \epsilon$, $x_D = -\epsilon$, $y_{DB} = -\epsilon$, $y_{DA} = \epsilon$, where ϵ is the absolute accuracy used to solve the algebraic equations in RAMSES.

algeq*algebraic equation*

Syntax: & algeq
 math expression

Internal states : none

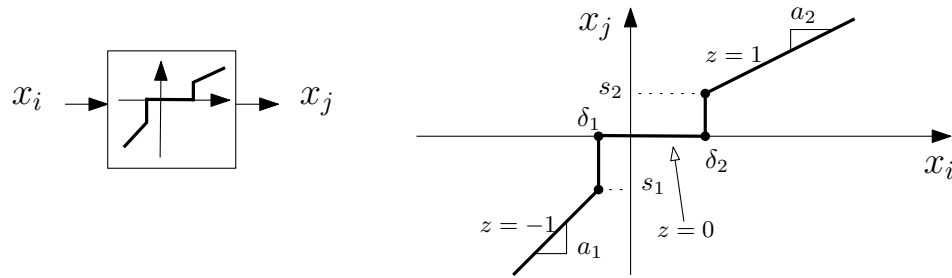
Discrete variables : none

This block forces an algebraic constraint (or equation) involving one of several states :

$$f(x_1, x_2, \dots, x_n) = 0$$

where n is the number of states ($n \geq 1$)

Note that this blocks does not really have “inputs” and “outputs”. The latter stem from the rest of the model involving the algebraic constraint.

db*Deadband.*

Syntax :

& db

name of variable x_i name of variable x_j data name, parameter name or math expression for δ_1 data name, parameter name or math expression for s_1 data name, parameter name or math expression for a_1 data name, parameter name or math expression for δ_2 data name, parameter name or math expression for s_2 data name, parameter name or math expression for a_2

Internal states : none

Discrete variables : $z \in \{0, 1, -1\}$

Equations :

$$0 = \begin{cases} x_j & \text{if } z = 0 \\ x_j - s_2 - a_2(x_i - \delta_2) & \text{if } z = 1 \\ x_j - s_1 - a_1(x_i - \delta_1) & \text{if } z = -1 \end{cases}$$

Discrete transitions :

```

if  $z \in \{0, 1\}$  then
  if  $x_i < \delta_1$  then
     $z \leftarrow -1$ 
  end if
else if  $z \in \{-1, 0\}$  then
  if  $x_i > \delta_2$  then
     $z \leftarrow 1$ 
  end if
else if  $z \in \{-1, 1\}$  then
  if  $\delta_1 < x_i < \delta_2$  then
     $z \leftarrow 0$ 
  end if
end if

```

Initialization of discrete variables :

```

if  $x_i > \delta_2$  then
   $z \leftarrow 1$ 
else if  $x_i < \delta_1$  then
   $z \leftarrow -1$ 
else
   $z \leftarrow 0$ 
end if

```

The data must obey $\delta_1 < \delta_2$, $a_1 \geq 0$ and $a_2 \geq 0$ (see for instance the diagram above).

A particular case is $s_1 = s_2 = 0$ and $a_1 = a_2 = 1$ (but all values are allowed for these four parameters).

The pwlin4, pwlin5 or pwlin6 block can be used as an alternative to this block.

f_inj

Computes f , an estimate of the frequency (in per unit) at a given bus, from the evolution of the rectangular components v_x and v_y of the bus voltage. A measurement time constant T is involved. Can be used in the model of an injector only.

Syntax: & f_inj
 name of variable f
 data name, parameter name or math expression for T

Internal states : v_{xm} and v_{ym} .

Discrete variables : none

Equations :

$$\dot{v}_{xm} = \frac{v_x - v_{xm}}{T} \quad (26.1)$$

$$\dot{v}_{ym} = \frac{v_y - v_{ym}}{T} \quad (26.2)$$

$$0 = \omega_{ref,pu} + \frac{(v_y - v_{ym})v_{xm} - (v_x - v_{xm})v_{ym}}{2\pi f_N T (v_{xm}^2 + v_{ym}^2)} - f \quad (26.3)$$

Initialization of internal states : $v_{xm} = v_x$ and $v_{ym} = v_y$.

Explanation of model

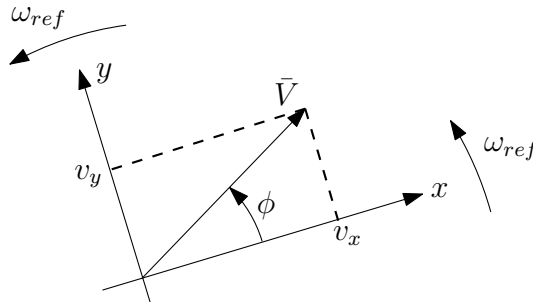
The model uses v_x and v_y as inputs but these variables are automatically inherited and must not be declared. Only the name given to the estimated frequency and the time constant T must be provided.

As shown in the figure below, v_x and v_y are the projections of the bus voltage phasor on the references axes x and y rotating at the angular speed ω_{ref} (rad/s). The latter is known from the settings of the simulation. The angular frequency of the corresponding rotating vector is given by :

$$\omega = \omega_{ref} + \frac{d\phi}{dt}$$

with:

$$\phi = \arctan \frac{v_y}{v_x}$$



The frequency in per unit is given by :

$$f = \frac{\omega}{2\pi f_N} = \frac{\omega_{ref}}{2\pi f_N} + \frac{1}{2\pi f_N} \frac{d\phi}{dt} = \omega_{ref,pu} + \frac{1}{2\pi f_N} \frac{d}{dt} \left(\arctan \frac{v_y}{v_x} \right)$$

where f_N is the nominal frequency (known from the system data).

By developing the last term, it is easily found that:

$$f = \omega_{ref,pu} + \frac{1}{2\pi f_N} \frac{\dot{v}_y v_x - \dot{v}_x v_y}{v_x^2 + v_y^2} \quad (26.4)$$

To filter the transients affecting v_x and v_y , a measurement device with a time constant T is simulated. The “measured” values, denoted v_{xm} and v_{ym} , evolve according to (26.1, 26.2). These measured values and their derivatives are then used in (26.4), which becomes:

$$f = \omega_{ref,pu} + \frac{1}{2\pi f_N} \frac{\dot{v}_{ym} v_{xm} - \dot{v}_{xm} v_{ym}}{v_{xm}^2 + v_{ym}^2}$$

Replacing $\dot{v}_{xm}$ and $\dot{v}_{ym}$ by their expressions (26.1,26.2) yields Eq. (26.3).

A recommended value for T is in the order to 0.05 – 0.10 s. A zero value for T is not allowed. If too small a value is specified for T , the solver may encounter a singularity and the simulation may not proceed.

f_twop_bus1

Similar to f_inj, computes f , an estimate of the frequency (in per unit) at the first bus of a given two-port. Can be used in the model of a two-port only.

Syntax: & f_twop_bus1
 name of variable f
 data name, parameter name or math expression for T

Please refer to f_inj.

f_twop_bus2

Similar to f_inj, computes f , an estimate of the frequency (in per unit) at the second bus of a given two-port. Can be used in the model of a two-port only.

Syntax: & f_twop_bus2
 name of variable f
 data name, parameter name or math expression for T

Please refer to f_inj.

fsa*Finite State Automaton*

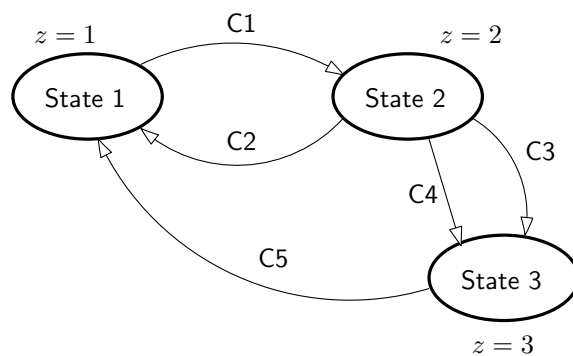
This block forces a set of n algebraic equations. There are s possible sets, each of them corresponding to a value of the discrete state z . The change from one set to another takes place when boolean expressions are true.

Syntax : see example below

Internal states : none

Discrete variables: z , the number of the state currently active

Example. Consider the example below with three states ($s = 3$). $n = 2$ is assumed.



```

& fsa
initial state of the system
# 1
algebraic constraint No. 1
algebraic constraint No. 2
-> 2
boolean expression C1
# 2
algebraic constraint No. 3
algebraic constraint No. 4
-> 1
boolean expression C2
-> 3
boolean expression C3
-> 3
boolean expression C4
# 3
algebraic constraint No. 5
algebraic constraint No. 6
-> 1
boolean expression C5
##
  
```

The # symbol indicates the start of the section relative to a state. It is followed by the number of the state. The states must be numbered consecutively from 1 to s , and they must be listed by increasing order. The ## symbol indicates the end of the list of states.

The initial state is specified as an integer in the second line.

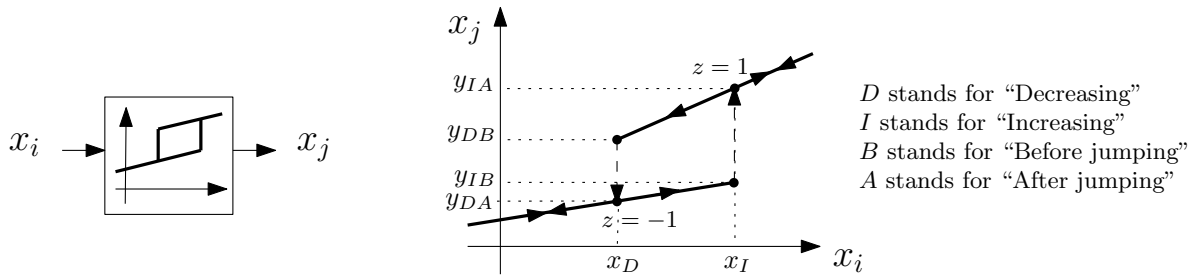
The number n of algebraic constraints must be the same in all states.

The -> symbol indicates a transition. It is followed by the number of the state reached after the transition has taken place. The next line is the corresponding boolean expression, involving states and possibly parameters.

In the example above, there are two possible transitions from State 2 to State 3. Alternatively a single transition can be specified with the combined condition "C3 or C4".

hyst

Hysteresis



Syntax :

& hyst

name of variable x_i

name of variable x_j

data name, parameter name or math expression for x_I

data name, parameter name or math expression for y_{IB}

data name, parameter name or math expression for y_{IA}

data name, parameter name or math expression for x_D

data name, parameter name or math expression for y_{DB}

data name, parameter name or math expression for y_{DA}

data name, parameter name or math expression for z_0

Internal states : none

Discrete variables : $z \in \{-1, 1\}$

Equations :

$$0 = \begin{cases} x_j - y_{IA} - \frac{y_{IA} - y_{DB}}{x_I - x_D}(x_i - x_I) & \text{if } z = 1 \\ x_j - y_{DA} - \frac{y_{IB} - y_{DA}}{x_I - x_D}(x_i - x_D) & \text{if } z = -1 \end{cases}$$

Discrete transitions :

```

if  $z = -1$  then
  if  $x_i > x_I$  then
     $z \leftarrow 1$ 
  end if
else
  if  $x_i < x_D$  then
     $z \leftarrow -1$ 
  end if
end if

```

Initialization of discrete variables :

```

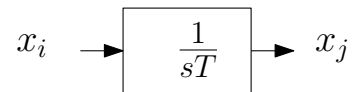
if  $x_i > x_I$  then
   $z \leftarrow 1$ 
else if  $x_i < x_D$  then
   $z \leftarrow -1$ 
else
  if  $z_0 \geq 0$  then
     $z \leftarrow 1$ 
  else
     $z \leftarrow 0$ 
  end if
end if

```

At $t = 0$, if $x_D < x_i(0) < x_I$ the initial state of the system is indeterminate, since it could operate on the (y_{IA}, y_{DB}) line (i.e. with an initial value of z equal to 1) as well as on the (y_{DA}, y_{IB}) line (i.e. with an initial value of z equal to -1). Hence, the user must specify z_0 , the initial value of z .

If $x_i(0) < x_D$ (resp. $x_i(0) > x_I$) the initial value is $z = -1$ (resp. $z = 1$) and z_0 is not used.

The data must obey $x_D < x_I$, otherwise the model would not correspond to hysteresis. The case $x_D = x_I$ is not allowed either, but it can be handled with the **pwlin4** model.

int*Integrator with (positive) time constant T* 

Syntax :

& int
name of variable x_i
name of variable x_j
data name, parameter name or math expression for T

Internal states : none

Discrete variables : none

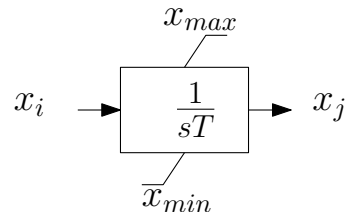
Equations :

$$T\dot{x}_j = x_i \quad (26.5)$$

A zero value for T is not allowed. If too small a value is specified for T , the solver may encounter a singularity and the simulation may not proceed.

inlim

Integrator with (positive) time constant T and non-windup limits on output



Syntax : & inlim
 name of variable x_i
 name of variable x_j
 data name, parameter name or math expression for T
 data name, parameter name or math expression for x_{min}
 data name, parameter name or math expression for x_{max}

Internal states : none

Discrete variables : $z \in \{0, 1, -1\}$

Equations :

$$\begin{aligned}
 T\dot{x}_j &= x_i && \text{if } z = 0 \\
 0 &= x_j - x_{min} && \text{if } z = -1 \\
 0 &= x_j - x_{max} && \text{if } z = 1
 \end{aligned}
 \tag{26.6}$$

Discrete transitions :

```

if  $z = 0$  then
  if  $x_j > x_{max}$  then
     $z \leftarrow 1$ 
  else if  $x_j < x_{min}$  then
     $z \leftarrow -1$ 
  end if
else if  $z = 1$  then
  if  $x_i < 0$  then
     $z \leftarrow 0$ 
  end if
else if  $z = -1$  then
  if  $x_i > 0$  then
     $z \leftarrow 0$ 
  end if
end if

```

Initialization of discrete variables :

```

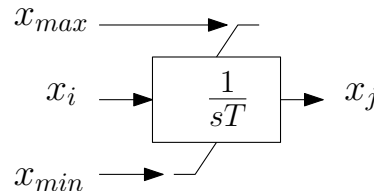
if  $x_j > x_{max}$  then
   $z \leftarrow 1$ 
else if  $x_j < x_{min}$  then
   $z \leftarrow -1$ 
else
   $z \leftarrow 0$ 
end if

```

A zero value for T is not allowed. If too small a value is specified for T , the solver may encounter a singularity and the simulation may not proceed.

invlim

Integrator with (positive) time constant T and non-windup limits on output. The lower and upper limits are variables.



Syntax: & invlim
 name of variable x_i
 name of variable x_{min}
 name of variable x_{max}
 name of variable x_j
 data name, parameter name or math expression for T

Internal states : none

Discrete variables : $z \in \{0, 1, -1\}$

Equations :

$$\begin{aligned}
 T\dot{x}_j &= x_i && \text{if } z = 0 \\
 0 &= x_j - x_{min} && \text{if } z = -1 \\
 0 &= x_j - x_{max} && \text{if } z = 1
 \end{aligned}
 \tag{26.7}$$

Discrete transitions :

```

if  $z = 0$  then
  if  $x_j > x_{max}$  then
     $z \leftarrow 1$ 
  else if  $x_j < x_{min}$  then
     $z \leftarrow -1$ 
  end if
else if  $z = 1$  then
  if  $x_i < 0$  then
     $z \leftarrow 0$ 
  end if
else if  $z = -1$  then
  if  $x_i > 0$  then
     $z \leftarrow 0$ 
  end if
end if

```

Initialization of discrete variables :

```

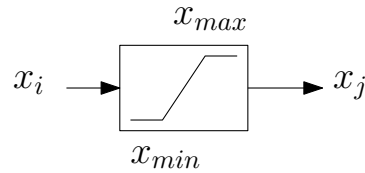
if  $x_j > x_{max}$  then
   $z \leftarrow 1$ 
else if  $x_j < x_{min}$  then
   $z \leftarrow -1$ 
else
   $z \leftarrow 0$ 
end if

```

A zero value for T is not allowed. If too small a value is specified for T , the solver may encounter a singularity and the simulation may not proceed.

lim

Limiter with constant bounds



Syntax :

- & lim
- name of variable x_i
- name of variable x_j
- data name, parameter name or math expression for x_{min}
- data name, parameter name or math expression for x_{max}

Internal states : none

Discrete variables : $z \in \{-1, 0, 1\}$

Equations :

$$0 = \begin{cases} x_j - x_i & \text{if } z = 0 \\ x_j - x_{max} & \text{if } z = 1 \\ x_j - x_{min} & \text{if } z = -1 \end{cases}$$

Discrete transitions :

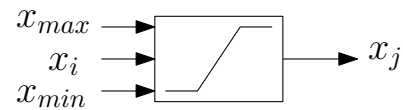
```
if  $z = 0$  then
  if  $x_i > x_{max}$  then
     $z \leftarrow 1$ 
  else if  $x_i < x_{min}$  then
     $z \leftarrow -1$ 
  end if
else if  $z = 1$  then
  if  $x_i < x_{max}$  then
     $z \leftarrow 0$ 
  end if
else if  $z = -1$  then
  if  $x_i > x_{min}$  then
     $z \leftarrow 0$ 
  end if
end if
```

Initialization of discrete variables :

```
if  $x_j > x_{max}$  then
   $z \leftarrow 1$ 
else if  $x_j < x_{min}$  then
   $z \leftarrow -1$ 
else
   $z \leftarrow 0$ 
end if
```

limvb

Limiter with variable bounds



Syntax :

```
& limvb
name of variable  $x_i$ 
name of variable  $x_{min}$ 
name of variable  $x_{max}$ 
name of variable  $x_j$ 
```

Internal states : none

Discrete variables : $z \in \{-1, 0, 1\}$

Equations :

$$0 = \begin{cases} x_j - x_i & \text{if } z = 0 \\ x_j - x_{max} & \text{if } z = 1 \\ x_j - x_{min} & \text{if } z = -1 \end{cases}$$

Discrete transitions :

```
if  $z = 0$  then
  if  $x_i > x_{max}$  then
     $z \leftarrow 1$ 
  else if  $x_i < x_{min}$  then
     $z \leftarrow -1$ 
  end if
else if  $z = 1$  then
  if  $x_i < x_{max}$  then
     $z \leftarrow 0$ 
  end if
else if  $z = -1$  then
  if  $x_i > x_{min}$  then
     $z \leftarrow 0$ 
  end if
end if
```

Initialization of discrete variables :

```
if  $x_i > x_{max}$  then
   $z \leftarrow 1$ 
else if  $x_i < x_{min}$  then
   $z \leftarrow -1$ 
else
   $z \leftarrow 0$ 
end if
```

max1v1c

Maximum between a state and a constant



Syntax: & max1v1c
 name of variable x_i
 name of variable x_j
 data name, parameter name or math expression for C

Internal states : none

Discrete variables : $z \in \{1, 2\}$

Equations :

$$0 = \begin{cases} x_j - C & \text{if } z = 1 \\ x_j - x_i & \text{if } z = 2 \end{cases}$$

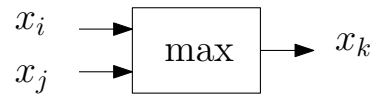
Discrete transitions :

```

if  $z = 1$  then
  if  $x_i > C$  then
     $z \leftarrow 2$ 
  end if
else if  $z = 2$  then
  if  $x_i \leq C$  then
     $z \leftarrow 1$ 
  end if
end if
  
```

Initialization of discrete variables :

```
if  $x_i < C$  then  
   $z \leftarrow 1$   
else  
   $z \leftarrow 2$   
end if
```

max2v*Maximum between two states*

Syntax: & max2v
 name of variable x_i
 name of variable x_j
 name of variable x_k

Internal states : none

Discrete variables : $z \in \{1, 2\}$

Equations :

$$0 = \begin{cases} x_i - x_k & \text{if } z = 1 \\ x_j - x_k & \text{if } z = 2 \end{cases}$$

Discrete transitions :

```

if  $z = 1$  then
  if  $x_i < x_j$  then
     $z \leftarrow 2$ 
  end if
else if  $z = 2$  then
  if  $x_j \leq x_i$  then
     $z \leftarrow 1$ 
  end if
end if

```

Initialization of discrete variables :

```

if  $x_i > x_j$  then
   $z \leftarrow 1$ 
else
   $z \leftarrow 2$ 
end if

```

min1v1c*Minimum between a state and a constant*

Syntax : & min1v1c
 name of variable x_i
 name of variable x_j
 data name, parameter name or math expression for C

Internal states : none

Discrete variables : $z \in \{1, 2\}$

Equations :

$$0 = \begin{cases} x_j - x_i & \text{if } z = 1 \\ x_j - C & \text{if } z = 2 \end{cases}$$

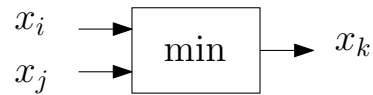
Discrete transitions :

```

if  $z = 1$  then
  if  $x_i > C$  then
     $z \leftarrow 2$ 
  end if
else if  $z = 2$  then
  if  $x_i \leq C$  then
     $z \leftarrow 1$ 
  end if
end if
  
```

Initialization of discrete variables :

```
if  $x_i < C$  then  
   $z \leftarrow 1$   
else  
   $z \leftarrow 2$   
end if
```


min2v*Minimum between two states*

Syntax: & min2v
 name of variable x_i
 name of variable x_j
 name of variable x_k

Internal states : none

Discrete variables : $z \in \{1, 2\}$

Equations :

$$0 = \begin{cases} x_i - x_k & \text{if } z = 1 \\ x_j - x_k & \text{if } z = 2 \end{cases}$$

Discrete transitions :

```

if  $z = 1$  then
  if  $x_i > x_j$  then
     $z \leftarrow 2$ 
  end if
else if  $z = 2$  then
  if  $x_j \geq x_i$  then
     $z \leftarrow 1$ 
  end if
end if
  
```

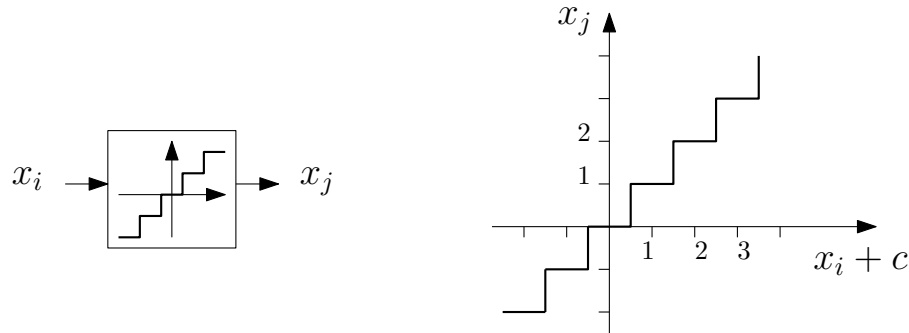
Initialization of discrete variables :

```

if  $x_i < x_j$  then
   $z \leftarrow 1$ 
else
   $z \leftarrow 2$ 
end if
  
```

nint

Integer nearest to the input shifted by a constant c .



Syntax: & nint
 name of variable x_i
 name of variable x_j
 data name, parameter name or math expression for c

Internal states : none

Discrete variables : $z \in \mathcal{I}$

Equations :

$$0 = x_j - z$$

Discrete transitions :

```

if nint( $x_i + c$ )  $\neq$   $z$  then
   $z \leftarrow$  nint( $x_i + c$ )
end if

```

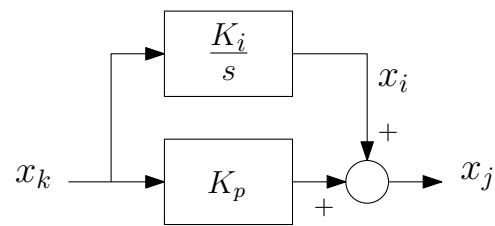
where the nint function returns the nearest integer.

Particular cases:

- with $c = 0$, the output is the integer nearest to x_i ;
- with $c = -0.5$, the output is the “floor” integer of x_i , i.e. the largest integer smaller or equal to x_i ;
- with $c = 0.5$, the output is the “ceiling” integer of x_i , i.e. the smallest integer larger or equal to x_i .

Initialization of discrete variables :

$$z \leftarrow \text{nint}(x_i + c)$$

pictl*Proportional-Integral (PI) controller*

Syntax :

& pictl

name of variable x_k name of variable x_j data name, parameter name or math expression for K_i data name, parameter name or math expression for K_p Internal states : x_i

Discrete variables: none

Equations :

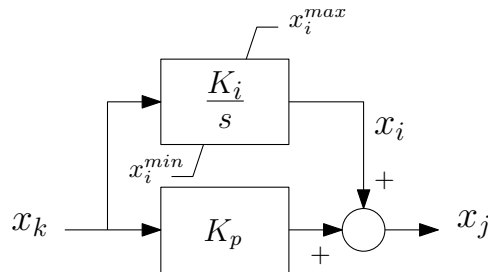
$$\dot{x}_i = K_i x_k$$

$$0 = K_p x_k + x_i - x_j$$

Initialization of internal states: $x_i = x_j$

pictllim

Proportional-Integral (PI) controller with non-windup limit on the integral term.



Syntax :

& pictllim

name of variable x_k

name of variable x_j

data name, parameter name or math expression for K_i

data name, parameter name or math expression for K_p

data name, parameter name or math expression for x_i^{min}

data name, parameter name or math expression for x_i^{max}

Internal states: x_i

Discrete variables : $z \in \{-1, 0, 1\}$

Equations :

$$\begin{cases} \dot{x}_i = K_i x_k & \text{if } z = 0 \\ 0 = x_i - x_i^{min} & \text{if } z = -1 \\ 0 = x_i - x_i^{max} & \text{if } z = 1 \end{cases}$$

$$0 = K_p x_k + x_i - x_j$$

Discrete transitions :

```

if  $z = 0$  then
  if  $x_i > x_i^{max}$  then
     $z \leftarrow 1$ 
  else if  $x_i < x_i^{min}$  then
     $z \leftarrow -1$ 
  end if
else if  $z = 1$  then
  if  $K_i x_k < 0$  then
     $z \leftarrow 0$ 
  end if
else if  $z = -1$  then
  if  $K_i x_k > 0$  then
     $z \leftarrow 0$ 
  end if
end if

```

Initialization of the internal state and the discrete variable:

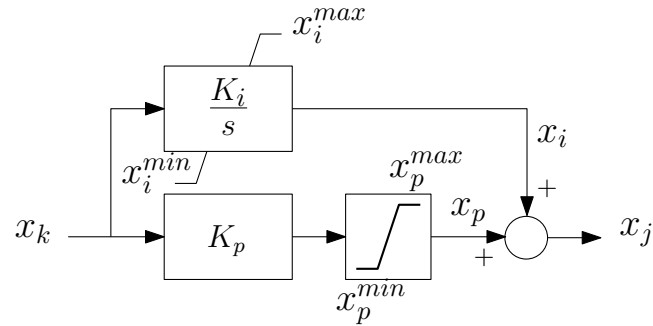
```

if  $K_i x_k > 0$  then
   $z \leftarrow 1$ 
   $x_i \leftarrow x_i^{max}$ 
else if  $K_i x_k < 0$  then
   $z \leftarrow -1$ 
   $x_i \leftarrow x_i^{min}$ 
else
   $z \leftarrow 0$ 
   $x_i \leftarrow x_j$ 
end if

```

pictl2lim

Proportional-Integral (PI) controller with non-windup limit on the integral term and limit on the proportional term.



Syntax :

& pictl2lim

name of variable x_k

name of variable x_j

data name, parameter name or math expression for K_i

data name, parameter name or math expression for K_p

data name, parameter name or math expression for x_i^{min}

data name, parameter name or math expression for x_i^{max}

data name, parameter name or math expression for x_p^{min}

data name, parameter name or math expression for x_p^{max}

Internal states : x_i and x_p

Discrete variables : $z_1 \in \{-1, 0, 1\}$ and $z_2 \in \{-1, 0, 1\}$

Equations :

$$\begin{cases} 0 &= K_p x_k - x_p & \text{if } z_1 = 0 \\ 0 &= x_p - x_p^{min} & \text{if } z_1 = -1 \\ 0 &= x_p - x_p^{max} & \text{if } z_1 = 1 \end{cases}$$

$$\begin{cases} \dot{x}_i &= K_i x_k & \text{if } z_2 = 0 \\ 0 &= x_i - x_i^{min} & \text{if } z_2 = -1 \\ 0 &= x_i - x_i^{max} & \text{if } z_2 = 1 \end{cases}$$

$$0 = x_p + x_i - x_j$$

Discrete transitions :

```

if  $z_1 = 0$  then
  if  $x_p > x_p^{max}$  then
     $z_1 \leftarrow 1$ 
  else if  $x_p < x_p^{min}$  then
     $z_1 \leftarrow -1$ 
  end if
else if  $z_1 = 1$  then
  if  $K_p x_k < x_p^{max}$  then
     $z_1 \leftarrow 0$ 
  end if
else if  $z_1 = -1$  then
  if  $K_p x_k > x_p^{min}$  then
     $z_1 \leftarrow 0$ 
  end if
end if
if  $z_2 = 0$  then
  if  $x_i > x_i^{max}$  then
     $z_2 \leftarrow 1$ 
  else if  $x_i < x_i^{min}$  then
     $z_2 \leftarrow -1$ 
  end if
else if  $z_2 = 1$  then
  if  $K_i x_k < 0$  then
     $z_2 \leftarrow 0$ 
  end if
else if  $z_2 = -1$  then
  if  $K_i x_k > 0$  then
     $z_2 \leftarrow 0$ 
  end if
end if

```

Initialization of the internal state x_p : $x_p = \min(x_p^{max}, \max(x_p^{min}, K_p x_k))$

Initialization of internal state x_i and the discrete variables:

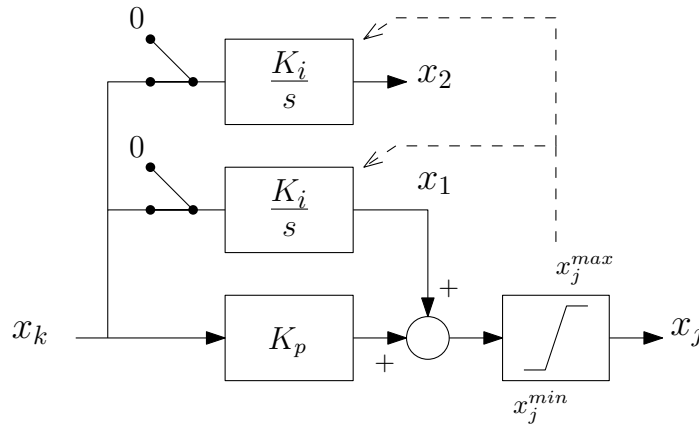
```

if  $K_p x_k > x_p^{max}$  then
   $z_1 \leftarrow 1$ 
else if  $K_p x_k < x_p^{min}$  then
   $z_1 \leftarrow -1$ 
else
   $z_1 \leftarrow 0$ 
end if
if  $K_i x_k > 0$  then
   $z_2 \leftarrow 1$ 
   $x_i \leftarrow x_i^{max}$ 
else if  $K_i x_k < 0$  then
   $z_2 \leftarrow -1$ 
   $x_i \leftarrow x_i^{min}$ 
else
   $z_2 \leftarrow 0$ 
   $x_i \leftarrow x_j - x_p$ 
end if

```


pictlieee

Proportional-Integral (PI) controller with non-windup limit on the integral term, compliant with IEEE standards.



This PI controller involves a limiter and a non-windup integrator compliant with the IEEE specifications in:

- IEEE recommended practice for excitation system models for power system stability studies, IEEE Std 421.5-1992
- IEEE recommended practice for excitation system models for power system stability studies, IEEE Std 421.5-2005 (Revision of IEEE Std 421.5-1992)
- IEEE recommended practice for excitation system models for power system stability studies, IEEE Std 421.5-2016 (Revision of IEEE Std 421.5-2005)

The IEEE standard specifies that the x_1 variable is frozen as soon as x_j reaches its (lower or upper) limit. This is done by merely setting the input of the integrator to zero.

However, the standard does not specify the condition under which x_1 is let to vary again, i.e. when the integrator is put back into service. A simple way would be to re-activate the integrator as soon x_j gets back within its limits. However, at that time instant, the x_k variable may have a value such that $x_j = K_p x_k + x_1$ is pushed again to its (lower or upper) limit. The system would then be trapped into a limit cycle (which would not match the behaviour of the system to model!).

With the implementation shown in the above block diagram, the limit cycle is avoided by relying on the upper integrator to decide when the lower integrator can be released. The upper integrator is used to observe what the evolution of the system would be with x_1 free to vary: the lower integrator is re-activated only when $K_p x_k + x_2$ is in $[x_j^{min}, x_j^{max}]$. As it is in “open loop”, the additional integrator does not interact with the rest of the system.

Syntax: & pictlieee
 name of variable x_k
 name of variable x_j
 data name, parameter name or math expression for K_i
 data name, parameter name or math expression for K_p
 data name, parameter name or math expression for x_j^{min}
 data name, parameter name or math expression for x_j^{max}

Internal states: x_1 and x_2

Discrete variables : $z \in \{-2, -1, 0, 1, 2\}$

Equations :

$$\text{if } z = 0 : \begin{cases} 0 & = K_p x_k + x_1 - x_j \\ \dot{x}_1 & = K_i x_k \\ 0 & = x_2 - x_1 \end{cases}$$

$$\text{if } z = 2 : \begin{cases} 0 & = x_j^{max} - x_j \\ \dot{x}_1 & = 0 \\ 0 & = x_2 - x_1 \end{cases} \quad \text{if } z = 1 : \begin{cases} 0 & = x_j^{max} - x_j \\ \dot{x}_1 & = 0 \\ \dot{x}_2 & = K_i x_k \end{cases}$$

$$\text{if } z = -2 : \begin{cases} 0 & = x_j^{min} - x_j \\ \dot{x}_1 & = 0 \\ 0 & = x_2 - x_1 \end{cases} \quad \text{if } z = -1 : \begin{cases} 0 & = x_j^{min} - x_j \\ \dot{x}_1 & = 0 \\ \dot{x}_2 & = K_i x_k \end{cases}$$

Discrete transitions (note the use of x_2 in the tests marked with (*)):

```

if  $z = 0$  then
  if  $x_j > x_j^{max}$  then
     $z \leftarrow 2$ 
  else if  $x_j < x_j^{min}$  then
     $z \leftarrow -2$ 
  end if
else if  $z = 2$  then
  if  $K_p x_k + x_1 < x_j^{max}$  then
     $z \leftarrow 1$ 
  end if
else if  $z = 1$  then
  if  $K_p x_k + x_1 > x_j^{max}$  then
     $z \leftarrow 2$ 
  else if  $K_p x_k + x_2 < x_j^{max}$  (*) then
     $z \leftarrow 0$ 
  end if
else if  $z = -2$  then
  if  $K_p x_k + x_1 > x_j^{min}$  then
     $z \leftarrow -1$ 
  end if
else if  $z = -1$  then
  if  $K_p x_k + x_1 < x_j^{min}$  then
     $z \leftarrow -2$ 
  else if  $K_p x_k + x_2 > x_j^{min}$  (*) then
     $z \leftarrow 0$ 
  end if
end if

```

Initialization of the internal state and the discrete variable:

```

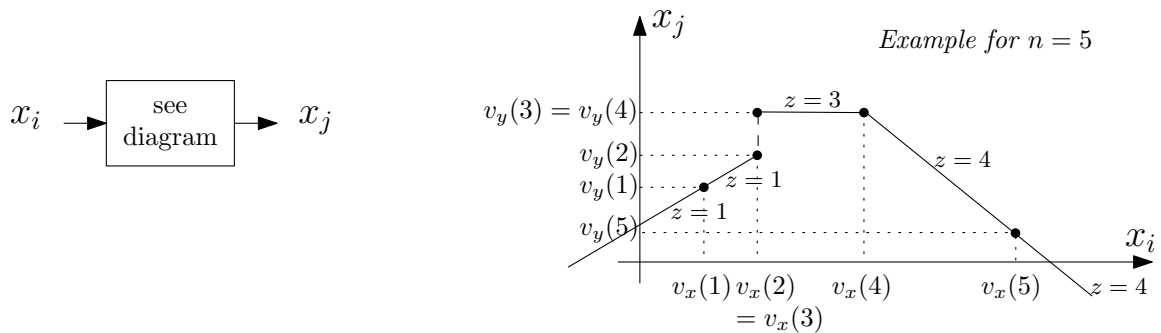
if  $x_j \geq x_j^{max}$  then
   $z \leftarrow 2$ 
   $x_1 \leftarrow x_j^{max} - K_p x_k$ 
   $x_2 \leftarrow x_1$ 
else if  $x_j \leq x_j^{min}$  then
   $z \leftarrow -2$ 
   $x_1 \leftarrow x_j^{min} - K_p x_k$ 
   $x_2 \leftarrow x_1$ 
else
   $z \leftarrow 0$ 
   $x_1 \leftarrow x_j$ 
   $x_2 \leftarrow x_1$ 
end if

```

pwlin*

Piece-wise linear function of input, defined by n points.

Separate blocks exist for $n = 3, 4, 5$ and 6 .



Syntax:

& pwlin3

name of variable x_i

name of variable x_j

data name, parameter name or math expression for $v_x(1)$

data name, parameter name or math expression for $v_y(1)$

data name, parameter name or math expression for $v_x(2)$

data name, parameter name or math expression for $v_y(2)$

data name, parameter name or math expression for $v_x(3)$

data name, parameter name or math expression for $v_y(3)$

& pwlin4

name of variable x_i

name of variable x_j

data name, parameter name or math expression for $v_x(1)$

data name, parameter name or math expression for $v_y(1)$

data name, parameter name or math expression for $v_x(2)$

data name, parameter name or math expression for $v_y(2)$

data name, parameter name or math expression for $v_x(3)$

data name, parameter name or math expression for $v_y(3)$

data name, parameter name or math expression for $v_x(4)$

data name, parameter name or math expression for $v_y(4)$

& pwlin5

name of variable x_i

name of variable x_j

data name, parameter name or math expression for $v_x(1)$

data name, parameter name or math expression for $v_y(1)$

data name, parameter name or math expression for $v_x(2)$

data name, parameter name or math expression for $v_y(2)$

data name, parameter name or math expression for $v_x(3)$

data name, parameter name or math expression for $v_y(3)$

data name, parameter name or math expression for $v_x(4)$

data name, parameter name or math expression for $v_y(4)$

data name, parameter name or math expression for $v_x(5)$

data name, parameter name or math expression for $v_y(5)$

& pwlin6
 name of variable x_i
 name of variable x_j
 data name, parameter name or math expression for $v_x(1)$
 data name, parameter name or math expression for $v_y(1)$
 data name, parameter name or math expression for $v_x(2)$
 data name, parameter name or math expression for $v_y(2)$
 data name, parameter name or math expression for $v_x(3)$
 data name, parameter name or math expression for $v_y(3)$
 data name, parameter name or math expression for $v_x(4)$
 data name, parameter name or math expression for $v_y(4)$
 data name, parameter name or math expression for $v_x(5)$
 data name, parameter name or math expression for $v_y(5)$
 data name, parameter name or math expression for $v_x(6)$
 data name, parameter name or math expression for $v_y(6)$

Internal states : none

Discrete variables : $z \in \{1, \dots, n - 1\}$

Equations :

$$0 = v_y(z) + \frac{v_y(z + 1) - v_y(z)}{v_x(z + 1) - v_x(z)} (x_i - v_x(z)) - x_j$$

Discrete transitions :

```

if  $x_i < v_x(1)$  then
   $z \leftarrow 1$ 
else if  $x_i \geq v_x(n)$  then
   $z \leftarrow n - 1$ 
else
  for  $k = 1$  to  $n - 1$  do
    if  $v_x(k) \leq x_i$  and  $x_i < v_x(k + 1)$  then
       $z \leftarrow k$ 
    end if
  end for
end if

```

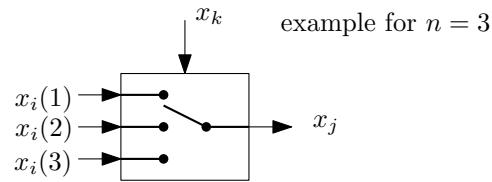
Initialization of discrete variables : same code as for the discrete transitions

The v_x values must be increasing, i.e. $v_x(1) < v_x(2) \leq v_x(3) \leq \dots \leq v_x(n-1) < v_x(n)$.

For x_i values smaller than $v_x(1)$ (resp. larger than $v_x(n)$), x_j is obtained by linear extrapolation based on the first two (resp. the last two) points. Hence, the data must be such that $v_x(1) \neq v_x(2)$ and $v_x(n-1) \neq v_x(n)$.

switch*

Set the output state to one among n input states, based on the value of a controlling state.
 Separate blocks exist for $n = 2, 3, 4$ and 5 .



Syntax :

& switch2

name of variable $x_i(1)$

name of variable $x_i(2)$

name of variable x_k

name of variable x_j

& switch4

name of variable $x_i(1)$

name of variable $x_i(2)$

name of variable $x_i(3)$

name of variable $x_i(4)$

name of variable x_k

name of variable x_j

& switch3

name of variable $x_i(1)$

name of variable $x_i(2)$

name of variable $x_i(3)$

name of variable x_k

name of variable x_j

& switch5

name of variable $x_i(1)$

name of variable $x_i(2)$

name of variable $x_i(3)$

name of variable $x_i(4)$

name of variable $x_i(5)$

name of variable x_k

name of variable x_j

Internal states : none

Discrete variables : $z \in \{1, 2, \dots, n\}$

Equations :

$$0 = x_j - x_i(z)$$

Discrete transitions :

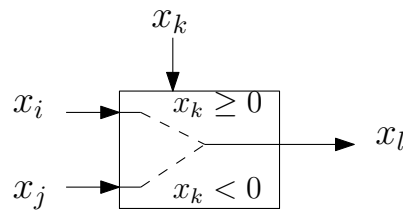
$$z \leftarrow \max(1, \min(n, \text{nint}(x_k)))$$

where the nint function returns the nearest integer.

Initialization of discrete variables : same code as for the discrete transitions

swsign

Switch between two input states, based on the sign of a third input state



Syntax: & swsign
 name of variable x_i
 name of variable x_j
 name of variable x_k
 name of variable x_l

Internal states : none

Discrete variables : $z \in \{1, 2\}$

Equations :

$$0 = \begin{cases} x_l - x_i & \text{if } z = 1 \\ x_l - x_j & \text{if } z = 2 \end{cases}$$

Discrete transitions :

```

if  $z = 1$  then
  if  $x_k < 0$  then
     $z \leftarrow 2$ 
  end if
else if  $z = 2$  then
  if  $x_k \geq 0$  then
     $z \leftarrow 1$ 
  end if
end if
  
```

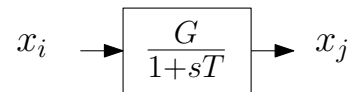
Initialization of discrete variables :

```

if  $x_k < 0$  then
   $z \leftarrow 2$ 
else
   $z \leftarrow 1$ 
end if
  
```


tf1p

Transfer function between input and output: one time constant



Syntax : & tf1p
 name of variable x_i
 name of variable x_j
 data name, parameter name or math expression for G
 data name, parameter name or math expression for T

Internal states : none

Discrete variables : none

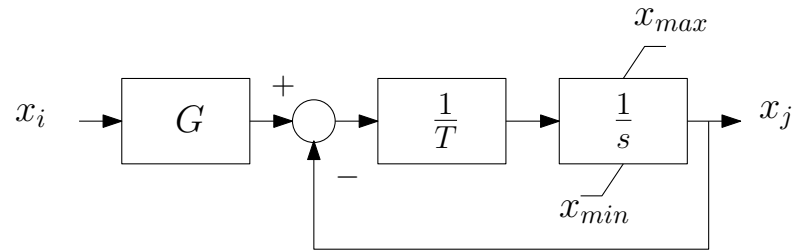
Equations :

$$T \dot{x}_j = -x_j + G x_i$$

The time constant T can be zero.

tf1plim

Transfer function between input and output: one time constant with non-windup limits on output.



Syntax :

& tf1plim

name of variable x_i

name of variable x_j

data name, parameter name or math expression for G

data name, parameter name or math expression for T

data name, parameter name or math expression for x_{min}

data name, parameter name or math expression for x_{max}

Internal states : none

Discrete variables : $z \in \{-1, 0, 1\}$

Equations :

$$\begin{cases} T \dot{x}_j &= G x_i - x_j & \text{if } z = 0 \\ 0 &= x_j - x_{max} & \text{if } z = 1 \\ 0 &= x_j - x_{min} & \text{if } z = -1 \end{cases}$$

Discrete transitions :

```

if  $z = 0$  then
  if  $x_j > x_{max}$  then
     $z \leftarrow 1$ 
  else if  $x_j < x_{min}$  then
     $z \leftarrow -1$ 
  end if
else if  $z = 1$  then
  if  $G x_i - x_j < 0$  then
     $z \leftarrow 0$ 
  end if
else if  $z = -1$  then
  if  $G x_i - x_j > 0$  then
     $z \leftarrow 0$ 
  end if
end if

```

The time constant T can be zero. In this case:

- the block behaves like a gain: $x_j = Gx_i$
- the limits x_{min} and x_{max} remain in effect.

Initialization of discrete variables :

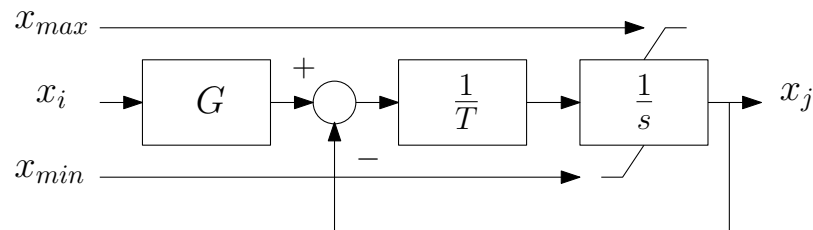
```

if  $x_j \geq x_{max}$  then
   $z \leftarrow 1$ 
else if  $x_j \leq x_{min}$  then
   $z \leftarrow -1$ 
else
   $z \leftarrow 0$ 
end if

```

tf1pvlm

Transfer function between input and output: one time constant with non-windup limits on output. The limits are variables.



Syntax:

```
& tf1plim
name of variable  $x_i$ 
name of variable  $x_j$ 
name of variable  $x_{min}$ 
name of variable  $x_{max}$ 
data name, parameter name or math expression for  $G$ 
data name, parameter name or math expression for  $T$ 
```

Internal states : none

Discrete variables : $z \in \{-1, 0, 1\}$

Equations :

$$\begin{cases} T \dot{x}_j = G x_i - x_j & \text{if } z = 0 \\ 0 = x_j - x_{max} & \text{if } z = 1 \\ 0 = x_j - x_{min} & \text{if } z = -1 \end{cases}$$

Discrete transitions :

```
if  $z = 0$  then
  if  $x_j > x_{max}$  then
     $z \leftarrow 1$ 
  else if  $x_j < x_{min}$  then
     $z \leftarrow -1$ 
  end if
else if  $z = 1$  then
  if  $G x_i - x_j < 0$  then
     $z \leftarrow 0$ 
  end if
else if  $z = -1$  then
  if  $G x_i - x_j > 0$  then
     $z \leftarrow 0$ 
  end if
end if
```

The time constant T can be zero. In this case:

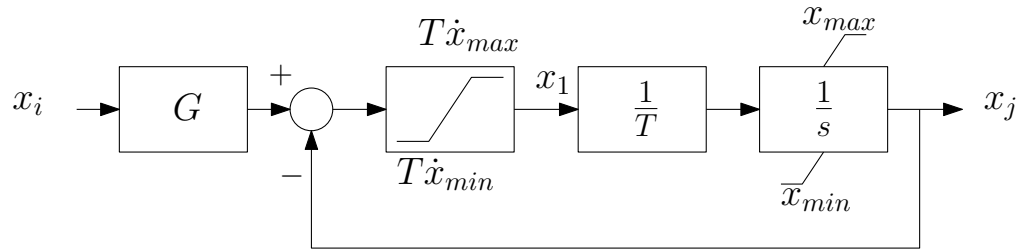
- the block behaves like a gain: $x_j = G x_i$
- the limits x_{min} and x_{max} remain in effect.

Initialization of discrete variables :

```
if  $x_j \geq x_{max}$  then  
   $z \leftarrow 1$   
else if  $x_j \leq x_{min}$  then  
   $z \leftarrow -1$   
else  
   $z \leftarrow 0$   
end if
```

tf1p2lim

Transfer function between input and output: one time constant, with limits on rate of change of output and non-windup limits on output



Syntax:

```
& tf1p2lim
name of variable  $x_i$ 
name of variable  $x_j$ 
data name, parameter name or math expression for  $G$ 
data name, parameter name or math expression for  $T$ 
data name, parameter name or math expression for  $x_{min}$ 
data name, parameter name or math expression for  $x_{max}$ 
data name, parameter name or math expression for  $\dot{x}_{min}$ 
data name, parameter name or math expression for  $\dot{x}_{max}$ 
```

One internal state : x_1 initialized at $\min[\max(0, T \dot{x}_{min}), T \dot{x}_{max}]$

Two discrete variables : $z_1 \in \{-1, 0, 1\}$ and $z_2 \in \{-1, 0, 1\}$

Two equations :

$$\begin{cases} 0 &= x_1 - G x_i + x_j & \text{if } z_1 = 0 \\ 0 &= x_1 - T \dot{x}_{max} & \text{if } z_1 = 1 \\ 0 &= x_1 - T \dot{x}_{min} & \text{if } z_1 = -1 \end{cases}$$

$$\begin{cases} T \dot{x}_j &= x_1 & \text{if } z_2 = 0 \\ 0 &= x_j - x_{max} & \text{if } z_2 = 1 \\ 0 &= x_j - x_{min} & \text{if } z_2 = -1 \end{cases}$$

$\dot{x}_{max}$ (resp. $\dot{x}_{min}$) is the maximum (resp. minimum) rate of change of x with time.

The time constant T can be zero. In this case:

- $x_1 = 0$ throughout the whole simulation
- the block behaves like a gain: $x_j = G x_i$
- both limiters are ignored: $x_{max} \rightarrow \infty$, $x_{min} \rightarrow -\infty$, $T \dot{x}_{max} \rightarrow \infty$, $T \dot{x}_{min} \rightarrow -\infty$.

Discrete transitions :

```

if  $z_1 = 0$  then
  if  $x_1 > T \dot{x}_{max}$  then
     $z_1 \leftarrow 1$ 
  else if  $x_1 < T \dot{x}_{min}$  then
     $z_1 \leftarrow -1$ 
  end if
else if  $z_1 = 1$  then
  if  $Gx_i - x_j < T \dot{x}_{max}$  then
     $z_1 \leftarrow 0$ 
  end if
else if  $z_1 = -1$  then
  if  $Gx_i - x_j > T \dot{x}_{min}$  then
     $z_1 \leftarrow 0$ 
  end if
end if
if  $z_2 = 0$  then
  if  $x_j > x_{max}$  then
     $z_2 \leftarrow 1$ 
  else if  $x_j < x_{min}$  then
     $z_2 \leftarrow -1$ 
  end if
else if  $z_2 = 1$  then
  if  $x_1 < 0$  then
     $z_2 \leftarrow 0$ 
  end if
else if  $z_2 = -1$  then
  if  $x_1 > 0$  then
     $z_2 \leftarrow 0$ 
  end if
end if

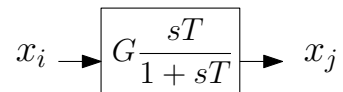
```

Initialization of discrete variables :

```
if  $Gx_i - x_j > T \dot{x}_{max}$  then  
   $z_1 \leftarrow 1$   
else if  $Gx_i - x_j < T \dot{x}_{min}$  then  
   $z_1 \leftarrow -1$   
else  
   $z_1 \leftarrow 0$   
end if  
if  $x_j > x_{max}$  then  
   $z_2 \leftarrow 1$   
else if  $x_j < x_{min}$  then  
   $z_2 \leftarrow -1$   
else  
   $z_2 \leftarrow 0$   
end if
```


tfder1p

Transfer function: derivative with one time constant



Syntax : & tfder1p
 name of variable x_i
 name of variable x_j
 data name, parameter name or math expression for G
 data name, parameter name or math expression for T

Internal states : x_1

Discrete variables : none

Equations :

$$T \dot{x}_1 = x_j \quad (26.1)$$

$$0 = G x_i - x_1 - x_j \quad (26.2)$$

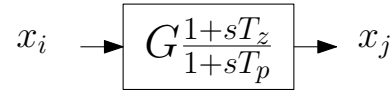
Initialization of internal states: $x_1 = G x_i$

The model allows $T = 0$. In this case:

- $x_j = 0$ as expected;
- Eq. (26.2) is useless but is integrated with the rest of the model.

tf1p1z

Transfer function between input and output: one zero and one pole



Syntax : & tf1p1z
 name of variable x_i
 name of variable x_j
 data name, parameter name or math expression for G
 data name, parameter name or math expression for T_z
 data name, parameter name or math expression for T_p

Internal states : x_1

Discrete variables : none

Equations :

$$\dot{x}_1 = G x_i - x_j \quad (26.3)$$

$$0 = T_p \dot{x}_j - G T_z x_i - x_1 \quad (26.4)$$

Initialization of internal states : $x_1 = G (T_p - T_z) x_i$

The model allows $T_z = 0$. In this case, simplifying Eq. (26.4) and combining it with Eq. (26.3) yields $T_p \dot{x}_j = -x_j + G x_i$, which corresponds to $x_j = \frac{G}{1 + sT_p} x_i$ as expected.

Formally, the model also allows $T_p = 0$. However, in this case, the transfer function between x_i and x_j becomes $G (1 + sT_z)$. **This involves a pure derivator. Hence, the solver may produce undesired transients when x_i or $\dot{x}_i$ undergoes a discontinuity.**

The model allows $T_p = T_z = T$. In this case, Equation (26.4) becomes $x_j - G x_i = \frac{x_1}{T}$, and replacing this result in Eq. (26.3) yields $\dot{x}_1 = -\frac{x_1}{T}$, showing that x_1 evolves independently of x_i and x_j . Furthermore, when $T_z = T_p$, x_1 is initialized to zero; hence, it remains equal to zero for the whole simulation. Replacing $x_1 = 0$ yields $x_j = G x_i$ as expected.

tf2p2z

Transfer function between input and output: two real zeros and two real poles

$$x_i \rightarrow \boxed{G \frac{1+n_1s+n_2s^2}{1+d_1s+d_2s^2}} \rightarrow x_j$$

Syntax :

& tf2p2z

name of variable x_i

name of variable x_j

data name, parameter name or math expression for G

data name, parameter name or math expression for n_1

data name, parameter name or math expression for n_2

data name, parameter name or math expression for d_1

data name, parameter name or math expression for d_2

Internal states : x_1 and x_2

Discrete variables : none

Equations : correspond to a second-order state space model with controllable canonical form:

$$\dot{x}_1 = x_2 \quad (26.1)$$

$$d_2 \dot{x}_2 = -x_1 - d_1 x_2 + d_2 x_i \quad (26.2)$$

$$0 = G(d_2 - n_2)x_1 + G(n_1 d_2 - d_1 n_2)x_2 + G n_2 d_2 x_i - d_2^2 x_j \quad (26.3)$$

Initialization of internal states : $x_2 = 0$ and $x_1 = d_2 x_i$

Exception. If a small value is specified for d_2 , namely if $d_2 < 0.005$, both d_2 and n_2 are set to zero and the transfer function becomes:

$$G \frac{1 + n_1 s}{1 + d_1 s}$$

as considered in the **tf1p1z** block.

If, in addition, a small value is specified for d_1 , namely if $d_1 < 0.005$, both d_1 and n_1 are set to zero and the block behaves as a simple gain:

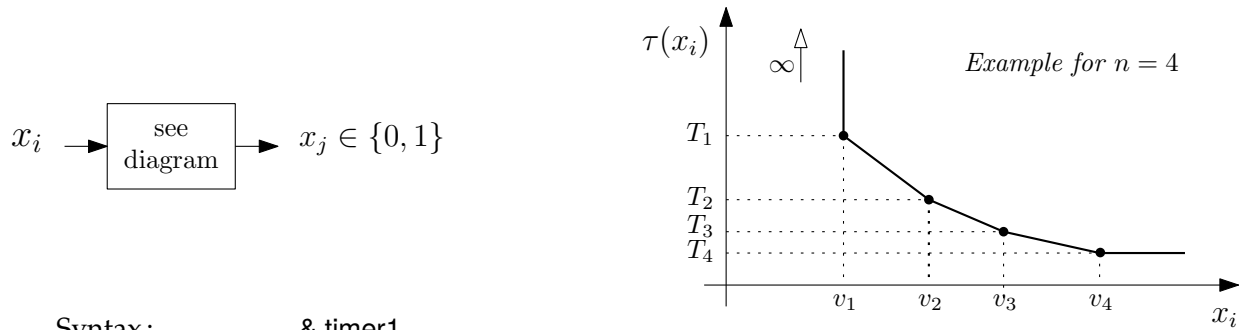
$$x_j = G x_i$$

timer*

Timer with varying delay. The latter is a piecewise linear function of the monitored variable

If x_i is smaller than a threshold v_1 , the output x_j is equal to zero. Otherwise, x_j changes from zero to one at time $t^ + \tau(x_i)$ where t^* is the time at which the input x_i became larger than v_1 and the delay $\tau(x_i)$ varies with x_i according to a piecewise linear characteristic involving n points (see diagram below).*

Separate blocks exist for $n = 1, 2, 3, 4$ and 5.



Syntax :

& timer1

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

& timer2

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

data name, parameter name or math expression for v_2

data name, parameter name or math expression for T_2

& timer4

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

data name, parameter name or math expression for v_2

data name, parameter name or math expression for T_2

data name, parameter name or math expression for v_3

data name, parameter name or math expression for T_3

data name, parameter name or math expression for v_4

data name, parameter name or math expression for T_4

& timer3

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

data name, parameter name or math expression for v_2

data name, parameter name or math expression for T_2

data name, parameter name or math expression for v_3

data name, parameter name or math expression for T_3

& timer5

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

data name, parameter name or math expression for v_2

data name, parameter name or math expression for T_2

data name, parameter name or math expression for v_3

data name, parameter name or math expression for T_3

data name, parameter name or math expression for v_4

data name, parameter name or math expression for T_4

data name, parameter name or math expression for v_5

data name, parameter name or math expression for T_5

Internal states : x_1

Discrete variables : $z \in \{-1, 0, 1\}$

Equations :

$$0 = \begin{cases} x_j & \text{if } z \in \{-1, 0\} \\ x_j - 1 & \text{if } z = 1 \end{cases}$$

$$\begin{cases} 0 &= x_1 & \text{if } z = -1 \\ x_1 &= 1 & \text{if } z = 0 \\ x_1 &= 0 & \text{if } z = 1 \end{cases}$$

Discrete transitions :

```

if  $z = -1$  then
  if  $x_i \geq v_1$  then
     $z \leftarrow 0$ 
  end if
else
  if  $x_i < v_1$  then
     $z \leftarrow -1$ 
  end if
end if
if  $z = 0$  then
  if  $x_1 \geq \tau(x_i)$  then
     $z \leftarrow 1$ 
  end if
end if

```

Initialization of internal states : $x_1 \leftarrow 0$

Initialization of discrete variables : $z \leftarrow -1$

The v_i values must be increasing, but two consecutive values may be equal, i.e. $v_1 \leq v_2 \leq v_3 \leq \dots \leq v_{n-1} \leq v_n$.

The piecewise linear characteristic is typically used to approximate an inverse-time characteristic, in which case the T values are decreasing, i.e. $T_1 \geq T_2 \geq T_3 \geq \dots \geq T_{n-1} \geq T_n$. Nevertheless, non decreasing values are also allowed.

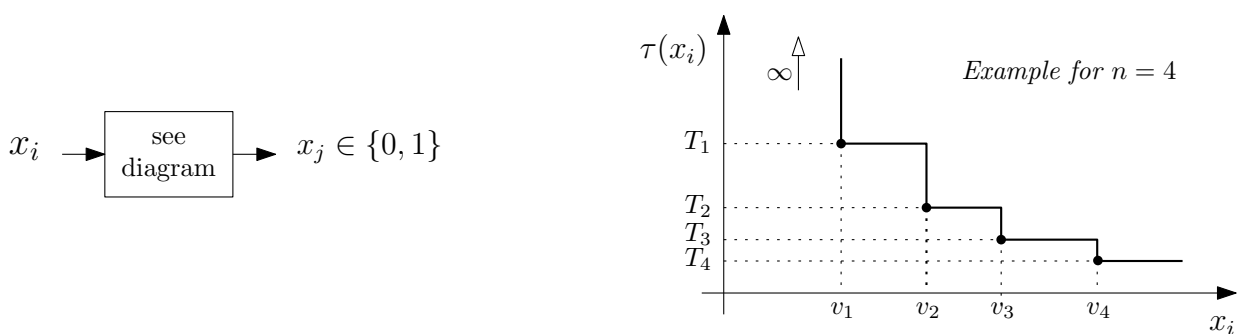
If the initial value of x_i is larger than v_1 , x_j will change to one after the time $\tau(x_i)$, unless x_i decreases below v_1 before the delay τ is elapsed.

timersc*

Timer with varying delay. The latter is a staircase function of the monitored variable

If x_i is smaller than a threshold v_1 , the output x_j is equal to zero. Otherwise, x_j changes from zero to one at time $t^ + \tau(x_i)$ where t^* is the time at which the input x_i became larger than v_1 and the delay $\tau(x_i)$ varies with x_i according to a staircase characteristic involving n points (see diagram below).*

Separate blocks exist for $n = 1, 2, 3, 4, 5$ and 6.



Syntax :

& timersc1

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

& timersc2

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

data name, parameter name or math expression for v_2

data name, parameter name or math expression for T_2

& timersc4

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

data name, parameter name or math expression for v_2

data name, parameter name or math expression for T_2

data name, parameter name or math expression for v_3

data name, parameter name or math expression for T_3

data name, parameter name or math expression for v_4

data name, parameter name or math expression for T_4

& timersc3

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

data name, parameter name or math expression for v_2

data name, parameter name or math expression for T_2

data name, parameter name or math expression for v_3

data name, parameter name or math expression for T_3

& timersc5

name of variable x_i

name of variable x_j

data name, parameter name or math expression for v_1

data name, parameter name or math expression for T_1

data name, parameter name or math expression for v_2

data name, parameter name or math expression for T_2

data name, parameter name or math expression for v_3

data name, parameter name or math expression for T_3

data name, parameter name or math expression for v_4

data name, parameter name or math expression for T_4

data name, parameter name or math expression for v_5

data name, parameter name or math expression for T_5

& timersc6
 name of variable x_i
 name of variable x_j
 data name, parameter name or math expression for v_1
 data name, parameter name or math expression for T_1
 data name, parameter name or math expression for v_2
 data name, parameter name or math expression for T_2
 data name, parameter name or math expression for v_3
 data name, parameter name or math expression for T_3
 data name, parameter name or math expression for v_4
 data name, parameter name or math expression for T_4
 data name, parameter name or math expression for v_5
 data name, parameter name or math expression for T_5
 data name, parameter name or math expression for v_6
 data name, parameter name or math expression for T_6

Internal states : x_1

Discrete variables : $z \in \{-1, 0, 1\}$

Equations :

$$0 = \begin{cases} x_j & \text{if } z \in \{-1, 0\} \\ x_j - 1 & \text{if } z = 1 \end{cases}$$

$$\begin{cases} 0 & = x_1 & \text{if } z = -1 \\ \dot{x}_1 & = 1 & \text{if } z = 0 \\ \dot{x}_1 & = 0 & \text{if } z = 1 \end{cases}$$

Discrete transitions :

```

if  $z = -1$  then
  if  $x_i \geq v_1$  then
     $z \leftarrow 0$ 
  end if
else
  if  $x_i < v_1$  then
     $z \leftarrow -1$ 
  end if
end if
if  $z = 0$  then
  if  $x_1 \geq \tau(x_i)$  then
     $z \leftarrow 1$ 
  end if
end if

```

Initialization of internal states : $x_1 \leftarrow 0$

Initialization of discrete variables : $z \leftarrow -1$

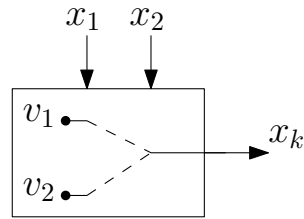
The v_i values must be increasing, but two consecutive values may be equal, i.e. $v_1 \leq v_2 \leq v_3 \leq \dots \leq v_{n-1} \leq v_n$.

The staircase characteristic is typically used to approximate an inverse-time characteristic, in which case the T values are decreasing, i.e. $T_1 \geq T_2 \geq T_3 \geq \dots \geq T_{n-1} \geq T_n$. Nevertheless, non decreasing values are also allowed.

If the initial value of x_i is larger than v_1 , x_j will change to one after the time $\tau(x_i)$, unless x_i decreases below v_1 before the delay τ is elapsed.

tsa

Two-state automaton with transitions based on signs of two inputs



The output state x_k can take two values: v_1 or v_2 . When $x_k = v_1$ the change to v_2 depends on the sign of x_1 ; when $x_k = v_2$ the change to v_1 depends on the sign of x_2 .

Syntax: & 2sa
 name of variable x_1
 name of variable x_2
 name of variable x_k
 data name, parameter name or math expression for v_1
 data name, parameter name or math expression for v_2

Internal states : none

Discrete variable : $z \in \{1, 2\}$ represents the state of the automaton

Equations :

$$0 = \begin{cases} x_k - v_1 & \text{if } z = 1 \\ x_k - v_2 & \text{if } z = 2 \end{cases}$$

Discrete transitions :

```

if  $z = 1$  and  $x_1 > 0$  then
   $z \leftarrow 2$ 
else if  $z = 2$  and  $x_2 > 0$  then
   $z \leftarrow 1$ 
end if
  
```

Initialization of discrete variable : the initial value is $z = 1$, i.e. $x_k = v_1$ is assumed initially.

26.4. Functions available in models

The functions documented in this section can be used in all mathematical expressions mentioned in Fig. 25.7.

The first two are general. The others are power system-oriented. Among them, some functions are restricted to some types of models.

```
double precision function equal(var1,var2)
```

```
double precision:: var1, var2
```

This function returns 1.d0 if var1 differs from var2 by less than 10^{-6} , and 0.d0 otherwise.

```
double precision function equalstr(var1,var2)
```

```
character:: var1, var2
```

This function returns 1.d0 if the non-blank parts of str1 and str2 are the same, and 0.d0 otherwise.

```
double precision ppower([vx],[vy],[ix],[iy])
```

Returns the active power injected into network

Can be used in models of type: inj

Computation :

$$P = v_x i_x + v_y i_y$$

```
double precision qpower([vx],[vy],[ix],[iy])
```

Returns the reactive power injected into network

Can be used in models of type: inj

Computation :

$$Q = v_y i_x - v_x i_y$$

```
double precision function vrectif([if],[vin},{kc})
```

```
double precision:: kc
```

This function is used to model the voltage drop in rectifiers. It gives the output voltage vrectif as a function of the output current if and the input voltage vin. It appears in some IEEE standard excitation models with $v_{rectif} = f_{ex} E_{FD}$

Can be used in models of type: exc

Computation :

```
in = kc if / max(vin, 10-3)
if in ≤ 0 then
    vrectif = vin
else if in ≤ 0.433 then
    vrectif = vin - 0.577 kc if
else if in ≤ 0.75 then
    vrectif = √(0.75 vin2 - (kc if)2)
else if in ≤ 1.00 then
    vrectif = 1.732(vin - kc if)
else
    vrectif = 0
end if
```

```
double precision function vinrectif([if],[vrectif},{kc})
```

```
double precision:: kc
```

This function is the inverse of `vrectif`. It is aimed at being called at initialization. For given values the field current `if` and rectifier output voltage `vrectif` it returns the value of the rectifier input voltage `vinrectif`. This determination is iterative because the portion of the nonlinear input-output characteristic on which the rectifier is operating is not known beforehand. The function does not work with a zero rectifier output `vrectif` since, in this case, the input voltage is indeterminate.

Can be used in models of type: `exc`

Computation :

```

nbtries = 1
vinest = vrectif + 0.577 kc if
loop
  in = kc if / max(vinest, 1d - 03)
  if in ≤ 0 then
    vinrectif = vrectif
  else if in ≤ 0.433 then
    vinrectif = vrectif + 0.577 kc if
  else if in ≤ 0.75 then
    vinrectif = √(vrectif2 + (kc if)2) / 0.75
  else
    vinrectif = (vrectif / 1.732) + kc if
  end if
  if vinrectif = vinest or nbtries > 5 then
    exit loop
  end if
  nbtries = nbtries + 1
  vinest = vinrectif
end loop

```

```
double precision function vcomp([v],[p],[q],[Kv],[Rc],[Xc])
```

```
double precision:: Kv, Rc, Xc
```

This function returns the magnitude of a combination of the terminal voltage and the current of a synchronous machine. It appears in some IEEE standard excitation models

Can be used in models of type: `exc`

Computation :

$$\begin{aligned}
 V_{comp} &= |K_v \bar{V} + (R_c + jX_c)\bar{I}| = |K_v V + (R_c + jX_c)(i_P - ji_Q)| \\
 &= |K_v V + (R_c + jX_c)(\frac{P}{V} - j\frac{Q}{V})| \\
 &= \frac{1}{V} \sqrt{(K_v V^2 + R_c P + X_c Q)^2 + (X_c P - R_c Q)^2}
 \end{aligned}$$

```
double precision function satur([ve],[ve1],[se1],[ve2],[se2])
```

```
double precision:: ve1, se2, ve2, se2
```

This function returns the increment of field current needed to obtain a given output voltage, taking saturation into account. It appears in the model of some excitation systems.

Can be used in models of type: `exc`

Computation :

$$satur = m ve^n$$

where:

$$n = \frac{\log_{10}(se1/se2)}{\log_{10}(ve1/ve2)}$$
$$m = \frac{se1}{ve1^n}$$

Exception. The function returns *satur* = 0 if any of the following conditions holds true:

$$ve \leq 0 \quad \text{or} \quad ve1 \leq 0 \quad \text{or} \quad ve2 \leq 0 \quad \text{or} \quad ve1 = ve2 \quad \text{or} \quad se1 = 0 \quad \text{or} \quad se2 = 0$$

27

Worked CODEGEN examples

This page provides complete, annotated CODEGEN model files for each of the four model categories. These are real models from the RAMSES model library and can serve as templates for developing your own models.

For the block reference, see CODEGEN Blocks. For the model framework (states, equations, initialization), see User-Defined Models.

27.1. Exciter Model: exc_ST1A

The IEEE ST1A is a bus-fed static excitation system. This model demonstrates the typical structure of an exc-type CODEGEN file: voltage compensation, AVR with lead-lag stages, field current limiting, UEL/OEL gating, and variable-limit output clamp.

Source: IEEE_models/exc_ST1A.txt (from the RAMSES model library)

Full Model File

```
exc
ST1A

%data

Kv
Rc
Xc
TR
UEL
VIMIN
VIMAX
VUEL    ERROR - Vuel is the UEL signal not a parameter!
TC
TB
TC1
TB1
KA
TA
VAMIN
VAMAX
VRMIN
```

```

VRMAX
KC
KF
TF
KLR
ILR

%parameters

VREF      = vcomp([v],[p],[q],[Kv],[Rc],[Xc])+([vf]/{KA})

%states

Vc1       = vcomp([v],[p],[q],[Kv],[Rc],[Xc])
Vc        = vcomp([v],[p],[q],[Kv],[Rc],[Xc])
deltaV    = ([vf]/{KA})
V1        = [vf]/{KA}
V2        = [vf]/{KA}
V3        = [vf]/{KA}
V4        = [vf]/{KA}
VA        = [vf]
VLR       = {KLR}*([if]-{ILR})
VLRlim    = 0.
V5        = [vf]
V6        = [vf]
V7        = [vf]
uplim     = [v]*{VRMAX}-{KC}*[if]
lolim     = [v]*{VRMIN}
VF        = 0.

VOEL      = 99999.

%observables

VA
Vc1
Vc
V1
V2
V3
V4
vf
p
q
v
VLRlim
deltaV

VREF
VOEL

%models
& algeq          fictitious over excitation limit
[VOEL]-99999.
& algeq          line drop compensation
[Vc1]-vcomp([v],[p],[q],[Kv],[Rc],[Xc])

```



```

& tf1p          voltage measurement time constant
Vc1
Vc
1.d0
{TR}
& algeq          summation point of AVR
[deltaV]-{VREF}+[Vc]-equal({UEL},1.d0)*{VUEL}+[VF]
& lim           limiter on deltaV
deltaV
V1
{VIMIN}
{VIMAX}
& max1v1c       HV gate for VUEL if UEL=2
V1
V2
{VUEL}*equal({UEL},2.d0) - 9999.*(1-equal({UEL},2.d0))
& tf1p1z        first lead-lag
V2
V3
1.
{TC}
{TB}
& tf1p1z        second lead-lag
V3
V4
1.
{TC1}
{TB1}
& tf1plim       amplifier
V4
VA
{KA}
{TA}
{VAMIN}
{VAMAX}
& algeq          immediate field current limiting signal
[VLR]-{KLR}*([if]-{ILR})
& lim           ... lower limited to zero
VLR
VLRlim
0.
99999.
& algeq          ... and subtracted from main signal
[V5]-[VA]+[VLRlim]
& max1v1c       HV gate for VUEL if UEL=3
V5
V6
{VUEL}*equal({UEL},3.d0) - 9999.*(1-equal({UEL},3.d0))
& min2v         LV gate for VOEL
V6
VOEL
V7
& algeq          variable lower limit of final limiter
[v]*{VRMIN}-[lolim]
& algeq          variable upper limit of final limiter
[v]*{VRMAX}-{KC}*[if]-[uplim]

```

```

& limvb                final limiter
V7
lolim
uplim
vf
& tfder1p              derivative feedback
V7
VF
{KF}/{TF}
{TF}

```

Structure Guide

27.1.1. File Header

```

exc          <- model type (exciter)
ST1A         <- model name (used in the dynamic data file as EXC ST1A)

```

27.1.2. %data Section

Lists the parameters that the user provides in the dynamic data file. Each name becomes a {name} reference in expressions. The order here defines the order of values in the data file.

27.1.3. %parameters Section

Operating-point parameters computed at initialization. The number of parameters must equal the number of output states (here 1: vf). VREF is the voltage setpoint, computed from the initial operating point so that the steady-state field voltage is matched.

27.1.4. %states Section

All internal and output states with their initial values. The initialization expressions use system variables ([v], [p], [q], [if], [vf]) and data values ({KA}, etc.). The last state vf is the output state (field voltage).

27.1.5. %observables Section

States that can be recorded during simulation.

27.1.6. %models Section: Signal Flow

1. algeq, OEL placeholder (set to large value, overridden by OEL model if present)
2. algeq, line drop compensation: $V_{c1} = \text{vcomp}(V, P, Q, K_v, R_c, X_c)$
3. tf1p, voltage measurement filter with time constant T_R
4. algeq, AVR summation: $\Delta V = V_{REF} - V_c - V_{UEL} + V_F$
5. lim, error limiter $[V_{IMIN}, V_{IMAX}]$
6. max1v1c, HV gate for UEL (active when UEL=2)
7. tf1p1z, first lead-lag: $(1 + sT_C)/(1 + sT_B)$
8. tf1p1z, second lead-lag: $(1 + sT_{C1})/(1 + sT_{B1})$
9. tf1plim, amplifier with gain K_A , time constant T_A , limits $[V_{AMIN}, V_{AMAX}]$
10. algeq + lim, instantaneous field current limiter $V_{LR} = K_{LR}(I_f - I_{LR})$, lower-limited to 0
11. algeq, subtract limiter signal from main path
12. max1v1c, HV gate for UEL (active when UEL=3)
13. min2v, LV gate for OEL
14. algeq $\times 2$, compute variable limits: $V_{RMIN} \cdot V_t$ and $V_{RMAX} \cdot V_t - K_C I_f$
15. limvb, bus-fed variable-limit output clamp $\rightarrow$ vf
16. tfder1p, derivative feedback: $sK_F/T_F/(1 + sT_F)$

27.2. Governor Model: `tor_hygov`

A hydraulic turbine-governor model with speed droop, PI controller, rate limiter, gate servo, and non-linear turbine (head-flow-power). This demonstrates a `tor`-type model with both differential and algebraic equations.

Source: `IEEE_models/tor_hygov.txt` (from the RAMSES model library)

Full Model File

```

tor
hygov

%data

R
r
Tr
Tf
Tg
VELM
Gmax
Gmin
Tw
At
Dturb
Qnl

%parameters

Po = ([p]/{At})+{Qnl}    ! power setpoint

%states

Q = {Po}    ! water flow
dH = 0.    ! deviation of head from 1.
g = {Po}    ! gate opening
c = {Po}    ! desired gate opening
x5 = {Po}    ! output of PI controller
x4 = {Po}    ! output of integrator of PI controller
x3 = 0.    ! output of rate limiter = input of integrator of PI controller
x2 = 0.    ! input of rate limiter of PI controller
e = 0.    ! output of Tf time constant block
x1 = 0.    ! input of Tf time constant block
Pm = [p]    ! mechanical power

%observables

Pm
Q
dH
g

%models

& algeq    ! input of Tf time constant block

```

```

1.-[omega]-{R}*([c]-{Po}) - [x1]
& tflp      ! Tf time constant
x1
e
1.
{Tf}
& algeq      ! input of rate limiter
[e]/({r}*{Tr}) - [x2]
& lim       ! rate limiter
x2
x3
-{VELM}
{VELM}
& inlim      ! integrator of PI controller
x3
x4
1.
{Gmin}
{Gmax}
& algeq      ! output of PI controller
[x4]+([e]/{r}) - [x5]
& lim       ! gate limiter
x5
c
{Gmin}
{Gmax}
& tflp      ! speed governor : time constant
c
g
1.
{Tg}

& algeq                      ! hydro turbine: head as function of water flow and gate
↪ opening
1. -([Q]/[g])**2 -[dH]
& int                      ! hydro turbine: water column inertia
dH
Q
{Tw}
& algeq                      ! hydro turbine: mechanical torque
{At}* (1.-[dH])*([Q]-{Qnl})-{Dturb}* (1.-[omega])*[g] - [tm]*[omega]
& algeq
[tm]*[omega] - [Pm]

```

Structure Guide

27.2.1. File Header

```

tor          <- model type (torque/governor)
hygov        <- model name (used as `TOR HYGOV` in the data file; dispatcher case is
↪ `tor_HYGOV`)

```

27.2.2. Input/Output States

- **Input:** [omega] (rotor speed), [p] (electrical power)
- **Output:** [tm] (mechanical torque)

27.2.3. Signal Flow

Governor section: 1. Speed error with droop: $x_1 = 1 - \omega - R \cdot (g - P_0)$ 2. Filter: tf1p with time constant T_f 3. PI controller path: rate limiter (lim) $\rightarrow$ integrator (inlim) with gate limits $\rightarrow$ proportional bypass 4. Gate servo: tf1p with time constant T_g

Turbine section: 5. Head-flow relation: $\Delta H = 1 - (Q/g)^2$ (non-linear algebraic) 6. Water column inertia: $T_w \dot{Q} = \Delta H$ (int block) 7. Mechanical torque: $T_m \omega = A_t(1 - \Delta H)(Q - Q_{nl}) - D_{turb}(1 - \omega)g$ 8. Mechanical power: $P_m = T_m \omega$

27.2.4. Key Parameters

Parameter	Description
R	Permanent droop
r	Temporary droop
Tr	Reset time
Tf	Filter time constant
Tg	Gate servo time constant
VELM	Gate velocity limit
Gmax/Gmin	Gate position limits
Tw	Water starting time
At	Turbine gain
Dturb	Turbine damping
Qnl	No-load flow

27.3. Injector Model: inj_vfd_load

A voltage- and frequency-dependent load model with exponential characteristics and low-voltage protection (constant admittance below a threshold). This is the most widely used load model in RAMSES. It demonstrates an inj-type model with current injection equations.

Source: custom_models/inj_vfd_load.txt (from the RAMSES model library)

Full Model File

```
inj
vfd_load

%data

Dp      ! sensitivity of active power to frequency
a1      ! proportion of type 1 in active power
alpha1  ! exponent of type 1 in active power
a2      ! proportion of type 2 in active power
alpha2  ! exponent of type 2 in active power
alpha3  ! exponent of type 3 in active power (proportion = 1-a1-a2)
Dq      ! sensitivity of reactive power to frequency
b1      ! proportion of type 1 in reactive power
beta1   ! exponent of type 1 in reactive power
b2      ! proportion of type 2 in reactive power
beta2   ! exponent of type 2 in reactive power
beta3   ! exponent of type 3 in reactive power (proportion = 1-b1-b2)
Vinit   ! initial voltage at equivalent distribution bus
Vlow    ! voltage below which the load behaves as constant admittance
foption ! set to 1 to use bus frequency; otherwise COI speed
Tmes    ! measurement time constant for bus frequency estimation
```

```

%parameters

Vo = equal({Vinit},0.d0)*dsqrt([vx]**2+[vy]**2) + {Vinit}
r = dsqrt([vx]**2+[vy]**2)/{Vo}
denomP = {a1}*{Vo}**{alpha1}+{a2}*{Vo}**{alpha2}+(1.d0-{a1}-{a2})*{Vo}**{alpha3}
denomQ = {b1}*{Vo}**{beta1}+{b2}*{Vo}**{beta2}+(1.d0-{b1}-{b2})*{Vo}**{beta3}
Po = -[vx]*[ix]-[vy]*[iy]
Qo = [vx]*[iy]-[vy]*[ix]
Vmin = max(1.d-01,{Vlow})
Geq = {Po}*({a1}*{Vmin}**{alpha1}+{a2}*{Vmin}**{alpha2}
        +(1.d0-{a1}-{a2})*{Vmin}**{alpha3})/({denomP}*{Vmin}**2)
Beq = -{Qo}*({b1}*{Vmin}**{beta1}+{b2}*{Vmin}**{beta2}
        +(1.d0-{b1}-{b2})*{Vmin}**{beta3})/({denomQ}*{Vmin}**2)

%states

Vreal = {Vo}      ! voltage at equivalent distribution bus
V = {Vo}          ! Vreal slightly delayed
P = {Po}          ! active power
Q = {Qo}          ! reactive power
fbus = 1.d0       ! bus frequency
df = 0.d0         ! frequency deviation
u = 1             ! model switch: =1 if V > Vmin, =0 otherwise

%observables

P
Q
df
u

%models

& f_inj
fbus
{Tmes}
& algeq
equal(1.d0,{foption})*([fbus]-1.d0)
    + (1.d0-equal(1.d0,{foption}))*(omega-1.d0) - [df]
& algeq      ! voltage at distribution bus = network voltage / trfo ratio
dsqrt([vx]**2+[vy]**2)/{r}-[Vreal]
& tf1p      ! small measurement time constant
Vreal
V
1.
0.003
& pwlin4      ! determine if u=1 or u=0
V
u
-1.d0
0.d0
{Vmin}
0.d0
{Vmin}
1.d0

```

```

2.d0
1.d0
& algeq      ! active power function of V, u and f
[u]*({Po}*(1.d0+{Dp}*[df])
  *({a1}*[V]**{alpha1}+{a2}*[V]**{alpha2}
    +(1.d0-{a1}-{a2})*[V]**{alpha3})/{denomP})
  + (1.d0-[u])*{Geq}*[V]**2 -[P]
& algeq      ! reactive power function of V, u and f
[u]*({Qo}*(1.d0+{Dq}*[df])
  *({b1}*[V]**{beta1}+{b2}*[V]**{beta2}
    +(1.d0-{b1}-{b2})*[V]**{beta3})/{denomQ})
  - (1.d0-[u])*{Beq}*[V]**2 -[Q]
& algeq      ! ix component of injected current
(-[P]*[vx] - [Q]*[vy])/max([V]**2,1.d-02) -[ix]
& algeq      ! iy component of injected current
([Q]*[vx] - [P]*[vy])/max([V]**2,1.d-02) -[iy]

```

Structure Guide

27.3.1. File Header

```

inj          <- model type (injector)
vfd_load     <- model name (used as INJEC vfd_load in the data file)

```

27.3.2. Input/Output States

- **Input:** [vx], [vy] (bus voltage components), [omega] (COI frequency)
- **Output:** [ix], [iy] (injected current components)

27.3.3. Model Logic

Frequency estimation: 1. f_{inj} , estimates bus frequency from voltage phasor derivatives 2. $algeq$, selects bus frequency or COI frequency based on f_{option}

Voltage processing: 3. $algeq$, compute distribution bus voltage: $V_{real} = \sqrt{v_x^2 + v_y^2}/r$ 4. $\tau f1p$, small delay (3 ms) for numerical stability 5. $pwlin4$, switch: $u = 1$ if $V > V_{min}$, $u = 0$ otherwise

Power computation: 6. Active power: $P = u \cdot P_0(1 + D_p \Delta f) \frac{a_1 V^{\alpha_1} + a_2 V^{\alpha_2} + (1 - a_1 - a_2) V^{\alpha_3}}{\text{denom}_P} + (1 - u) G_{eq} V^2$ 7. Reactive power: analogous with b/β coefficients

Current injection: 8–9. Convert P, Q to rectangular currents: $i_x = (-PV_x - QV_y)/V^2$, $i_y = (QV_x - PV_y)/V^2$

27.3.4. Key Design Patterns

- **Low-voltage protection:** below V_{min} , the load switches to constant admittance (G_{eq}, B_{eq}) to prevent numerical issues
- **Distribution transformer:** the ratio r models the voltage step-down to the load bus
- **Frequency dependence:** D_p, D_q scale power with frequency deviation

27.4. Two-Port Model: twop_HVDC_LCC

A Line-Commutated Converter (LCC) HVDC link model connecting two AC buses through a DC line. This demonstrates a `twop`-type model with complex AC/DC interaction, multiple control modes (current control, voltage control, gamma control), and non-linear commutation equations.

Source: custom_models/twop_HVDC_LCC.txt (from the RAMSES model library)
Full Model File

```

twop
HVDC_LCC

%data

p          ! number of poles (1 or 2)
rec_ctl    ! rectifier control: 1=current, 2=power, -1=alpha min, 3=blocked
inv_ctl    ! inverter control: 1=voltage, 2=gamma, -2=gamma min, -1=reduced current,
↳ 3=blocked
Br         ! number of bridges in series in rectifier
Bi         ! number of bridges in series in inverter
Er         ! phase-to-phase open-circuit voltage (kV) on DC side of rectifier
↳ transformer
Ei         ! phase-to-phase open-circuit voltage (kV) on DC side of inverter
↳ transformer
Rcr        ! rectifier commutating resistance per bridge (ohm)
Xcr        ! rectifier commutating reactance per bridge (ohm)
Rci        ! inverter commutating resistance per bridge (ohm)
Xci        ! inverter commutating reactance per bridge (ohm)
RL         ! DC line series resistance (ohm)
Rcomp      ! compounding resistance for inverter voltage control (ohm)
Tp         ! time constant of power control (s)
Idset      ! initial DC current setpoint (kA)
nr         ! initial rectifier transformer ratio (pu/pu)
ni         ! initial inverter transformer ratio (pu/pu)

%parameters

Pset = -([vx1]*[ix1]+[vy1]*[iy1])*sbase1
Vdr0 = {Pset}/{p}*{Idset}
alpha0 = dacos( ([nr]/(1.3504745*{Er}*dsqrt([vx1]**2+[vy1]**2)))
               * (({Vdr0}/{Br})+(0.9549297*{Xcr}+2*{Rcr})*{Idset}) )
mur0 = dacos( dcos({alpha0})
              - 1.4142136*{Xcr}*{nr}*{Idset}
              /(dsqrt([vx1]**2+[vy1]**2)*{Er})) - {alpha0}
Qhvdc1 = {Pset}*(2.d0*{mur0}+dsin(2*{alpha0})-dsin(2d0*({mur0}+{alpha0})))
          /(dcos(2.d0*{alpha0})-dcos(2d0*({mur0}+{alpha0})))
B1 = ({Qhvdc1}/sbase1-[vx1]*[iy1]+[vy1]*[ix1])/([vx1]**2+[vy1]**2)
Vdi0 = {Vdr0}-{RL}*{Idset}
gamma0 = dacos( ({ni}/(1.3504745*{Ei}*dsqrt([vx2]**2+[vy2]**2)))
               * (({Vdi0}/{Bi})+(0.9549297*{Xci}+2*{Rci})*{Idset}) )
mui0 = dacos( dcos({gamma0})
              - 1.4142136*{Xci}*{ni}*{Idset}
              /(dsqrt([vx2]**2+[vy2]**2)*{Ei})) - {gamma0}
P2in = {p}*{Vdi0}*{Idset}
G2 = ({P2in}/sbase2-[vx2]*[ix2]-[vy2]*[iy2])
     /(dsqrt([vx2]**2+[vy2]**2))
Qhvdc2 = {P2in}*(2.d0*{mui0}+dsin(2*{gamma0})-dsin(2d0*({mui0}+{gamma0})))
          /(dcos(2.d0*{gamma0})-dcos(2d0*({mui0}+{gamma0})))
B2 = ({Qhvdc2}/sbase2-[vx2]*[iy2]+[vy2]*[ix2])/([vx2]**2+[vy2]**2)
Vset = {Vdi0}+{Rcomp}*{Idset}
alphamin = 0.d0
Idred = 0.d0

```



```

gammamin = 0.d0
Td = 0.02

%states

Vdr = {Vdr0}          ! rectifier DC voltage (kV)
alpha = {alpha0}      ! firing angle (rad)
mur = {mur0}          ! overlap angle (rad)
Id = {Idset}          ! DC current (kA)
Vdi = {Vdi0}          ! inverter DC voltage (kV)
gamma = {gamma0}      ! extinction angle (rad)
mui = {mui0}          ! inverter overlap angle (rad)
Ides = {Idset}        ! desired DC current (kA)
Iord = {Idset}        ! DC current order (kA)
P1 = (-[vx1]*[ix1]-[vy1]*[iy1])*sbase1
Q1 = ([vx1]*[iy1]-[vy1]*[ix1])*sbase1
P2 = ([vx2]*[ix2]+[vy2]*[iy2])*sbase2
Q2 = ([vx2]*[iy2]-[vy2]*[ix2])*sbase2
V1 = dsqrt([vx1]**2+[vy1]**2)
V1d = dsqrt([vx1]**2+[vy1]**2)
V2 = dsqrt([vx2]**2+[vy2]**2)
V2d = dsqrt([vx2]**2+[vy2]**2)
alphad = {alpha0}*57.2957795
gammad = {gamma0}*57.2957795

%observables

Vdr
alphad
mur
Vdi
gammad
mui
Ides
Iord
Id
P1
Q1
P2
Q2
Pset

%models

& algeq          ! Vdr: rectifier DC voltage equation
[Vdr] - {Br}*( 1.3504745*[V1d]*{Er}*dcos([alpha])/[nr]
- (0.9549297*[Xcr]+2*[Rcr])*[Id] )
& algeq          ! overlap angle at rectifier
dcos([alpha]+[mur]) - dcos([alpha])
+ 1.4142136*[Xcr]*[nr]*[Id]/(max([V1d],1.d-03)*{Er})
& algeq          ! ix1: rectifier AC current (x-component)
{B1}*[vy1]-([vx1]+[vy1]*(2*[mur]+dsin(2*[alpha])
-dsin(2*([mur]+[alpha])))/max((dcos(2*[alpha])
-dcos(2*([mur]+[alpha]))),1d-03))
*[p]*[Vdr]*[Id]/(sbase1*max([V1d]**2,1d-06)) - [ix1]
& algeq          ! iy1: rectifier AC current (y-component)

```

```

-[B1]*[vx1]-([vy1]-[vx1]*(2*[mur]+dsin(2*[alpha])
  -dsin(2*([mur]+[alpha])))/max((dcos(2*[alpha])
  -dcos(2*([mur]+[alpha]))),1d-03))
  *[p]*[Vdr]*[Id]/(sbase1*max([V1d]**2,1d-06)) - [iy1]
& algeq          ! Vdi: inverter DC voltage equation
[Vdi] - {Bi}*( 1.3504745*[V2d]*{Ei}*dcos([gamma])/[ni]
  - (0.9549297*[Xci]+2*[Rci])*[Id] )
& algeq          ! overlap angle at inverter
dcos([gamma]+[mui]) - dcos([gamma])
  + 1.4142136*[Xci]*[ni]*[Id]/(max([V2d],1.d-03)*{Er})
& algeq          ! ix2: inverter AC current (x-component)
-[G2]*[vx2]+[B2]*[vy2]+([vx2]-[vy2]*(2*[mui]+dsin(2*[gamma])
  -dsin(2*([mui]+[gamma])))/max((dcos(2*[gamma])
  -dcos(2*([mui]+[gamma]))),1d-03))
  *[p]*[Vdi]*[Id]/(sbase2*max([V2d]**2,1d-06)) - [ix2]
& algeq          ! iy2: inverter AC current (y-component)
-[B2]*[vx2]-[G2]*[vy2]+([vy2]+[vx2]*(2*[mui]+dsin(2*[gamma])
  -dsin(2*([mui]+[gamma])))/max((dcos(2*[gamma])
  -dcos(2*([mui]+[gamma]))),1d-03))
  *[p]*[Vdi]*[Id]/(sbase2*max([V2d]**2,1d-06)) - [iy2]
& algeq          ! DC link: Kirchhoff voltage law
[Vdr]-[Vdi]-[RL]*[Id]

& algeq          ! desired DC current (depends on control mode)
(equal({rec_ctl},1.d0)*{Idset})
  +(equal({rec_ctl},2.d0)*({Pset}/max([Vdr],1d-02)))
  -(equal({rec_ctl},3.d0)*50.d0)
  -(equal({rec_ctl},-1.d0)*50.d0) -[Ides]
& tfplim          ! current order (filtered and limited)
Ides
Iord
1.d0
Tp
0.d0
9999.d0

& algeq          ! rectifier control equation
equal({rec_ctl},1.d0)*([Iord]-[Id])
  +equal({rec_ctl},2.d0)*([Iord]-[Id])
  +equal({rec_ctl},-1d0)*({alphamin}-[alpha])
  +equal({rec_ctl},3.d0)*[Id]
& algeq          ! inverter control equation
equal({inv_ctl},1.d0)*([Vdi]+[Rcomp]*[Id]-{Vset})
  +equal({inv_ctl},2.d0)*({gamma0}-[gamma])
  +equal({inv_ctl},-2d0)*({gammamin}-[gamma])
  +equal({inv_ctl},-1d0)*({Idred}-[Id])
  +equal({inv_ctl},3.d0)*(1.570796327-[gamma])

& algeq          ! AC voltage at bus 1
dsqrt([vx1]**2+[vy1]**2) - [V1]
& tf1p          ! delayed AC voltage at bus 1
V1
V1d
1.
Td
& algeq          ! AC voltage at bus 2

```

```

dsqrt([vx2]**2+[vy2]**2) - [V2]
& tflp          ! delayed AC voltage at bus 2
V2
V2d
1.
Td
& algeq          ! observable: P1
(-[vx1]*[ix1]-[vy1]*[iy1])*sbase1-[P1]
& algeq          ! observable: Q1
([vx1]*[iy1]-[vy1]*[ix1])*sbase1-[Q1]
& algeq          ! observable: P2
([vx2]*[ix2]+[vy2]*[iy2])*sbase2-[P2]
& algeq          ! observable: Q2
([vx2]*[iy2]-[vy2]*[ix2])*sbase2-[Q2]
& algeq          ! alpha in degrees
[alphad]-[alpha]*57.2957795
& algeq          ! gamma in degrees
[gammad]-[gamma]*57.2957795

```

Structure Guide

27.4.1. File Header

```

twop      <- model type (two-port)
HVDC_LCC  <- model name (used as `TWOP HVDC_LCC ...` in data file)

```

27.4.2. Input/Output States

- **Input:** [vx1], [vy1], [vx2], [vy2] (voltage at both buses)
- **Output:** [ix1], [iy1], [ix2], [iy2] (current injected at both buses)
- System variables: sbase1, sbase2 (base power at each bus)

27.4.3. Model Structure

DC equations (algebraic): - Rectifier DC voltage: $V_{dr} = B_r \left(\frac{3\sqrt{2}}{\pi} V_1 E_r \cos \alpha / n_r - (R_{cr} + \frac{3}{\pi} X_{cr}) I_d \right)$
 - Inverter DC voltage: analogous with γ , E_i , n_i - Overlap angles: from commutation equations -
 DC link: $V_{dr} - V_{di} = R_L I_d$

AC current injection: - Includes fundamental-frequency reactive power from commutation overlap - Shunt compensation (B_1 , B_2 , G_2) to match initial operating point

Control modes (selected by rec_ctl and inv_ctl using equal() function): - Rectifier: constant current, constant power, minimum alpha, blocked - Inverter: constant voltage, constant gamma, minimum gamma, reduced current, blocked

27.4.4. Key Design Patterns

- **equal()** function: used to switch between control modes without discrete variables, acts as a continuous selector
- **sbase1/sbase2:** system variables for per-unit conversion at each bus
- **Td delay:** small artificial time constant on terminal voltages for numerical robustness
- **Initialization:** complex parameter expressions compute initial angles, overlap, and compensation from the power flow solution

28

CODEGEN Studio

CODEGEN Studio is a browser-based visual editor for assembling User-Defined Models using the CODEGEN block library. Instead of writing DSL files by hand, you drag blocks onto a canvas, connect them, fill in metadata, and export a valid `.txt` file ready for the codegen binary.

28.1. Installation

pip install stepss-cg-studio, then run `cg-studio`. See Installing CODEGEN Studio for the Python version it needs, the address it serves, and why the **Run Codegen** button needs a codegen executable you supply yourself.

28.1.1. CLI options

```
cg-studio --port 9000      # use a different port
cg-studio --host 0.0.0.0   # allow network access
cg-studio --no-browser     # start server without opening browser
```

28.1.2. Install from source (development)

```
git clone https://github.com/SPS-L/stepss-cg-studio
cd stepss-cg-studio
pip install -e ".[dev]"
cg-studio
```

28.2. Interface Overview

The interface has three columns:

Area	Location	Purpose
Block palette	Left sidebar	Searchable catalogue of all CODEGEN blocks, grouped by category
Canvas	Center	Drag-and-drop block diagram with connection wires
Metadata tabs	Below canvas	Editable tables for %data, %parameters, %states, %observables
Block inspector	Right panel (top)	Edit output signal names and block arguments for the selected block
DSL preview	Right panel (bottom)	Live syntax-highlighted preview of the generated DSL code
Topbar	Top	Model type/name selectors, file operations, and settings

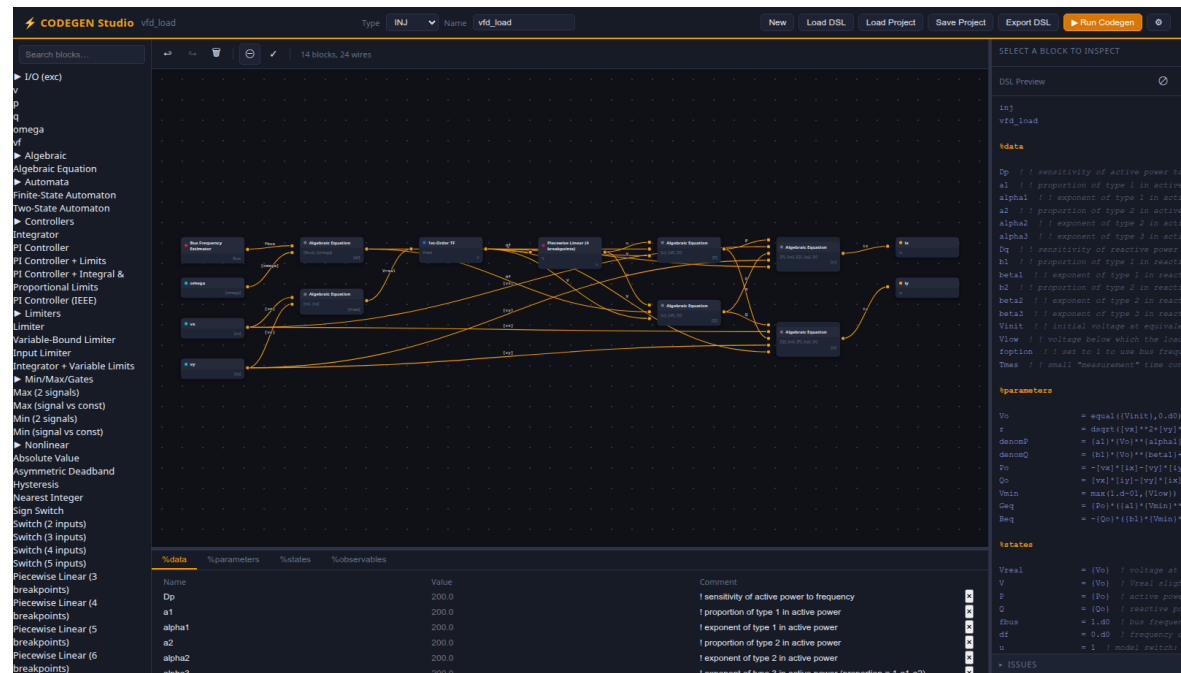


Figure 28.1: The CODEGEN Studio editor with the vfd_load injector loaded: the searchable block palette on the left, a canvas of 14 blocks joined by 24 wires in the centre with a first-order transfer function selected, the block inspector and a syntax-highlighted DSL preview on the right, and the %data metadata table along the bottom.

28.3. Creating a Model

28.3.1. 1. Choose the Model Type and Name

Use the **Type** dropdown in the topbar to select the model category:

Type	Description	Mandatory outputs	Colour theme
EXC	Excitation controller	vf	Blue
TOR	Torque controller	tm	Green
INJ	Current injector	ix, iy	Orange
TWOP	Two-port device	ix1, iy1, ix2, iy2	Purple

New in the topbar asks for the type before it clears the canvas, so a fresh model starts with the right RAMSES inputs and outputs already seeded.

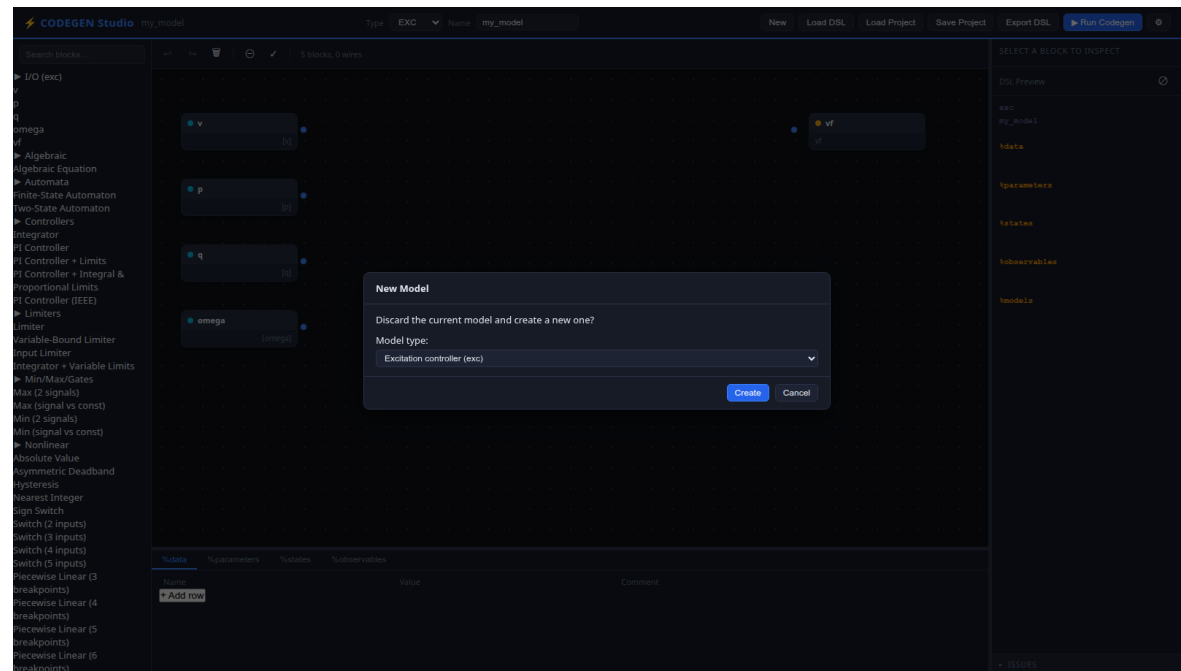


Figure 28.2: The New Model dialog over the editor.

The `exc` type also reserves `if` as a RAMSES name (usable in DSL expressions such as `[if]`) but it is not a palette input. See User-Defined Models for the complete list of reserved names.

28.3.2. 2. Add Blocks to the Canvas

The left palette lists all available blocks from the CODEGEN Blocks reference, organised by category (Transfer Functions, Limiters, Controllers, etc.).

1. **Search** by typing in the search box at the top of the palette
2. **Expand** a category by clicking its header
3. **Drag** a block from the palette onto the canvas

Each block appears as a node with input ports on the left and output ports on the right. A coloured dot indicates the block category.

28.3.3. 3. Connect Blocks

Click and drag from an **output port** (right side) of one block to an **input port** (left side) of another. A wire appears showing the signal flow. The connection assigns a signal name that links the two blocks in the generated DSL.

- Each input port accepts exactly one incoming wire
- Each output port can fan out to multiple inputs
- The status bar below the canvas toolbar shows the current block and wire count

Cycle detection

If your connections form a cycle, the status bar shows “**Cycle detected**” in red in place of the block and wire count. Feedback loops in control systems are valid in RAMSES, but the loop must be broken by declaring the fed-back signal as a `%state` with an explicit initial value. See User-Defined Models, States for details.

The warning is advisory: it does not stop you exporting or running the model. It also does not yet account for a loop that a `%state` already breaks, so a valid model with feedback can show it. Two of the editor’s own example models do, `exc_AC1A` among them. Read it as a prompt to check your states rather than as a verdict.

28.3.4. 4. Edit Block Properties

Click a block on the canvas to select it. The **Block Inspector** (right panel) shows:

- **Output signal names:** one editable field per output port. The default auto-generated name (e.g., `tf11`, `int2`) can be renamed to any valid identifier. This is the state name that appears in the DSL.
- **Arguments:** block-specific parameters such as gain K , time constant T , or limiter bounds. Values can be:
 - A numeric literal: `1.0`
 - A data reference: `{KA}` (refers to an entry in `%data`)
 - A Fortran expression: `{1/{omegac}}`

The `algeq` (algebraic equation) block has a special **Expression** field for the equation set to zero.

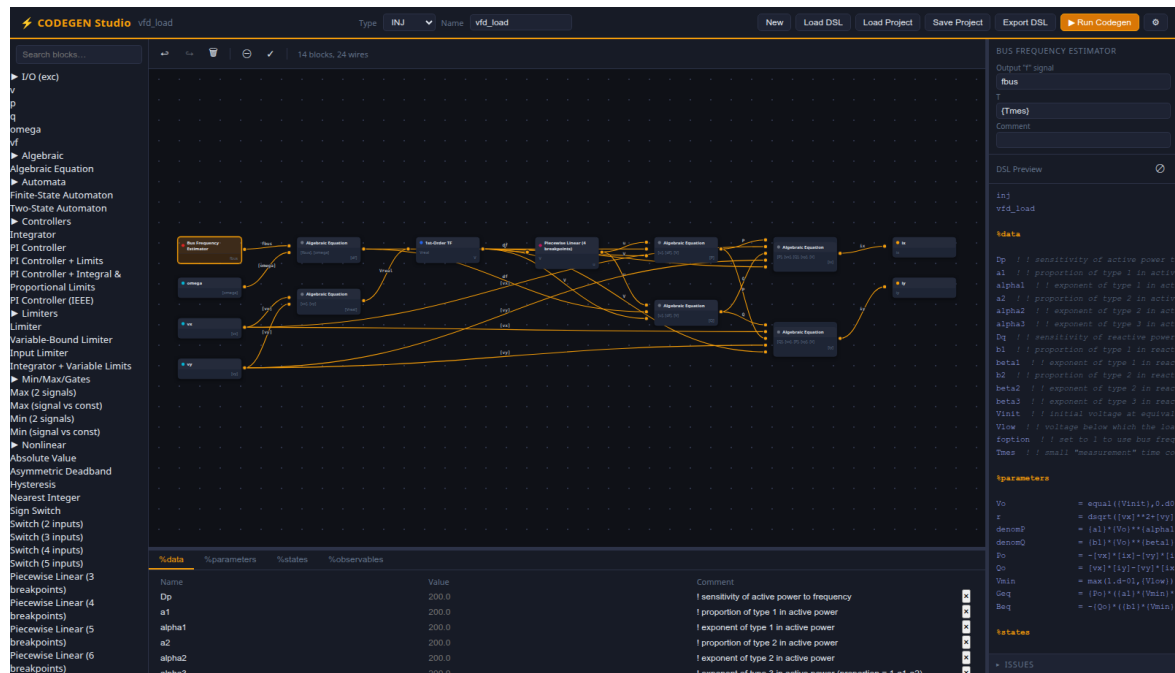


Figure 28.3: The CODEGEN Studio editor with a bus frequency estimator block selected on the canvas.

28.3.5. 5. Fill in the Metadata Tables

The four tabs below the canvas correspond to the four metadata sections of a CODEGEN DSL file:

%data

Purpose: Declare named constants whose values are read from the RAMSES data record at runtime.

Column	Description	Example
Name	Identifier (max 20 chars)	KA
Value	Optional default	200.0
Comment	Annotation	AVR gain

In block arguments and expressions, reference data values with braces: {KA}.

%parameters

Purpose: Define derived quantities computed once at initialisation from data and initial state values.

Column	Description	Example
Name	Identifier	Vo
Expr	Fortran expression	[vf]/{KE}+[vf]/{KA}
Comment	Annotation	initial voltage

Use [name] to reference state/input variables and {name} to reference data constants. Multi-line expressions are supported with the & continuation marker.

%states

Purpose: Declare the internal state variables of the model and their initial-condition expressions.

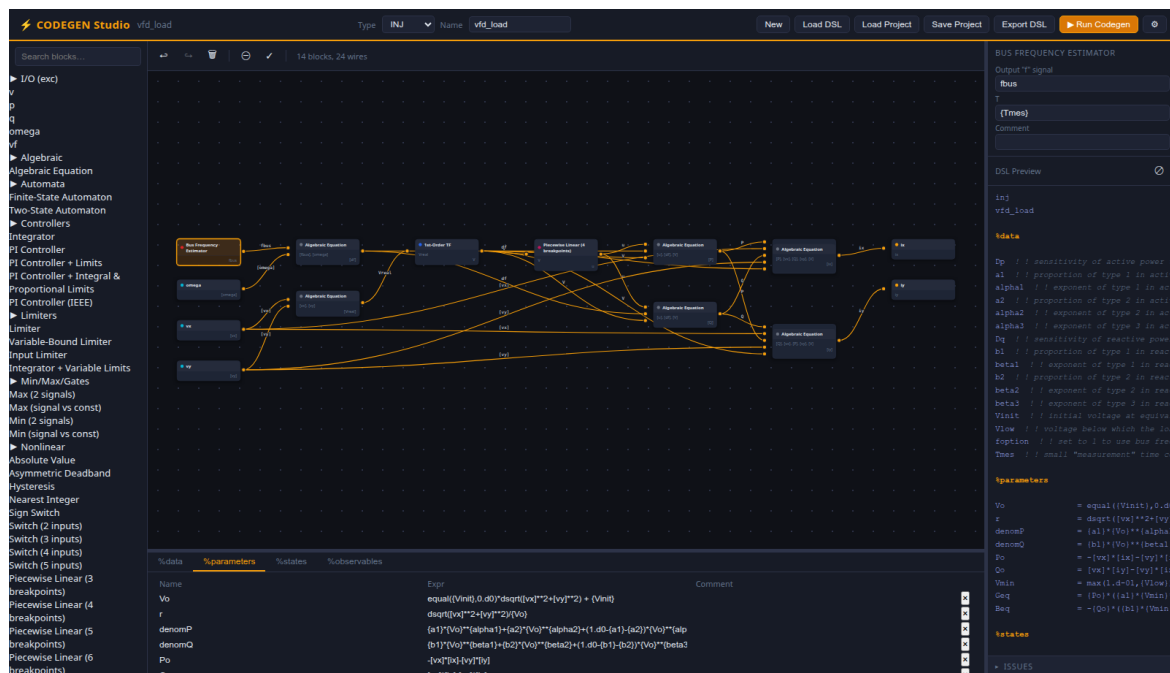


Figure 28.4: The %parameters tab of the metadata panel below the canvas, on the vfd_load injector.

Column	Description	Example
Name	State identifier	avr1
Init	Initial value expression	vfd/{KE}
Comment	Annotation	AVR integrator

Every block output signal that represents a dynamic state should have a corresponding entry here. The mandatory output states for the selected model type (e.g., vf for EXC) must appear either as block outputs or in %states.

%observables

Purpose: List variables to be recorded during simulation. These are typically the states you want to monitor.

Column	Description	Example
Name	Observable name	vf

Observables do not need expressions. They refer to state names already defined elsewhere in the model.

Click “+ Add row” to append a row. Click the x button on a row to remove it. All edits update the DSL preview in real time.

28.3.6. 6. Review the DSL Preview

The bottom-right panel shows a **live syntax-highlighted preview** of the generated DSL code. It updates automatically (with a 600 ms debounce) after every change. The preview uses colour coding:

Colour	Meaning
Orange, bold	Section headers (%data, %parameters, etc.)
Gray, italic	Comments (! ...)
Blue	Numeric literals

Colour	Meaning
Purple	Data/parameter references ($\{KA\}$)
Cyan	State/input references ($[\omega]$)

Click the **copy button** in the preview header to copy the DSL text to your clipboard.

28.4. Saving a Project

Click “**Save Project**” in the topbar (or press **Ctrl+S** / **Cmd+S**).

The browser downloads a `.json` file named after your model (e.g., `simple_avr.json`) containing the complete project state:

- Model type and name
- All blocks with their positions, arguments, and output signal names
- All wire connections
- All metadata (data, parameters, states, observables)

This is a **lossless format**, every aspect of your canvas layout and configuration is preserved. Use this format for work-in-progress models that you intend to continue editing.

28.5. Loading a Project

Click “**Load Project**” in the topbar and select a previously saved `.json` file.

The editor restores the full project state: - Canvas blocks appear at their saved positions - All wires are reconnected - Metadata tables repopulate - The model type and name update in the topbar - The colour theme switches to match the model type

A toast notification confirms the load with the model name.

New vs. Load

Clicking “**New**” resets the editor to a blank state. A confirmation dialog (“Discard current model?”) prevents accidental loss of work. If you want to keep your current model, save it first.

28.6. Importing a DSL File

Click “**Load DSL**” and select a `.txt` file written in the CODEGEN DSL format.

The backend parser extracts all sections and blocks from the file, and the editor reconstructs the canvas:

1. Each `& blockname` in `%models` becomes a node on the canvas
2. Signal references are resolved into wire connections
3. Metadata sections populate the corresponding tabs
4. A **Sugiyama auto-layout** algorithm assigns node positions automatically:
 - Blocks are arranged left-to-right by dependency order
 - A three-pass crossing-reduction minimises wire overlaps
 - Feedback loops are detected and handled gracefully

This lets you visually inspect and edit any existing CODEGEN model, including the example models.

28.7. Checking the Model

The **yes** button on the canvas toolbar runs every structural check at once, without waiting for an export. Its findings go to the **Issues** panel in the right sidebar, which stays there rather than appearing as a dialog, so the list can be worked through while the canvas is edited. A model with nothing wrong reports that too, which is the useful half: it is the confirmation that the two export gates below will not stop you.

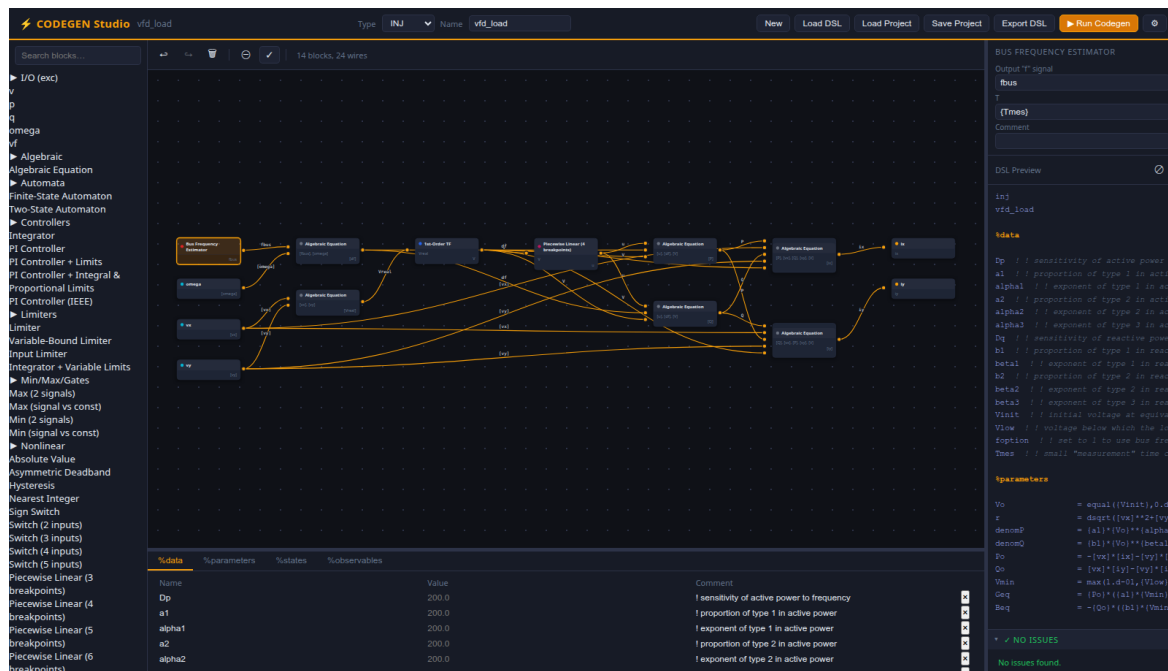


Figure 28.5: The CODEGEN Studio editor after pressing the check button on the canvas toolbar.

28.8. Exporting a DSL File

Click “Export DSL” to download the generated .txt file.

Export runs two checks, in order. Each raises a dialog you can override with “Export Anyway” or abandon with “Cancel”.

Mandatory outputs. That every mandatory output state for the selected model type is produced by at least one block on the canvas:

Missing Mandatory Outputs

Missing mandatory outputs for **exc**: **vfd**

The codegen binary will reject this model.

Floating state connectors. That no state is referenced by several blocks without a wire joining them. A state literal sitting on one block’s port while the same name appears elsewhere in the project usually means a connection was left undrawn on the canvas:

Floating State Connectors

The following states are referenced by multiple blocks but the connections are not wired on the canvas:

- Vc referenced by 2 blocks without a shared wire

The emitted DSL may not compile. Wire the connectors up before exporting.

Note that **Run Codegen** applies only the first of the two. A model that exports with a floating-state warning will be sent to the codegen binary without one.

The exported file follows the strict DSL section order: model type, model name, %data, %parameters, %states, %observables, %models. Blocks are emitted in topologically-sorted order (dependencies before dependents).

Round-trip fidelity

You can import an exported DSL file back into the editor. Unknown block types (not in the catalogue) are preserved through `rawArgLines` and re-emitted verbatim, so the structure survives a round trip even for custom or future blocks.

One exception, in **comments**: the parser keeps the leading `!` as part of the comment text and the emitter adds another, so `! gain` comes back as `! ! gain`, and a further marker accrues on each pass. It is cosmetic, and codegen ignores comments, but strip the extra markers before keeping a re-exported file as the master copy.

28.9. Running CODEGEN

Click **“Run Codegen”** (the accent-coloured button with the play icon) to compile the model into Fortran.

The same mandatory-output validation runs first. If validation passes:

1. The editor generates the DSL text
2. The backend writes it to a temporary file and invokes the bundled CODEGEN
3. On **success**, a modal displays the generated `.f90` source code with options to **“Download .f90”** or **“Close”**
4. On **failure**, a modal shows the error output from CODEGEN

The downloaded `.f90` file follows the naming convention `exc_simple_avr.f90` (type prefix + model name) and is ready to compile and link with RAMSES.

Nothing to install or configure

CODEGEN ships inside the wheel, for Linux x86-64, Windows x86-64 and macOS Apple Silicon, so this works on a fresh `pip install stepss-cg-studio`. There is no binary to obtain and no path to set; Settings shows which CODEGEN release this install runs, and the package version names it too.

macOS is the one platform that needs anything else: `brew install gcc`, because Apple does not support fully static executables and that build links against `libgfortran`.

28.10. File Formats Summary

Format	Extension	Use case	Lossless?
Project	.json	Save/resume editing	Yes, includes canvas positions, all metadata
DSL	.txt	Input to codegen binary	Structure only, no canvas layout
Fortran	.f90	Output from codegen	Export only, cannot be re-imported

28.11. Keyboard Shortcuts

Shortcut	Action
Ctrl+S / Cmd+S	Save project
Ctrl+Z / Cmd+Z	Undo
Ctrl+Y / Cmd+Shift+Z	Redo
Delete / Backspace	Delete selected block
Escape	Close any open modal

Shortcuts are disabled while typing in text fields.

28.12. Settings

Click the **gear icon** in the topbar to configure:

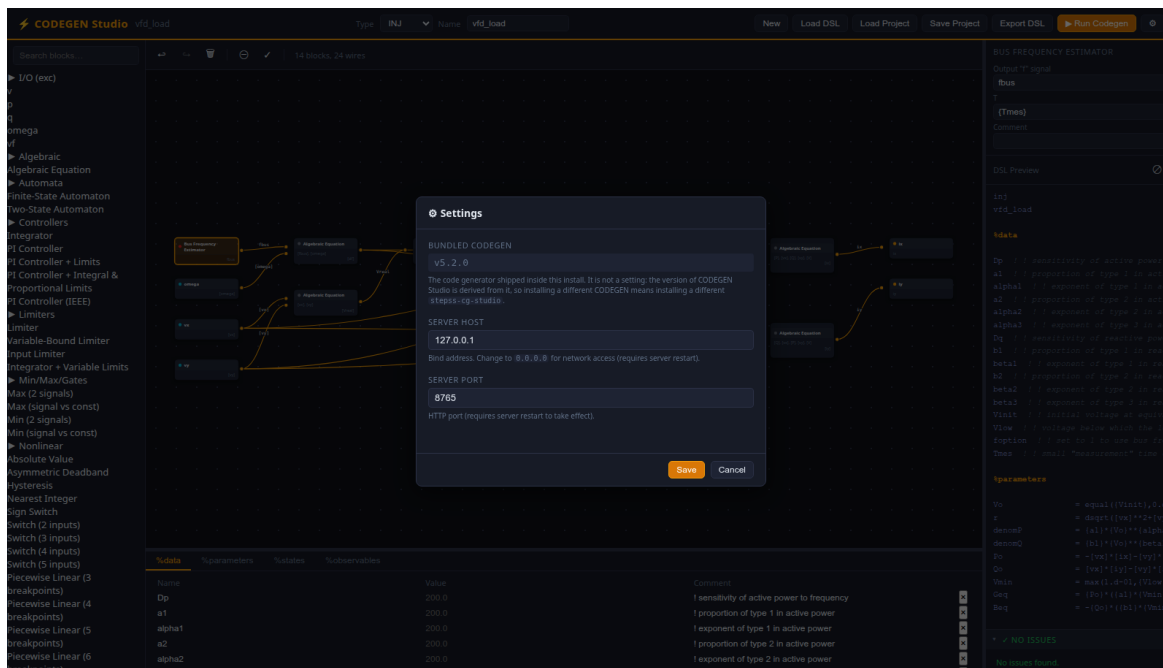


Figure 28.6: The Settings dialog over the editor, with three rows: Bundled CODEGEN showing v5.2.0 as flat monospace text rather than an editable field, server host set to 127.0.0.1, and server port set to 8765.

Row	Default	Description
Bundled CODEGEN	(read-only)	Which CODEGEN release this install runs
Server host	127.0.0.1	Bind address (change to 0.0.0.0 for network access)
Server port	8765	HTTP port

Changes to host and port require a server restart.

Bundled CODEGEN is not a setting. It reports what the wheel carries, and the two leading components of the package's own version name the same release: `stepss-cg-studio` 5.3.0 and 5.3.1 both run CODEGEN 5.3, and 5.4.0 is the first release on CODEGEN 5.4. Running a different generator therefore means installing a different `stepss-cg-studio`, not repointing the editor. Earlier versions had a "Codegen binary path" field, which existed only because no CODEGEN was shipped; an upgrade drops it from `config.json` automatically.

Settings are stored in a platform-specific config file:

- **Windows:** %LOCALAPPDATA%\cg-studio\config.json
- **Linux/macOS:** ~/.config/cg-studio/config.json

They can also be edited via the REST API at /docs (Swagger UI).

28.13. Workflow Example: Building an AVR Model

This walkthrough creates a simplified Automatic Voltage Regulator (AVR) as an EXC model.

28.13.1. Step 1: Set up the model

1. Select **EXC** from the Type dropdown
2. Type `simple_avr` in the Name field

28.13.2. Step 2: Add data constants

Switch to the **%data** tab and add rows:

Name	Value	Comment
KA		AVR gain
TA		AVR time constant
VREF		Reference voltage

28.13.3. Step 3: Place blocks

Drag the following blocks from the palette onto the canvas:

1. `algeq`, to compute the voltage error
2. `tf1p`, a first-order transfer function for the AVR

28.13.4. Step 4: Connect and configure

1. Connect the `algeq` output to the `tf1p` input
2. Select the `algeq` block and set its **Expression** to `{VREF}-[v]-avr1`
3. Select the `tf1p` block and:
 - Rename its output signal to `vf` (this is the mandatory EXC output)
 - Set **K** to `{KA}` and **T** to `{TA}`

28.13.5. Step 5: Define states

Switch to the `%states` tab and add:

Name	Init	Comment
avr1	<code>vf/{KA}</code>	AVR integrator state

28.13.6. Step 6: Add observables

Switch to the `%observables` tab and add `vf`.

28.13.7. Step 7: Export

1. Click “**Export DSL**”, validation passes because `vf` is produced by the `tf1p` block
2. The downloaded `simple_avr.txt` is ready for codegen `-tsimple_avr.txt`
3. Or click “**Run Codegen**” to generate `exc_simple_avr.f90` directly

28.14. See Also

- User-Defined Models, DSL file format and model framework
- CODEGEN Blocks, complete block reference
- CODEGEN Model Examples, annotated model files for all four types

Part VII

STEPSS in Python

29

STEPSS in Python

stepss is the package that delivers STEPSS in Python, one of the platform's two editions. It provides an interface to the RAMSES dynamic simulator and the Helios AC power-flow engine, covering the full workflow: defining test cases, launching simulations, querying system states at runtime, extracting and plotting results, and running power flows. The package embeds pre-compiled RAMSES and Helios libraries for Windows, Linux, and macOS and exposes them through a clean Python API.

29.1. Package-Level Attributes

After importing the package, the following attributes are available:

Attribute	Description
<code>stepss.__version__</code>	Current version string
<code>stepss.__ramses_version__</code>	Version of the bundled RAMSES library
<code>stepss.__helios_version__</code>	Version of the bundled Helios library
<code>stepss.__url__</code>	Documentation URL

29.2. Main Classes

Class	Description
<code>stepss.cfg</code>	Defines a test case: data files, disturbance file, output files, observables, and runtime options.
<code>stepss.sim</code>	Runs simulations. Supports start/pause/continue, runtime queries, and disturbance injection.
<code>stepss.extractor</code>	Extracts and visualises time-series results from trajectory files produced by a simulation.
<code>stepss.monitor</code>	Plots chosen quantities while a simulation runs, one panel per observable.
<code>stepss.helios.HeliosSession</code>	Runs AC power flows with the Helios engine: load, modify with redispatch, solve, contingency screening, and file exports.

29.3. Platform Support

stepss supports Windows, Linux, and macOS for both dynamic simulation and power flows. All binaries are bundled directly in the package, no separate simulator installation is required. See the installation guide for per-platform system prerequisites.

29.4. Further Reading

- API Reference, Detailed documentation for `cfg`, `sim`, `extractor` and `monitor`
- Power Flow (Helios), Running AC power flows from Python with `HeliosSession`
- Examples, Practical simulation examples and notebooks

29.5. Repository

Source code: `SPS-L/stepss-python-ui`

30

Installing the Python package

stepss is a Python interface to the RAMSES dynamic simulator and the Helios AC power-flow engine, enabling users to define test cases, run simulations, extract results, and perform post-processing, all within Python scripts or Jupyter notebooks.

stepss supports Windows, Linux, and macOS, for both dynamic simulation and the bundled Helios power-flow engine, and integrates with standard scientific Python libraries (NumPy, SciPy, Matplotlib).

30.1. Installation

Install stepss and all recommended dependencies via pip:

```
pip install jupyter ipython stepss
```

Required dependencies (matplotlib, scipy, numpy) are installed automatically.

For a minimal installation (no plotting or notebook support):

```
pip install stepss
```

30.1.1. Version numbers

A stepss version is the bundled RAMSES version, optionally followed by a counter: version 3.59 carries RAMSES 3.59. The counter is appended when stepss is released without a new RAMSES behind it, so a 3.59.1 would bundle RAMSES 3.59 as well. A new RAMSES release restarts the sequence with a bare version.

stepss on PyPI starts at 3.59; nothing below that can be installed.

To pin an engine version, pin the matching stepss release:

```
pip install "stepss~=3.59.0"    # any 3.59.x, all bundling RAMSES 3.59
```

Check what an installed copy carries:

```
import stepss
print(stepss.__version__, stepss.__ramses_version__, stepss.__helios_version__)
```

30.2. Linux System Prerequisites

On Linux, the following system libraries must be installed before running stepss:

```
sudo apt install libopenblas0 libgfortran5 libgomp1
```

These packages provide:

Package	Purpose
libopenblas0	OpenBLAS BLAS/LAPACK routines used by the solver
libgfortran5	GNU Fortran runtime required by the Fortran components of RAMSES
libgomp1	OpenMP runtime for multi-core parallel execution

On most desktop Linux distributions these are already present. If `stepss` fails to import with a shared-library error, install the packages above and retry.

30.3. macOS System Prerequisites

On macOS (Apple Silicon), install the GNU compiler runtimes and OpenBLAS via Homebrew before running `stepss`:

```
brew install gcc openblas
```

These provide the `libgfortran`, `libgomp`, and `libopenblas` dylibs that the bundled arm64 binaries link against. Intel Macs are not supported.

30.4. Run-time observables

`case.addRunObs(...)` asks the engine to record chosen quantities while a simulation runs, and RAMSES writes them to a curve file as it goes. Nothing has to be installed for this.

The file is plain text, with a `#` header naming the columns and time in column one, so any reader takes it:

```
import numpy as np
data = np.loadtxt('temp_display.cur')
```

30.5. Live plotting

`stepss.monitor` plots chosen quantities *while* the simulation runs, one panel per observable:

```
import stepss

ram = stepss.sim()
case = stepss.cfg('cmd.txt')
ram.execSim(case, 0.0)                # initialise, paused at t = 0

mon = stepss.monitor(ram, ['BV 4044', 'MS g6'])
curves = mon.run(step=0.5)            # run to the end of the scenario
```

It polls the engine in process and draws with `matplotlib`, which `pip` installs with the package, so nothing further is needed. The API reference lists the descriptor vocabulary and the rest of the methods.

30.6. Platform Support

Platform	Library	Notes
Windows	<code>ramses.dll</code>	Primary platform, full support
Linux	<code>ramses.so</code>	Full support (requires the system libraries above)

Platform	Library	Notes
macOS	<code>ramses.so</code>	Apple Silicon (arm64) only; requires the Homebrew runtimes above

30.7. Next Steps

- API Reference, Full API documentation
- Examples, Practical usage examples

31

Python examples

All examples assume the working directory contains the relevant data and disturbance files.

31.1. Running a Basic Simulation

Define the test case, run the simulation, and extract results:

```
import stepss

case = stepss.cfg()
case.addData('dyn_A.dat')      # dynamic models
case.addData('volt_rat_A.dat') # power-flow initialisation
case.addData('settings1.dat')  # solver settings
case.addDst('short_trip_branch.dst')
case.addInit('init.trace')
case.addTrj('output.trj')      # save trajectories for post-processing
case.addObs('obs.dat')         # define which observables to record
case.addCont('cont.trace')
case.addDisc('disc.trace')
case.addRunObs('BV 4044')      # record bus voltage while the run proceeds
case.addRunObs('BV 1041')
case.writeCmdFile('cmd.txt')   # save for future reuse

ram = stepss.sim()
ram.execSim(case)              # run to completion

ext = stepss.extractor(case.getTrj())
ext.getBus('1041').mag.plot()  # voltage magnitude at bus 1041
```

Note

Set `$NB_THREADS 0` ; in the solver settings file to use all available CPU cores for parallel simulation. The free version uses at most **2 cores** whatever this is set to; a `$LICENSE` record in the data files lifts the cap. See License.

31.2. Pause and Continue

Pause the simulation at intermediate time points to inspect state or add disturbances:

```

ram = stepss.sim()
ram.execSim(case, 0.0)           # initialise, paused at t=0
ram.contSim(10.0)               # simulate to t=10 s
ram.contSim(ram.getSimTime() + 60.0) # advance 60 s from current time
ram.contSim(ram.getInTime())     # run to end of time horizon

```

31.3. Querying System State

Query bus voltages, branch flows, observables, and parameters while paused:

```

ram.execSim(case, 10.0)

# Bus voltages
busNames = ['g1', 'g2', 'g3']
voltages = ram.getBusVolt(busNames) # list of voltage magnitudes (pu)
phases   = ram.getBusPha(busNames)  # list of phase angles (deg)

# Branch power flows
branch_pq = ram.getBranchPow(['1041-01']) # [[P_from, Q_from, P_to, Q_to]]

# Component observables: P of injector L_11, vf of exciter g2, Pm of governor g3
comp_type = ['INJ', 'EXC', 'TOR']
comp_name = ['L_11', 'g2', 'g3']
obs_name  = ['P', 'vf', 'Pm']
obs = ram.getObs(comp_type, comp_name, obs_name)

# Component parameters: V0 of exciter g1, KPSS of exciter g2
comp_type = ['EXC', 'EXC']
comp_name = ['g1', 'g2']
prm_name  = ['V0', 'KPSS']
prms = ram.getPrm(comp_type, comp_name, prm_name)

# All component names of a type
all_buses = ram.getAllCompNames('BUS')
all_gens  = ram.getAllCompNames('SYNC')

```

31.4. Adding Disturbances at Runtime

Inject disturbances and parameter changes while the simulation is running:

```

ram.execSim(case, 80.0)

# LTC voltage setpoint changes at t=100 s (ramp)
for i in range(1, 6):
    ram.addDisturb(100.0, f'CHGPRM DCTL {i}-104{i} Vsetpt -0.05 0')

# Fault and clearance
ram.addDisturb(100.0, 'FAULT BUS 4032 0. 0.') # 3-phase short circuit
ram.addDisturb(100.1, 'CLEAR BUS 4032')      # clear fault
ram.addDisturb(100.1, 'BREAKER BRANCH 4032-4044 0 0') # trip line

# Generator trip
ram.addDisturb(100.0, 'BREAKER SYNC_MACH g7 0')

ram.contSim(ram.getInTime())

```

For full disturbance syntax, see the Disturbances reference.

31.5. Plotting Results

Extract and plot time-series results after the simulation:

```
import stepss
ext = stepss.extractor('output.trj')

# Single curve
ext.getSync('g5').S.plot()      # rotor speed of generator g5
ext.getBus('4044').mag.plot()   # voltage magnitude at bus 4044

# Multiple curves on the same plot
curves = [ext.getSync(f'g{i}').S for i in range(1, 5)]
stepss.curplot(curves)

# Exciter and governor outputs
ext.getExc('g1').vf.plot()      # field voltage
ext.getTor('g1').Pm.plot()      # mechanical power

# Branch power flows
ext.getBranch('1041-01').PF.plot()

# Injector (wind/PV/BESS)
ext.getInj('WT1a').Pw.plot()

# Two-port (HVDC)
ext.getTwop('hvdc1').P1.plot()
```

Every one of those calls opens a matplotlib figure. Run against the bundled Kundur two-area example, whose disturbance steps load L9 at $t = 1$ s, the first three look like this.

`ext.getSync('G1').P.plot()`, one curve, labelled from the trajectory:

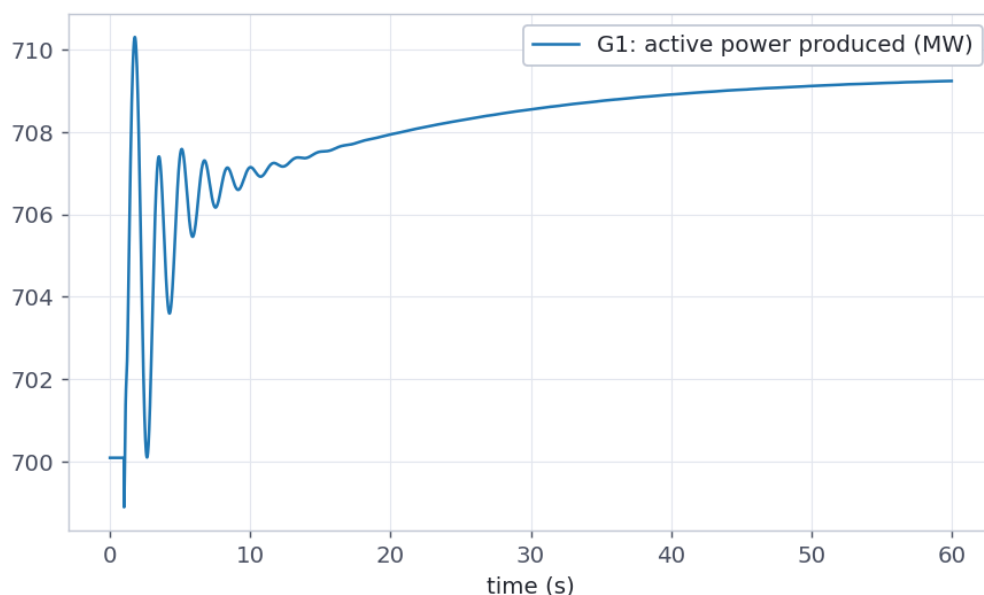


Figure 31.1: Active power produced by generator G1 against time, in megawatts over 60 seconds.

`stepss.curplot([...])`, several curves on one set of axes. Putting all four machines together is what separates the areas: G3 starts 19 MW above the rest and stays there, while G1, G2 and G4 sit on top of one another.

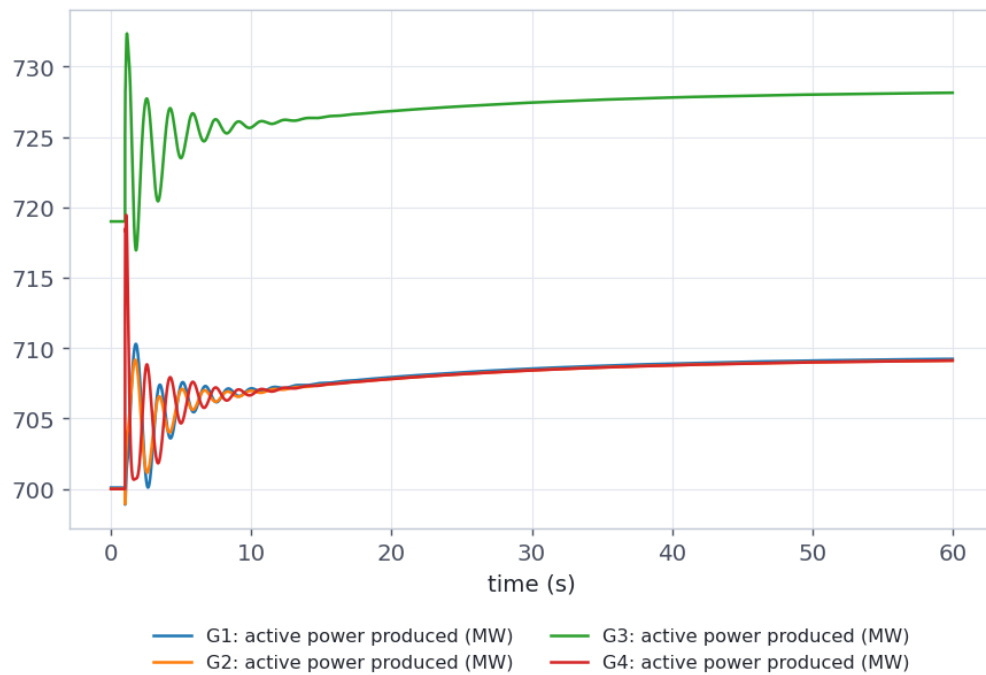


Figure 31.2: Active power produced by generators G1 to G4 against time, four curves on one set of axes with a legend below them.

`ext.getBus('9').mag.plot()`, the same run seen as a voltage rather than a power:

These are ordinary matplotlib figures, so the usual `savefig`, styling and subplot handling all apply to them.

31.6. Live Plotting

Watch chosen quantities as the simulation computes them, one panel per observable:

```
import stepss

ram = stepss.sim()
ram.execSim(case, 0.0)                                # initialise, paused at t = 0

mon = stepss.monitor(ram, [
    'BV 4044',                                         # voltage magnitude of bus 4044
    'MS g6',                                           # rotor speed of machine g6
    'BPO 4041-4044',                                   # active power at the branch origin
    'RT RT',                                           # wall clock, to gauge simulation speed
], title='Nordic')

curves = mon.run(step=0.1)                            # to the end of the scenario
mon.savefig('live.png')
```

Each observable gets a panel, the panels share the time axis, and the figure is redrawn as the run advances. `RT RT` is the one to include when you want to see how fast the run itself is going: a straight line means the engine is keeping a steady pace, and a knee in it is where the case got expensive.

`run` returns the same `cur` objects the extractor produces, so the collected data goes straight into the post-processing above:

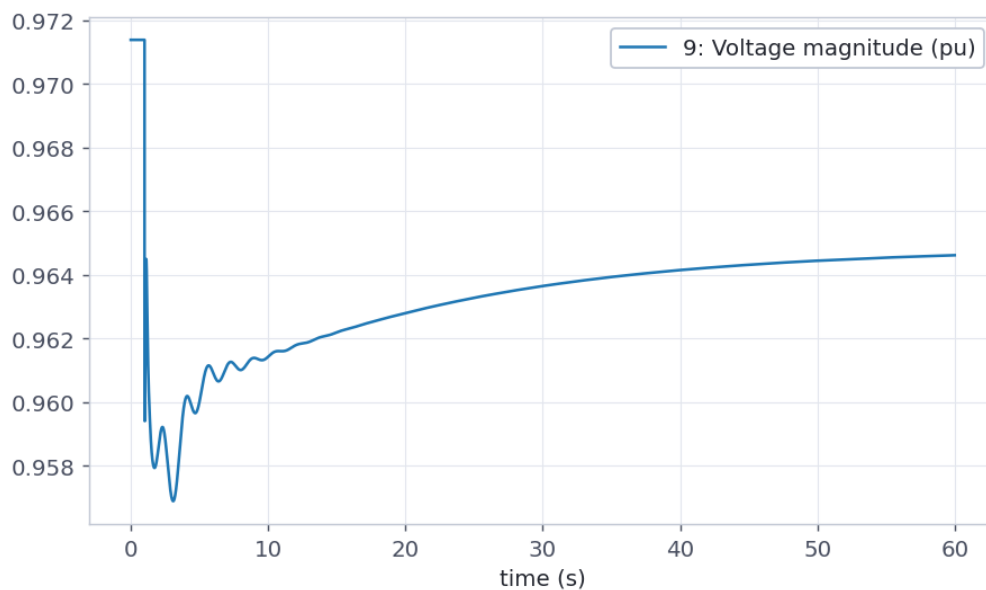


Figure 31.3: Voltage magnitude at bus 9 against time, in per unit over 60 seconds.

```
stepss.curplot(curves)
```

Drive the stepping yourself when the run needs disturbances injected along the way:

```
mon = stepss.monitor(ram, ['BV 4044', 'BV 1041'])
for target in range(10, 200, 10):
    ram.contSim(float(target))
    mon.sample()
    mon.refresh()
    if target == 100:
        ram.addDisturb(105.0, 'CHGPRM DCTL 1-1041 Vsetpt -0.05 0')
```

31.7. Parameter Sweep

Run multiple simulations with varying parameters and collect results:

```
import stepss
import numpy as np

results = {}
for disturbance_time in [5.0, 10.0, 20.0]:
    case = stepss.cfg('cmd.txt')
    trj_file = f'output_{disturbance_time:.0f}.trj'
    case.addTrj(trj_file)
    case.addObs('obs.dat')

    ram = stepss.sim()
    ram.execSim(case, 0.0)
    ram.addDisturb(disturbance_time, 'BREAKER SYNC_MACH g7 0')
    ram.contSim(ram.getInfTime())
    ram.endSim()

    ext = stepss.extractor(trj_file)
    min_freq = np.min(ext.getSync('g5').S.value)
```

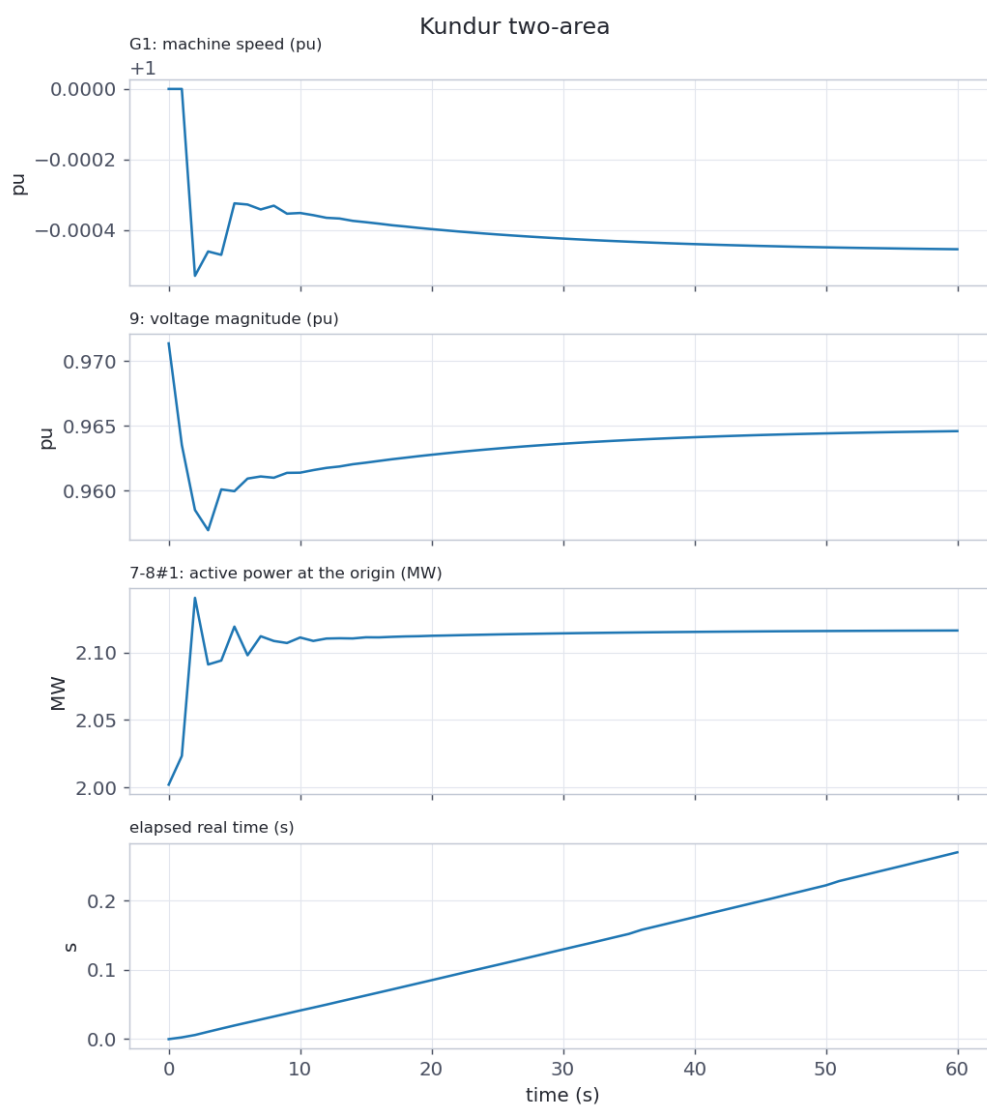


Figure 31.4: A live monitor figure titled Kundur two-area, four panels stacked on a shared time axis running to 60 seconds.

```
results[disturbance_time] = min_freq
print(f't_dist={disturbance_time:5.1f}s  min_speed={min_freq:.5f} pu')
```

31.8. Eigenanalysis Workflow

RAMSES performs the small-signal analysis itself. Schedule an EIG event at the operating point and it writes the modes, participation factors and mode shapes:

```
import stepss

case = stepss.cfg('cmd.txt')
ram = stepss.sim()

ram.execSim(case, 0.0)          # pause at the steady-state operating point
ram.addDisturb(0.001, "EIG 'ssa'") # writes ssa_modes.dat, ssa_pf.dat, ssa_ms.dat
ram.contSim(0.01)              # advance past the event so it fires
ram.endSim()
```

Reading the result needs nothing beyond numpy:

```
import numpy as np

m = np.loadtxt('ssa_modes.dat', comments='#')
freq, zeta = m[:, 4], m[:, 3]

interarea = m[(freq > 0.4) & (freq < 0.9) & (m[:, 2] > 0)]
print('inter-area mode: %.4f Hz, zeta = %+.4f' % (interarea[0, 4], interarea[0, 3]))
```

A complete annotated walkthrough ships with the package under `examples/eigenanalysis/`.

To drive your own solver instead, `getJac()` returns the descriptor-form pair as SciPy sparse matrices. This is the route for systems above `$EIG_MAX_STATES`, where the engine's dense solve is not practical and sparse shift-invert methods are needed:

```
ram.execSim(case, 0.0)
A, E = ram.getJac() # scipy.sparse.csc_matrix
ram.endSim()
```

Note

Both routes need `$OMEGA_REF SYN` ; and `$SCHEME DE` ; in the solver settings. EIG refuses without them, exiting 78 with the reason in the log; the JAC export skips with a warning under COI. See Eigenanalysis.

31.9. Test System Examples

The following examples use the ready-to-run test systems. For system descriptions, data files, and disturbance scenarios, see the Test Systems section.

31.9.1. Nordic Test System: Generator Trip

Trips generator g7 at $t = 10$ s on the heavily-stressed Operating Point B and observes the voltage collapse dynamics over 150 seconds. See the Nordic Test System page for full system details and file descriptions.

```

import stepss
import os

case = stepss.cfg()
case.addData('dyn_B.dat')
case.addData('volt_rat_B.dat')
case.addData('settings1.dat')
case.addDst('nothing.dst')
case.addObs('obs.dat')
case.addTrj('output.trj')

for f in os.listdir('.'):
    if f.endswith('.trj') or f.endswith('.trace'):
        os.remove(f)

ram = stepss.sim()
ram.execSim(case, 0.0)
ram.addDisturb(10.0, 'BREAKER SYNC_MACH g7 0')
ram.contSim(150.0)
ram.endSim()

ext = stepss.extractor(case.getTrj())
ext.getSync('g7').S.plot() # rotor speed
ext.getBus('1041').mag.plot() # voltage at central bus

```

31.9.2. 5-Bus System: Exciter Parameter Change

Applies a step change to the exciter voltage setpoint at $t = 1$ s and plots the generator response. See the 5-Bus Test System page for details.

```

import stepss
import os

case = stepss.cfg()
case.addData('dyn.dat')
case.addData('lflsolv.dat')
case.addData('solveroptions.dat')
case.addDst('nothing.dst')
case.addObs('obs.dat')
case.addTrj('output.trj')

for f in os.listdir('.'):
    if f.endswith('.trj') or f.endswith('.trace'):
        os.remove(f)

ram = stepss.sim()
ram.execSim(case, 0.0)
ram.addDisturb(1.0, 'CHGPRM EXC G Vo 0.05 2')
ram.contSim(60.0)
ram.endSim()

ext = stepss.extractor(case.getTrj())
ext.getSync('G').P.plot()
ext.getSync('G').Q.plot()

```

The exciter set point rises by 0.05 pu, so the machine's reactive output is what moves: active power holds at its scheduled 450 MW while reactive power climbs from 68 to about 120 Mvar

in a couple of seconds and settles there.

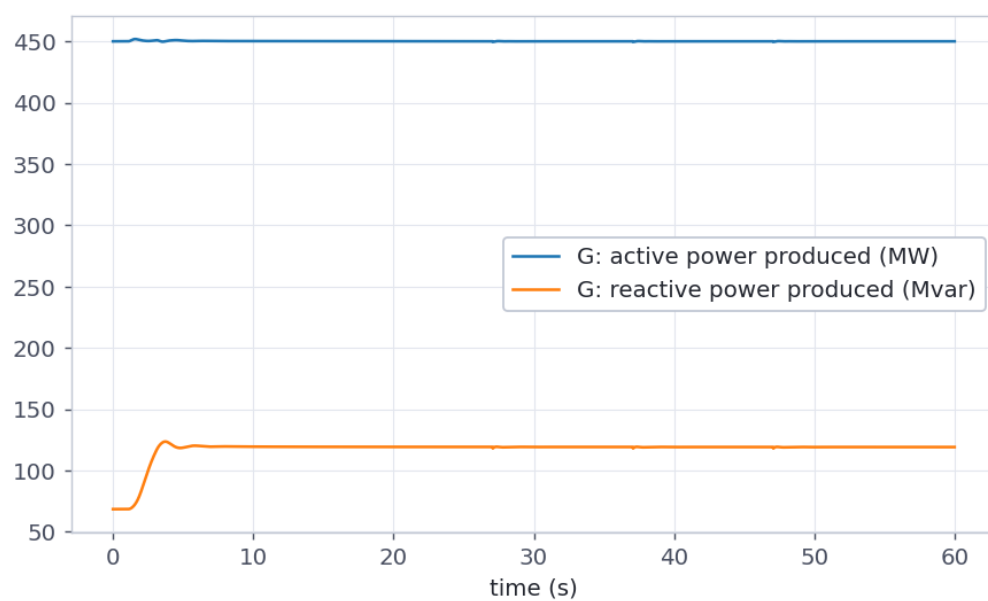


Figure 31.5: Active and reactive power produced by the 5-bus system's single generator against time, over 60 seconds.

32

Python API reference

32.1. `stepss.cfg`: Test Case Configuration

The `stepss.cfg` class defines a simulation scenario: data files, disturbance file, output files, observables, and runtime options.

32.1.1. Initializing

Create an empty configuration or load one from a previously saved command file:

```
import stepss

case = stepss.cfg()          # empty configuration
case = stepss.cfg("cmd.txt") # load from command file
```

Load multiple cases in a loop:

```
import stepss

list_of_cases = []
for i in range(12):
    list_of_cases.append(stepss.cfg('cmd' + str(i) + '.txt'))
```

`writeCmdFile(filename=None)`

Save the current configuration to a command file. Useful for reproducing a run later. When called without an argument, it returns the command-file content as a string instead of writing a file.

```
case.writeCmdFile('cmd.txt')
```

Save multiple cases in a loop:

```
for i in range(12):
    list_of_cases[i].writeCmdFile('cmd' + str(i) + '.txt')
```

32.1.2. Data Files

Data files describe the network topology, dynamic models, and solver settings. At least one must be provided.

addData(filename)

Add a data file to the case.

```
case.addData('dyn_A.dat')
case.addData('settings1.dat')
```

delData(filename)

Remove a specific data file from the case.

```
case.delData('dyn_A.dat')
```

getData()

Return the list of currently registered data files.

```
files = case.getData()
```

clearData()

Remove all data files from the case.

```
case.clearData()
```

Caution

At least one data file must be provided, otherwise the simulator will raise an exception.

32.1.3. Initialization File

Specifies where the simulator writes initialization procedure output.

addInit(filename)

```
case.addInit('init.trace')
```

getInit()

Return the currently registered initialization file path.

```
path = case.getInit()
```

Note

This is optional. If omitted, the simulator skips writing initialization output.

32.1.4. Disturbance File

Describes the disturbances to be simulated (generator trips, faults, parameter changes, etc.).

addDst(filename)

```
case.addDst('events.dst')
```

getDst()

Return the currently registered disturbance file path.

```
path = case.getDst()
```

clearDst()

Remove the disturbance file from the case.

```
case.clearDst()
```

Caution

A disturbance file must be provided, otherwise the simulator will raise an exception.

32.1.5. Trajectory File

Specifies the file where time-series simulation results (trajectories) are saved for post-processing. This file is used by `stepss.extractor` to access results after the simulation completes.

addTrj(filename)

```
case.addTrj('output.trj')
```

getTrj()

Return the currently registered trajectory file path.

```
path = case.getTrj()
```

Note

This is optional. If omitted, no trajectory file is written and result extraction via `stepss.extractor` will not be possible.

32.1.6. Observables File

Defines which components and quantities are recorded in the trajectory file.

addObs(filename)

```
case.addObs('obs.dat')
```

getObs()

Return the currently registered observables file path.

```
path = case.getObs()
```

Note

Required when a trajectory file is specified. Defines what data is stored in the trajectory.

32.1.7. Output/Trace Files**addOut(filename)**

Set the main output trace file for simulation progress logging.

```
case.addOut('output.trace')
```

getOut()

Return the currently registered output trace file path.

```
path = case.getOut()
```

addCont(filename)

Set the continuous trace file. Records Newton solver convergence information at each step. Useful for debugging but can slow down the simulation.

```
case.addCont('cont.trace')
```

getCont()

Return the currently registered continuous trace file path.

```
path = case.getCont()
```

addDisc(filename)

Set the discrete trace file. Records discrete events: switching actions from disturbance files, discrete controllers, or discrete variables in injector/torque/exciter/two-port models.

```
case.addDisc('disc.trace')
```

getDisc()

Return the currently registered discrete trace file path.

```
path = case.getDisc()
```

clearDisc()

Remove the discrete trace file from the case.

```
case.clearDisc()
```

Note

All output/trace files are optional.

32.1.8. Runtime Observables

Runtime observables are recorded by the engine while a simulation runs, into a curve file you can read with `cur` and plot with `curplot`. Nothing has to be installed for this.

addRunObs(obs_string)

Add a runtime observable. The following observable types are supported:

BV **BUSNAME**, Voltage magnitude of a bus:

```
case.addRunObs('BV 1041')
```

MS **MACHINE_NAME**, Rotor speed of a synchronous machine:

```
case.addRunObs('MS g1')
```

BPE / BQE / BPO / BQO **BRANCH_NAME**, Active (P) or reactive (Q) power at the origin (O) or extremity (E) of a branch:

```
case.addRunObs('BPO 1041-01') # active power at origin of branch 1041-01
case.addRunObs('BQO 1041-01') # reactive power at origin
case.addRunObs('BPE 1041-01') # active power at extremity
case.addRunObs('BQE 1041-01') # reactive power at extremity
```

ON INJECTOR_NAME OBSERVABLE_NAME, Named observable from an injector model:

```
case.addRunObs('ON WT1a Pw') # observable Pw from injector WT1a
```

TO TWOP_NAME OBSERVABLE_NAME, Named observable from a two-port model:

```
case.addRunObs('TO hvdc1 P1') # observable P1 from two-port hvdc1
```

RT RT, Real-time versus simulated-time plot (useful to gauge simulation speed):

```
case.addRunObs('RT RT')
```

clearRunObs()

Remove all runtime observables.

```
case.clearRunObs()
```

Note

Nothing needs to be installed for run-time observables.

32.2. `stepss.sim`: Simulation Control

The `stepss.sim` class runs simulations. It wraps the RAMSES dynamic library and supports start/pause/continue, runtime queries, and disturbance injection.

32.2.1. Initializing

```
import stepss
```

```
ram = stepss.sim() # use bundled RAMSES libraries
ram = stepss.sim(custLibDir='/path/to/') # use custom library directory
```

Parameter	Type	Description
<code>custLibDir</code>	str or None	Custom path to the RAMSES library directory. Default: use bundled libraries.

32.2.2. Running Simulations

A properly configured `stepss.cfg` test case is required before running a simulation.

execSim(case): run to completion

```
ram.execSim(case)
```

execSim(case, t): start and pause at time t

Start the simulation and pause at a specific time (in seconds):

```
ram.execSim(case, 10.0)    # start and pause at t = 10 s
```

contSim(t): continue to time t

Resume a paused simulation until a specified time:

```
ram.contSim(20.0)          # resume until t = 20 s
ram.contSim(ram.getSimTime() + 60.0) # advance by 60 s from current time
ram.contSim(ram.getInfTime()) # run to the end of the time horizon
```

endSim(): terminate early

Terminate the simulation before reaching the time horizon:

```
ram.endSim()
```

pauseSim(t_pause): schedule a pause

Schedule a pause at a given simulated time; it takes effect on the next `execSim()` or `contSim()` call:

```
ram.pauseSim(10.0)
ram.execSim(case)    # will pause at t = 10 s
```

getEndSim()

Return 0 while the simulation is still running, 1 once it has ended:

```
if ram.getEndSim():
    print('simulation finished')
```

getLastErr()

Return the last error message issued by RAMSES as a string. Useful after catching a `RAMSESError`:

```
msg = ram.getLastErr()
```

32.2.3. Querying State

When the simulation is paused, the following methods query the current system state.

getSimTime()

Return the current simulation time in seconds.

```
t = ram.getSimTime()
```

getInfTime()

Return the value used as “infinity” (i.e., the end of the simulation time horizon). Pass this to `contSim()` to run to completion.

```
t_inf = ram.getInfTime()
ram.contSim(ram.getInfTime())
```


getAllCompNames(type)

Return a list of all component names of the given type.

```
buses      = ram.getAllCompNames('BUS')      # list of all bus names
gens       = ram.getAllCompNames('SYNC')     # list of all generator names
injs       = ram.getAllCompNames('INJ')      # list of all injector names
dctls      = ram.getAllCompNames('DCTL')     # list of all discrete controller names
branches   = ram.getAllCompNames('BRANCH')   # list of all branch names
twops      = ram.getAllCompNames('TWOP')     # list of all two-port names
shunts     = ram.getAllCompNames('SHUNT')    # list of all shunt names
loads      = ram.getAllCompNames('LOAD')     # list of all load names
```

Supported component types: BUS, SYNC, INJ, DCTL, BRANCH, TWOP, SHUNT, LOAD.

getCompName(comp_type, num)

Return the name of the num-th component (1-based) of the given type. Accepts the same component types as `getAllCompNames()`.

```
first_bus = ram.getCompName('BUS', 1)  # e.g. 'B1'
```

getBusVolt(names)

Return voltage magnitudes (in pu) for a list of bus names.

```
ram.execSim(case, 10.0)
bus_names = ['g1', 'g2', '4032']
voltages = ram.getBusVolt(bus_names)
```

getBusPha(names)

Return voltage phase angles (in degrees) for a list of bus names.

```
phases = ram.getBusPha(bus_names)
```

getBranchPow(names)

Return power flows for a list of branch names. Each entry is [P_from, Q_from, P_to, Q_to] in MW and Mvar.

```
powers = ram.getBranchPow(['1041-01'])
# powers[0] == [P_from, Q_from, P_to, Q_to]
```

getBranchCur(names)

Return branch currents for a list of branch names. Each entry contains the x-y current components at the origin and extremity: [ix_orig, iy_orig, ix_extr, iy_extr].

```
currents = ram.getBranchCur(['1011-1013', '1012-1014'])
```

getObs(comp_types, comp_names, obs_names)

Get the current value of named observables for a list of components. Lists must be the same length.

```
comp_type = ['INJ', 'EXC', 'TOR']
comp_name = ['L_11', 'g2', 'g3']
obs_name  = ['P', 'vf', 'Pm']
obs = ram.getObs(comp_type, comp_name, obs_name)
```

Supported model types: EXC (exciter), TOR (governor), INJ (injector), TWOP (two-port), DCTL (discrete controller), SYN (synchronous generator).

getPrm(comp_types, comp_names, prm_names)

Get parameter values for a list of components. Lists must be the same length.

```
comp_type = ['EXC', 'EXC']
comp_name = ['g1', 'g2']
prm_name = ['V0', 'KPSS']
prms = ram.getPrm(comp_type, comp_name, prm_name)
```

getPrmNames(comp_types, comp_names)

List the parameter names of one or more model instances. Accepts single strings or equal-length lists; component types are EXC, TOR, INJ, DCTL, TWOP.

```
names = ram.getPrmNames('EXC', 'g1') # list of parameter names of g1's exciter
```

32.2.4. Runtime Observable Recording

Observables can also be selected programmatically after the simulation starts, instead of via an observables file. Call the three methods in order, after `execSim(case, 0.0)`:

initObserv(traj_filename)

Initialize the runtime observable recording system and set the output trajectory file.

addObserv(string)

Register one observable selector in RAMSES format (e.g. 'BUS *', 'SYNC g1'). Call once per selector.

finalObserv()

Finalize the selection, allocate the recording buffers, and write the trajectory file header.

```
ram.execSim(case, 0.0)
ram.initObserv('obs.trj')
ram.addObserv('BUS *') # record all bus voltages
ram.addObserv('SYNC g1') # record machine g1
ram.finalObserv()
ram.contSim(ram.getInTime())
```

32.2.5. Subsystems

Subsystems select a group of buses for aggregate queries.

defineSS(ssID, filter1, filter2, filter3)

Define subsystem `ssID` as the intersection of three filters: voltage levels, zones, and bus names. Each filter is a list of strings; an empty list deactivates that filter.

```
ram.defineSS(1, ['735'], [], []) # subsystem 1 = all 735 kV buses
```

getSS(ssID)

Return the list of buses belonging to a subsystem.

```
buses = ram.getSS(1)
```

getTrfoSS(ssID, location, in_service, rettype)

Return transformer information for a subsystem. `location`: 1 = both ends inside the subsystem, 2 = tie transformers, 3 = both. `in_service`: 1 = in-service only, 2 = all. `rettype` selects the returned quantity: NAME, From, To, Status, Tap, Currentf, Currentt, Pf, Qf, Pt, Qt.

```
status = ram.getTrfoSS(1, 3, 2, 'Status')
```

32.2.6. Runtime Disturbances

Disturbances can be added dynamically while the simulation is paused, enabling interactive scenario analysis.

addDisturb(time, description)

Schedule a disturbance to occur at a given simulation time.

The disturbance description string follows the same syntax as the disturbance file format. See Disturbances for the complete reference.

```
ram.execSim(case, 80.0)

# Trip generator g7 at t = 100 s
ram.addDisturb(100.0, 'BREAKER SYNC_MACH g7 0')

# Apply a 3-phase fault at bus 4032 and clear it 100 ms later
ram.addDisturb(100.0, 'FAULT BUS 4032 0. 0.')
ram.addDisturb(100.1, 'CLEAR BUS 4032')

# Step change in an LTC setpoint
ram.addDisturb(100.0, 'CHGPRM DCTL 1-1041 Vsetpt -0.05 0')

ram.contSim(ram.getInTime())
```

32.2.7. Jacobian Export**getJac()**

Export the system Jacobian in descriptor form at the current pause point. Returns a tuple (A, E) of `scipy.sparse.csc_matrix` objects, where A holds the numerical Jacobian values and E is the structural incidence matrix of the descriptor system. Two intermediate text files are also written to the working directory:

File	Contents
<code>py_val.dat</code>	Jacobian values (parsed into A)
<code>py_eqs.dat</code>	Structural incidence matrix (parsed into E)

```
ram.execSim(case, 10.0)
A, E = ram.getJac()
```

Use this when you want to drive your own solver, for instance the sparse shift-invert methods in `scipy.sparse.linalg` that large systems need. To have RAMSES do the analysis instead, schedule an EIG disturbance; see Eigenanalysis.

Note

Set `$OMEGA_REF SYN` ; in the solver settings data file when exporting the Jacobian for eigenanalysis.

32.3. stepss.extractor: Result Extraction

The `stepss.extractor` class extracts and visualises time-series results from a trajectory file produced during simulation.

32.3.1. Initializing

Pass the trajectory file path to the extractor:

```
import stepss

case = stepss.cfg('cmd.txt')
# ... run simulation ...
ext = stepss.extractor(case.getTrj())
```

Or provide the file path directly:

```
ext = stepss.extractor('output.trj')
```

32.3.2. Curve Objects

All extraction methods return objects whose attributes are **curve objects** (`stepss.cur` named tuples). Every curve object has:

Attribute	Type	Description
time	<code>numpy.ndarray</code>	Time values in seconds
value	<code>numpy.ndarray</code>	Observable values
msg	<code>str</code>	Description string (used as plot legend label)

.plot()

Display the curve using Matplotlib:

```
bus = ext.getBus('4044')
bus.mag.plot()
```

32.3.3. Extraction Methods

getBus(name)

Retrieve voltage time series for a bus. Returns an object with:

Attribute	Description
<code>.mag</code>	Voltage magnitude (pu)

Attribute	Description
<code>.pha</code>	Voltage phase angle (deg)

```
bus = ext.getBus('4044')
bus.mag.plot() # voltage magnitude (pu)
bus.pha.plot() # voltage phase angle (deg)
```

getSync(name)

Retrieve the full set of synchronous machine observables. Returns an object with:

Attribute	Description
<code>.P</code>	Active power (MW)
<code>.Q</code>	Reactive power (Mvar)
<code>.S</code>	Rotor speed (pu; 1 = nominal)
<code>.A</code>	Rotor angle w.r.t. COI (deg)
<code>.FV</code>	Field voltage (pu)
<code>.FC</code>	Field current (pu)
<code>.T</code>	Mechanical torque (pu)
<code>.ET</code>	Electromagnetic torque (pu)
<code>.FW</code>	Field winding flux
<code>.DD</code>	d1 damper flux
<code>.QD</code>	q1 damper flux
<code>.QW</code>	q2 winding flux
<code>.SC</code>	COI speed deviation (pu; 0 = nominal)

Note

The two speed observables use different conventions: `.S` is the whole rotor speed (1 = nominal), while `.SC` is the deviation of the centre-of-inertia (COI) speed from nominal (0 = nominal). The system frequency in Hz is $f_{nom} * (1 + SC)$ or, per machine, $f_{nom} * S$.

```
gen = ext.getSync('g1')
gen.P.plot() # active power (MW)
gen.Q.plot() # reactive power (Mvar)
gen.S.plot() # rotor speed (pu)
gen.A.plot() # rotor angle w.r.t. COI (deg)
gen.FV.plot() # field voltage (pu)
gen.FC.plot() # field current (pu)
gen.T.plot() # mechanical torque (pu)
gen.ET.plot() # electromagnetic torque (pu)
```

getExc(name)

Retrieve exciter observables. Available observables depend on the exciter model.

Attribute	Description
<code>.obsdict</code>	dict mapping observable name → description
<i>(model-dependent)</i>	Access by observable name, e.g. <code>.vf</code>

```
exc = ext.getExc('g1')
print(exc.obsdict) # list available observables for this model
exc.vf.plot() # field voltage (model-dependent name)
```

getTor(name)

Retrieve governor/torque model observables. Available observables depend on the governor model.

Attribute	Description
<code>.obsdict</code> (<i>model-dependent</i>)	dict mapping observable name → description Access by observable name, e.g. <code>.Pm</code>

```
gov = ext.getTor('g1')
print(gov.obsdict)  # list available observables for this model
gov.Pm.plot()       # mechanical power (pu)
```

getInj(name)

Retrieve injector observables. Injectors include renewable energy sources (wind, PV, BESS), loads, and other single-bus components.

Attribute	Description
<code>.obsdict</code> (<i>model-dependent</i>)	dict mapping observable name → description Access by observable name, e.g. <code>.Pw</code>

```
inj = ext.getInj('WT1a')
print(inj.obsdict)  # list available observables
inj.Pw.plot()       # wind power (model-dependent name)
```

getTwop(name)

Retrieve two-port model observables. Two-port models include HVDC links (LCC and VSC), SVCs, and DC systems.

Attribute	Description
<code>.obsdict</code> (<i>model-dependent</i>)	dict mapping observable name → description Access by observable name, e.g. <code>.P1</code> , <code>.P2</code>

```
twop = ext.getTwop('hvdc1')
print(twop.obsdict)  # list available observables
twop.P1.plot()       # active power at terminal 1
twop.P2.plot()       # active power at terminal 2
```

getDctl(name)

Retrieve discrete controller observables. Discrete controllers include LTC transformers, under-voltage load shedding, phase shifters, etc.

Attribute	Description
<code>.obsdict</code>	dict mapping observable name → description

```
dctl = ext.getDctl('1-1041')
print(dctl.obsdict)  # list available observables
```

getBranch(name)

Retrieve branch (line/transformer) power flow time series.

Attribute	Description
.PF	Active power at FROM end (MW)
.QF	Reactive power at FROM end (Mvar)
.PT	Active power at TO end (MW)
.QT	Reactive power at TO end (Mvar)
.RM	Transformer ratio magnitude
.RA	Transformer phase angle (deg)

```
branch = ext.getBranch('1041-01')
branch.PF.plot() # active power at FROM end (MW)
branch.QF.plot() # reactive power at FROM end (Mvar)
branch.PT.plot() # active power at TO end (MW)
branch.QT.plot() # reactive power at TO end (Mvar)
branch.RM.plot() # transformer ratio magnitude
branch.RA.plot() # transformer phase angle (deg)
```

getShunt(name)

Retrieve shunt compensation time series.

Attribute	Description
.Q	Reactive power produced (Mvar)

```
ext.getShunt('sh1').Q.plot()
```

getLoad(name)

Retrieve load time series.

Attribute	Description
.P	Active power consumed (MW)
.Q	Reactive power consumed (Mvar)

```
ext.getLoad('L_1').P.plot()
```

32.3.4. Multi-Curve Plotting**stepss.curplot(curves)**

Display multiple curve objects on the same axes. Each curve's msg field is used as the legend label.

```
import stepss
```

```
ext = stepss.extractor(case.getTrj())
```

```
curves = [
    ext.getSync('g1').S,
```

```

        ext.getSync('g2').S,
        ext.getSync('g3').S,
    ]
    stepss.curplot(curves)

```

32.4. stepss.monitor: Live Plotting

`monitor` plots chosen quantities while a simulation runs. It steps the engine forward in slices, reads those quantities at every pause, and redraws a stacked figure: one panel per observable, all sharing the time axis.

The engine is polled in process, so nothing is read from disk and nothing has to be installed beyond matplotlib, which pip installs with the package. Runtime observables, above, are a separate mechanism in which the engine writes a curve file of its own; the two are independent and can be used together.

```

import stepss

ram = stepss.sim()
case = stepss.cfg('cmd.txt')
ram.execSim(case, 0.0)                                # initialise, paused at t = 0

mon = stepss.monitor(ram, ['BV 4044', 'MS g6', 'RT RT'], title='Nordic')
curves = mon.run(step=0.5)                             # run to the end of the scenario

```

The simulation must already be initialised and paused, which `execSim` does when given a pause time.

32.4.1. monitor(ram, observables, title=None, refresh=0.2, show=True)

Argument	Description
<code>ram</code>	The <code>stepss.sim</code> driving the run.
<code>observables</code>	What to plot: descriptor strings, (label, callable) pairs, or bare callables. A single observable need not be wrapped in a list.
<code>title</code>	Figure title.
<code>refresh</code>	Minimum wall-clock seconds between redraws. Samples are never skipped, only draws; 0 redraws at every sample.
<code>show</code>	False collects the samples and builds no figure.

32.4.2. Observable descriptors

The vocabulary is the one `addRunObs` uses, plus BA and the generic OBS:

Descriptor	Quantity	Unit
BV BUSNAME	Voltage magnitude of a bus	pu
BA BUSNAME	Voltage phase of a bus	deg
MS MACHINE_NAME	Rotor speed of a synchronous machine	pu
BPO / BQO / BPE / BQE	Active or reactive power at the origin or extremity of a branch	MW, Mvar
BRANCH_NAME	Named observable of an injector model	set by the model
ON INJECTOR_NAME	Named observable of an injector model	set by the model
OBSERVABLE_NAME	Named observable of a two-port model	set by the model
TO TWOP_NAME	Named observable of a two-port model	set by the model
OBSERVABLE_NAME	Named observable of a two-port model	set by the model

Descriptor	Quantity	Unit
OBS TYPE NAME OBSERVABLE_NAME	Named observable of any component type getObs accepts	set by the model
RT RT	Elapsed wall-clock time, to gauge simulation speed	s

Anything the descriptors do not cover is a callable, taking the simulator and returning one number:

```
mon = stepss.monitor(ram, [
    'BV 4044',
    ('4044 reactive reserve (Mvar)', lambda r: r.getObs('SYN', 'g6', 'Q')[0]),
])
```

32.4.3. `run(step=1.0, until=None)`

Advance the simulation, sampling and redrawing at each pause, and return one cur per observable. Returns when the engine reports the end of the scenario, when `until` is reached, or when a slice fails to advance the simulated time.

```
curves = mon.run(step=0.1, until=30.0)    # 30 s of simulated time, sampled every 0.1
↪ s
```

`step` is simulated seconds per slice, so it sets both the sampling resolution and the redraw cadence. The engine pauses at the first internal step at or after the requested time, so the samples land on or just after the multiples of `step`.

If the engine raises, the samples taken up to that point stay available from `curves()`:

```
from stepss.globals import RAMSESError

try:
    mon.run(step=1.0)
except RAMSESError:
    stepss.curplot(mon.curves())           # everything up to the failure
```

32.4.4. `curves()`

Return everything sampled so far, one cur per observable, in the order given to the constructor. These are the same objects `extractor` produces, so `curplot` and the rest of the post-processing accept them unchanged.

32.4.5. `sample()`

Read every observable once, at the current simulated time. Call it directly when driving the simulation yourself:

```
mon = stepss.monitor(ram, ['BV 4044'])
for target in [10.0, 20.0, 30.0]:
    ram.contSim(target)
    ram.addDisturb(target + 1.0, 'CHGPRM DCTL 1-1041 Vsetpt -0.01 0')
    mon.sample()
    mon.refresh()
```

32.4.6. `refresh(force=False)`

Push the samples into the figure and redraw it. A draw falling inside the `refresh` interval is skipped unless `force` is set.

32.4.7. savefig(fname, **kwargs)

Redraw and write the figure to a file, forwarding to `matplotlib.figure.Figure.savefig`.

32.4.8. close()

Close the figure. The samples are unaffected: `curves()` keeps working.

Note

On a file-only matplotlib backend such as Agg, and in Jupyter's inline mode, there is no window to animate: the monitor collects the samples and builds the figure, and `savefig` writes the same chart a window would have shown. Closing the window mid-run stops the redraws and leaves the run going.

32.5. Complete Example

```
import stepss
```

```
# --- Build test case ---
case = stepss.cfg()
case.addData('dyn_A.dat')
case.addData('settings1.dat')
case.addInit('init.trace')
case.addDst('events.dst')
case.addTrj('output.trj')
case.addObs('obs.dat')
case.addOut('output.trace')
case.addCont('cont.trace')
case.addDisc('disc.trace')

# Runtime observables, recorded by the engine into a curve file
case.addRunObs('BV 1041')
case.addRunObs('MS g1')
case.addRunObs('RT RT')

# Save configuration
case.writeCmdFile('cmd.txt')

# --- Run simulation ---
ram = stepss.sim()

# Start and pause at t = 80 s
ram.execSim(case, 80.0)

# Inject a disturbance dynamically
ram.addDisturb(100.0, 'FAULT BUS 4032 0. 0.')
ram.addDisturb(100.1, 'CLEAR BUS 4032')

# Export Jacobian at this operating point
A, E = ram.getJac()

# Run to end
ram.contSim(ram.getInfTime())

# --- Extract results ---
```

```
ext = stepss.extractor(case.getTrj())

# Bus voltages
ext.getBus('4044').mag.plot()

# Generator observables
gen = ext.getSync('g1')
gen.S.plot()    # rotor speed
gen.A.plot()    # rotor angle

# Branch flows
ext.getBranch('1041-01').PF.plot()

# Plot multiple rotor speeds together
stepss.curplot([
    ext.getSync('g1').S,
    ext.getSync('g2').S,
    ext.getSync('g3').S,
])
```

32.6. Next Steps

- Examples, Practical simulation examples and workflows
- Test Systems, Ready-to-run benchmark systems

33

Power flow from Python

The `stepss.helios` module wraps Helios, the STEPSS AC power-flow engine. Pre-compiled libraries are bundled with the package for Windows, Linux, and macOS, so no separate installation is required.

Unlike the RAMSES classes (which follow the historical camelCase conventions), this module uses PEP 8 snake_case naming. Errors are raised as `stepss.HeliosError`, carrying the engine's diagnostic message.

33.1. Basic Workflow

`HeliosSession` follows a load → (modify) → solve → query lifecycle and works as a context manager:

```
from stepss.helios import HeliosSession

with HeliosSession() as pf:
    pf.load_file('network.dat')
    converged = pf.solve()

    v, angle = pf.get_bus_voltage('1041')    # one bus (pu, rad)
    p, q = pf.get_branch_flow('1042-1044')    # from-end flow (MW, Mvar)
```

Solve diagnostics are available after `solve()`: `pf.converged`, `pf.solver_status`, `pf.iterations`, and `pf.max_mismatch`.

33.2. Solver Options

Options map to the \$PARAM records of the data file and can be overridden after loading (loading overwrites them with the file's values, so set options *after* `load_file`):

```
from stepss.helios import Option

pf.load_file('network.dat')
pf.set_option(Option.TOLAC, 0.001)    # MW
pf.set_option(Option.MAX_ITER, 30)
pf.solve()
```

33.3. Vectorized Results

Bulk getters return NumPy arrays indexed like the corresponding `*_names()` lists, with no text parsing needed:

```

v_pu, angle_rad = pf.get_bus_voltages()
p_load, q_load = pf.get_bus_loads()
p_from, q_from, p_to, q_to = pf.get_branch_flows()
p_gen, q_gen, status = pf.get_generator_outputs()

names = pf.bus_names()
print(f"lowest voltage: {v_pu.min():.4f} pu at {names[v_pu.argmin()]}")

```

Per-element dataclasses are available via `get_bus_info()`, `get_branch_info()`, and `get_generator_info()`.

33.4. Modifying the System

Modifications accumulate an active-power imbalance that `apply_changes()` settles: connectivity check, redispatch onto the remaining generators, and a re-solve (the same workflow as the interactive modify menu). `change_*` methods apply increments; `set_load()` sets absolute values:

```

pf.trip_branch('1042-1044')
pf.change_load('1041', 50.0, 10.0)      # +50 MW, +10 Mvar
pf.set_generator_voltage('g6', 1.02)
pf.apply_changes()                      # redispatch + re-solve

pf.reset()                             # back to the state saved at load

```

33.5. Contingency Screening

Run automatic N-1 over selected equipment classes, or a Fortran-format contingency file (BT/GT/ST/HC actions). Each result is a dataclass with convergence, acceptance, voltage/loading extremes, and violation strings; the base case is untouched afterwards:

```

results = pf.run_contingencies(branches=True, generators=True,
                               v_min=0.95, v_max=1.05, overload=100.0)
for r in results:
    if not r.accepted:
        print(r.name, r.violations)

results = pf.run_contingencies(file='contingencies.txt')

```

33.6. Exporting Results

```

pf.write_dump('solved_case.dat')        # re-loadable data file
pf.write_voltrat('volt_rat.dat')        # LFRESV + TRANSFO records, RAMSES initial
↪ conditions
pf.write_matlab('system.m')              # operating point + Y-bus script
pf.write_diagram('template.svg', 'diagram.svg')

```

`volt_rat.dat` is the natural bridge to dynamic simulation: solve a power flow with Helios, export it, and pass it to a RAMSES case via `case.addData('volt_rat.dat')`.

Note

`write_voltrat()` is the API form of the VT menu command, and writes the same records. For what those records contain, and how they relate to the hand-written TRFO style, see Exported Operating Point.

33.7. Runnable Examples

Five self-contained scripts ship in the repository under `examples/helios/`: basic solve, options and arrays, modify and re-solve, contingency screening, and exports.

33.8. Further Reading

- Power Flow user guide, data format, solver parameters, engine details
- Full method-level documentation: the docstrings on `HeliosSession`, e.g. `help(stepss.HeliosSession)`

34

uRAMSES, the C and Fortran interface

URAMES provides the framework and tools needed to compile and link custom Fortran models with stepss and the STEPSS GUI.

This is the route for user models. RAMSES resolves a model name against the models compiled into it, so your own models are linked into the engine and reach it the same way the built-in ones do. There is no way to load a model library while a simulation is running.

34.1. Prerequisites

Linux (Debian, Ubuntu)

- **gfortran** (GNU Fortran compiler)
- **OpenBLAS** (optimized BLAS library)
- **stepss**

```
sudo apt install gfortran make libopenblas-dev
```

Debian and Ubuntu are the Linux distributions STEPSS is built and tested on; see Installation. The kit itself needs only gfortran, GNU make and OpenBLAS, so it will build wherever those exist, but the compiler has to match the ABI of the bundled .mod files and that is checked against the distributions above.

macOS

- **gfortran** from Homebrew (macOS ships no Fortran compiler)
- **OpenBLAS**, or Apple's Accelerate framework
- **stepss**

```
brew install gcc openblas
```

Apple Silicon only: the pre-compiled modules_m/ kit is arm64.

Windows (MinGW)

- **MSYS2** with the MinGW-w64 toolchain
- **OpenBLAS**
- **stepss**

```
pacman -S mingw-w64-x86_64-gcc-fortran mingw-w64-x86_64-openblas
```

Build from the **MSYS2 MinGW 64-bit** shell. The plain MSYS shell links against the `msys-2.0` runtime and produces a DLL that CPython cannot load.

Windows (Intel)

- **Microsoft Visual Studio 2019** or later
- **Intel oneAPI Fortran Compiler**, installed from Intel's own distribution; the Installation page covers the gfortran toolchains only
- **stepss**

Match gfortran to the kit

Fortran `.mod` files can only be read by the gfortran release that wrote them. Each kit records the exact module ABI version it was built with in its own `BUILDINFO.txt` (`cat modules_l/BUILDINFO.txt`), and the same information appears in the toolchain table at the top of every uRAMSES release. Run `check-deps` on any Makefile to compare your compiler against the kit before building; it fails early and names both versions on a mismatch.

If your distribution's default gfortran is the wrong generation, install a matching one and pass it explicitly, e.g. `make -f build/Makefile.linux FC=gfortran-11`.

34.2. Project Structure

Every platform carries one suffix throughout: `l` for Linux, `m` for macOS, `wg` for Windows/MinGW (gfortran) and `wi` for Windows/Intel. A module kit and the output directory it builds into always share it.

```

URAMES/
+-- src/                                # Framework sources (all platforms)
|   +-- c_interface.f90                 # C interface for Python integration
|   +-- main.f90                       # Main entry point (executable only)
|   +-- FUNCTIONS_IN_MODELS.f90        # Helper functions for models
|   +-- usr_exc_models.f90             # Exciter model associations
|   +-- usr_inj_models.f90             # Injector model associations
|   +-- usr_tor_models.f90             # Torque model associations
|   +-- usr_twop_models.f90            # Two-port model associations
|   +-- usr_dctl_models.f90            # Discrete controller associations
+-- custom_models/                     # Your own models (all platforms)
|   +-- exc_*.f90                      # Exciter models
|   +-- inj_*.f90                      # Injector models
|   +-- tor_*.f90                      # Torque models
|   +-- twop_*.f90                     # Two-port models
|   +-- *.txt                          # Model parameter files
+-- modules_l/                         # Pre-compiled kit (Linux/gfortran)
|   +-- *.mod                          # Module interface files
|   +-- libramses.a                    # Pre-compiled RAMSES library
+-- modules_m/                         # Pre-compiled kit (macOS arm64/gfortran)
|   +-- *.mod                          # Module interface files
|   +-- libramses.a                    # Pre-compiled RAMSES library
+-- modules_wg/                       # Pre-compiled kit (Windows/MinGW gfortran)
|   +-- *.mod                          # Module interface files
|   +-- libramses.lib                  # Pre-compiled RAMSES library
+-- modules_wi/                       # Pre-compiled kit (Windows/Intel Fortran)
|   +-- *.mod                          # Module interface files
|   +-- libramses.lib                  # Pre-compiled RAMSES library
+-- build/                             # All build files

```

```
|  +-- Makefile.linux          # Linux builds
|  +-- Makefile.macos         # macOS builds (Apple Silicon)
|  +-- Makefile.windows       # Windows/MinGW builds
|  +-- msvs/                  # Visual Studio files (Windows/Intel)
|      +-- URAMESSES.sln      # Visual Studio solution
+-- tools/                    # Kit-verification and release helpers
+-- Release_l/                # Build output (Linux)
+-- Release_m/                # Build output (macOS)
+-- Release_wg/               # Build output (Windows/MinGW)
+-- Release_wi/               # Build output (Windows/Intel)
```

Each platform links its own pre-compiled RAMSES kit, published with the matching RAMSES release as `urameses-modules_{l,m,wg}-<version>.zip`.

Note

The Makefiles live in `build/` but are always run **from the repository root**, `make -f build/Makefile.linux`. `-f` does not change make's working directory, so every path inside them stays root-relative. Do not `cd build`.

34.3. Model Types

Type	Prefix	Description
Exciters	<code>exc_*</code>	Generator excitation system models
Injectors	<code>inj_*</code>	Current/voltage injection models
Torque	<code>tor_*</code>	Mechanical torque models
Two-port	<code>twop_*</code>	Two-port network models (SVC, STATCOM, HVDC)
Discrete Control	<code>dctl_*</code>	Discrete control system models

34.4. Building

Linux

Run from the repository root:

```
# Build shared library + executable
make -f build/Makefile.linux all

# Build only the shared library (for stepss)
make -f build/Makefile.linux dll

# Build only the executable
make -f build/Makefile.linux exe

# Verify gfortran, OpenBLAS and the module kit
make -f build/Makefile.linux check-deps

# Clean build artifacts
make -f build/Makefile.linux clean
```

The shared library (`ramses.so`) will be in `Release_l/`.

macOS

Run from the repository root:

```
# Build shared library + executable
make -f build/Makefile.macos all
```

```
# Build only the shared library (for stepss)
make -f build/Makefile.macos dll

# Build only the executable
make -f build/Makefile.macos exe

# Verify gfortran, BLAS, the architecture and the module kit
make -f build/Makefile.macos check-deps

# Clean build artifacts
make -f build/Makefile.macos clean
```

The shared library (ramses.so) will be in Release_m/.

Link against Apple's Accelerate framework instead of OpenBLAS with `make -f build/Makefile.macos BLAS=accelerate`, and select a versioned Homebrew compiler with `FC=gfortran-15`.

Note

The macOS library is named `ramses.so`, not `ramses.dylib`. `stepss` separates platforms using the `libs/lin`, `libs/mac` and `libs/win` folders rather than by filename, so Linux and macOS share the same name.

Windows (MinGW)

From the **MSYS2 MinGW 64-bit** shell, at the repository root:

```
# Build shared library + executable
make -f build/Makefile.windows all

# Build only the shared library (for stepss)
make -f build/Makefile.windows dll

# Build only the executable
make -f build/Makefile.windows exe

# Verify gfortran, OpenBLAS and the module kit
make -f build/Makefile.windows check-deps

# Clean build artifacts
make -f build/Makefile.windows clean
```

The shared library (ramses.dll) will be in Release_wg/.

This route is independent of the Intel Visual Studio projects; both can coexist in the same clone, writing to `Release_wg/` and `Release_wi/`.

Note

The Windows gfortran kit ships its archive as `libramses.lib` rather than `libramses.a`. MinGW's linker does not probe that name for `-lramses`, so the Makefile passes `modules_wg/libramses.lib` by explicit path. Do not replace that with `-lramses`.

Note

The resulting DLL depends on the MSYS2 runtime libraries (`libgfortran`, `libgcc_s`, `libgomp`, `libopenblas`). Keep the MinGW bin directory on `PATH`, or copy those DLLs next to `ramses.dll`, when loading it from Python.

Windows (Intel)

1. Open `build/msvs/URAMSES.sln` in Visual Studio
2. Select the desired project:
 - `dllramses`: builds `ramses.dll` (for `stepss`)
 - `exeramses`: builds `dynsim.exe` (standalone)
3. Build the solution (Release x64 configuration)
4. Output will be in `Release_wi/`

34.5. Adding Custom Models

1. **Write the model source:** Create a `.f90` file in `custom_models/` following the naming convention (`exc_`, `inj_`, `tor_`, `twop_`, or `dctl_` prefix)
2. **Register the model:** Add the model association in the corresponding `usr_*_models.f90` file in `src/`
3. **Rebuild:** Compile and link the modified URAMSES with the Makefile for your platform (`build/Makefile.linux`, `build/Makefile.macos` or `build/Makefile.windows`), which picks up new `.f90` files in `custom_models/` automatically. Only the Intel Visual Studio route needs the file added to the project by hand.

34.5.1. Example: Registering an Exciter Model

In `src/usr_exc_models.f90`, declare the subroutine and point the model pointer at it. The case label carries the `exc_` prefix, because the router has already added it to whatever name the data file used:

```
external :: exc_MY_EXCITER
...
select case (modelname4)
  case('exc_MY_EXCITER')
    exc_ptr=>exc_MY_EXCITER
```

The other four routers follow the same shape, with `tor_`, `inj_`, `twop_` and `dctl_` prefixes and their own pointer names. A data file then refers to the model as either `MY_EXCITER` or `exc_MY_EXCITER`.

Up to RAMSES 3.78 this was also how you reached a model that RAMSES compiled but registered under no name, such as `exc_AC8B` or `twop_HVDC_VSC`. From 3.79 every compiled model is registered, so that step is no longer needed. The technique still works for anything you want to expose under a second name: the subroutine is already in `libramses`, so no `external` declaration or source file of your own is required.

34.5.2. Included Example Models

The repository includes these example models:

File	Description
<code>exc_ENTSOE_lim.f90</code>	ENTSO-E exciter with limits
<code>inj_AIR_COND1_mod.f90</code>	Air conditioning load injector

Do not duplicate library models

Models that the linked RAMSES library already provides, such as `exc_ST1A`, `exc_GENERIC` and `tor_ENTSOE_simp`, must not be copied into `custom_models/`. Defining a subroutine that the library also exports produces a duplicate-symbol link error. Register those by name in the `usr_*_models.f90` routers instead.

34.6. Using Custom Models

34.6.1. With stepss

Point stepss to your custom library:

```
import stepss
ram = stepss.sim(custLibDir="path/to/Release_l")    # Linux
# ram = stepss.sim(custLibDir="path/to/Release_m")  # macOS
# ram = stepss.sim(custLibDir=r"path\to\Release_wg") # Windows (MinGW)
# ram = stepss.sim(custLibDir=r"path\to\Release_wi") # Windows (Intel)
```

stepss loads `ramses.so` on Linux and macOS, and `ramses.dll` on Windows.

34.6.2. With STEPSS GUI

The **Codegen** tab does this for you: it runs CODEGEN on your model description, then drives these same Makefiles against the bundled URAMSES kit to produce a custom `dynsim`, which it uses for subsequent simulations. You need `gfortran`, GNU `make` and `OpenBLAS` installed; see Installation. Building by hand and replacing the simulator manually is only necessary for models the Codegen tab cannot generate.

34.7. Helper Functions

The `FUNCTIONS_IN_MODELS.f90` module provides utility functions available in all models. See the Functions Reference for the complete list with signatures.

34.8. Repository

Source code: `SPS-L/stepss-uramses`

Part VIII

Test systems and resources

35

Test systems

Four benchmark networks ship as ready-to-run RAMSES data sets, each in its own repository. They are ordered here from smallest to largest, which is also the order in which they are worth meeting.

System	Buses	Machines	Frequency	Best for
5-Bus	5	1	50 Hz	Learning the tools
Kundur Two-Area	11	4	60 Hz	Inter-area oscillations, PSS tuning
Nordic	74	20	50 Hz	Long-term voltage stability
GB Network	87	37	50 Hz	HVDC, wide-area monitoring

35.1. Which One to Start With

- **New to STEPSS:** the 5-bus system is small enough to trace every computation by hand while still exercising the whole workflow, from power flow through disturbance to trajectory extraction.
- **Small-signal and damping studies:** the Kundur two-area system ships two dynamic data variants, with and without PSS, so the inter-area mode can be compared directly. Pair it with Eigenanalysis.
- **Voltage stability and long-term dynamics:** the Nordic system is the IEEE PES-TR19 benchmark, with several operating points and a Jupyter tutorial that walks through a voltage collapse.
- **Converter-dominated and HVDC studies:** the GB network models its interconnections as two-port converters and carries ready-made disturbance templates.

35.2. Running Any of Them

All four follow the same pattern: clone the repository, then point a stepss cfg at its data files.

```
import stepss
```

```
case = stepss.cfg()
case.addData('dyn.dat')      # dynamic data
case.addData('lf.dat')       # power-flow solution
case.addData('solveroptions.dat') # solver settings
case.addDst('disturb.dst')    # disturbance scenario
case.addObs('obs.dat')       # observables to record
```

```
sim = stepss.sim()  
sim.execSim(case)
```

The exact file names differ per system; each page lists them. See Python API Examples for complete worked scripts, and Installation if stepss is not set up yet.

35.3. Next Steps

- Python API Examples, full simulation workflows
- Disturbances, write your own scenarios
- Model Reference, the models these systems use

36

The 5-bus tutorial system

The 5-bus test system is a minimal power system suitable for learning the STEPSS tools and experimenting with dynamic simulation concepts. It is small enough to trace every computation step while still demonstrating the full simulation workflow, and runs in either edition: in Python as shown below, or in STEPSS GUI, which ships it as a bundled example.

Repository: SPS-L/stepss-5-bus-test-system

36.1. One-line Diagram

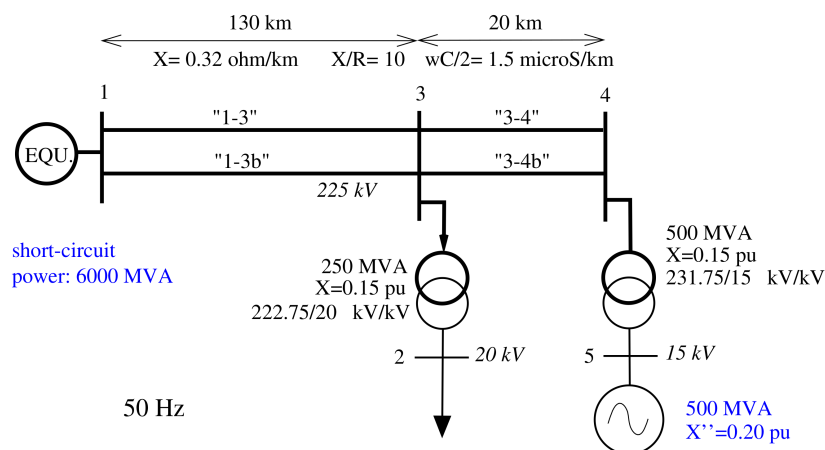


Figure 36.1: One-line diagram of the 5-bus test system

36.2. Quick Start

```
import stepss
import os

# Configure the test case
case = stepss.cfg()
case.addData('dyn.dat')           # dynamic model data
case.addData('lflsolv.dat')       # power-flow solution
case.addData('solveroptions.dat') # solver settings
```

```

case.addDst('nothing.dst')      # no pre-defined disturbances
case.addObs('obs.dat')          # observables to record
case.addTrj('output.trj')      # trajectory output file

# Remove stale output files from previous runs
for f in os.listdir('.'):
    if f.endswith('.trj') or f.endswith('.trace'):
        os.remove(f)

# Run simulation with exciter setpoint change
ram = stepss.sim()
ram.execSim(case, 0.0)
ram.addDisturb(1.0, 'CHGPRM EXC G Vo 0.05 2') # +0.05 pu step on Vo at t=1 s
ram.contSim(60.0)
ram.endSim()

# Extract and plot results
ext = stepss.extractor(case.getTrj())
ext.getSync('G').P.plot()      # active power
ext.getSync('G').Q.plot()      # reactive power

```

36.3. Download

The test system files are available in the `stepss-5-bus-test-system` repository.

36.4. See Also

- Test Systems, the other benchmark networks
- Python API Examples, Complete Python simulation workflow with this test system

37

The Kundur two-area system

The Kundur two-area system is the standard benchmark for inter-area oscillation studies: two symmetric areas connected by a weak tie, each with two 900 MVA synchronous generators (11 buses, 4 machines, 60 Hz). The RAMSES implementation uses the Kundur-book machine models with `exc_kundur` AVR/PSS and thermal governors, and ships two dynamic data variants, with and without power system stabilizers, so the damping of the inter-area mode can be compared directly.

Repository: SPS-L/stepss-Kundur-Two-Area-System

37.1. Quick Start

```
import stepss
```

```
case = stepss.cfg()
case.addData('lf.dat')           # power-flow data
case.addData('dyn.dat')         # dynamic data (use dyn_noPSS.dat for the no-PSS
    ↪ variant)
case.addData('solveroptions.dat') # solver settings
case.addDst('disturb.dst')       # +0.5 pu load step on L9 at t = 1 s, 60 s horizon
case.addObs('obs.dat')          # observables to record

sim = stepss.sim()
sim.execSim(case)
```

Running the same disturbance with `dyn.dat` and `dyn_noPSS.dat` (PSS gain set to zero) demonstrates the poorly damped inter-area oscillation and its stabilization by the PSS.

37.2. Download

The test system files are available in the `stepss-Kundur-Two-Area-System` repository.

37.3. Citation

If you use this test system in your research, please cite the original source of the system data:

P. Kundur, *Power System Stability and Control*, McGraw-Hill, 1994 (two-area system, Example 12.6).

37.4. See Also

- Test Systems, the other benchmark networks
- Eigenanalysis, analyse the inter-area mode this system is built to show
- Custom Exciters, the `exc_kundur` AVR/PSS used here

38

The IEEE Nordic test system

The Nordic test system is a well-established benchmark for voltage stability and long-term dynamics studies, originally defined in the IEEE PES Technical Report on Test Systems for Voltage Stability Analysis and Security Assessment. It represents a realistic multi-area transmission network and is used extensively in research and education.

38.1. System Overview

Property	Value
Nominal frequency	50 Hz
Buses	74
Synchronous generators	20 (g1–g20)
Transformers	50 (TRF0 records, step-up and distribution)
Transmission lines	52
Voltage levels	15 kV, 20 kV, 130 kV, 220 kV, 400 kV
Installed generation	16,550 MVA (11,500 MW dispatched at operating point A)

The system is divided into several areas interconnected through a meshed 400 kV transmission network. Lower voltage levels (130 kV, 220 kV) serve sub-transmission, while 15 kV and 20 kV buses connect generators and distribution loads.

38.2. One-line Diagram

38.3. Dynamic Models

Each generator is modeled as a detailed synchronous machine (SYNC_MACH) with:

- **Machine model:** Subtransient model with d- and q-axis dynamics (round-rotor or salient-pole depending on unit type)
- **Excitation system:** EXC GENERIC1. a generic AVR with field current limiter, OEL, and optional PSS (SPEEDIN type)
- **Governor/turbine:**

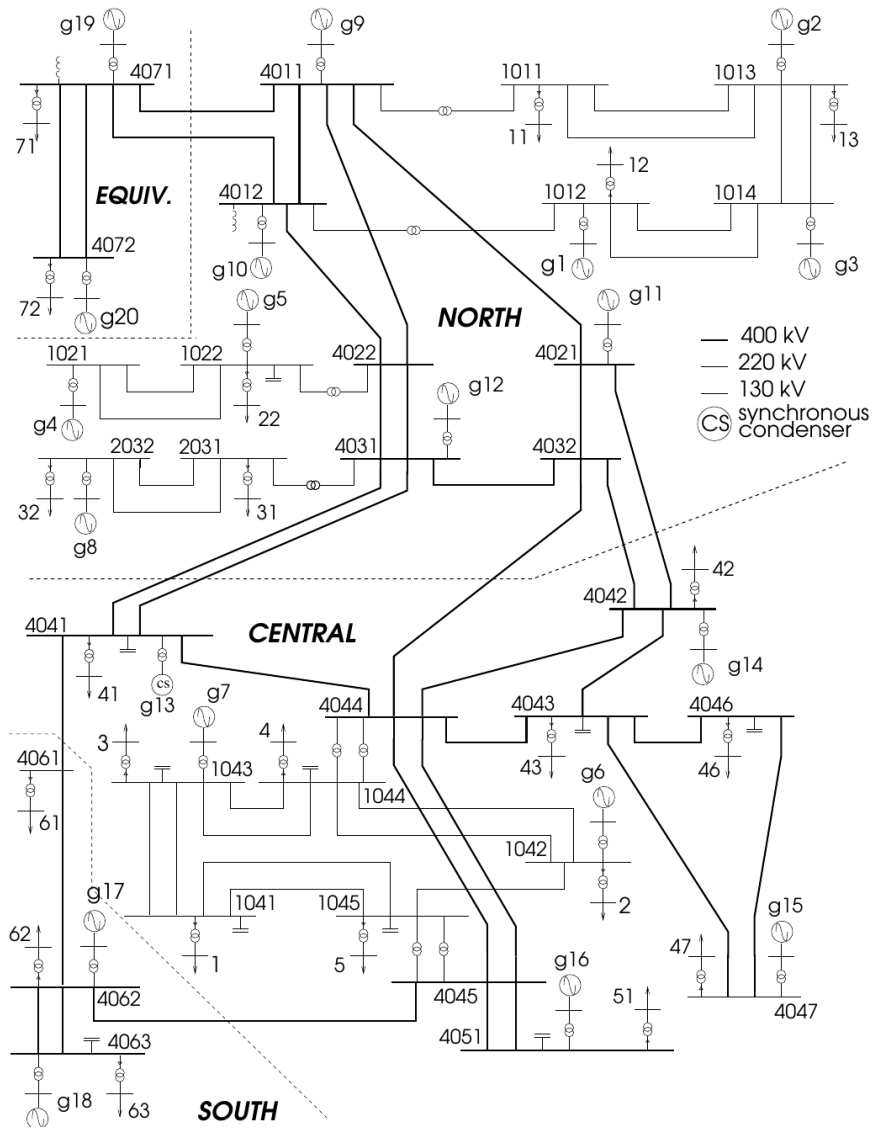


Figure 38.1: One-line diagram of the IEEE Nordic test system (from the Nordic test system report, T.

- Hydro units (g1–g5, g8–g12, g19–g20): TOR HYDRO_GENERIC1, hydraulic turbine with governor
- Thermal units (g6–g7, g13–g18): TOR CONSTANT: constant mechanical torque

Loads are represented using the INJEC vfd_load model (variable-frequency-dependent exponential recovery load) with voltage-dependent active and reactive power characteristics.

Under-load tap changers are modeled with DCTL LTC2 discrete controllers on all distribution transformers.

38.4. Operating Points

The repository provides two main operating points plus a family of load-increase variants:

File	Description
lf_A.dat / dyn_A.dat / volt_rat_A.dat	Operating Point A, base case, moderately stressed
lf_B.dat / dyn_B.dat / volt_rat_B.dat	Operating Point B, heavily stressed, closer to voltage collapse
lf_B_plus_*.dat / volt_rat_B_plus*.dat	Operating Point B with total load increased by 25 to 500 MW

Operating Point B is the primary case for voltage stability studies; it features higher load levels in the central area and reduced reactive power reserves. The B_plus variants (documented in doc/variants.pdf) progressively stress the system further and are useful for tracing the loadability limit.

38.5. Repository Contents

Path	Description
lf_*.dat / dyn_*.dat / volt_rat_*.dat	Load-flow data, dynamic data, and power-flow solutions for all operating points
settings1.dat	Solver configuration (time step, tolerances, threading)
obs.dat	Observation file, monitors all buses, branches, machines, injectors
uvls.dat	Undervoltage load-shedding (UVLS) controllers
nothing.dst	Empty disturbance file (undisturbed simulation)
trip_gen.dst / trip_branch.dst	Generator and branch trip disturbance scenarios
short_trip_branch.dst	Short-circuit followed by branch trip (_changeLTCs variant modifies tap-changer behavior)
eigen.dst / dampJac.dst	Jacobian export runs for eigenanalysis
cmd.txt / sim_*.cfg	RAMSES command file and STEPSS GUI simulation configurations (sim_nothing, sim_trip, sim_short_trip)
doc/	Detailed system report (Nordic_test_system_V6.pdf) and operating-point variants description (variants.pdf)
jupyterhub-tutorial/	Self-contained tutorial with Execute.ipynb, a step-by-step voltage collapse notebook

38.6. Disturbance Scenarios

38.6.1. Generator Trip (`trip_gen.dst`)

Trips generator g2 at $t = 1$ s and observes the long-term system response:

```
0.000 CONTINUE SOLVER BD 0.020 0.001 0.0 ABL
1.000 BREAKER SYNC_MACH g2 0
100.000 STOP
```

38.6.2. Short-Circuit with Branch Trip (`short_trip_branch.dst`)

Applies a three-phase fault on bus 4032 at $t = 1$ s, cleared after 100 ms by tripping the faulted branch:

```
0.000 CONTINUE SOLVER BD 0.020 0.001 0.00 ABL
1.000 FAULT BUS 4032 0. 0.
1.100 CLEAR BUS 4032
1.100 BREAKER BRANCH 4032-4044 0 0
200.000 STOP
```

The `jupyterhub-tutorial/` folder contains its own variants of these scenarios (e.g. with shunt compensation switching) used by the tutorial notebook.

38.7. Quick Start

38.7.1. Prerequisites

- Python 3 with `stepss` installed
- JupyterLab (recommended) or any Python environment

Install `stepss` following the installation guide.

38.7.2. Running a Simulation

1. Clone the repository:

```
git clone https://github.com/SPS-L/stepss-IEEE-Nordic-Test-system.git
cd stepss-IEEE-Nordic-Test-system
```

2. Open `jupyterhub-tutorial/Execute.ipynb` in Jupyter and run cells sequentially, or use the following Python script from the repository root:

Python Script

```
import stepss

case = stepss.cfg()
case.addData("dyn_B.dat")
case.addData("volt_rat_B.dat")
case.addData("settings1.dat")
case.addObs("obs.dat")
case.addDst("trip_gen.dst")

ram = stepss.sim()
ram.execSim(case, 150.0)

# Extract and plot results
ext = stepss.extractor(case.getTrj())
```

Jupyter Notebook

Open `Execute.ipynb` directly. It contains a complete guided tutorial with inline plots for: - Bus voltage evolution - Generator frequency deviations - Governor valve output and mechanical power - Active and reactive power output

38.8. What to Observe

After a generator trip on Operating Point B, the simulation demonstrates:

- **Frequency transient:** immediate frequency drop followed by primary governor response from hydro units
 - **Voltage dynamics:** progressive voltage decline in the central area as load restoration (tap changers, thermostatic loads) increases demand beyond available reactive reserves
 - **Long-term voltage instability:** if the system lacks sufficient reactive support, voltages collapse over tens of seconds. a classic long-term voltage stability phenomenon
-

38.9. License

The stepss-IEEE-Nordic-Test-system repository is licensed under the Apache License 2.0. The IEEE PES-TR19 report itself is not redistributed; obtain it from the IEEE Resource Center link below.

38.10. References

- IEEE PES Task Force on Test Systems for Voltage Stability Analysis and Security Assessment (chaired by T. Van Cutsem), "Test Systems for Voltage Stability Analysis and Security Assessment," IEEE PES Technical Report PES-TR19, Aug. 2015. Available from the IEEE Resource Center
- STEPSS project page at the Sustainable Power Systems Lab

38.11. See Also

- Python API Examples, Complete Python simulation workflow with this test system
- Python API Reference, Full API documentation for scripting simulations

39

The Great Britain network

The GB Network is a 50 Hz reduced-order representation of the Great Britain transmission grid with 87 buses and 37 synchronous machines (thermal and CCGT units with AVR/PSS and governor models), shunt compensation, and HVDC interconnections modelled as two-port converter injectors. It is used for dynamic simulation and wide-area monitoring/control studies; a one-line diagram is included in the repository.

Repository: SPS-L/stepss-GB-Network

39.1. One-line Diagram

39.2. Quick Start

```
import stepss

case = stepss.cfg()
case.addData('GBdyn.dat')      # dynamic data: machines, exciters/PSS, governors,
    ↪ loads
case.addData('GBhvd.c.txt')    # HVDC interconnection two-port models
case.addData('GBvoltr.dat')    # power-flow solution
case.addData('settings.dat')   # solver settings
case.addDst('disturb.dst')     # 180 s run with commented disturbance templates
case.addObs('obs.dat')        # observables to record

sim = stepss.sim()
sim.execSim(case)
```

The disturbance file contains ready-made (commented) templates for line and machine trips, bus faults, and HVDC parameter changes; uncomment and adapt them as needed. Alternatively, run the RAMSES executable directly with the included `cmd.txt`.

39.3. Download

The test system files are available in the stepss-GB-Network repository.

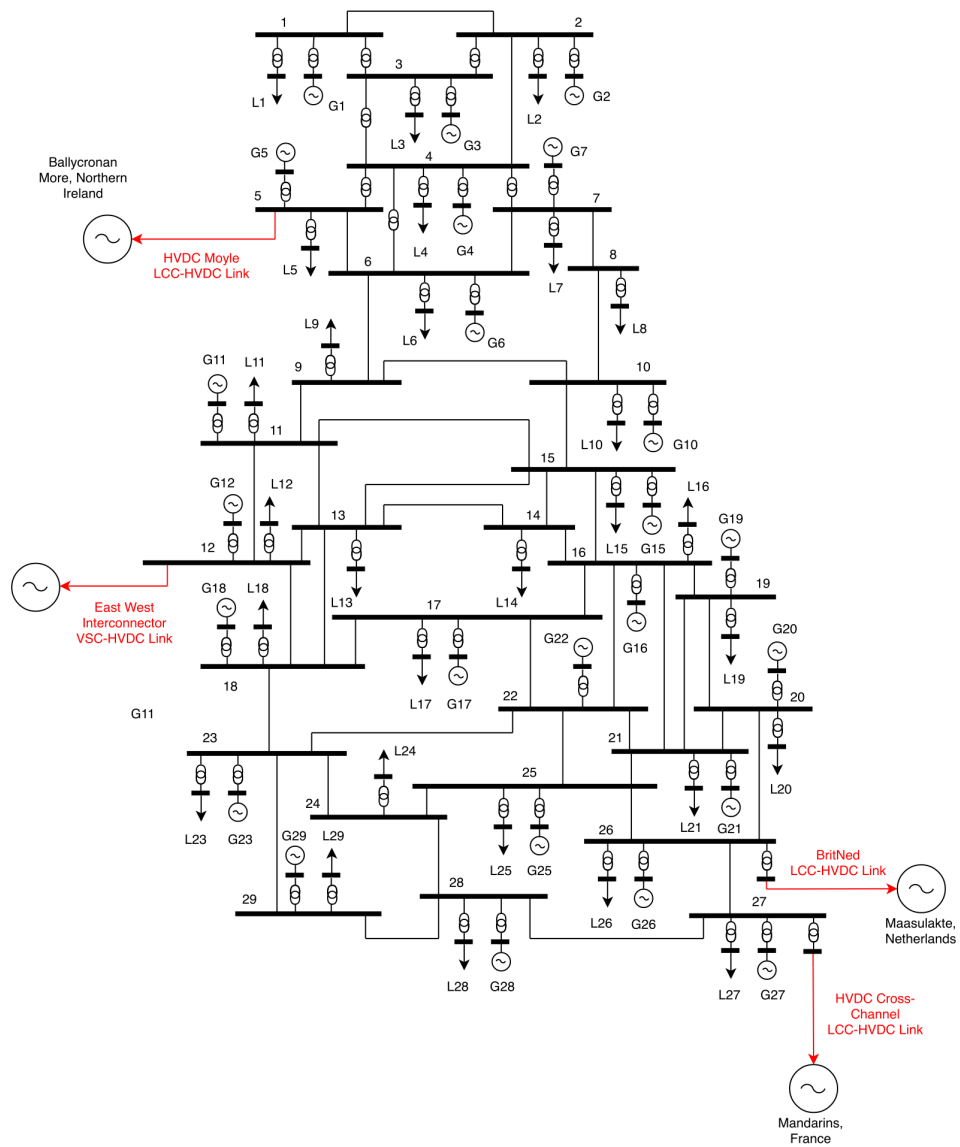


Figure 39.1: One-line diagram of the GB network with HVDC-LCC links

39.4. See Also

- Test Systems, the other benchmark networks
- Two-Port Models, the HVDC converter models used here
- Disturbances, syntax for the commented templates in `disturb.dst`

40

Component repositories

All STEPSS repositories are hosted under the SPS-L GitHub organization.

40.1. Repository Index

Repository	Role	Language	Visibility
stepss-java-ui	Java-based GUI front-end, bundling the whole toolchain	Java	Public
stepss-python-ui	Python API/wrapper for RAMSES and the Helios power-flow engine	Python	Public
stepss-userguide	LaTeX user documentation and models reference	LaTeX	Public
stepss-uramses	User-defined device models framework	Fortran	Public
stepss-eigenanalysis	Reference data and validation suite for small-signal analysis	Python	Public
stepss-cg-studio	Visual block diagram editor for CODEGEN models	Python/JS	Public
stepss-ramses	Core RAMSES simulation engine	Fortran	Private
stepss-test-systems	Curated collection of test cases and network models	RAMSES data	Private
stepss-helios	AC power-flow engine (Newton-Raphson), with a C API shared library wrapped by stepss (stepss.helios); releases ship CLI and C API binaries for Linux/macOS/Windows	C++	Private
stepss-dyngraph	Dynamic graph / topology module	Fortran	Private
stepss-Codegen	DSL-to-Fortran model code generator	Fortran	Private
stepss-RamsesNN	Physics-informed neural network experiments on RAMSES models	Python	Public
stepss-docs	This documentation website	Astro/Starlight	Public

40.2. Public Repositories

40.2.1. STEPSS (GUI)

The main graphical user interface for STEPSS, built with Java (Swing/AWT) and the Ant build system.

- **Repository:** github.com/SPS-L/stepss-java-ui

- **License:** Apache License 2.0
- **Requirements:** 64-bit Java 11 or later to run the jar, none at all to run an installer, JDK plus Apache Ant to build
- **Build:** `ant jar`
- **Run:** `java -jar dist/stepss.jar`, or `ant bundle` for a native installer

40.2.2. stepss-python-ui

Python interface to the RAMSES simulator providing scripting access to simulations, plus the `stepss.helios` module for AC power flows with the bundled Helios engine (Windows, Linux, and macOS).

- **Repository:** github.com/SPS-L/stepss-python-ui
- **Install:** `pip install stepss`
- **Documentation:** Python API section on this site
- **Includes:** five runnable power-flow examples under `examples/helios/`

40.2.3. stepss-userguide

The original LaTeX source for the STEPSS documentation (models reference and user guide).

- **Repository:** github.com/SPS-L/stepss-userguide
- **Build:** `pdflatex stepss_doc.tex` (run twice)
- **Main file:** `stepss_doc.tex`

40.2.4. stepss-uramses

Framework for compiling and linking custom Fortran models with RAMSES.

- **Repository:** github.com/SPS-L/stepss-uramses
- **Platforms:** Linux (gfortran), macOS (gfortran, Apple Silicon), Windows (MinGW gfortran or Intel Fortran)
- **Build (Linux):** `make -f build/Makefile.linux all`
- **Build (macOS):** `make -f build/Makefile.macos all`
- **Build (Windows/MinGW):** `make -f build/Makefile.windows all`
- **Build (Windows/Intel):** Visual Studio with `build/msvs/URAMES.sln`

40.2.5. stepss-eigenanalysis

Reference spectra and the validation suite for RAMSES's built-in small-signal analysis. The analysis itself runs in the engine, documented under Eigenanalysis; this repository holds the independently captured reference data the engine is checked against, so its tests need neither a RAMSES licence nor the engine itself.

- **Repository:** github.com/SPS-L/stepss-eigenanalysis
- **Requirements:** Python with numpy and pytest

40.2.6. stepss-cg-studio (CODEGEN Studio)

Browser-based visual editor for building CODEGEN user-defined models with drag-and-drop blocks.

- **Repository:** github.com/SPS-L/stepss-cg-studio
- **Requirements:** Python 3.10 or later. On macOS also `brew install gcc`, which the bundled CODEGEN links against.

- **Install:** `pip install stepss-cg-studio`
- **Run:** `cg-studio` → open `http://localhost:8765`
- **Documentation:** CODEGEN Studio guide on this site

The wheel bundles the CODEGEN executables for Linux, Windows and macOS, so Run Codegen works on a fresh install with nothing to configure. That makes it mixed-licence: see License. Its version names the CODEGEN it carries, so 5.3.0 and 5.3.1 both run CODEGEN 5.3.

Note

Test-system data repositories (Nordic, 5-bus, Kundur, GB Network, and others) are not listed here; see the Test Systems section of this site.

41

References

41.1. Core RAMSES / STEPSS Publications

1. P. Aristidou, D. Fabozzi, and T. Van Cutsem, "Dynamic simulation of large-scale power systems using a parallel Schur-complement-based decomposition method," *IEEE Transactions on Parallel and Distributed Systems*, vol. 25, no. 10, pp. 2561–2570, Oct. 2014. Doi: 10.1109/TPDS.2013.252
2. P. Aristidou, S. Lebeau, and T. Van Cutsem, "Power system dynamic simulations using a parallel two-level Schur-complement decomposition," *IEEE Transactions on Power Systems*, vol. 31, no. 5, pp. 3984–3995, Sept. 2016. Doi: 10.1109/TPWRS.2015.2509023
3. D. Fabozzi, A. Chieh, B. Haut, and T. Van Cutsem, "Accelerated and localized Newton schemes for faster dynamic simulation of large power systems," *IEEE Transactions on Power Systems*, vol. 28, no. 4, pp. 4936–4947, Dec. 2013. Doi: 10.1109/TPWRS.2013.2251915
4. P. Aristidou and T. Van Cutsem, "A parallel processing approach to dynamic simulations of combined transmission and distribution systems," *International Journal of Electrical Power & Energy Systems*, vol. 72, pp. 58–65, Nov. 2015. Doi: 10.1016/j.ijepes.2015.02.011

41.2. Reference Frame and Solver

5. D. Fabozzi and T. Van Cutsem, "On angle references in long-term time-domain simulations," *IEEE Transactions on Power Systems*, vol. 26, no. 1, pp. 483–484, Feb. 2011.

41.3. Project Pages

- STEPSS, SPS-Lab, Main project page with overview and downloads
- Thierry Van Cutsem, Software, Co-author's software page

41.4. External Resources

- Python API Documentation, API reference and examples
- Sustainable Power Systems Lab, Research group homepage